

11 - Data Scientist:

**Inference
Specialist**

Welcome to the Data Scientist: Inference Specialist Career Path

Your data science journey begins here!

Have you ever wondered how things affect each other? Do you think about how we can determine if x caused y , or if they are just randomly related phenomena? Have you ever heard someone say “correlation isn’t causation”, and thought to yourself: “well, then what is causation?” If you’ve answered ‘yes’ to any of these questions, you are in the right place!

Data Scientists who specialize in inference can answer all of these questions and more. In this Career Path, you will specialize in statistics, hypothesis testing, A/B tests, and sophisticated statistical analysis. As an Inference-based Data Scientist, you might work on research, insights, or experimentation teams. You will often work to find root causes for why things happen, and calculate the influence that different factors have on outcomes.

About the Data Scientist Career Paths

The Data Scientist Career Paths are designed for you to gain the knowledge, skills, and abilities (KSAs) you need to earn a data scientist job. You will develop the statistical knowledge, programming skills, and visualization techniques to work with data responsibly. You will also build a portfolio along the way to showcase your abilities to future employers.

You are currently enrolled in the Data Scientist: Inference Specialist Career Path. Data Scientists who focus on statistical analysis of data. As an Inference-based Data Scientist, you might work on experimental research/insights teams, or specialize in statistics, hypothesis testing, A/B tests, and sophisticated analysis.

This Career Path is divided into 2 parts. The first section covers all the basics every Data Scientist needs. We call this the Foundations of Data Science. The second section is the specialized KSAs you need to go from being a data generalist to an Inference-based Data Scientist.

If after learning a bit more about data science, you decide that you’d like to change your specialty, your progress will transfer, allowing you to pick up where you left off. To learn more about the specialties, check out [this blog post](#).

Overview

In the first half of this Career Path, you will learn about Python, SQL, statistics, and data best practices. You will start with a code-free introduction to working with data in the Data Literacy Track and then dive into *SQL* and *Python* in the SQL Fundamentals and Python Fundamentals Tracks. SQL and Python are essential tools that will give you the technical foundation you need to work with data programmatically.

Then you will dive into Data Manipulation with Python Pandas. Pandas is a Python library at the heart of Data Science. It is an essential tool for working with data and has dozens of useful built-in functions. Once you have mastered *Pandas*, you will apply your new programming skills to Exploratory Data Analysis where you will learn to summarize datasets before mastering the Fundamentals of *Statistics* where you will learn to spot trends and patterns.

Next, you will learn how to transform those numbers and analyses into meaningful *Data Visualizations*. Then you will apply your new skills to Wrangle and Tidy Datasets and prepare real data for analysis. Finally, you will bring it all together to Communicate Data Science Findings to various groups and stakeholders.

At this point in your journey, you will have completed the Data Foundations and be ready to move into your specialty.

Inference Specialty

In the Inference Specialty, you will learn about the most commonly used statistical techniques to test hypotheses, isolate variables, and conduct experiments. You'll be able to dig into data to find out both what happened and why! You'll build on your programming foundations by learning R and applying it to Causal Inference.

But, if you decide that you want to switch specialties after you learn more about Data Science, you can pick from any of our four specialties – and your progress will transfer!

By the end of the Data Scientist: Inference Specialist Career Path, you will be able to:

Along the way, you will build a portfolio of projects to showcase your data science KSA's, and prove your job-ready skills!

Structure

Throughout this Path, you will see lessons, quizzes, articles, and projects. Lessons and articles introduce new tools and concepts, quizzes give you a chance to quickly check your understanding, and projects give you a chance to apply your new skills.

There are a lot of projects, but pay special attention to Portfolio Projects and Cumulative Projects because they give you a chance to apply your skills and build up your portfolio.

The content in this Career Path is cumulative, and we recommend that you take the courses in order. However, if you are familiar with a given technology or idea, feel free to jump ahead, but be aware that skipping modules will affect your completion percentage.

Finally, we are constantly updating our content to be sure it is up to date and of the highest quality. Keep an eye on the notification bells, which we will use to alert you to new content. Sometimes these updates might affect your completion percentage. We will always do our best to let you know with a notification bell when that might happen.

Community

Finally, learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

Data Scientist: Inference Specialization

Inference Data Scientists go by a lot of different titles. This role can be called a

Preview: Docs Loading link description

[Statistics](#)

Data Scientist, Data Analyst, Causal Scientist, Marketing Analysts, etc. It is essential to read the job description to understand what a given job will entail.

Overview of the Role

Data Scientists who specialize in Inference leverage statistics to understand data. As an Inference Data Scientist, you will work with and influence decision-making by developing and testing hypotheses and conducting experiments. You may conduct and own A/B tests, explore datasets to identify patterns and causal relationships. The key differentiator of an Inference Data Scientist is the emphasis on statistical methods to evaluate the relationship between

Preview: Docs Loading link description

[variables](#)

and how manipulating real-world products or features can be measured in the resulting data.

How much will you make?

Estimated Avg. Total Pay \$123,325 (Glassdoor, 2022) Estimated Salary Range with 0-1 years experience:

\$54,000 - \$299,000 (Glassdoor, 2022) *Note that we find both ends of this range to be uncommon but have never heard of someone new to Data Science making \$299k, so interpret these estimates cautiously.*

Sample Job Description

- Use statistical methodologies to analyze datasets, build new products and evolve existing models.
- Apply stratification, variance reduction, and other advanced experiment design techniques to create A/B tests to efficiently measure the impact of your innovations while minimizing risk to the broader system.
- Apply various statistical methods, including causal inference, to understand the intricate ecosystem of our clients, customers, users, products, partners, and revenue.
- Generate and test hypotheses and analyze and interpret the results of experiments.

- Use statistical tools to identify specific, high-impact opportunities to build new and improve existing solutions.
- Build key data sets to empower operational and exploratory analysis.
- Define, forecast, evaluate and report on key business metrics.
- Work closely with adjacent teams (product engineers, marketing, sales, HR, and leadership) to understand their use cases, develop models, and balance scientific rigor and strategic considerations.
- Provide data visualization support to help the team make decisions.
- Find new ways to drive business impact and solve complex problems.
- Learn new tools, systems, and languages quickly.

What the Inference Specialization covers

- Data Science Foundations
- Advanced Statistics
- Causal Inference
- Deeper dive into A/B Testing & Hypothesis Testing
- Experimental Design
- Advanced SQL
- Advanced Data Visualization
- Data Visualization with Tableau

Analyzing Spreadsheets with Excel

Inferential Analysis

A/B tests are a popular business tool that data scientists use to optimize websites and other online platforms. A/B tests are a type of **inferential analysis**. Inferential analysis lets us test a hypothesis on a sample of a population and then extend our conclusions to the whole population.

Imagine we want to know if a blue or a green button will get more clicks on a website. We hypothesize that the green button will be more successful and we run an A/B test on a sample of people that visit our site. Half the sample sees the blue button (option A) and half see the green button (option B). At the end of the test, 90% of people that saw the green button clicked it, whereas 60% of the people that saw the blue button clicked it.

Now we need to ask, "If color wasn't related to click rate, how likely was a 30% difference just by chance?" We can use

Preview: Docs Loading link description

[statistics](#)

to calculate that probability. If there is less than a 5% probability that our results happened by chance, we have good evidence that green buttons get more clicks! We can extend these results to everyone that visits our site (the whole population), so it makes sense to use green buttons on our website.

This is a powerful thing to be able to do! But since it's so powerful, there are some rules about how to do it:

- Sample size must be big enough compared to the total population size (10% is a good rule-of-thumb).

- Our sample must be randomly selected and representative of the total population.

We can only test one hypothesis at a time.

Instructions

1. Based on what we know about A/B tests, consider the following scenarios:

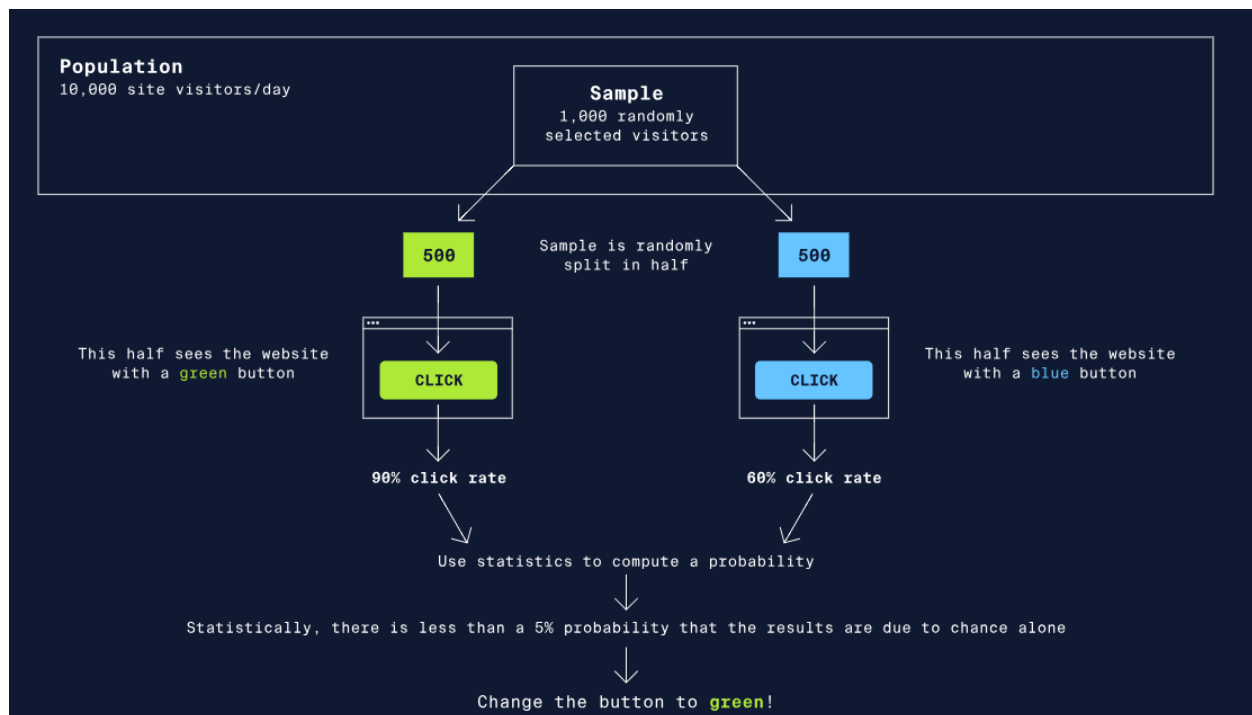
If the A/B test was being run for a site with 1 million visitors per day, would a sample size of 100 be big enough to extend conclusions from the sample to the whole population?

The site in the A/B test has visitors from all over the planet. If you ran the A/B test for 12 hours when only one-half of the planet was awake, would your sample be representative of the whole population?

When assigning people to see either the blue or green button, would it be better to randomly assign people or only assign people to the green group if they say their favorite color is green?

If instead of testing a green button versus a blue button, the test was of a blue button with small font vs. a green button with large font, would we be able to conclude that the color of the button was influencing whether visitors clicked the button?

Next up, causal analysis!



Causal Analysis

We know that correlation does not mean causation. This is an important limitation in data analysis. We should be cautious to believe any studies or headlines claiming that one thing caused another without knowing their research methods. However, we often really want to know why something happened. In these cases, we turn to **causal analysis**. Causal analysis generally relies on carefully designed experiments, but we can sometimes also do causal analysis with observational data.

Experiments that support causal analysis:

- Only change one variable at a time
- Carefully control *all* other variables
- Are repeated multiple times with the same results

These are pretty high standards to meet. However, experiments that meet these standards are the clearest way to figure out if one thing caused another.

Instructions

1. Take a look at the breakdown of good experimental design in the learning environment and think about the following questions:

Could you determine if drinking more water caused better sleep by monitoring your own sleep for a month and comparing results from days that you drank 8 glasses of water and days that you did not?

Could you determine if drinking more water caused better sleep if you expanded your study to include 1,000 subjects and asked all men to record their sleep on days they drank 8 glasses of water and all women to record their sleep on days that they did not drink 8 glasses of water?

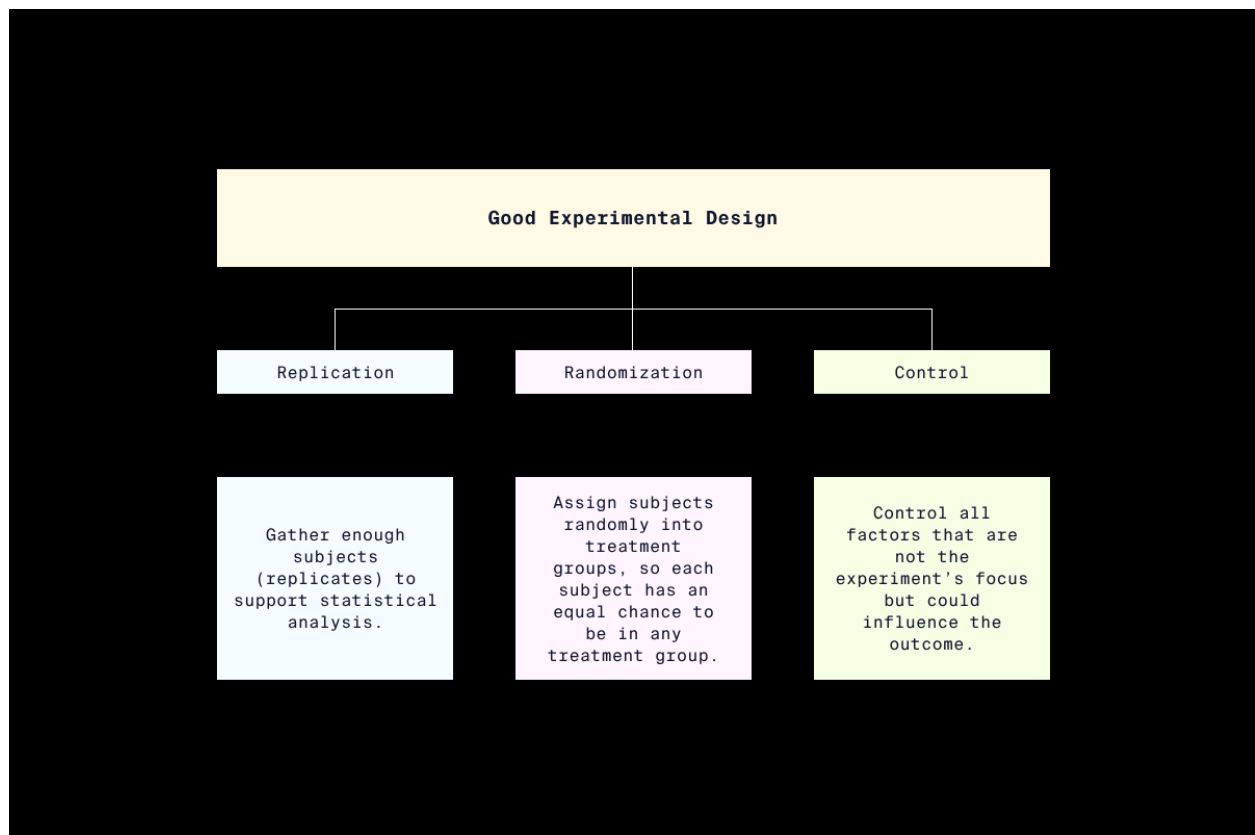
2. Remember:

Correlation does not equal causation.

Proving causation is tricky and generally requires very careful experimental design.

Replication, randomization, and control are key components of good experimental design.

In the next exercise, we will consider when and how to do causal analysis if an experiment is impossible.



Causal Analysis with Observational Data

Sometimes we need to establish causation when actual experimentation is impossible. This could be due to a variety of reasons. For example, we might want to know why something happened that we really don't want to repeat (e.g., why did a product flop?), or the necessary experiments may be too difficult, too expensive, or unethical. In such cases, we can sometimes apply causal inference techniques to observational data, but we need to be very careful.

Causal inference with observational data requires:

- Advanced techniques to identify a causal effect
- Meeting very strict conditions
- Appropriate statistical tests

Let's think about this in terms of climate change. It is important to know if climate change is causing more frequent and intense hurricanes. But, we can't do a controlled, replicated experiment on multiple planets with and without climate change. Instead, climate scientists carefully use causal analysis on observational data to determine whether climate change contributes to bigger hurricanes happening more often.

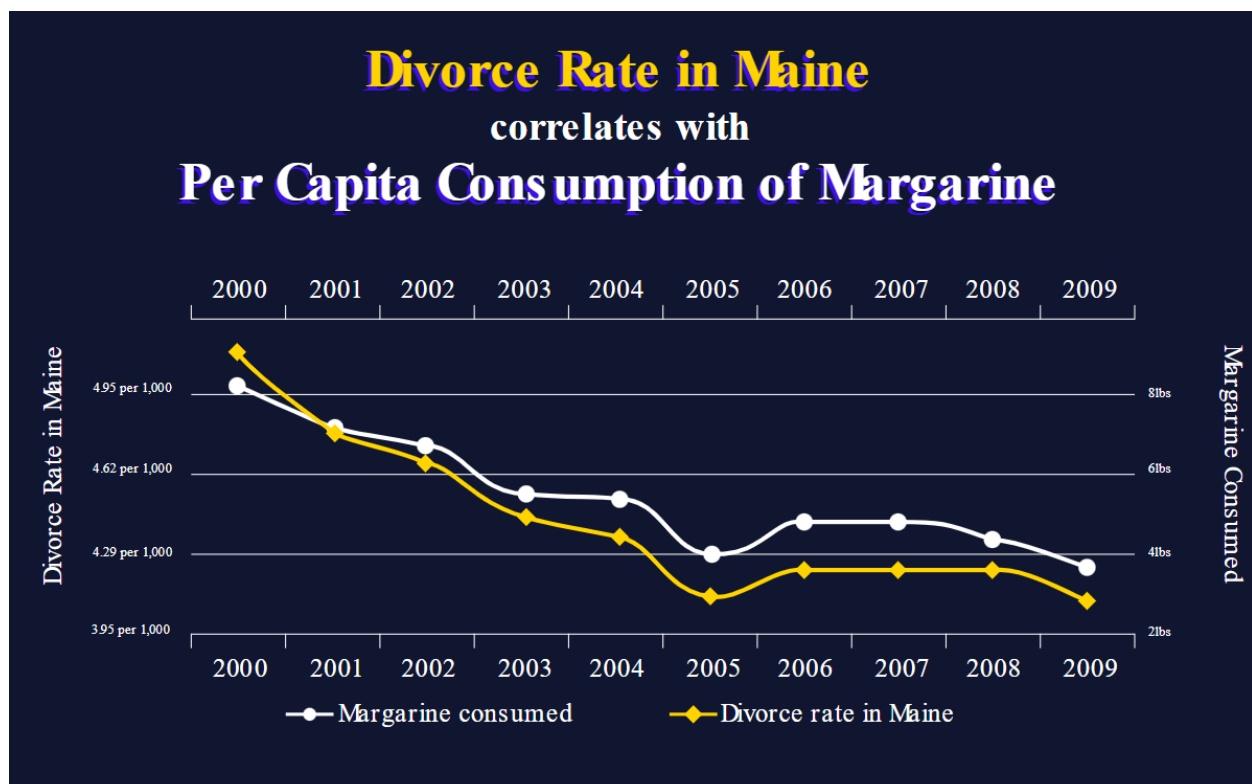
Instructions

1. Take a look at the observational data in the learning environment that comes from the site [Spurious Correlations](#). Consider the following questions:

Is margarine consumption correlated with divorce rate?

Does consuming less margarine *cause* fewer people to get divorced in Maine? Why or why not?

Next, we will discover a popular form of analysis that is used to make predictions!



Why Data Science?

Begin Your Journey

Catherine learned and applied a lot during the first two weeks of her job. She:

- Explored churn data using SQL

- Explored Codecademy's datasets

- Analyzed data using pandas, a Python library

- Visualized data using Matplotlib, a Python library

Now it's your turn. By the end of the Data Science Path, you'll be able to do all of the things that Catherine can do. We will also be adding new content every month.

Good luck and happy coding!



Data Types and Quality

Review of Data Types and Quality

Congratulations! You've done the hard work of collecting your data and preparing it for analysis. Along the way you've addressed some data quality issues and considered the relationship between your data and your questions.

While preparing the North Atlantic Tree Census, you've:

- Defined your

- [variables](#)

- Preview: Docs Loading link description to create tidy datasets

- Classified the variables based on their types

- Reconciled messy data

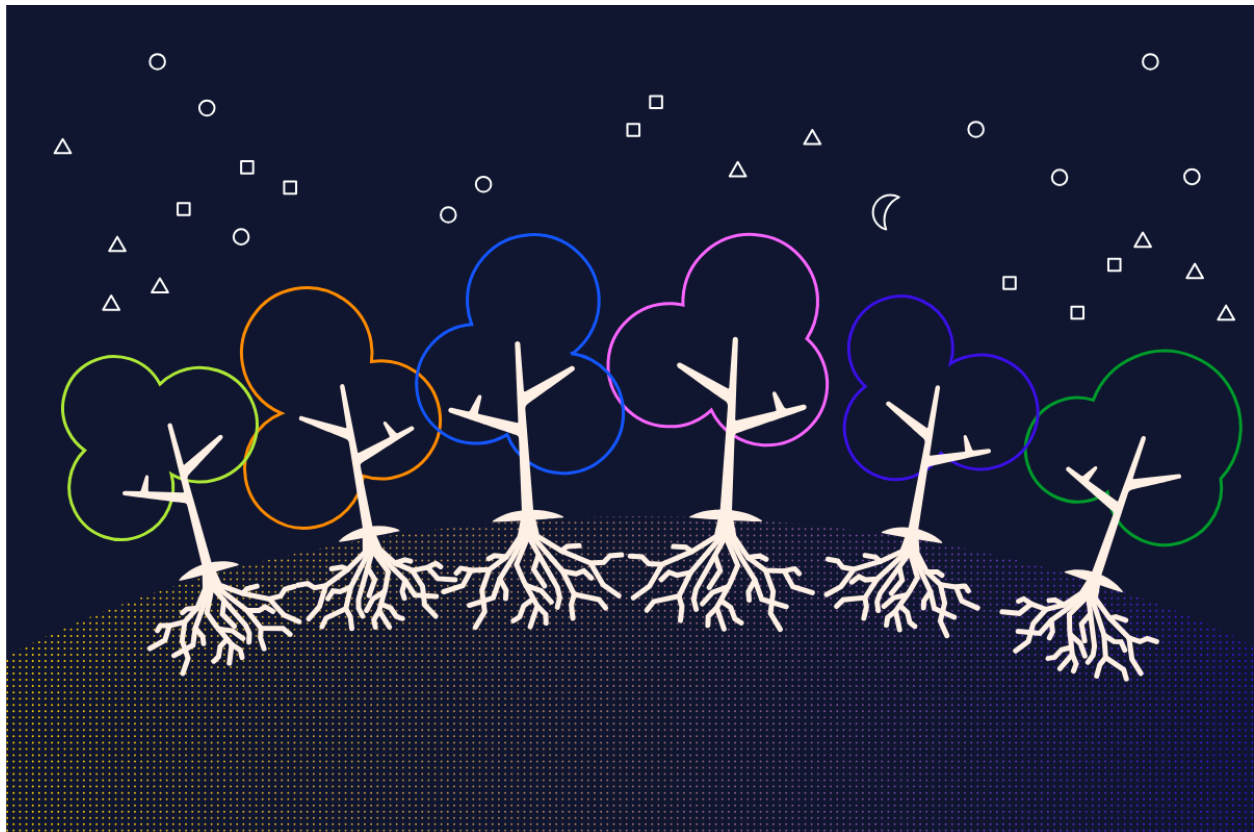
- Decided how to deal with missing data

- Addressed issues of accuracy

- Aligned your questions to the available data to ensure validity

- Created representative samples

These are important techniques for anyone working with data to always be conscious of. In the end, your human oversight and critical consideration of the data will have the biggest impact on data quality.



Welcome to Data Literacy

Establishing how to think about data will set you up for success when you start analyzing it.

Every job is a data job!

– every Data Scientist ever

Organizations are fueled by data, and data scientists and analysts are at the forefront of working with it. But they don't work alone, and even the most technically sophisticated data scientist will need to communicate about data.

This unit will help you build the conceptual foundation you need both to work with data yourself and present your insights to adjacent teams.

There's no programming yet, but we will get there. For now, join us on a journey to explore the building blocks of data, and establish common vocabulary and ideas.

What will Data Literacy cover?

After this unit, you will be able to:

We're excited for you to start your journey into Data Literacy!

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

How to Showcase your Data Science Skills

And start building your data science portfolio.

If a tree falls in the forest and no one is around to hear it, does it make a sound?

– Philosophy students everywhere

Hiring managers have their own version of that question: If a data scientist doesn't have a portfolio, do they really have the skills?

You need to have a portfolio. A portfolio is evidence that you have the technical skills and knowledge that you say that you do. But what goes into a portfolio and what does that mean in data science?

What Goes Into a Portfolio

Ultimately, exactly what you put in your portfolio is up to you. This project at the end of our Interview Prep Skill Path will help guide you through making sure that you've covered all of the major categories, but the projects themselves are completely up to you.

We recommend that you demonstrate the ability to:

Formulate questions that can be answered by real data

Clean and transform a real dataset to prepare it for analysis

Apply the right statistical techniques for the data to answer the question

Apply the right machine learning models where applicable

Support your analysis with visualizations

Summarize the main takeaways from your project

Throughout this Career Path, you will encounter Capstone and Portfolio projects. These can absolutely be put into your portfolio. However, thousands of people take this Career Path. To really make yourself stand out, you will want to add projects about things you are interested in.

If you like sports, use sports data, if you like science, use scientific data, etc. Showing a little bit of your interests and what you are curious about will not only make the project more fun for you, but also let you showcase your own domain knowledge or curiosity. Interest and enthusiasm in your projects will help set you apart from other candidates when you are applying and interviewing.

There are a lot of places to look for datasets. We have an entire lesson on where to find data later in this Career Path. For right now, the best thing you can do is be curious about the world around you, and keep track of some things that you might want to explore as you build your portfolio.

What a Data Science Portfolio Looks Like

A data science portfolio can take a lot of shapes, you could create your own professional website, upload documents to your LinkedIn or other social media site, or create a repository on GitHub. We highly recommend GitHub. It is simple, lightweight, and by far the most popular place to house a portfolio.

You have a few choices in terms of what kinds of files you want to upload. A general rule is the simpler, the better. You can upload Jupyter notebooks of your projects, .Rmd or Python files of your code, or PDFs, or presentations of summary reports.

The kind of files you upload will depend on the kind of Data Scientist role you are applying for. A role that is more programming-focused (i.e., a Machine Learning specialization) should have a more programming-heavy portfolio. A role that is more focused on analysis and communication should have at least one summary report.

The Culture of Portfolios

There are a few unspoken rules about portfolios that you should keep in mind. Throughout the Career Path, we've tried to be explicit about when these things come up, and we have presented best practices throughout. However, we want to call out some of the potential pitfalls and make sure you avoid them.

Datasets

There are a few datasets that are used for teaching a lot. Specifically, the iris dataset and the titanic dataset are the most common. We use them in the Path, and you will likely find them in the wild. These are so popular that almost everyone who works in data science has seen them. These do not make good portfolio projects because employers might read them as highly scaffolded student projects or as simply replicating a tutorial, and they won't set you apart from other candidates.

Formatting

It is important to pay close attention to presentation. Using the right file types, and presenting content in the right format is essential to getting that first interview.

For example, if you have written some code to analyze a dataset, it should be stored as a .py, .ipynb, .R, or .Rmd file. We teach you how to do this in the Career Paths, but it is worth mentioning again that code should almost never be stored in a text document (e.g., .txt, .doc, etc.) unless it is formatted as a code snippet and used as an example or evidence.

Another example is Tableau (or PowerBI) visualizations and dashboards. There are two ways to store these that demonstrate proficiency with these tools. First, both platforms offer repositories. Tableau Public is both a great place to show off your work and see what other people are making. The other way is to embed them into a website with an iframe.

You should not take screenshots, download them as jpegs, pdfs, or otherwise transform the rich, interactive dashboards you've made into static images.

You can (and should) embed static visualizations (that you make in Python, R, or even Excel or Tableau) into written reports, but the skill you are demonstrating is communicating visually about data (not proficiency in Tableau).

Finally, reports and summaries can be woven into your Jupyter Notebooks, presented as written documents (and stored as PDFs), or compiled into slide decks (also stored as PDFs).

Code Hygiene

It takes a long time and a lot of practice to write clear, well-structured code. However, you can take massive steps in that direction by writing well-commented and organized code. Just like writing an essay, writing good code often requires editing. Not all of your projects need to be edited. However, if you choose to highlight one or two (which you can do by pinning them in GitHub), you'll want to be sure that you've written in some comments and reorganized your code for clarity as much as possible.

Grammar

Just like in your resume and cover letter, you want to proofread your portfolio projects to be sure they are free from grammatical or spelling errors. Unlike in your resume/cover letter, a couple typos probably won't disqualify you. However, every Data Scientist needs to communicate about data, and you want to demonstrate that you can do that clearly. Even simply running any reports you write through a spellchecker will help in this respect.

Conclusion

Ultimately, your portfolio is how you verify that you have all of the skills you've worked so hard to acquire. Have fun with it by including some projects that excite you, and be sure to be ready to talk about those projects with recruiters when you get that first interview!

Introduction to Advanced EDA

Explore what you will learn in Advanced Exploratory Data Analysis!

Exploration is great, but exploration with a map is even better. This unit provides a map to get you started.

What will Advanced Exploratory Data Analysis cover?

After this unit, you will be able to:

Why did we build this?

Exploratory Data Analysis is the backbone of most data analysis. It's all about discovering what is in your data. It's the best place to start formulating questions and getting curious and excited about the potential of your dataset. Likewise, EDA can itself be the answer to a lot of data questions. Often, all we are looking for is a description of the world we collected data about, and the process of EDA can deliver that.

Review

In the last exercise you saw that LeBron James was 0.55 standard deviations above the mean of NBA player heights. He's taller than average, but compared to the other NBA players, he's not absurdly tall.

However, compared to the OkCupid dating pool, he is extremely rare! He's almost three full standard deviations above the mean. You'd expect only about 0.15% of people on OkCupid to be more than 3 standard deviations away from the mean.

This is the power of standard deviation. By taking the square root of the variance, the standard deviation gives you a statistic about spread that can be easily interpreted and compared to the mean.

Instructions

We've created a visualization that shows the mean, first, second, and third standard deviations of each dataset.

Enter your height (in inches) into the variable named `your_height` to see how you compare to the heights of NBA players and OkCupid daters!

You're likely below the first standard deviation for the NBA. Are you below the second? How unusual is your height compared to the OkCupid members?

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
from data import nba_data, okcupid_data

nba_mean = np.mean(nba_data)
okcupid_mean = np.mean(okcupid_data)

#Change this variable to your height (in inches)!
your_height = 0

nba_standard_deviation = np.std(nba_data)
okcupid_standard_deviation = np.std(okcupid_data)

plt.subplot(211)
```

```

plt.title("NBA Player Heights")
plt.xlabel("Height (inches)")

plt.hist(nba_data)

plt.axvline(nba_mean, color='#FD4E40', linestyle='solid', linewidth=2, label = "Mean")

plt.axvline(nba_mean + nba_standard_deviation, color='#FFB908', linestyle='solid',
linewidth=2, label = "Standard Deviations")
plt.axvline(nba_mean - nba_standard_deviation, color='#FFB908', linestyle='solid',
linewidth=2)

plt.axvline(nba_mean + nba_standard_deviation * 2, color='#FFB908', linestyle='solid',
linewidth=2)
plt.axvline(nba_mean - nba_standard_deviation * 2, color='#FFB908', linestyle='solid',
linewidth=2)

plt.axvline(nba_mean + nba_standard_deviation * 3, color='#FFB908', linestyle='solid',
linewidth=2)
plt.axvline(nba_mean - nba_standard_deviation * 3, color='#FFB908', linestyle='solid',
linewidth=2)

plt.axvline(your_height, color='#62EDBF', linestyle='solid', linewidth=2, label = "You")

plt.xlim(55, 90)
plt.legend()

plt.subplot(212)
plt.title("OkCupid Profile Heights")
plt.xlabel("Height (inches)")

plt.hist(okcupid_data)

plt.axvline(okcupid_mean, color='#FD4E40', linestyle='solid', linewidth=2, label = "Mean")

plt.axvline(okcupid_mean + okcupid_standard_deviation, color='#FFB908', linestyle='solid',
linewidth=2, label = "Standard Deviations")
plt.axvline(okcupid_mean - okcupid_standard_deviation, color='#FFB908', linestyle='solid',
linewidth=2)

plt.axvline(okcupid_mean + okcupid_standard_deviation * 2, color='#FFB908',
linestyle='solid', linewidth=2)
plt.axvline(okcupid_mean - okcupid_standard_deviation * 2, color='#FFB908',
linestyle='solid', linewidth=2)

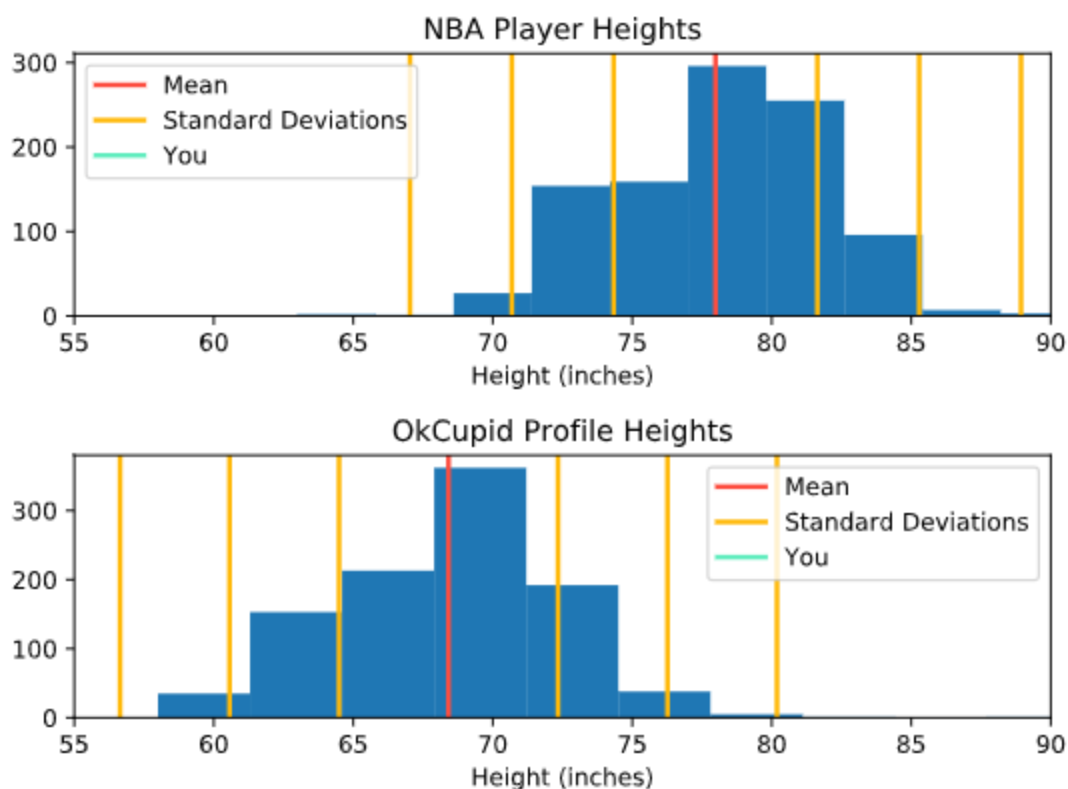
plt.axvline(okcupid_mean + okcupid_standard_deviation * 3, color='#FFB908',
linestyle='solid', linewidth=2)
plt.axvline(okcupid_mean - okcupid_standard_deviation * 3, color='#FFB908',
linestyle='solid', linewidth=2)

plt.axvline(your_height, color='#62EDBF', linestyle='solid', linewidth=2, label = "You")

plt.xlim(55, 90)
plt.legend()

plt.tight_layout()
plt.show()

```



Describe a Histogram

Review

In this lesson, you learned a description for describing the distribution of a dataset, which includes the following five features:

- Center
- Spread
- Skew
- Modality
- Outliers

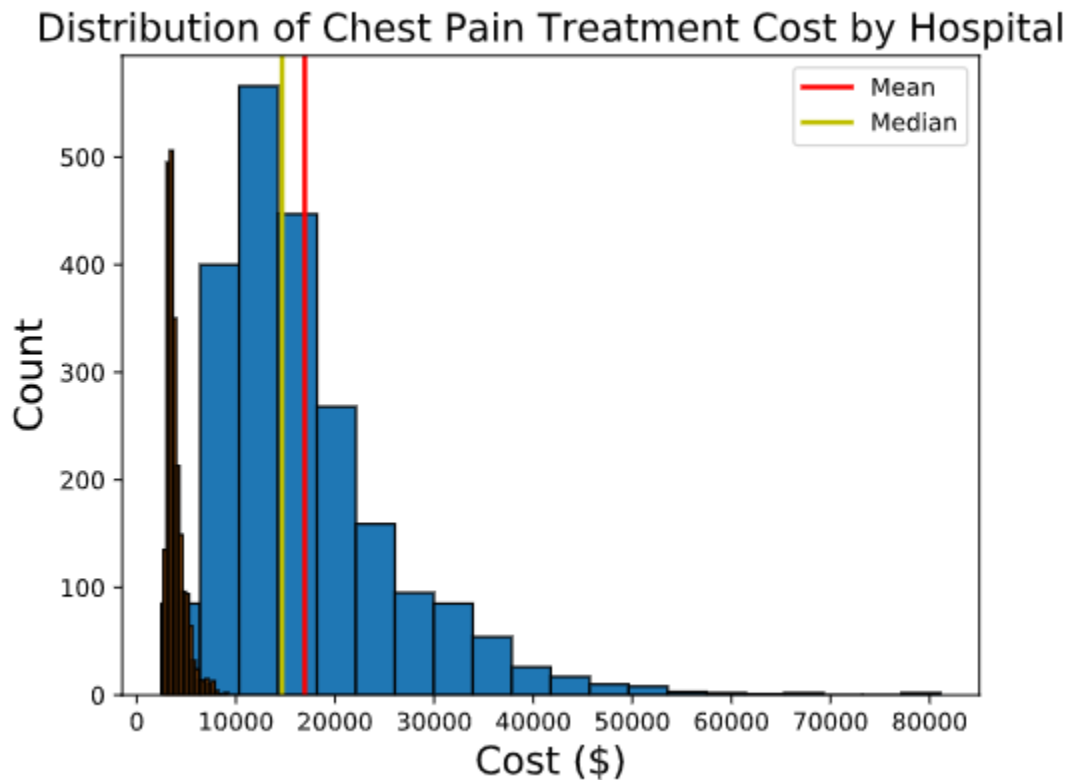
If you're curious, while the dataset in this study showed the average amount charged by hospitals for a given condition, the amount paid is far less. This is often due to nature of the United States healthcare system, where private insurers negotiate down the charges levied by hospitals.

The small histogram on the left side of the plot displays the actual amount of money paid to the hospital. *This histogram displays the distribution of chest pain cost for over 2,000 hospitals across the United States.*

The average and median costs are \$16,948 and \$14,659.6, respectively. Given that the data is unimodal, with

one local maximum and a right skew, the fact that the average is greater than the median, matches our expectation.

The range of costs is very large, \$78,623, with the smallest cost equal to \$2,459 and the largest cost equal to \$81,083. There is one hospital, Bayonne Hospital Center, that charges far more than the rest at \$81,083.



Quiz questions

1.

Which of the following best describes the skew of this histogram?

2.

True or False: the median of this distribution is greater than the mean.

3.

Which of the following is not a feature you should consider when interpreting a histogram?

4.

Which of the following is an important measure of a dataset's center?

5.

Which of the following is the best estimate of this dataset's range?

6.

Which of the following best describes the following distribution?

7.

Which of the following distributions is the most symmetric?

8.

Which of the following best describes an outlier?

9.

Which of the following best describes what the red arrow is pointing to in this distribution?

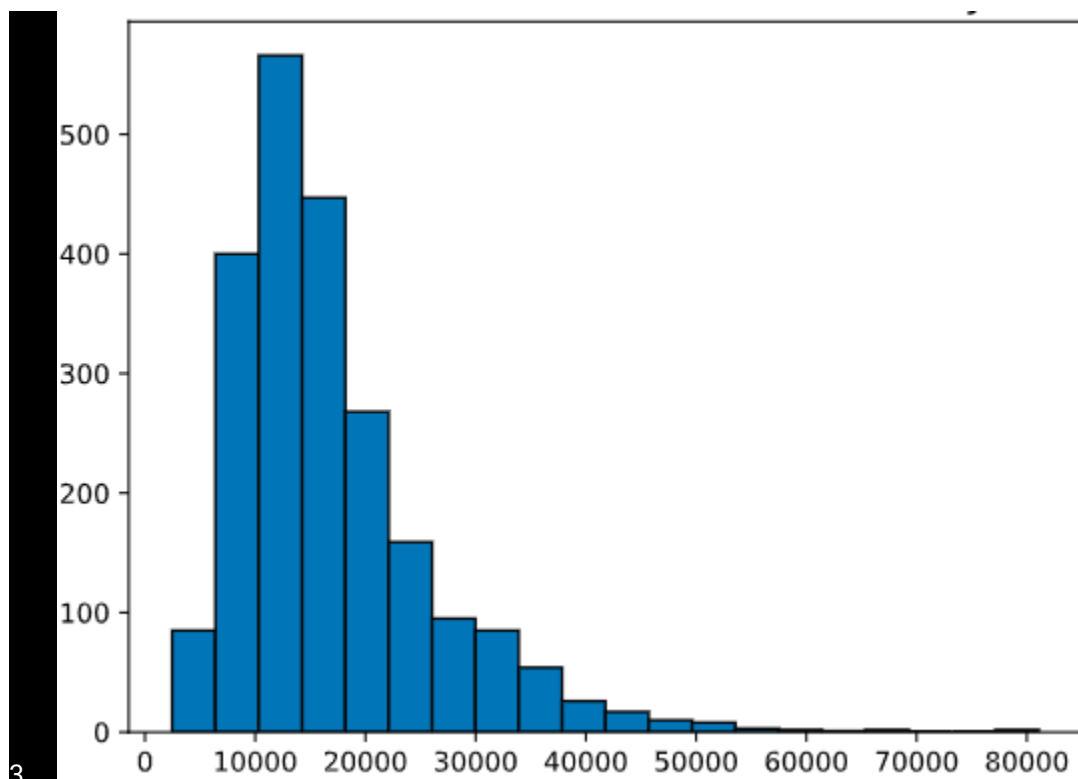
10.

What is the modality of this distribution? Roughly, where are the peaks?

Review your answers

1. 1.

2. Which of the following best describes the skew of this histogram?



4. Correct answer

5. Right-skew

6. There is no skew. The distribution is symmetric.

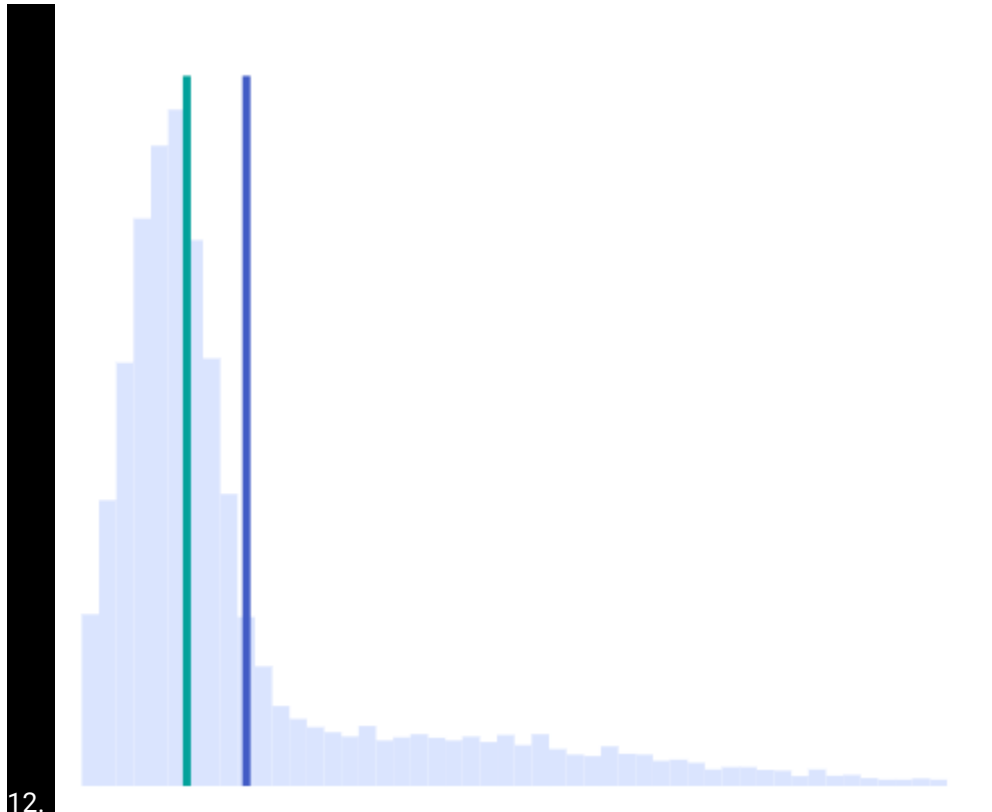
7. Left-skew

8. Explanation is currently unavailable.

9.

10. 2.

11. True or False: the median of this distribution is greater than the mean.



12.

13. Correct answer

14. False

15. True

16. Explanation is currently unavailable.

17.

18. 3.

19. Which of the following is not a feature you should consider when interpreting a histogram?

20. Correct answer

21. Gradient

22. Modality

23. Centrality

24. Outliers

25. Explanation is currently unavailable.

26.

27. 4.

28. Which of the following is an important measure of a dataset's center?

29. Correct answer

30. Mean

31. Standard Deviation

32. Mode

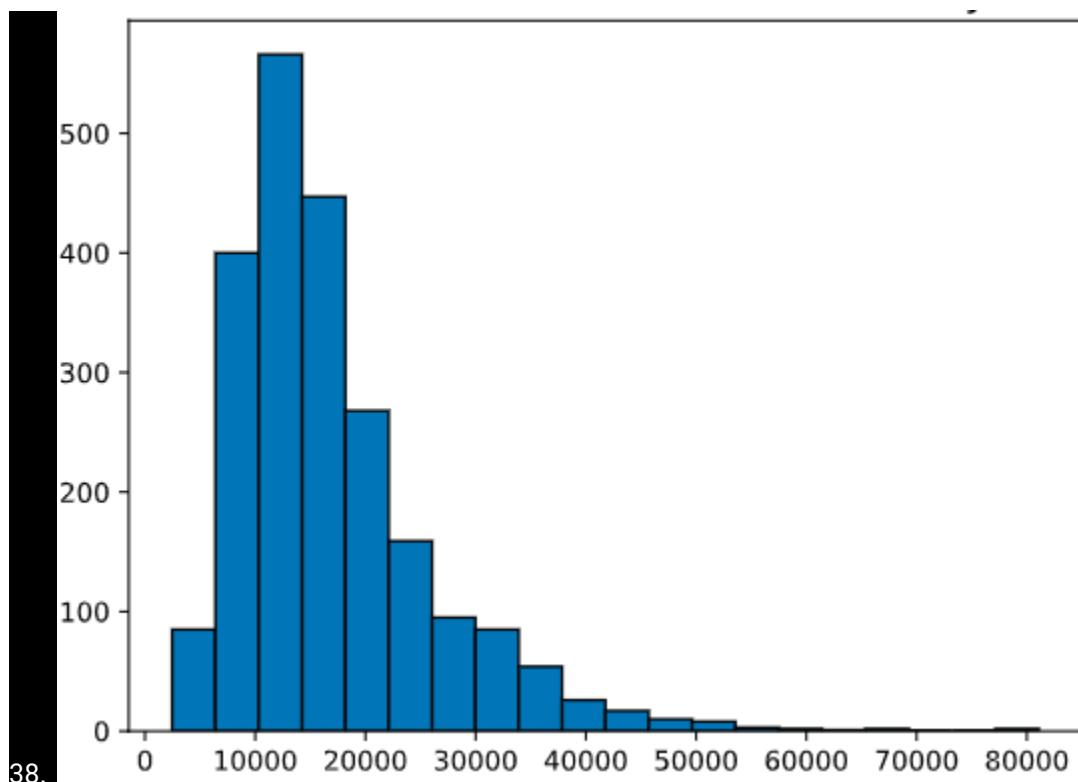
33. Range

34. Explanation is currently unavailable.

35.

36. 5.

37. Which of the following is the best estimate of this dataset's range?



39. Correct answer

40. 75,000

41. 45,000

42. 580

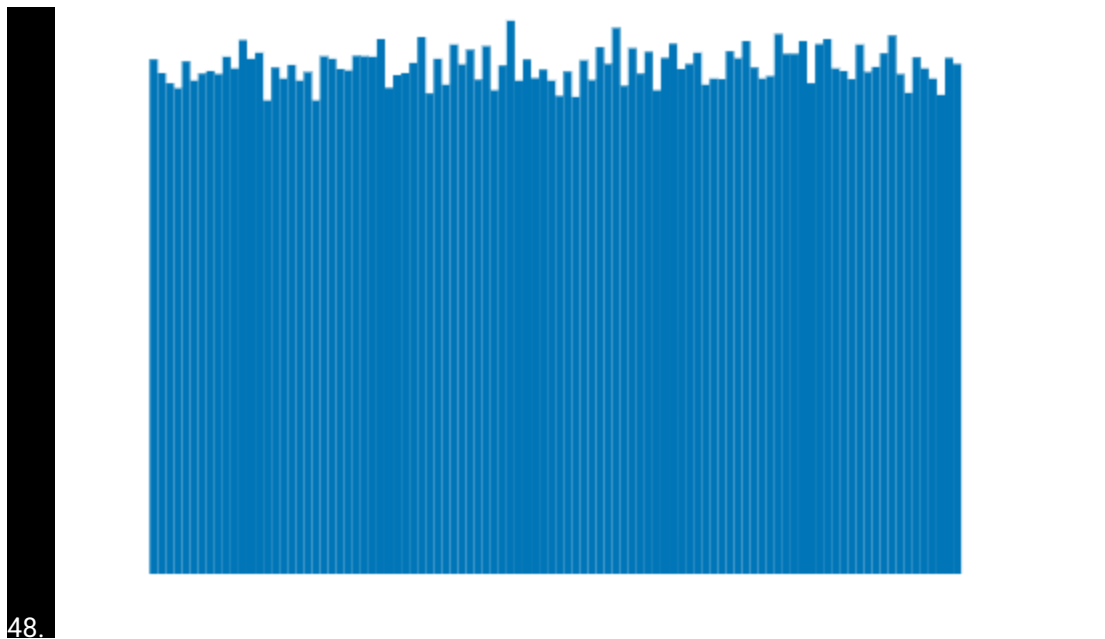
43. 5,000

44. Explanation is currently unavailable.

45.

46. 6.

47. Which of the following best describes the following distribution?



49. Correct answer

50. Symmetric and uniform

51. Bimodal and uniform

52. Right-skewed and uniform

53. Multimodal and uniform

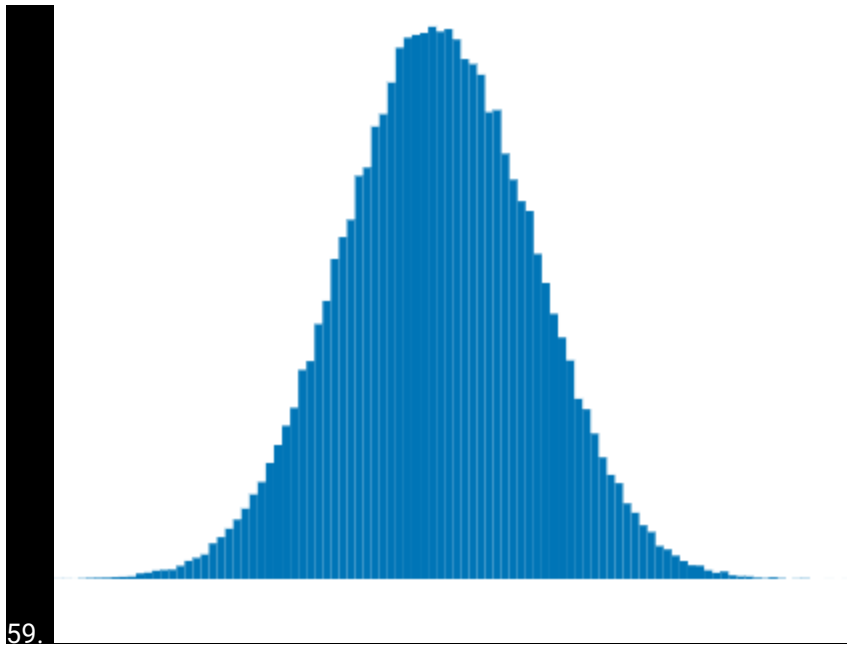
54. Explanation is currently unavailable.

55.

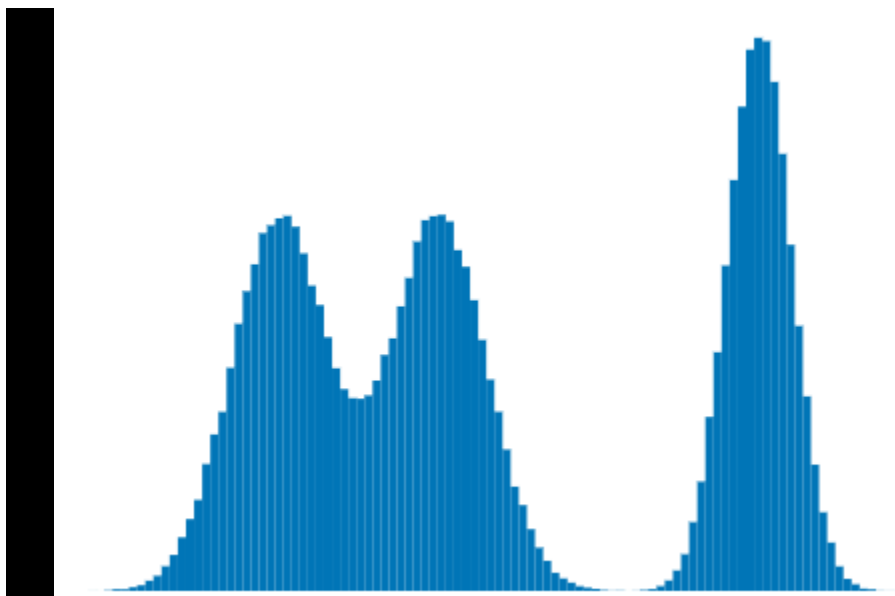
56. 7.

57. Which of the following distributions is the most symmetric?

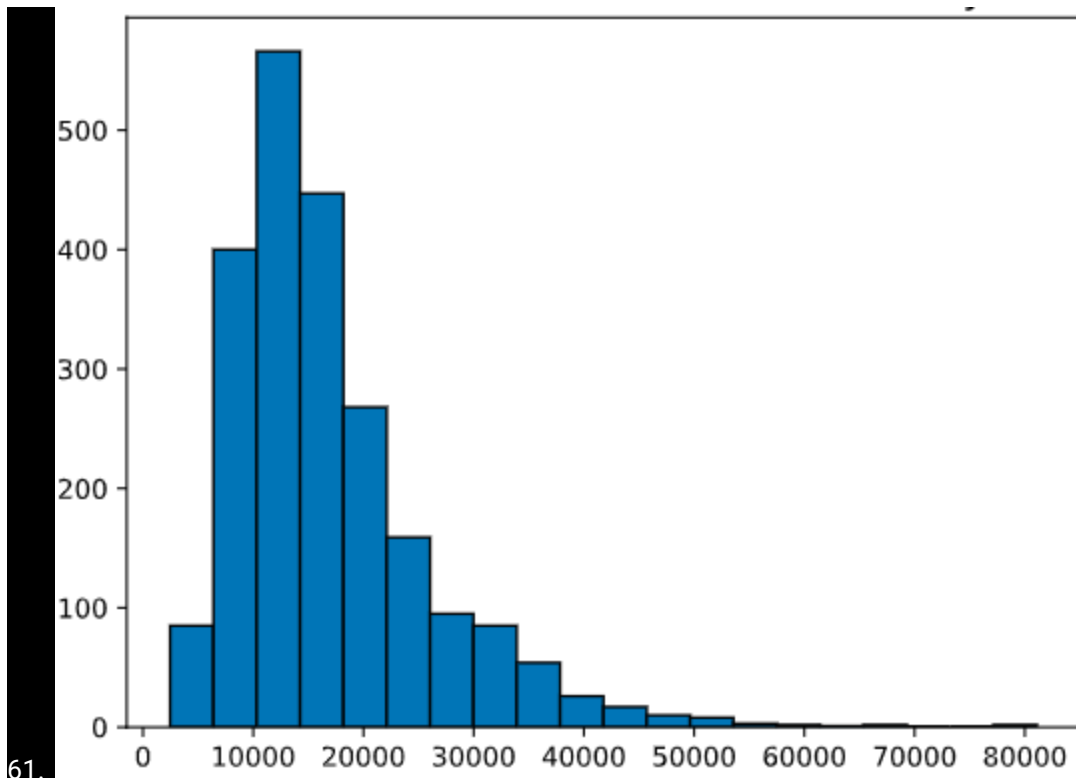
58. Correct answer



59.



60.



62. Explanation is currently unavailable.

63.

64. 8.

65. Which of the following best describes an outlier?

66. Correct answer

67. An outlier is a data point that is far away from the rest of the dataset.

68. Outliers are visualized as points in a histogram.

69. All points that are more than one standard deviation away from the mean are outliers.

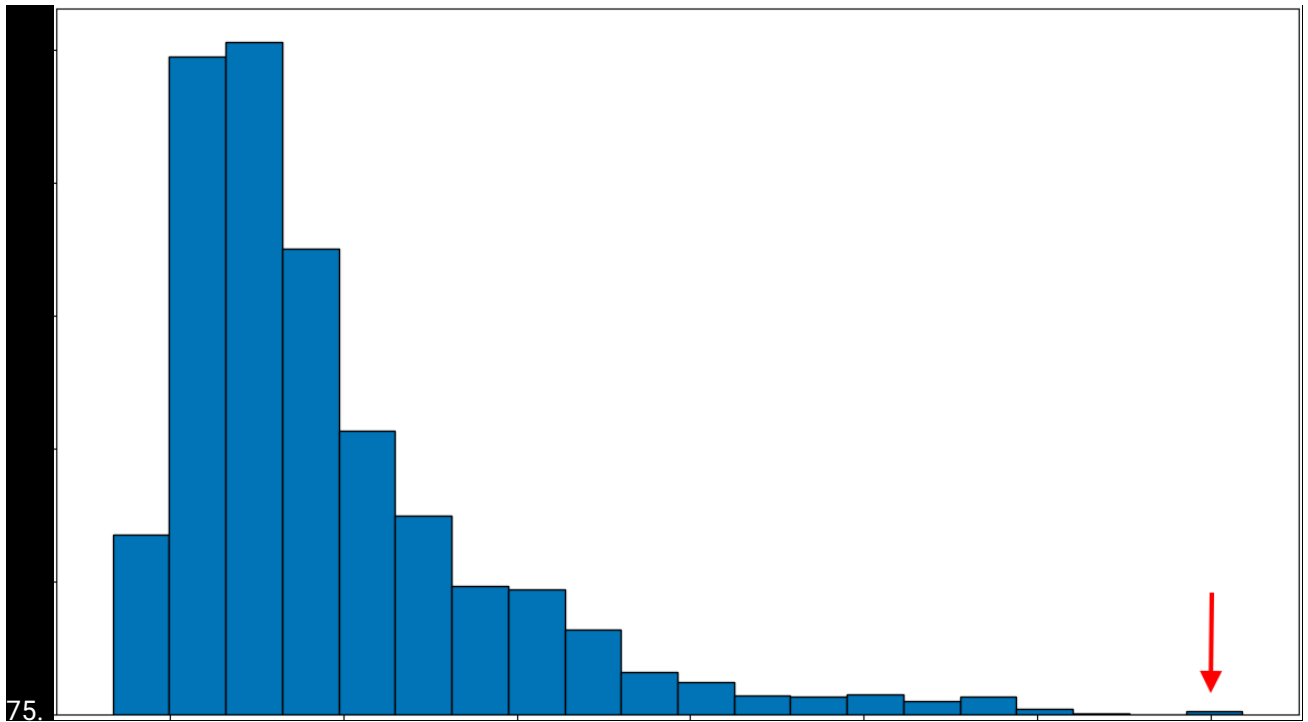
70. The maximum and minimum values of a dataset are always the outliers of a dataset.

71. Explanation is currently unavailable.

72.

73. 9.

74. Which of the following best describes what the red arrow is pointing to in this distribution?



75.

76. Correct answer

77. An outlier

78. The smallest peak in the dataset

79. The right-skew of the dataset

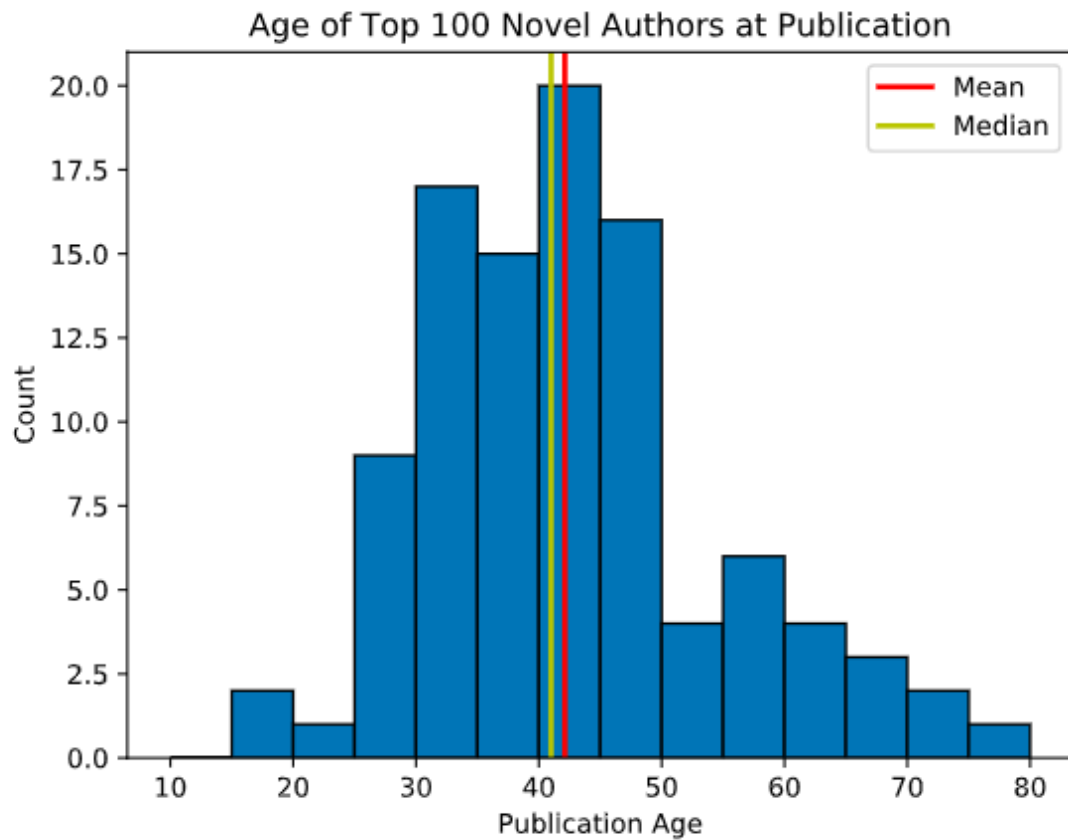
80. The range of the dataset

81. Explanation is currently unavailable.

82.

83. 10.

84. What is the modality of this distribution? Roughly, where are the peaks?



85.

86. Correct answer

87. One, centered at 41.

88. Three, with one centered at 20, one centered at 40, and one at 60.

89. Two, with one centered at 20 and one centered at 40.

90. Explanation is currently unavailable.

Quartiles

Quartiles Review

Great work! You now know how to calculate the quartiles of any dataset by hand and with NumPy.

Quartiles are some of the most commonly used descriptive

[statistics](#)

Preview: Docs Loading link description

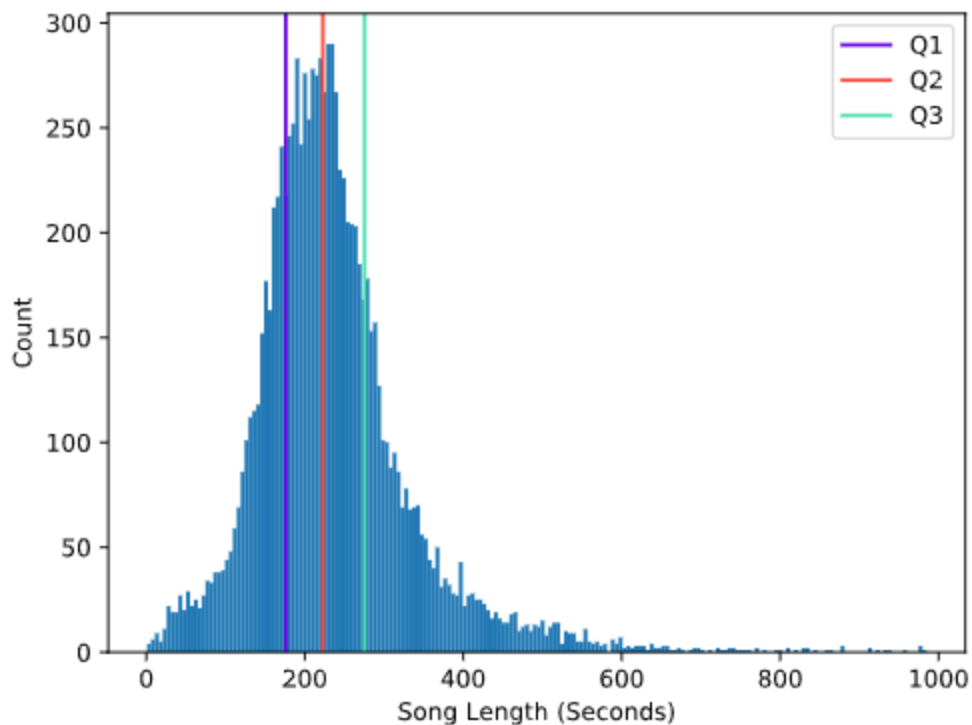
. For example, You might see schools or universities think about quartiles when considering which students to accept. Businesses might compare their revenue to other companies by looking at quartiles.

In fact quartiles are so commonly used that the three quartiles, along with the minimum and the maximum values of a dataset, are called the five-number summary of the dataset. These five numbers help you quickly get a sense of the range, centrality, and spread of the dataset.

```
import codecademylib3_seaborn
from song_data import songs
import matplotlib.pyplot as plt
import numpy as np

q1 = np.quantile(songs, 0.25)
q2 = np.quantile(songs, 0.5)
q3 = np.quantile(songs, 0.75)

plt.hist(songs, bins = 200)
plt.axvline(x=q1, label="Q1", c = '#6400e4')
plt.axvline(x=q2, label="Q2", c = '#fd4d3f')
plt.axvline(x=q3, label="Q3", c = '#4fe0b0')
plt.xlabel("Song Length (Seconds)")
plt.ylabel("Count")
plt.legend()
plt.show()
```



Quantiles

Quantiles Review

Nice work! Here are some of the major takeaways about quantiles:

Quantiles are values that split a dataset into groups of equal size.

If you have n quantiles, the dataset will be split into $n+1$ groups of equal size.

The median is a quantile. It is the only 2-quantile. Half the data falls below the median and half falls above the median.

Quartiles and percentiles are other common quantiles. Quartiles split the data into 4 groups while percentiles split the data into 100 groups.

Instructions

To the right, we've shown three different histograms along with the deciles. Each histogram shows the SAT scores of the students that a *fake* university has accepted.

If you had an SAT score of 1350, which tenth of the data would you be in for each school? Which schools should you apply to? Would any of the schools be unrealistic options?

```
import codecademylib3_seaborn
import matplotlib.pyplot as plt
import numpy as np
```

```

from data import school_one, school_two, school_three

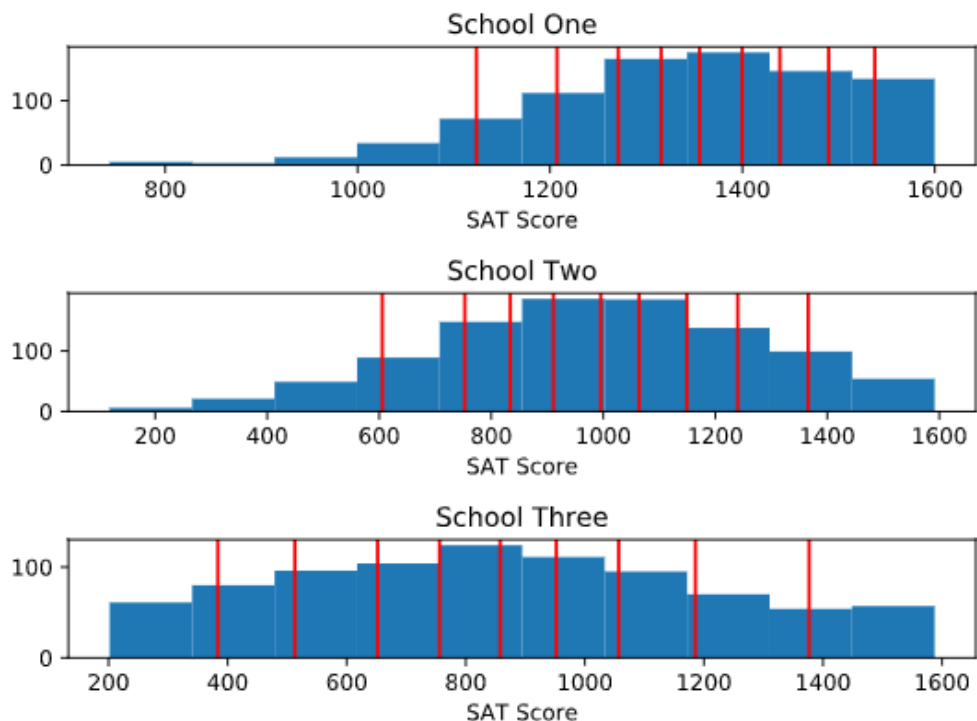
deciles_one = np.quantile(school_one, [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8,
0.9])
deciles_two = np.quantile(school_two, [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8,
0.9])
deciles_three = np.quantile(school_three, [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8,
0.9])

plt.subplot(311)
plt.hist(school_one)
for decile in deciles_one:
    plt.axvline(x=decile, c = 'r')
plt.title("School One")
plt.xlabel("SAT Score")

plt.subplot(312)
plt.hist(school_two)
for decile in deciles_two:
    plt.axvline(x=decile, c = 'r')
plt.title("School Two")
plt.xlabel("SAT Score")

plt.subplot(313)
plt.hist(school_three)
for decile in deciles_three:
    plt.axvline(x=decile, c = 'r')
plt.title("School Three")
plt.xlabel("SAT Score")
plt.tight_layout()
plt.show()

```



Interquartile Range

Review

Nice work! You can now calculate the Interquartile Range of a dataset by using the SciPy library. The main takeaway of the IQR is that it is a statistic, like the range, that helps describe the spread of the center of the data.

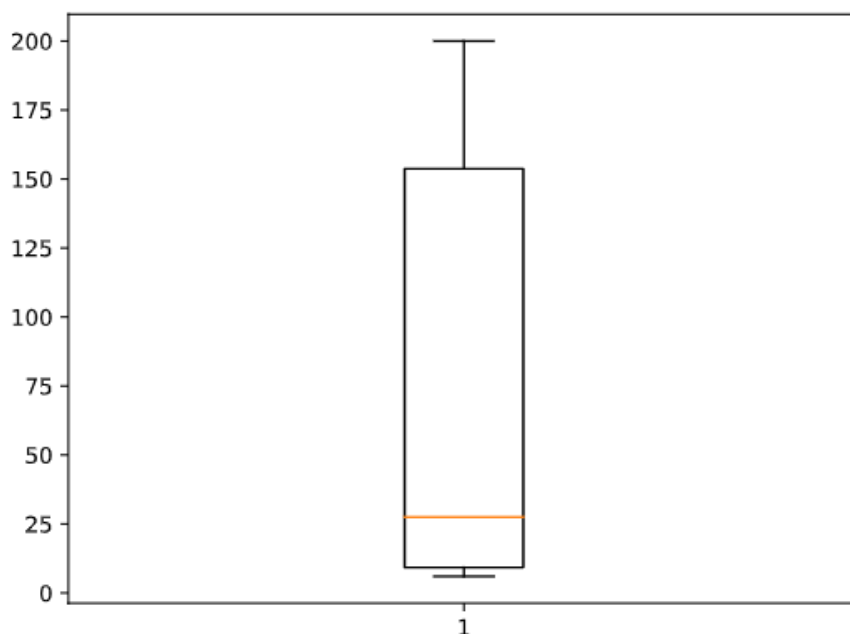
However, unlike the range, the IQR is robust. A statistic is robust when outliers have little impact on it. For example, the IQRs of the two datasets below are identical, even though one has a massive outlier.

```
dataset_one = [6, 9, 10, 45, 190, 200] #IQR is 144.5  
dataset_two = [6, 9, 10, 45, 190, 2000000] #IQR is 144.5
```

Copy to Clipboard

By looking at the IQR instead of the range, you can get a better sense of the spread of the *middle* of the data.

The interquartile range is displayed in a commonly-used graph — the box plot.



In a box plot, the ends of the box are Q1 and Q3. So the length of the box is the IQR. You can learn more about box plots in our upcoming course!

Instructions

We've set up a small `dataset` and are printing its range and IQR.
Try changing the maximum number in the dataset to different values.
What happens to the range when you make the maximum value `100000`? What happens to the IQR?
Try changing the minimum value to be more of an outlier as well.

Quiz summary

Quantiles, Quartiles, and IQR

Boxplots

Review

Great work! Here are some of the major takeaways from boxplots:

- The box of a boxplot visualizes the median, first quartile, and third quartile of a dataset.
- The length of the box in a boxplot visualizes the interquartile range.
- The whiskers extend from the box 1.5 times the size of the interquartile range.
- Outliers are points that fall outside of the whiskers. They're represented by dots.
- Boxplots are especially useful for comparing the spread of multiple datasets.

Instructions

- On this page, we've imported five different test datasets. We've drawn both the boxplot and the histogram for each dataset (you might need to scroll down to see all of the histograms). What are some of the advantages and disadvantages of each type of graph? Think about your ability to articulate your understanding of things like:
 - The spread of the data
 - The central tendency of the data
 - Your ability to compare different datasets
 - The number of data points in each dataset

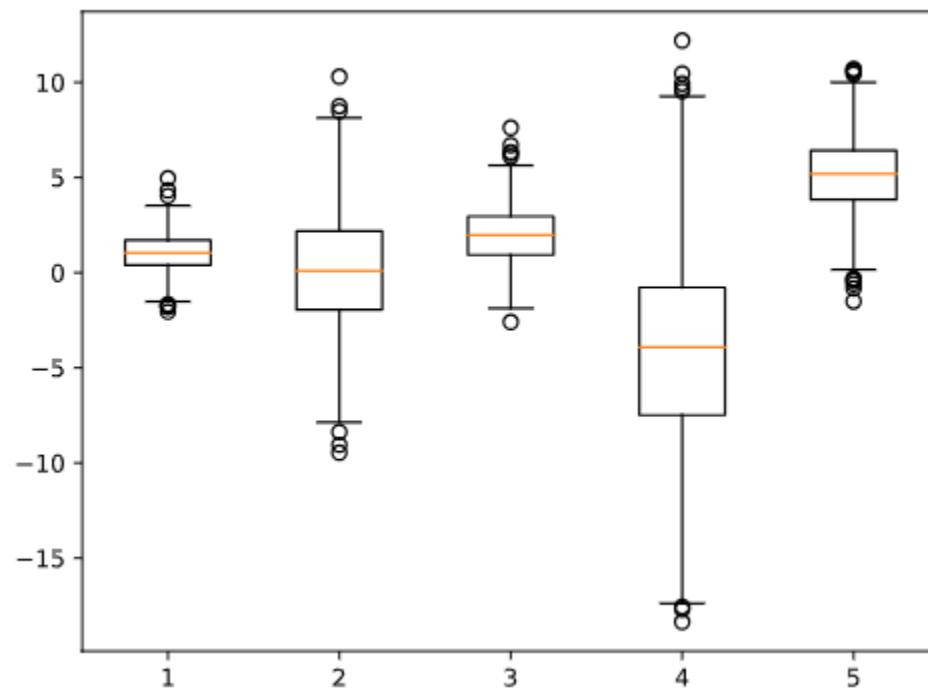
```
import codecademylib3_seaborn
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)

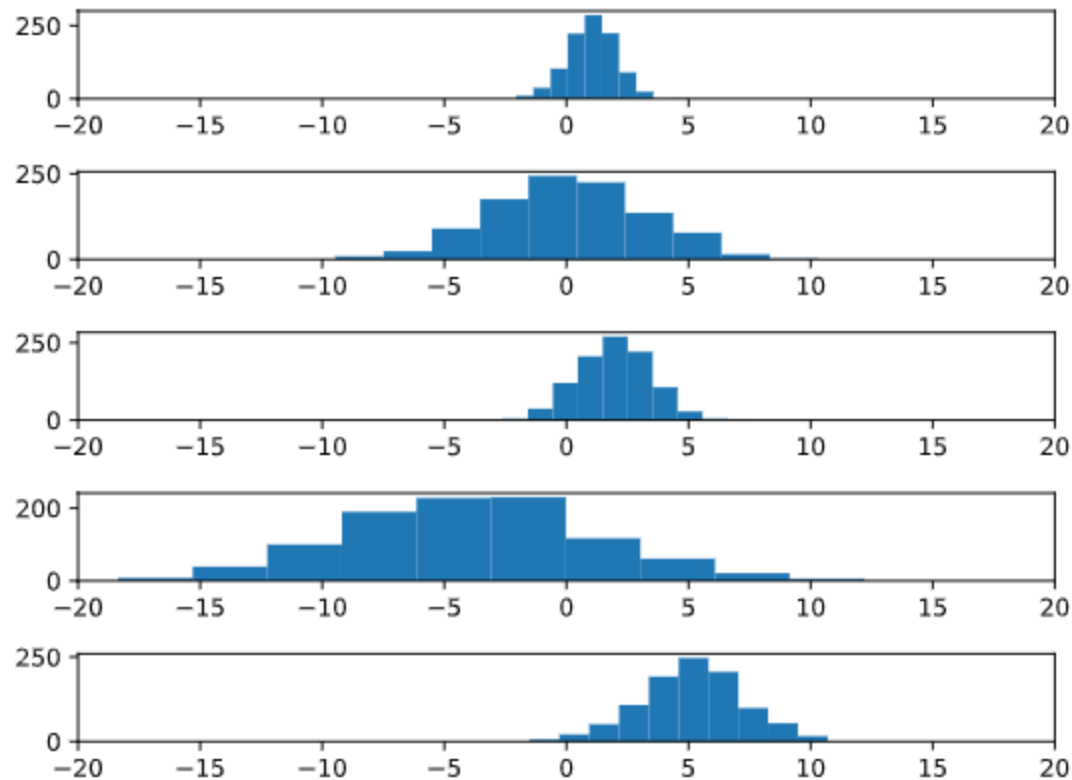
a = np.random.normal(1, 1, 1000)
```

```

b = np.random.normal(0, 3, 1000)
c = np.random.normal(2, 1.5, 1000)
d = np.random.normal(-4, 5, 1000)
e = np.random.normal(5, 2, 1000)
plt.boxplot([a,b,c,d,e])
plt.show()
plt.subplot(511)
plt.hist(a)
plt.xlim([-20,20])
plt.subplot(512)
plt.hist(b)
plt.xlim([-20,20])
plt.subplot(513)
plt.hist(c)
plt.xlim([-20,20])
plt.subplot(514)
plt.hist(d)
plt.xlim([-20,20])
plt.subplot(515)
plt.hist(e)
plt.xlim([-20,20])
plt.tight_layout()
plt.show()

```





Summary Statistics for Categorical Variables

Review

In this lesson you've learned the steps you can take to summarize and interpret summaries of *nominal categorical* and *ordinal categorical*

[variables](#)

Preview: Docs Loading link description

- For *nominal categorical* variables, there is no ordering to the categories. Because of this, we're limited to using the *mode* to describe central tendency and there is no way to summarize the spread.
- For *ordinal categorical* variables, there is an implied ordering to the categories. In Python, we can use `pd.Categorical()` to transform a variable to a categorical type. The Categorical type allows us to access a numeric value for each category by using `.cat.codes`. From there, we may perform operations on this variable as if it were a regular, numeric variable.

- However, when calculating [Statistics](#) or an *ordinal categorical variable* we should be mindful that some numeric statistics rely on the assumption of **equal spacing** between categories.
- For ordinal categorical variables, *median* and *mode* can be used to summarize the central tendency, and the IQR (or any difference between percentiles) can be used to summarize the spread.
- Certain summary statistics (e.g. frequencies and proportions), can be used for all categorical variables. You can create true/false columns and `np.sum()` and `np.mean()` to quickly summarize what proportion of your data meets certain criteria.

Instructions

1. As a final exercise, a new dataset has been loaded for you in the workspace. Follow the instructions below to inspect and summarize the categorical variables in this data.

`film_permits` contains a sample of NYC filming permits. Inspect the first few rows. Think about how you might explore and summarize this data. Some exercises you might wish to work through:

Which variables in this data are nominal? Which are ordinal?

Which Boroughs are granted permits for the most TV pilot episodes?

Summarize the types (`Category`) and subtypes (`SubCategoryName`) of projects that get filming permits granted.

Solution code is available in `solutions.py`.

SOLUTIONS:

```
import pandas as pd
import codecademylib3

# Read CSV
film_permits = pd.read_csv('film_permits.csv')

# Look first few rows
print(film_permits.head())

# Nominal Vars
nominalvars = ['EventType', 'Borough', 'Category', 'SubCategoryName']

# Ordinal Vars - We might consider an Id like 'EventID' an ordinal variable in
some situations

# Borough with the most permits for pilot episodes
print(film_permits[film_permits.SubCategoryName ==
'Pilot'].Borough.value_counts())

# Summarize the Top Categories
print(film_permits.Category.value_counts())

# Summarize the Top Subcategories
print(film_permits.SubCategoryName.value_counts())
```

Data Centering and Scaling

An introduction to data centering, min-max normalization, and standardization

As a data scientist, you will often need to transform your data before visualizing, analyzing, or modeling it. In this article, you will learn how to center and scale your data using common techniques. You will learn:

- How to center data and interpret centered data
- Why scaling your data is important
- How to scale data using two common methods:
 - Min-max Normalization
 - Standardization
- When to choose normalization vs. standardization

Let's get started!

Data Centering

Data centering involves subtracting the mean of a data set from each data point so that the new mean is 0. Mathematically, this looks like:

$$X_{centered_i} = X_i - \mu$$

where x_i is a datapoint and the Greek letter μ is the mean of all the x values.

For example, let's take a look at a data set of ages for five individuals:

```
ages = [24, 40, 28, 22, 56]
```

The mean age in this data set is 34 years old.

To center our data, we subtract the mean from each data point in `ages`:

```
centered_ages = [-10, 6, -6, -12, 22]
```

This centered data is useful because it tells us how far above or below the mean each data point is, giving us additional insight that we can't get just by looking at the initial data set. For example, the age of the first individual is ten years below the average.

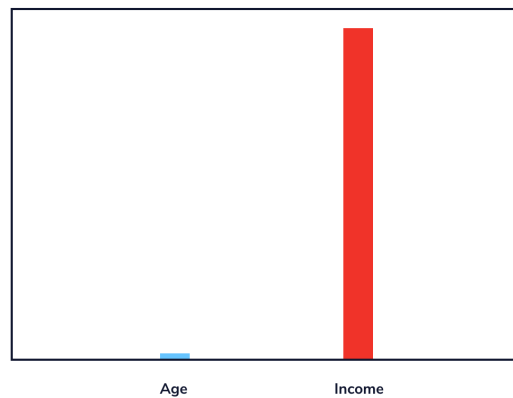
Note that, because the sum of the centered values is 0 ($-10 + 6 - 6 - 12 + 22 = 0$), the mean of the centered data is 0.

Data Scaling

A common task for data analysts and scientists is to find trends in data by comparing features of data points. However, this task is made difficult when the features are on drastically different scales.

For instance, let's consider a data set containing two features, **age** and **income**.

In general, a person's age usually ranges from 0 to about 100 years. A person's income, on the other hand, usually ranges from 0 to large amounts measured in the thousands of dollars. Clearly, age and income are two features that have vastly different ranges.



This presents issues when trying to use many machine learning algorithms, which treat all dimensions equally regardless of their scale. The difference in one year of age is interpreted as exactly equal to the difference in one dollar of income. That makes no sense!

In other words, the income feature outweighs the importance of age because income is on a relatively huge scale. Take a look at the following image:



From this chart, it is impossible to notice any relationship between income and age – all of the data is squished to the left. This is because the feature with the larger scale (income) dominates the smaller feature (age).

We would like every datapoint to have the same scale so each feature contributes equally to the relationship. Data scaling lets us achieve this.

Two of the most commonly used data scaling techniques are:

- Min-max normalization
- Standardization

Min-Max Normalization

Min-max normalization is one of the most simple and common ways to scale data.

For every feature in a data set, the minimum value of that feature is transformed into 0, the maximum value is transformed into 1, and every other value is transformed into a decimal between 0 and 1.

For example, if the minimum value of a feature is 10, and the maximum value is 30, then 20 would be transformed to 0.5 since it is halfway between 10 and 30.

The formula for *min-max normalization* is as follows:

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Using min-max normalization, our previous chart of income vs. age looks like the following:



Notice that the data is now properly scaled – both age and income fall in the range of [0,1].

Let's get some practice implementing min-max normalization in Python:

Coding question

Questions

Complete the implementation of min-max normalization in the function `min_max_normalize()` using the formula given above.

One downside of min-max normalization is that it does not handle outliers very well. For example, if you have 99 values between 0 and 20, and one value is 100, then the 99 values will all be transformed to a value between 0 and 0.2 while the outlier is transformed to 1. This results in skewed data:



In this example, normalizing the data fixed the skewing problem on the x-axis, but now the y-axis is causing problems.

As we'll see shortly, standardization is a more robust data scaling method for dealing with outliers.

Standardization

Standardization (also known as *Z-score normalization*) is another common data scaling technique.

Standardization involves subtracting the mean of each observation and then dividing by the standard deviation:

$$z = \frac{\text{value} - \text{mean}}{\text{stdev}}$$

$z =$

$\frac{\text{value} - \text{mean}}{\text{stdev}}$

$\frac{\text{value} - \text{mean}}{\text{stdev}}$

Once standardization is complete, all the features will have a mean of zero, a standard deviation of one, and therefore, the same scale.

Unlike normalization, standardization does not have a bounding range. This means that even if you have outliers in your data, your standardized data will not be affected. Therefore, if your dataset has outliers, standardization is the preferred scaling technique.

Let's see if you can use the formula above to implement standardization in Python:

Coding question

Questions

Complete the implementation of standardization in the function `standardize()` using the formula given above.

When to Normalize vs. Standardize?

Min-max normalization and standardization both have a similar goal of transforming features in data to have the same scale so that each feature is equally important. So when should you use min-max normalization vs. standardization?

There is not always a clear answer. Both normalization and standardization have their strengths as well as their drawbacks. For example, if you need your data to be on a 0-1 scale, then it makes sense to use min-max normalization. If you have outliers in your data, then it is best to use standardization (Z-score *normalization*) since it does not have a bounding range like min-max normalization does.

Keep in mind that not every data set requires normalization or standardization. If your data features do not have vastly different ranges, then scaling your data might not be necessary.

Python Implementation

As you saw, it is possible to implement min-max normalization and standardization by writing your own Python functions. However, in practice, most data analysts and scientists use popular libraries such as `scikit-learn`, which makes it very easy to scale your data.

For example, to normalize your data you can import `MinMaxScaler` from the `sklearn.preprocessing` package and then make a simple function call:

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

# read in data
data = pd.read_csv('data.csv')

# normalize data
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(data)
```

Likewise, standardizing your data is easy to do in just a few lines of code:

```
from sklearn.preprocessing import StandardScaler
import pandas as pd

# read in data
data = pd.read_csv('data.csv')
```

```
# standardize data
scaler = StandardScaler()
standardized_data = scaler.fit_transform(data)
```

Later on, you'll get practice with transforming your data in Python using real-world data. For now, let's test your knowledge of min-max normalization and standardization with a few questions:

Multiple choice

Questions

Which of the following is **not true** about *min-max normalization*?

Answer Choices

Min-max normalization does not handle outliers well.

Min-max normalization handles outliers well.

After min-normalization, every feature has the same scale

Min-max normalization transforms features so that the maximum value is 1.

Multiple choice

Questions

Which of the following is correct about *standardization*?

Answer Choices

Standardization involves subtracting the standard deviation and then dividing by the mean.

Standardization is the same thing as normalization.

Standardization involves subtracting the mean of each observation and then dividing by the standard deviation.

Standardization does not handle outliers well.

Multiple choice

Questions

Which of the following is correct about min-max normalization?

Answer Choices

Min-max normalization is more widely used than standardization.

Min-max normalization is also called Z-score normalization.

For every feature in a data set, the minimum value of that feature gets transformed into a -1, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between -1 and 1.

For every feature in a data set, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.

Review

As you learned in this article, normalization and standardization are scaling techniques that are slightly different but have a similar motive: to put data features in the same scale so that no feature is dominated by the other. These techniques are widely used by data analysts and scientists.

In this article, you learned:

Data centering:

Data centering involves subtracting the mean of a data set from each data point so that the new mean is 0.
Centered data is useful because it tells us how far above or below the mean each data point is.

Min-max normalization:

The goal of normalization is to put features with different ranges onto the same scale.
For every feature in a data set, the minimum value of that feature gets transformed into a 0, the maximum value gets transformed into a 1, and every other value gets transformed into a decimal between 0 and 1.
A downside of normalization is that it does not handle outliers well
Formula:

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Standardization:

Standardization, also known as *Z-score normalization*, involves subtracting the mean of each observation and then dividing by the standard deviation:

$$z = \frac{\text{value} - \text{mean}}{\text{stdev}}$$

$$z =$$

$$\text{stdev}$$

$$\frac{\text{value} - \text{mean}}$$

Once standardization is complete, all standardized features will have a mean of zero, a standard deviation of one, and therefore, the same scale.
Unlike normalization, standardization does not have a bounding range. This means standardization can deal with outliers.

As a data analyst or scientist, it's important to understand the data you're working with and how to transform that data. You now have a better grasp of common data transformation techniques and why it's important to transform your data in the first place.

Discretizing Numerical Data and Collapsing Categories

An introduction to binning numerical data and combining categorical data

In this article, you will learn simple transformation techniques for numerical and categorical data to help make your data easier to analyze, including:

- What it means to bin numerical data and why it's important
- How to bin numerical data in Python
- Why combining categorical data can be useful for analysis/visualization
- How to combine categorical data in Python

Let's get started!

Binning Data

As a data analyst or scientist, you will sometimes deal with numerical data that you would like to arrange into different categories. For example, imagine you are looking at the ages of a set of individuals. You might care more about the age category each person falls under (20-29, 30-39, 40-49, etc) than the exact age. The process of transforming numerical variables into categorical counterparts is called "binning."

Binning is a way to group a number of continuous values into a smaller number of "bins". We see this in the real world quite often. For example, a student's letter grade is determined by the percentage "bin" they fall under. Moreover, an individual's tax rate is determined by the income "bin" they fall under.

Why bin data?

Binning data can be useful in many situations, whether it be to create more appealing visualizations or to improve a machine learning model.

In areas like machine learning, binning data can be helpful to improve the accuracy of predictive models by reducing the noise of the data. For example, if you're a machine learning engineer working at an investment firm, you might use binning as a technique to improve your stock prediction models by smoothing the data to reduce the impact of small, short term price changes.

Now that you have a better understanding of why we bin data, let's explore how to bin numerical data in Python!

Binning Numerical Data in Python

In this example, we will look at the ages of students in a dance class (stored in a file called `dance_class_data.csv`).

First, we import pandas – a Python library to perform data manipulation and analysis. Then, we load in our data and take a peek at the first 10 rows of the data:

```
import pandas as pd

# Load in data
dance_class = pd.read_csv('dance_class_data.csv')

# Print the first 10 rows
print(dance_class.head(10))
```

The output looks like:

	Name	Gender	Age	Experience
0	Chris Shelton	M	23	beginner
1	Douglas Watson	M	28	intermediate
2	Martha Gomez	F	45	beginner
3	Amos Moore	M	63	beginner
4	Valentina Sen	F	35	beginner
5	Billy Woods	M	53	advanced
6	Oscar Barker	M	43	intermediate
7	Marie Sandoval	F	23	beginner
8	Nancy McBride	F	35	beginner
9	Lindsay Bowen	F	27	beginner

The `dance_class` dataframe has four columns – `Name`, `Gender`, `Age`, and `Experience`. We can find out the data types of these columns by using the `dtypes` property:

```
print(dance_class.dtypes)
```

The output looks like:

```
Name      object
Gender     object
Age        int64
Experience  object
```

Notice that the `Age` column is of type `int`. This is the column we'll be exploring.

We're curious to see which age range each student in the class falls under. To do this, we can bin the values to specific age groups. Before we do that, let's store the age of the students and find out the minimum and maximum ages:

```
# Store the ages
student_ages = dance_class['Age']

# min() method returns the lowest value
print(student_ages.min()) # 23

# max() method returns the highest value
print(student_ages.max()) # 65
```

The youngest student in the class is 23 years old and the oldest is 65 years old. We can therefore define our bin boundaries by decade (20-29, 30-39, 40-49, 50-59, 60-69).

Let's first create a `bins` variable to store these values:

```
# Store the boundaries
bins = [20, 30, 40, 50, 60, 70]
```

Next, we can create the bins using `pd.cut(df['column_name'], bins)` where `bins` is either:

Let's bin the value of the `Age` column into equally sized bins in a new *column* called `binned_age`. Then, let's print out the first few rows of the data:

```
# Create new binned_age column that bins the values of the 'Age' column
dance_class['binned_age'] = pd.cut(dance_class['Age'], bins)

# Print the first few rows of the data
print(dance_class[['binned_age', 'Age']].head())
```

The output looks like:

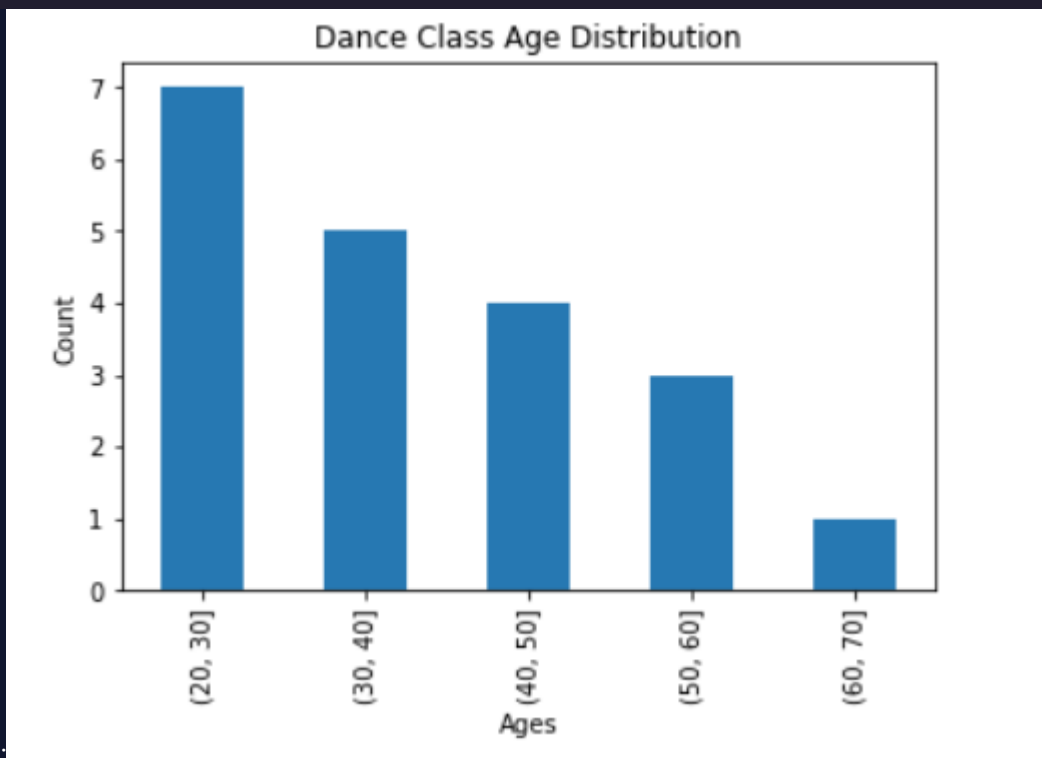
	binned_age	Age
0	(20, 30]	23
1	(20, 30]	28
2	(40, 50]	45
3	(60, 70]	63
4	(30, 40]	35

Notice how each age now falls under a specific age group. We can also plot our data in a bar graph to get a better visualization:

```
# Plot the bar graph of binned ages
dance_class['binned_age'].value_counts().plot(kind='bar')

# Label the bar graph
plt.title('Dance Class Age Distribution')
plt.xlabel('Ages')
plt.ylabel('Count')

# Show the bar graph
plt.show()
```



Output:

Notice how the data is categorized into different age groups, allowing us to see the big picture of our data without focusing too much on the precise ages of each individual. Based on this plot, we can see that the class skews toward younger ages.

If we want, we can also specify labels for our bins using a `labels` argument:

```
# Store the boundaries
bins = [20, 30, 40, 50, 60, 70]

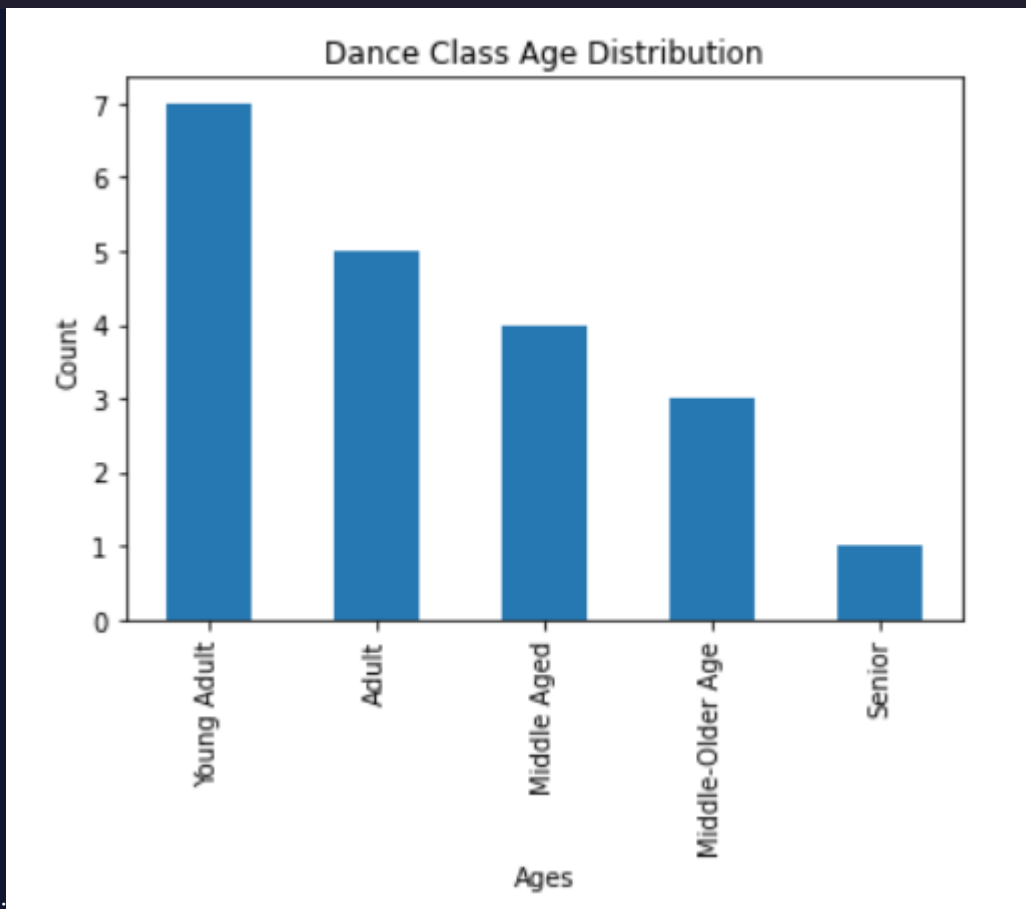
# Store the labels for our bins
age_labels = ['Young Adult', 'Adult', 'Middle Aged', 'Middle-Older Age', 'Senior']

# Bin the values of the 'Age' column and specify the labels
dance_class['binned_age'] = pd.cut(dance_class['Age'], bins, labels = age_labels)
```

Now when we print the first few rows of our data or plot a bar graph of the data, each numeric range that we specified earlier is replaced with the assigned label:

	binned_age	Age
0	Young Adult	23
1	Young Adult	28

```
2 Middle Aged 45
3 Senior 63
4 Adult 35
```



Output:

Multiple choice

Questions

Which of the following is the correct way to bin the value of the `Grades` column in dataframe `student_data` into five equal bins?

Answer Choices

```
pd.cut(student_data, 5)
```

```
pd.bin(student_data, 5)
```

```
pd.cut(student_data['Grades'], 5)
```

```
pd.bin(student_data['Grades'], 5)
```

Combining Categorical Data

As a data analyst or scientist, you will sometimes work with categorical data. Just as a refresher, categorical data are variables that contain label values rather than numeric values. For example,

When dealing with categorical data, you may sometimes want to combine/remove categories for easier analysis. For example, imagine you're looking at a data set of the most popular sports in a region:

Sport	Count
Basketball	500
Football	400
Baseball	8
Tennis	7
Cricket	4
Sailing	3

Notice that there are several categories of sports here but a pretty uneven distribution of their occurrences. Basketball and football make up an overwhelming majority of the data, while the other categories only have a few occurrences. Therefore, it may make sense to combine all of the other sports into a category called "Other."

Sport	Count
Basketball	500
Football	400
Other	22

Combining Categories in Python

Let's explore how to combine categories of categorical data in Python. We'll be working with a data set called `election_poll.csv` to see the most popular candidates for a city council election. This data set has a column called `votes` that we will use.

First, let's import our data and see the counts for each of the candidates (categories) in our data:

```
import pandas as pd

# read in data
election_data = pd.read_csv('election_data.csv')

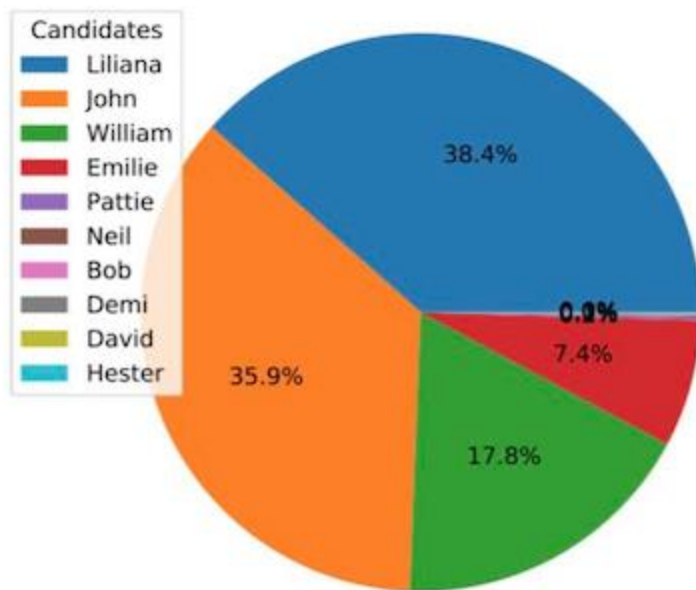
# get the counts for each candidate
votes = election_data['Vote'].value_counts()
print(votes)
```

The output looks like:

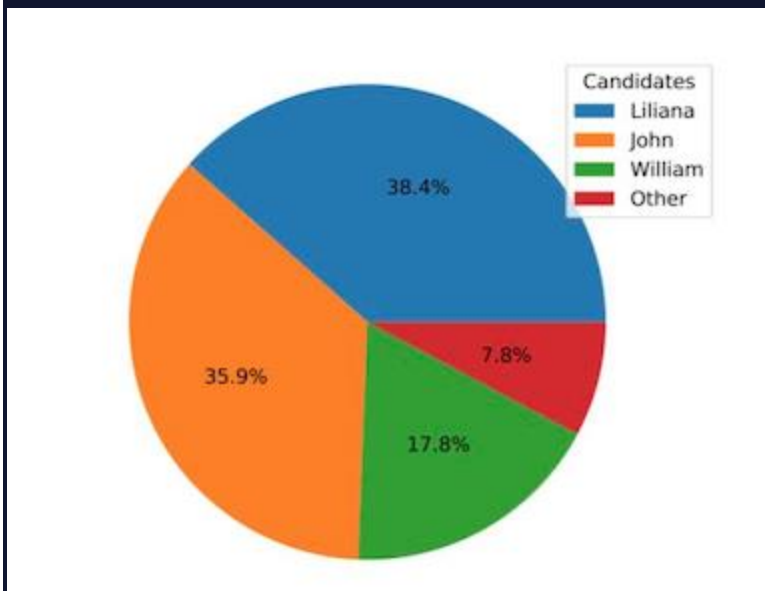
Liliana	1067
John	998
William	494
Emilie	196
Pattie	6
Neil	3
Bob	2
Demi	1
David	1
Hester	1

What do you notice about the data? Are there specific candidates we should focus more on and others who we should focus less on?

You may notice that `Liliana`, `John`, and `William` make up most of the data while the other candidates make up a smaller percentage of the overall vote count. If we're trying to understand the likelihood that each of these candidates is elected in an upcoming vote, it makes sense to collapse the other candidates into a category called "Other." If we decided not to do this, then our analysis might be more difficult to parse. For example, if we were to plot this data in a pie chart, it would look like this:



That isn't a pleasant pie chart to look at. Liliana, John, and William make up 92% of the data, while the other seven categories make up just 8% of the data. Once we combine those seven categories in an "Other" category, the pie chart looks like:



How can we do this in Python? We can create a mask for values occurring less than a specific number of times in `votes`. In our case, we'll check for values occurring less than 200 times. We do this using the `isin()` function:

```
mask = election_data.isin(votes[votes < 200].index)
```

Next, we can label these other categories as "Other" and print the updated vote count:

```
election_data[mask] = 'Other'
```

```
print(election_data['Vote'].value_counts())
```

Output:

```
Liliana    1067
John       998
William    494
Other      210
Name: Vote, dtype: int64
```

Notice how much cleaner the data is now that we have combined the seven low frequency categories into one "Other" category.

Remember that the cleaner your data is, the simpler it will be to analyze and visualize, so taking the time to understand your data and apply simple transformations like this when appropriate will go a long way.

Advanced Data Transformations

[An introduction to log transformation and other advanced data transformations](#)

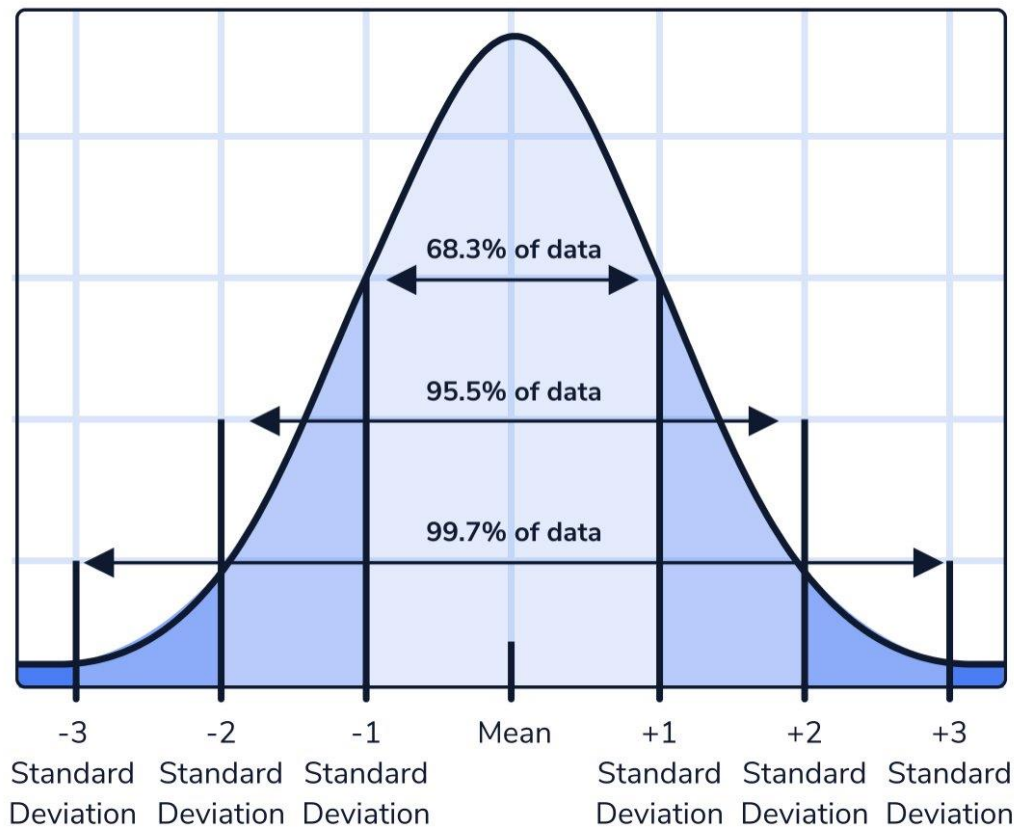
In this article, you will learn how to transform data in particular situations, such as when the data you're working with is skewed. You will learn:

- How to transform data that is skewed
- What is log transformation and why is it useful?
- How to implement log transformation in Python

Let's get started!

Skewed Data

As a data analyst and scientist, you will often work with normally distributed data. Just as a refresher, normally distributed data is data where most data points are close to a central value with fewer instances as you get further away from the center. Visually, this looks like a "bell curve":



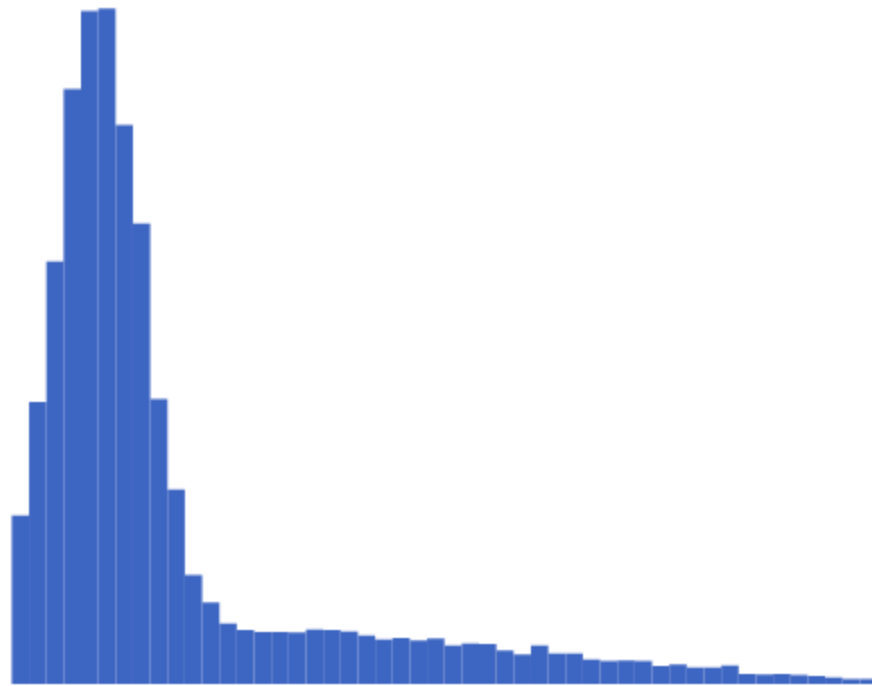
There are several reasons why having normally distributed data is useful:

Normal distributions are symmetric around their mean. Moreover, the mean, median, and mode of a normal distribution are equal. These properties make the data easier to analyze because we can make some assumptions about how many datapoints are on either side of the mean. As shown above, approximately 68% of the data falls within 1 standard deviation of the mean, and approximately 95% of the data falls within 2 standard deviations of the mean. Again, these properties make it easier to make generalizations about the data you're working with. Many machine learning algorithms (such as linear regression) assume that the data distribution is close to normal. Therefore, transforming data so that it is normally distributed can enhance your ability to fit an accurate predictive model.

Oftentimes, real-life data is messy and does not conform to a normal distribution. Instead, the data might be skewed to the left or right.

For example, imagine you're a data analyst looking at income data in a population. You may find that a lot of the data is centered around lower values but that there are high values in the data as well. This results in a skewed distribution:

Skew-Right



This is an example of a *right-skewed* data set. Lower incomes, which make up most of the data, fall on the left side of the diagram, while higher incomes fall on the right.

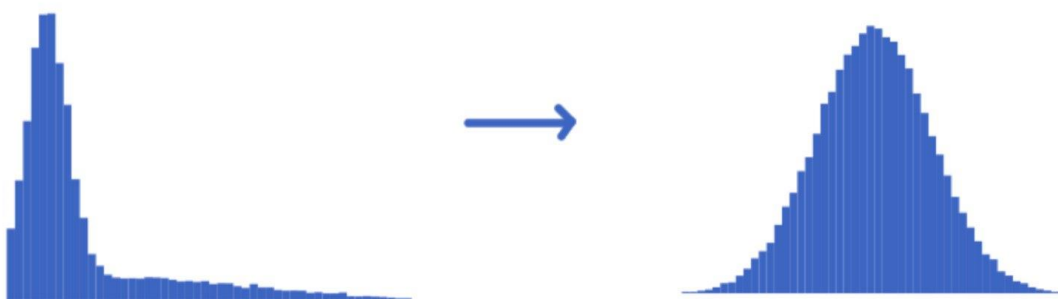
Feeding this data to some machine learning or statistical models will present issues if the model assumes normality, because the model will be trained on a much larger number of lower incomes. The skewness of the data violates the assumptions of the model.

In situations like this, we would like to make our data less skewed so that it conforms to model assumptions and is easier to work with. To help with this issue, we can use *Log Transformation*.

Log Transformation

Log transformation is a data transformation method that replaces each variable x with $\log(x)$.

When log transformation is applied to data that is not normally distributed, the result is that the data will be less skewed, or more “normal” than before. For example, after applying log transformation to the right-skewed data set above, we would see something similar to the following:



Notice how the shape of the data changed. The logarithm function squeezes together the larger values in your data set and stretches out the smaller values, making the transformed data more “normal” than before.

Let’s see how we can apply this on real data in Python.

Python Implementation

Python contains libraries that make it simple to transform your data using log transformation.

One quick method is to use [NumPy](#), a Python library that makes it easy to work with arrays of data and perform statistical analysis. Numpy has a natural log transformation function that you can apply to an array of data, like so:

```
import numpy as np

data = [1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4]

# log transform
log_data = np.log(data)

print(log_data)
# 0, 0, 0, 0, 0.693, 0.693, 0.693, 1.098, 1.098, 1.386, 1.386
```

Data scientists and machine learning engineers will generally usually use powerful machine learning libraries such as [scikit-learn](#) to apply log transformation. You can import [PowerTransformer](#) from sklearn’s [preprocessing](#) module and then pass in your data to a simple function call:

```
from sklearn.preprocessing import PowerTransformer

# log transform
log_transform = PowerTransformer()
log_transform.fit_transform(data)
```

Let’s try to perform log transformation on real data in Python. We’ll be working with home price data from Kaggle.

First, we will need to import several Python libraries:

`numpy` (to perform log transformation)

pandas (to read in a CSV file and perform data analysis)
seaborn (to plot our data)

```
import numpy as np
import pandas as pd
import seaborn as sns
```

Next, we need to read in our data file and store the home price data in a new variable:

```
import numpy as np
import pandas as pd
import seaborn as sns

# read in data file
home_data = pd.read_csv('home_data.csv')

# store home price data
home_prices = home_data['SalePrice']
```

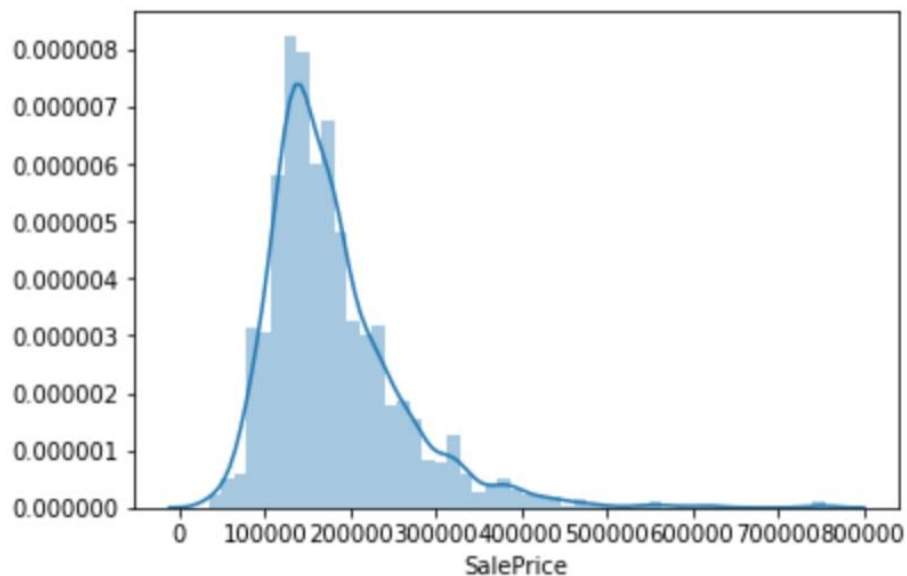
Now that we have our data stored, we can plot it using `seaborn`:

```
import numpy as np
import pandas as pd
import seaborn as sns

home_data = pd.read_csv('home_data.csv')
home_prices = home_data['SalePrice']

# plot data
sns.distplot(home_prices)
```

Distribution plot:



The first thing we notice is that our data is right skewed. Most of our data consists of lower and medium home prices, while higher home prices make up a small part of the data. Let's see exactly how skewed our data is:

```
print(home_prices.skew())
```

Output:

```
1.88
```

Generally, if skewness is less than -1 or greater than 1, the distribution is highly skewed. Our distribution has a skewness of 1.88, so it is highly skewed.

If we were to use this skewed data to predict home prices, we would run into some issues. The prediction model would be trained on a much larger number of lower and moderately priced homes so it would have trouble predicting the prices of expensive homes.

To help with this, we can apply log transformation to our data:

```
import numpy as np
import pandas as pd
import seaborn as sns

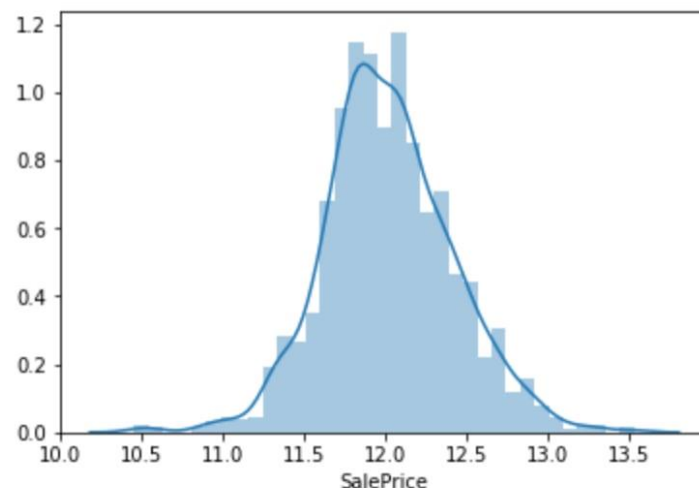
home_data = pd.read_csv('home_data.csv')
home_prices = home_data['SalePrice']

sns.distplot(home_prices)

# apply log transformation
log_home_prices = np.log(home_prices)
```

Next, let's plot the transformed data to see the shape of the distribution:

```
sns.distplot(log_home_prices)
```



Distribution plot:

Notice how the shape of the distribution is much more symmetrical (normal) than before. Let's see how skewed our data is now:

```
print(home_prices.skew())
```

Output:

```
0.12133506220520406
```

Applying log transformation reduced the skewness from 1.88 to 0.12 – that’s a significant change!

Now that our transformed data is less skewed and more normally distributed, we can use this data in our prediction model to more accurately predict home prices.

Other Transformations

In this article, we discussed only one advanced data transformation method – log transformation. But different situations call for other types of transformations.

In the examples above, we dealt with income and price data, which cannot be negative. But what happens if you have negative numbers in your data? You cannot take the log of a negative number, so log transformation will not work.

In this case, you would have to explore other data transformation techniques, such as *cube root transformation*, which involves converting x to $x^{1/3}$. This transformation reduces the right skewness but also has the benefit of working with zero and negative values.

To reduce left skewness, you might look at a technique such as *square transformation*, which involves converting x to x^2 .

An important part of your job as a data analyst and scientist is understanding the data you’re working with and the shape of that data. Once you know this, it becomes easier to figure out how you should transform that data.

As you progress in your journey, you’ll learn many techniques to deal with unique situations. For now, feel free to explore other data transformation methods [here](#).

Introduction: Statistics Fundamentals Part II

Discover what you will learn in Statistics Fundamentals Part II

The 'Science' part of Data Science comes from using the scientific method to get results. A lot of what Data Scientists do is conduct experiments with data that already exists. This presents some unique challenges compared to creating a completely new data set.

For starters, the data might not address exactly what you want it to, it might be incidental data from multiple sources that you have to reconcile. Whatever the case may be, it's the responsibility of the Data Scientist to still analyze that data with the same rigor as they might analyze the results of a lab experiment.

This mostly comes down to testing hypotheses and designing experiments. Sometimes those experiments are in the shape of A/B tests - like when evaluating two variables. Sometimes they are testing the strength of a relationship between two variables.

Though you've had an introduction to Hypothesis testing already, to work as an Inferential or Machine Learning Data Scientist, we will go a little deeper to develop a more sophisticated toolset in this area.

What will Statistics Fundamentals Part II cover?

After this unit, you will be able to:

- Test the strength of an association between two variables
- Design experiments
- Conduct A/B Tests
- Calculate the necessary sample size for an experiment
- Apply your hypothesis testing skills to new projects

Introduction

In this lesson, we'll use hypothesis tests to make inference about population-level associations between two

Preview: Docs Loading link description

[variables](#)

.

We will cover four different hypothesis tests:

Two Sample T-Tests (for an association between a quantitative variable and a

Preview: Docs Loading link description

[binary](#)

categorical variable)

ANOVA and Tukey Tests (for an association between a quantitative variable and a non-binary categorical variable)

Chi-Square Tests (for an association between two categorical variables)

Hypothesis Testing: Associations

Two-Sample T-Test

Suppose that a company is considering a new color-scheme for their website. They think that visitors will spend more time on the site if it is brightly colored. To test this theory, the company shows the old and new versions of the website to 50 site visitors, each – and finds that, on average, visitors spent 2 minutes longer on the new version compared to the old. Will this be true of future visitors as well? Or could this have happened by random chance among the 100 people in this sample?

One way of testing this is with a *2-sample t-test*. The null hypothesis for this test is that average length of a visit does not differ based on the color of the website. In other words, if we could observe all site visitors in two alternate universes (one where they see each version of the site), the average visiting times in these universes would be equal.

We can use SciPy's `ttest_ind()` function to perform a 2-sample t-test. It takes the values for each group as inputs and returns the t-statistic (not covered in this course) and a p-value:

```
from scipy.stats import ttest_ind
tstat, pval = ttest_ind(times_version1, times_version2)
```

Copy to Clipboard

By default, `ttest_ind()` runs a two-sided test.

QUESTIONS:

The company randomly sampled 100 site visitors. They showed the old version of their website to half of their sample and the new version to the other half. The amount of time (in minutes) that each visitor spent on the website was recorded.

An overlaid histogram showing the amount of time spent on the website by visitors to the new and old versions has already been created for you in **script.py**. Press “Run” and inspect the histograms. Based on this picture, do you think there is a significant association between the version of the website a visitor saw and how long they spent on the site?

Stuck? Get a hint

Checkpoint 2 Passed

2.

The data from this study has already been saved for you in **script.py**: the time spent by the 50 visitors to the old version is saved as `old`; the time spent by visitors to the new version is saved as `new`. Run a two-sample t-test comparing these groups and save the p-value as `pval`, then print it out.

Stuck? Get a hint

Checkpoint 3 Passed

3.

Using a significance threshold of 0.05, is there a significant difference between the average amount of time visitors are spending on the old and new versions of the website? In **script.py** set the value of `significant` equal to `True` if there is a significant difference and `False` if not.

SOLUTION

```
import pandas as pd
import matplotlib.pyplot as plt
import codecademylib3
from scipy.stats import ttest_ind
```



```

data = pd.read_csv('version_time.csv')

#separate out times for two versions
old = data.time_minutes[data.version=='old']
new = data.time_minutes[data.version=='new']

#run the t-test here:
tstat, pval = ttest_ind(old, new)
print(pval)

#determine significance
significant = True

#plot overlapping histograms
plt.hist(old, alpha=.8, label='old')
plt.hist(new, alpha=.8, label='new')
plt.legend()
plt.show()

```

Multiple Tests

In the previous exercise, we used a two-sample t-test to investigate an association between a quantitative variable (time spent on a website) and a

Preview: Docs Binary is a number system of 1s and 0s which serves as a textual representation method for giving instructions to a computer's CPU. In computing and telecommunications, binary codes are used for various methods of encoding data such as turning the character strings of source code, into bit strings or instruction sets for the CPU to execute.

[binary](#)

categorical variable (an old color scheme or a new color scheme).

In some circumstances, we might instead care about an association between a quantitative variable and a **non-binary** categorical variable (non-binary means more than two categories).

For example, suppose that we own a chain of stores that sell ants, called VeryAnts. There are three different locations: A, B, and C. We want to know whether customers are spending a significantly different amount per order at any of the locations.

There are three different comparisons we could make: A vs. B, B vs. C, and A vs. C. One way to answer our question is to simply run three separate 2-sample t-tests.

QUESTIONS:

We have created samples `a`, `b`, and `c`, representing the amount (in U.S.D) spent on orders at VeryAnts at locations A, B, and C, respectively. We want to see if there's a significant difference in the average spending per order at the three locations.

Code has been provided for you to generate side by side box plots of the sales at each of these stores.

Based on this visualization, are there any stores where customers appear to be spending more or less money?

Stuck? Get a hint

Checkpoint 2 Passed

2.

Perform a 2-Sample T-test between each pair of location data.

Store the p-values in variables called `a_b_pval`, `a_c_pval`, and `b_c_pval`. Print them to the console.

Note that you may see numbers in scientific notation in the print out of one or more p-values. If you see

something like `e-05` at the end of a number, that means that the preceding number is multiplied by $10^{(-5)}$. In other words, `2.5134230524e-05` is equal to `0.000025134230524`.

Stuck? Get a hint

Checkpoint 3 Passed

3.

Inspect the p-values that you printed out. Using a significance level of 0.05, for which pairs of stores did you find a significant difference between the average sale price? Assign the values of `a_b_significant`, `a_c_significant`, `b_c_significant` to `True` if the p-value indicates a significant difference and `False` if the p-value does not indicate a significant difference.

SOLUTIONS:

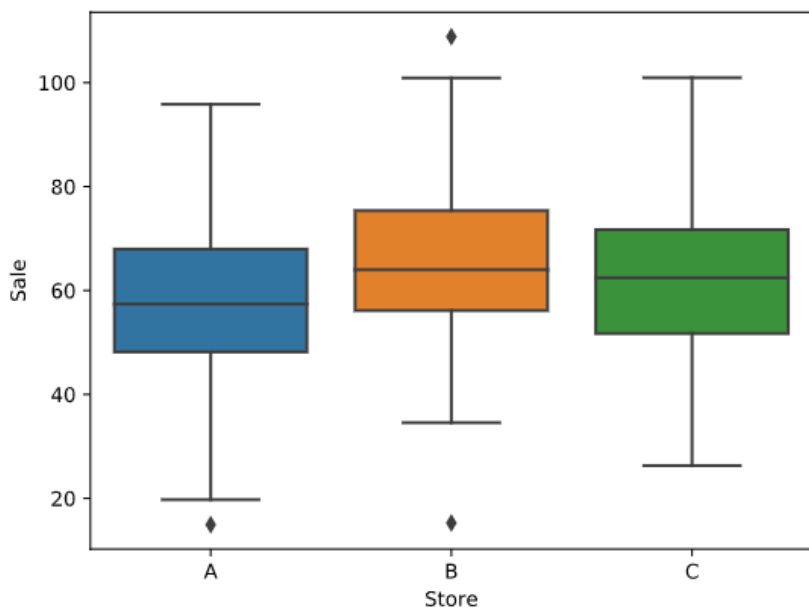
```
from scipy.stats import ttest_ind
import codecademylib3
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')
a = veryants.Sale[veryants.Store == 'A']
b = veryants.Sale[veryants.Store == 'B']
c = veryants.Sale[veryants.Store == 'C']

# run t-tests
a_b_tval, a_b_pval = ttest_ind(a,b)
a_c_tval, a_c_pval = ttest_ind(a,c)
b_c_tval, b_c_pval = ttest_ind(b,c)
print("A to B comparison:", a_b_pval)
print("A to C comparison:", a_c_pval)
print("B to C comparison:", b_c_pval)

# determine significance
a_b_significant = True
a_c_significant = True
b_c_significant = False

# create plot
sns.boxplot(data=veryants, x='Store', y='Sale')
plt.show()
```



ANOVA

In the last exercise, we ran three separate 2-sample t-tests to investigate an association between a quantitative variable (amount spent per sale) and a non-binary categorical variable (location of VeryAnts visited, with options A, B, and C). The problem with this approach is that it inflates our probability of a type I error; the more tests we run, the worse the problem becomes!

In this situation, one approach is to instead use *ANOVA* (Analysis of Variance). ANOVA tests the null hypothesis that all groups have the same population mean (eg., the true average price of a sale is the same at every location of VeryAnts).

In Python, we can use the SciPy function `f_oneway()` to perform an ANOVA. `f_oneway()` has two outputs: the F-statistic (not covered in this course) and the p-value. If we were comparing scores on a video-game for math majors, writing majors, and psychology majors, we could run an ANOVA test with this line:

```
from scipy.stats import f_oneway
fstat, pval = f_oneway(scores_mathematicians, scores_writers, scores_psychologists)
```

Copy to Clipboard

If the p-value is below our significance threshold, we can conclude that at least one pair of our groups earned significantly different scores on average; however, we won't know which pair until we investigate further!

QUESTIONS:

The same data from the previous exercise is available to you in the workspace: costs of sales made at three locations of VeryAnts (saved as `a`, `b`, and `c`).

Perform an ANOVA test on `a`, `b`, and `c` and store the p-value in a variable called `pval`, then print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

At a .05 significance level, does this p-value lead you to reject the null hypothesis (and conclude that at least one pair of stores have significantly different average sales)?

Change the value of `significant` to `True` if the p-value indicates at least one pair of stores have significantly different sales and `False` otherwise.

SOLUTIONS:

```
from scipy.stats import f_oneway
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')
a = veryants.Sale[veryants.Store == 'A']
b = veryants.Sale[veryants.Store == 'B']
c = veryants.Sale[veryants.Store == 'C']

# run ANOVA
Fstat, pval = f_oneway(a, b, c)
print(pval)

# determine significance
significant = True
```

Tukey's Range Test

Let's say that we have performed an ANOVA to compare sales at the three VeryAnts stores. We calculated a p-value less than 0.05 and concluded that there is a significant difference between at least one pair of stores.

Now, we want to find out **which** pair of stores are different. This is where Tukey's range test comes in handy!

In Python, we can perform Tukey's range test using the `statsmodels` function `pairwise_tukeyhsd()`. For example, suppose we are again comparing video-game scores for math majors, writing majors, and psychology majors. We have a dataset named `data` with two columns: `score` and `major`. We could run Tukey's range test with a type I error rate of 0.05 as follows:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey_results = pairwise_tukeyhsd(data.score, data.major, 0.05)
print(tukey_results)
```

Copy to Clipboard

Output:

```
Multiple Comparison of Means - Tukey HSD,FWER=0.05
=====
group1 group2 meandiff lower upper reject
-----
math psych 3.32 -0.11 6.74 False
math write 5.23 2.03 8.43 True
psych write -2.12 -5.25 1.01 False
=====
```

Tukey's range test is similar to running three separate 2-sample t-tests, except that it runs all of these tests simultaneously in order to preserve the type I error rate.

The function output is a table, with one row per pair-wise comparison. For every comparison where `reject` is `True`, we "reject the null hypothesis" and conclude there *is* a significant difference between those two groups. For example, in the output above, we would conclude that there is a significant difference between scores for math and writing majors, but no significant difference in scores for the other comparisons.

QUESTIONS:

The `veryants` dataset is provided for you once again in `script.py`. The `Store` column represents the store that a sale was made at ('A', 'B', or 'C') and the `Sale` column represents the cost of a sale in U.S.D.

Run Tukey's range test with a type I error rate of 0.05 to determine whether average sales are different at any pair of two stores and save the result as `tukey_results`, then print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Inspect the output from the test you just ran. For which pairs of stores did you find a significant difference in average sales?

Assign the values of `a_b_significant`, `a_c_significant`, `b_c_significant` to `True` if the test indicates a significant difference in sales at the indicated pair of stores and `False` if not.

Recall that when we ran three t-tests, we found significant differences for the A vs. B and A vs. C comparisons. Do we get the same result with this test?

Stuck? Get a hint

SOLUTIONS:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import pandas as pd

# store the data
veryants = pd.read_csv('veryants.csv')

# run tukey's test
tukey_results = pairwise_tukeyhsd(veryants.Sale, veryants.Store, 0.05)
print(tukey_results)

# determine significance
a_b_significant = True
a_c_significant = False
b_c_significant = False
```

RESULT:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj lower upper reject
-----
A    B    7.2763 0.001 3.2263 11.3264  True
A    C    4.011 0.0529 -0.0391  8.0611  False
B    C   -3.2653 0.141 -7.3154  0.7847  False
-----
```

Assumptions of T-Tests, ANOVA, and Tukey

Before we use a two sample t-test, ANOVA, or Tukey's range test, we need to be sure that the following things are true:

1. The observations should be independently randomly sampled from the population

Suppose the population we are interested in is all visitors to a website. Random sampling will help ensure that our sample is representative of the population we care about. For example, if we only sample site visitors on Halloween, those visitors may behave differently from the general population. In practice, this can be a challenging assumption to meet, but it's important to be aware of.

2. The standard deviations of the groups should be equal

For example, if we're comparing time spent on a website for two versions of a homepage, we first want to make sure that the standard deviation of time spent on version 1 is roughly equal to the standard deviation of time spent on version 2. To check this assumption, it is normally sufficient to divide one standard deviation by the other and see if the ratio is "close" to 1. Generally, a ratio between 0.9 and 1.1 should suffice.

That said, there is also a way to run a 2-sample t-test without assuming equal standard deviations — for example, by setting the `equal_var`

Preview: Docs A parameter is the name of a variable passed into a function. Parameters allow functions to accept inputs. An argument, on the other hand, is the actual value of the variable (also known as the parameter) passed into a function. Suppose we have a function called `tripleThis()`: `js function tripleThis(x) { return x * 3; } tripleThis(6);`

[parameter](#)

in the `scipy.stats.ttest_ind()` function equal to `False`. Running the test in this way has some disadvantages (it essentially makes it harder to reject the null hypothesis even when there is a true difference between groups), so it's important to check for equal standard deviations before running the test.

3. The data should be normally distributed...ish

Data analysts in the real world often still perform these tests on data that are not normally distributed. This is usually not a problem if sample size is large, but it depends on how non-normal the data is. In general, the bigger the sample size, the safer you are!

4. The groups created by the categorical variable must be independent

Here are some examples where the groups are **not** independent:

the number of goals scored per soccer player before, during, and after undergoing a rigorous training regimen (not independent because the same players are measured in each category)

years of schooling completed by a group of adults compared to their parents (not independent because kids and their parents can influence one another)

QUESTIONS:

Suppose that we want to run a two-sample t-test to compare the sample means for two groups (saved as `dist_1` and `dist_2` in the workspace).

Before running the test, we want to check whether the ratio of standard deviations for these groups is equal. Use the NumPy `std()` function to divide the standard deviation of `dist_1` by the standard deviation of `dist_2` and save the result as `ratio`, then print it out.

Are the standard deviations roughly equal?

Stuck? Get a hint

Checkpoint 2 Passed

2.

Code has already been provided for you in the workspace to create an overlaid histogram of these two distributions. Inspect the distributions. Are they approximately normal? If so, set the value of `normal_assumption` equal to `True` in **script.py**; otherwise set `normal_assumption` equal to `False`.

SOLUTIONS:

```
import codecademylib3
import numpy as np
import matplotlib.pyplot as plt

dist_1 = np.genfromtxt("1.csv")
dist_2 = np.genfromtxt("2.csv")

#calculate ratio of standard deviations:
ratio = np.std(dist_1)/np.std(dist_2)
print(ratio)

#check normality assumption
normal_assumption = True

#plot histograms of each distribution
plt.hist(dist_1, alpha = .8, normed = True, label = 'dist 1')
plt.hist(dist_2, alpha = .8, normed = True, label = 'dist 2')
plt.legend()
plt.show()
```

Chi-Square Test

If we want to understand whether the outcomes of two categorical

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

[variables](#)

are associated, we can use a Chi-Square test. It is useful in situations like:

An A/B test where half of users were shown a green submit button and the other half were shown a purple submit button. Was one group more likely to click the submit button?

People under and over age 40 were given a survey asking “Which of the following three products is your favorite?” Did these age groups have significantly different preferences?

In `SciPy`, we can use the function `chi2_contingency()` to perform a Chi-Square test. The input to `chi2_contingency` is a contingency table, which can be created using the `pandas crosstab()` function as follows:

```
#create table:
import pandas as pd
table = pd.crosstab(variable_1, variable_2)

#run the test:
from scipy.stats import chi2_contingency
```

```
chi2, pval, dof, expected = chi2_contingency(table)
```

For example, suppose we want to know whether gender is associated with the probability of a website visitor making a purchase. The null hypothesis is that there's no association between the variables (eg. males, females, and non-binary people are all equally likely to make a purchase on the website, so gender and purchase-status are not associated). If the p-value is below our chosen threshold (often 0.05), we reject the null hypothesis and can conclude there is a statistically significant association between the two variables (eg. men, women, and non-binary people appear to have different probabilities of making a purchase, so gender is associated with purchase-status).

QUESTIONS:

The management at the VeryAnts ant store wants to know if their two most popular species of ants, the Leaf Cutter and the Harvester, vary in popularity between 1st, 2nd, and 3rd graders. We have provided a dataset named `ants` with a sample of 108 sales to 1st, 2nd, and 3rd grade teachers. The dataset has two columns: `Grade` (equal to '1st', '2nd', or '3rd') and `Ant` (equal to 'Leaf Cutter' or 'Harvester').

Use this data to create a contingency table of the `Grade` and `Ant` columns, and save the table as `table`.

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `chi2_contingency()` function from SciPy to run a Chi-Square test using the contingency table you just created (saved as `table`). Save the p-value as `pval` and print it out.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Are certain types of ants more popular among specific grades (is there an association between grade and ant type)? Using a significance threshold of 0.05, indicate your answer by changing the value of `significant` to `True` if there is a significant association between these variables and `False` otherwise.

SOLUTIONS:

```
import pandas as pd
from scipy.stats import chi2_contingency

# read in and print data
ants = pd.read_csv("ants_grade.csv")
print(ants.head())

# create contingency table
table = pd.crosstab(ants.Grade, ants.Ant)
print(table)

# run Chi-Square test and print p-value
chi2, pval, dof, expected = chi2_contingency(table)
print(pval)

# determine significance
significant = False
```

RESULTS:

Output-only Terminal

```
Grade      Ant
0  1st      harvester
1  2nd      harvester
2  2nd      leaf cutter
3  1st      harvester
4  3rd      leaf cutter
Ant      harvester  leaf cutter
Grade
1st      28          8
2nd      31          5
3rd      23          13
0.08356116834982279
```

Assumptions of a Chi-Square Test

Before we use a Chi-Square test, we need to be sure that the following things are true:

1. The observations should be independently randomly sampled from the population

This is also true of 2-sample t-tests, ANOVA, and Tukey. The purpose of this assumption is to ensure that the sample is representative of the population of interest.

2. The categories of both

Preview: Docs Loading link description

[variables](#)

must be mutually exclusive

In other words, individual observations should only fall into one category per variable. This means that categorical variables like “college major”, where students can have multiple different college majors, would not be appropriate for a Chi-Square test.

3. The groups should be independent

Similar to 2-sample t-tests, ANOVA, and Tukey, a Chi-Square test also shouldn’t be used if either of the categorical variables splits observations into groups that can influence one another. For example, a Chi-Square test would **not** be appropriate if one of the variables represents three different time points.

QUESTIONS:

Researchers are running a study to test a new vaccine for Covid-19 in adults. A sample of 1000 adults (you can assume that they are randomly sampled adults, or at least representative of the population) are randomly split into two groups: half get a vaccine, while the other half get a placebo. Everyone is monitored for six months to see if they develop symptoms of Covid-19. The first few rows of the resulting dataset looks like this:

Group

Outcome

vaccine	not sick
---------	----------

vaccine	not sick
---------	----------

placebo	sick
---------	------

placebo	not sick
---------	----------

The researchers want to use this data to determine whether their vaccine will be effective at preventing illness in the general population of adults (eg., is whether or not someone got a vaccine associated with whether or not they got sick?).

Is a Chi-Square test appropriate to address this question? Change the value of `checkpoint_1` to `True` if a Chi-Square test is appropriate and `False` if it is not.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Researchers are interested in studying the effect of a 10 minute yoga regimen on self-reported mood in adults. In order to test this, a representative sample of 1000 adults are asked to complete a survey where they rate their current happiness level as “very low”, “low”, “neutral”, “high”, or “very high”. Each person then completes a 10 minute yoga regimen, then responds to the same survey once again. The first few rows of data from this study look like this:

Person ID	Time	Happiness
1	before yoga	low
1	after yoga	neutral
2	before yoga	neutral
2	after yoga	high

The researchers want to know if 10 minutes of yoga can help improve self-reported mood for adults in the general population (eg., is whether or not someone has just completed 10 minutes of yoga associated with their self-reported happiness level?).

Is a Chi-Square test appropriate to address this question? Change the value of `checkpoint_2` to `True` if a Chi-Square test is appropriate and `False` if it is not.

Stuck? Get a hint

SOLUTIONS

```
checkpoint_1 = True  
checkpoint_2 = False
```

Review

In this lesson, we have reviewed a few different ways to run a hypothesis test for an association between two variables:

Two Sample T-Tests (for an association between a quantitative variable and a

Preview: Docs Loading link description

[binary](#)

categorical variable)

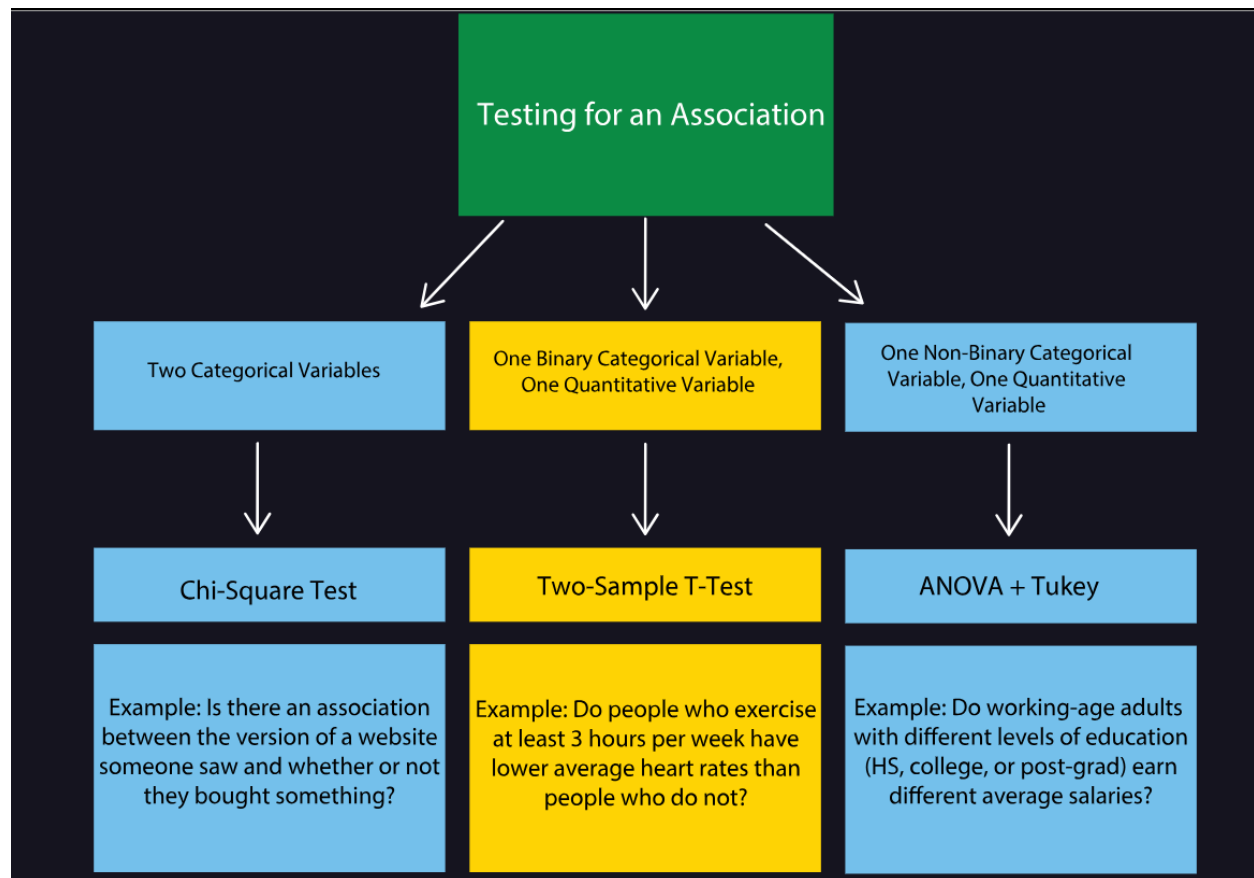
ANOVA and Tukey Tests (for an association between a quantitative variable and a non-binary categorical variable)

Chi-Square Tests (for an association between two categorical variables)

You are now equipped to use and interpret the results of each of these tests!

Hypothesis Testing for an Association - VIDEO- YouTube

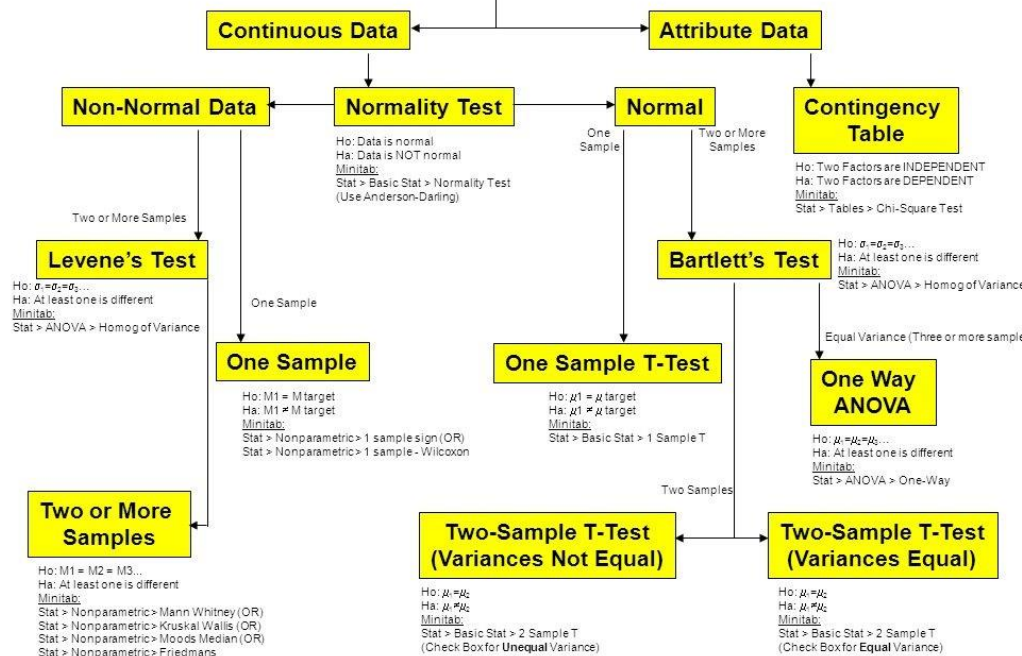
- <https://www.youtube.com/watch?v=w-7dSKL0UHs>
- <https://www.codecademy.com/paths/data-science/tracks/dsml-statistics-fundamentals-part-ii/modules/dsaly-hypothesis-testing-testing-an-association/informationals/live-stats-hp-association>
-



For all tests:
 $p > 0.05$ Fail to Reject H_0 (Null)
 $p < 0.05$ Reject H_0

Hypothesis Testing

“Roadmap”



How to Choose a Hypothesis Test

Learn how to choose an appropriate hypothesis test for your data and questions.

Introduction

In this module we'll cover some of the key considerations for designing an experiment. One of the first things to consider is **which statistical test** is appropriate for the question you are trying to answer. Sometimes this will depend on the data you have available, and sometimes you will have the opportunity to decide this before starting to collect data.

There are many different types of hypothesis tests that should be used in different situations. In this article, we are going to briefly review some of them and when they are appropriate.

Here is a summary table of some of the hypothesis tests we'll cover:

	Comparing a sample statistic to a hypothesized population value for a...	Testing an association between a binary categorical variable (2 categories) and a...	Testing an association between a categorical variable with 3 or more categories and a...
Quantitative Variable	<i>One sample t-test</i>	<i>Two-Sample t-test</i>	<i>ANOVA with Tukey's Range Test</i>
Categorical Variable	<i>Binomial Test</i>	<i>Chi-Square Test</i>	

Comparing a sample statistic to a hypothesized population value

In some situations, we may want to compare a **sample statistic** to a **hypothesized population value**. In this scenario, we have a sample of data from some population and have calculated a sample statistic (eg., a sample mean, frequency, or proportion). Based on this observed sample statistic, we then want to know whether the sample was likely to be drawn from a population with some hypothesized or target value of that same statistic.

For example, suppose we hypothesize that 5% of all people who access a website will buy a product. In order to understand if this is the case, we might find a sample of people who accessed that website and calculate that 4.3% of our sample bought a product. Finally, we can use a hypothesis test to determine the likelihood that, if we could observe ALL people who ever access the website, 5% of them will buy a product (even though only 4.3% of our sample did).

If we have a sample of quantitative data, such as height, weight, or amount spent, we should use a one-sample t-test. If we have a sample of binary data, such as whether or not someone made a purchase or clicked a link, we should use a binomial test.

Example: one sample t-test

Suppose we want to compare exam scores for students who attended a test prep program to the global average score of 35 points. Do students who attend this program score higher than 35 points? The global average is the **hypothesized population value** and the average of the exam scores of students who attended the program is the **sample statistic** (in this case, sample mean).

Below is the code to run a one-sample t-test to address the above question. In this example the **alternative hypothesis** is that the sample mean is significantly different than 35, and the **null hypothesis** is that the sample mean is 35.

```
from scipy.stats import ttest_1samp

global_average_score = 35
sample_scores = [12, 42, 37, 18, 23, 39, 45, ... , 52]

t_stat, p_value = ttest_1samp(sample_scores, global_average_score)
```

Example: binomial test

If we instead have a sample of **binary data** and want to compare a sample proportion/frequency to an **underlying probability (population value)**, a binomial test is appropriate. The classic example of a binomial test is tossing a coin to determine if it's fair (fair means that the probability of either heads or tails is exactly 50%).

For example, suppose that you collect sample data from a coin by tossing it 100 times, and find that 45 flips result in heads. Based on this sample, what is the probability that the coin is actually fair (if you flipped it infinitely many times, exactly half those flips would be heads)? The following code runs the binomial test to answer this question:

```
from scipy.stats import binom_test

p_value = binom_test(45, 100, p = 0.50)
```

The alternative hypothesis for this test is that the probability is **different** than $p = 0.50$, and the null is that it is equal to 0.50 .

Here are some other examples of situations where a binomial test would be useful

Fill in the blank

Questions

Quality Assurance at the Dairy Culture yogurt company

Code

```
You are a data analyst on the quality assurance team of a yogurt company, "The Dairy Culture". The team studies samples of its yogurt on a regular basis to make sure it meets its nutritional facts and collects data on the number of discarded yogurts during production. Their nutritional facts declare that a 6oz yogurt has 20g of Carbohydrates. The Dairy Culture also has a goal of discarding bad yogurts at a rate of 4% or lower.
```

To test both of these, you take a sample of 1000 6oz yogurts and find that those yogurts have an average of 23g of Carbohydrates. You also find that 48 of the 1000 yogurts you sampled were discarded (or 4.8%).

To check the claim that 6oz yogurts made by The Dairy Culture have 20g of carbohydrates on average, you would run a

blank 1

with the null hypothesis that all yogurts (the "population") have an average of

blank 2

of carbohydrates.

To check whether The Dairy Culture met their discarding goal of 4% or lower, you would run a

blank 3

with the null hypothesis that the discarding goal is less than

blank 4

for all yogurts (the population).

Answer Choices

6g

two-sample t-test

20%

4.8%

one sample t-test

4%

binomial test

20g

Click or drag and drop to fill in the blank

Testing for an association between two variables at the population level

When we have a sample of data with two variables, and want to know if there is an association between those variables at the population level, we'll need a different set of hypothesis tests. For example, we might be interested in the relationship between:

In each of these examples, we cannot observe the entire population of interest (eg., the entire wild population of each species of bear). Instead, we can collect data for a smaller sample and then use a hypothesis test to understand the likelihood that a similar association exists among the population we care about.

Example: two-sample t-test

A two-sample t-test is used to investigate an association between a quantitative variable and a binary categorical variable. For example, suppose we want to test if there is an association between claw size (quantitative) and species: black or grizzly bear (binary categorical). To answer this question, we could sample a selection of black bears and grizzly bears, then calculate the average claw size for each species. Then, we can use a two-sample t-test to determine the probability that the claw sizes for these two species are significantly different (among the entire population of black and grizzly bears). The data might look like this:

Species		Claw size (in)
0	black	1.48

1	grizzly	2.01
2	grizzly	1.89
3	black	1.65
4	black	1.58

And the code to run the two-sample t-test will look like this:

```
from scipy.stats import ttest_ind

#separate out claw lengths for two species
grizzly_bear = data.claw_length[data.species=='grizzly']
black_bear = data.claw_length[data.species=='black']

#run the t-test here:
tstat, pval = ttest_ind(grizzly_bear, black_bear)
```

Other examples of two-sample t-tests include studies like drug trials or psychology studies with a control and experimental group or A/B Testing with quantitative data like “time spent on a website”.

Example: ANOVA and Tukey's range test

In cases similar to the two-sample t-test, but when the categorical variable has three or more categories, an ANOVA can be used to see if there is a significant difference between *any* of the groups. Then, if at least one pair of groups are significantly different, Tukey's range test can be used to determine **which** groups are different. This is better than running multiple two-sample t-tests because it leads to a lower probability of making a type I error.

For example, if we want to compare the heights of **three different tree species**, in order to test the hypothesis that average tree heights vary by species, we can use an ANOVA. Then, if the p-value from the ANOVA is below our significance threshold, we can run Tukey's range test to determine which tree species have significantly different heights.

The code to run these tests is as follows:

```
# ANOVA Test
from scipy.stats import f_oneway
fstat, pval = f_oneway(heights_pine, heights_oak, heights_spruce)

# Tukey's Range Test
from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey_results = pairwise_tukeyhsd(tree_data.height, tree_data.species, 0.05)
```

Example: Chi-Square test

When looking at the relationship between two categorical variables, we can run a Chi-Square Test to see if there is a significant association between the variables. Both variables can have any number of categories. Some examples include testing an association between:

For example, suppose we are interested in understanding whether there is an association between the version of a website someone sees and whether or not they subscribe. The data may look something like this:

Web Version		Subscribed?
0	A	yes
1	A	yes
2	B	yes
3	B	no
4	A	no

These variables can then be summarized using a contingency table:

Subscribed		no	yes
Web Version			
A		23	27
B		16	34

Finally, a Chi-Square test evaluates whether the observed contingency table is significantly different from the table that would be expected if there were no association between the variables. The code to create the contingency table and run the Chi-Square test looks something like this:

```
import pandas as pd
from scipy.stats import chi2_contingency

# create contingency table
ab_contingency = pd.crosstab(data.Web_Version, data.Subscribed)

# run a Chi-Square test
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
```

Finding a representative sample

Beyond choosing a hypothesis test, it is important to understand whether the data you have meets the assumptions of the test you want to run. Each hypothesis test has a unique set of assumptions, which we won't cover in detail in this article; however, there is one assumption that all hypothesis tests share: the data was randomly sampled from the population of interest.

This is important because random sampling ensures that the sample is representative of the population in terms of observed (and unobserved) characteristics. Unfortunately, there may be situations where random sampling is impossible, but it is important to understand how this can bias results of a test.

For example, let's return to the example with the yogurt company "The Dairy Culture". Let's say the company had multiple factories, but the quality assurance team only collected yogurts from one specific factory. The data is thus **not** randomly sampled from the entire population that we care about (all factories), and could be biased if the quality of yogurt differs at each one.

There can also be ethical issues that arise when a sample is not representative of a population. When developing and testing a vaccine, for example, researchers must make sure to find volunteers from an appropriate proportion of genders, races, age ranges, pre-existing conditions, and so on to test efficacy for the entire population that the vaccine will be used on. If the vaccine manufacturers test on a sample that doesn't include sufficient data for one race, there is a risk that there could be reduced (if during the initial research phase) or unknown efficacy for that group.

It can often be challenging to find a representative sample or even to recognize when there is biased data, but it is essential to think about when designing an experiment.

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-statistics-fundamentals-part-ii/modules/dsinf-experimental-design/lessons/experimental-design/exercises/review>

A/B Testing: Sample Size Calculators

Baseline Conversion Rate

A/B tests usually compare an option that we're currently using to a new option that we suspect might be better. In order to compare the two options, we need a metric. Often, our metric will be the percent of users who take a certain action after interacting with one of our options. For instance:

The percent of customers who **buy a t-shirt** after visiting one of two versions of a website
The percent of users who **click** on one of two versions of an ad

In the t-shirt example above, the *baseline conversion rate* is our estimate for the percent of people who will buy a shirt under the current website design.

We can generally calculate a baseline by looking at historical data for the option that we're currently using. For example, suppose that 2000 people visited a website over the past three months and 320 of those visitors purchased a shirt. We could estimate the baseline rate as follows:

```
baseline = 320/2000*100
print(baseline) #output: 16.0
```

Copy to Clipboard

This number may be written as a proportion (eg., 0.16) or a percent (eg., 16%).

QUESTIONS:

Suppose that an online media company wants to run an A/B test to see if a re-designed subscribe button will make site visitors more likely click “subscribe”. Some historical data is saved for you in **script.py**:

`number_of_site_visitors` is the number of people who visited the site over the past month
`number_of_converted_visitors` is the number who clicked “subscribe”.

Use these two variables to calculate the baseline conversion rate and save the result as a variable named `baseline_rate`, then print it out. Report the percentage, not the proportion.

Hint

Divide the number of converted visitors by the number of total visitors and multiply by 100.

SOLUTIONS:

```
number_of_site_visitors = 2000.0
number_of_converted_visitors = 1300.0

# calculate baseline_rate and print it out:
baseline_rate = number_of_converted_visitors/number_of_site_visitors*100

print(baseline_rate)
```

Minimum Detectable Effect

Suppose we’re running an A/B Test to find out if a new website layout drives more subscriptions than the current one. If the new layout is only a tiny percent better, would we really care?

In order to detect precise differences, we need a very large sample size. In order to choose a sample size, we need to know the smallest difference that we actually care to measure. This “smallest difference” is our desired *minimum detectable effect*. This is also sometimes referred to as *desired lift*.

Minimum detectable effect or lift is generally expressed as a percent of the baseline conversion rate. Suppose that 6% of customers currently subscribe to our website (that’s our baseline conversion rate). Changing a website layout is hard, so we only think that it’s worth doing if at least 8% of our customers would subscribe with the new layout. To calculate this as a percentage of our baseline:

```
baseline = 6
new = 8
min_detectable_effect = (new - baseline) / baseline * 100
print(min_detectable_effect) #output: 33.0
```

Copy to Clipboard

Our minimum detectable effect/desired lift is 33%.

Instructions

Checkpoint 1 Passed

1.

Suppose you have a baseline conversion rate of 8% and want to implement a new website design if it will increase the conversion rate to 12%. What is your minimum detectable effect? Save that number into a variable called `mde` and print it out.

Significance Threshold

When we run an A/B test, we usually want to use the results of the test to make a decision: use version A or B? In order to make that decision, many data scientists use a pre-determined significance threshold for their hypothesis test. For example, if we set a significance threshold of 0.05 (a commonly chosen value), we'll "reject the null hypothesis" and conclude that the conversion rate for version B is significantly different from version A if we get a p-value less than 0.05.

It turns out that this significance threshold is the *false positive rate* for the test: the probability of finding a significant difference when there really is none. As a business owner, we don't want to make this kind of mistake, because then we might invest money in a change that doesn't actually make a difference!

Unfortunately, there's a trade-off between false positives and false negatives. A false negative occurs when there is a difference between version A and B, but the test doesn't detect it. This is a potential missed opportunity for a business owner!

Most A/B test sample size calculators estimate the sample size needed for a 20% false negative rate; while a data scientist needs to choose the false positive rate they are comfortable with. The lower the false positive rate, the larger the sample size will need to be!

Don't Interfere With Your Tests

Suppose that a Product Manager is running an A/B Test for a redesign of a landing page. Before starting the test, she used a sample size calculator to determine the sample size: 2,200 total website visitors. After reaching 2,200 visits, she ran a Chi-Square Test. The new website design performed slightly better, but the results were not statistically significant.

It might be tempting to run the test for another week to see if the difference becomes significant, but that would be a big mistake! By choosing to extend the A/B test past the original sample size, the project manager would introduce personal bias to the results of the test; she will be more likely to get the results she wants, regardless if these results reflect reality.

Here are two important rules for making sure that A/B tests remain unbiased:

- Don't continue to run the test after the predetermined sample size, until "significant" results are found.
- Don't stop a test before reaching the predetermined sample size, just because your results reach significance early (unless there are ethical reasons that require you to stop, like a prescription drug trial).

Test data is sensitive to changes in sample size, which is why it is important to calculate beforehand.

Sample Size Determination with Simulation

Introduction

In this lesson, we will use simulation to understand some of the considerations for setting up an A/B test: sample size, power, and the false positive rate. But before we think about designing an A/B test, let's first remind ourselves how to conduct the test itself, **after** planning and collecting data.

Suppose that a media company currently has a weekly newsletter email and wants to see if using the recipient's first name in the email subject will cause more people to open the email (ie. "Bob! Checkout this week's updates" vs "Checkout this week's updates"). They randomly assign a group of 100 recipients to receive one of the two email subjects and record whether or not each recipient opened the email. The first few rows of their data might look something like this:

Email	Opened
name	yes
name	no
control	yes
control	yes
name	no

In order to run a hypothesis test to decide whether there is a significant difference in the open rate for these emails, we would run a Chi-Square test. To accomplish this, we would first create a contingency table for the `Email` and `Opened`

Preview: Docs Loading link description

[variables](#)

in the above table:

```
X = pd.crosstab(data.Email, data.Opened)
print(X)
```

Copy to Clipboard

Output:

	Opened	no	yes
Email			
control		23	27

We would then use this table to run a Chi-Square test and get a p-value:

```
chi2, pval, dof, expected = chi2_contingency(X)
print(pval) #Output: 0.2186
```

Copy to Clipboard

Based on the p-value, we would make a decision about which email to use; a small p-value would provide evidence that the open rates are significantly different for the two groups, while a large p-value would suggest no significant difference.

QUESTIONS

Run the code in **script.py** to see the first five rows of data.

Checkpoint 2 Passed

2.

Suppose that you are running an A/B test comparing two versions of a checkout page (version A or version B) to see whether there is a significantly different purchase rate for one version compared to the other. Data from this experiment has been loaded for you in **script.py** as a dataframe named **data**. Use this data to create a contingency table and save the result as **ab_contingency**, then print out the result.

Hint

Use the `pd.crosstab()` function with the two columns of the dataframe `Web_Version` and `Purchased` as the two input variables.

Checkpoint 3 Passed

3.

Use **ab_contingency** to run a Chi-Square test using `chi2_contingency()` and save the p-value as a variable named **pval**. Print out **pval**.

SOLUTIONS

```
import pandas as pd

from scipy.stats import chi2_contingency

data = pd.read_csv("ab_data.csv")
print(data.head())

# calculate contingency table here
ab_contingency = pd.crosstab(data.Web_Version, data.Purchased)
print(ab_contingency)

# run your chi square test here
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
```

```
print('P-value')  
print(pval)
```

Simulating Data for a Chi-Square test

In the last exercise, we used some data from an A/B test to run a Chi-Square test. In the next few exercises, we'll build up a simulation to understand the considerations that go into choosing a sample size for that test.

Again consider the A/B test example from the previous exercise, comparing email subjects with and without the recipient's first name. Suppose we know that visitors have a 50% chance of opening the control email and a 65% chance of opening the name email (30% lift!).

Here we use **lift** to refer to the inherent difference in the distributions of our two groups of data. In the *A/B Testing: Sample Size Calculators* lesson, we learned that **minimum detectable effect** is the smallest size of the difference between the two groups that we want our test to be able to detect. If we set up our experiment with a minimum detectable effect of at least 20%, our statistical test should detect a difference with a "lift" or "effect" of 20% or greater. In this lesson we are going to simulate data that has a lift of 30% to demonstrate how the inherent lift impacts the power of our statistical test.

We can use the aforementioned probabilities to simulate a dataset of 100 email recipients as follows:

```
sample_control = np.random.choice(['yes', 'no'], size=50, p=[.5, .5])  
sample_name = np.random.choice(['yes', 'no'], size=50, p=[.65, .35])
```

Copy to Clipboard

This gives us two simulated samples, of 50 recipients each, who hypothetically saw the name or control email subject. Each one looks something like `['yes' 'no' 'no' 'no' 'yes' 'yes' ...]`, where 'yes' corresponds to an opened email.

Next, we can assemble these arrays into a data frame that looks a lot like the one we saw in exercise 1:

```
group = ['control']*50 + ['name']*50  
outcome = list(sample_control) + list(sample_name)  
sim_data = {"Email": group, "Opened": outcome}  
sim_data = pd.DataFrame(sim_data)  
print(sim_data.head())
```

QUESTIONS

1.

In **script.py**, you'll see the code from the narrative, which can be used to simulate a dataset for a Chi-Square test. You'll notice that we've replaced all hard-coded numbers with the following variables: `sample_size`, `control_rate`, and `name_rate` (which is calculated using `control_rate` and `lift`).

Change the sample size to `4` and press "Run". Inspect the output. Does it look as expected?

Stuck? Get a hint

Checkpoint 2 Enabled

2.

Press “Run” a few more times and notice how the data changes each time even though you haven’t changed the code. This happens because we’ve provided probabilities for the outcomes; (opened or not), rather than specific values.

SOLUTIONS

```
import numpy as np
import pandas as pd

sample_size = 4
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate,1-control_rate])
sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])

group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
outcome = list(sample_control) + list(sample_name)
sim_data = {"Button": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)
print(sim_data)
```

Determining Significance

Now that we’ve practiced simulating data for an A/B test, let’s actually run a Chi-Square test for each simulated dataset and consider the decision we would make based on the outcome.

If we were really running this test, we would want to use the data to make a decision about whether to use the control (old) or name (new) email subject. To make that decision, we can use a significance threshold. For example, if we’re using a significance threshold of 0.05, we’ll “reject the null hypothesis” for any p-value less than 0.05. In this context, rejecting the null would mean that we conclude that there **is** a significant difference between the open rates for the two email subjects and therefore we **should** switch to the email subject that uses the recipient’s first name.

We can use the following Python statement to record whether a particular p-value is significant or not, based on a threshold of 0.05:

```
result = ('significant' if pval < 0.05 else 'not significant')
print(result)
```

QUESTIONS:

The code from the previous exercises is provided for you in **script.py**. This code generates a simulated dataset named `sim_data` and then runs a Chi-Square test for that data, saving the p-value as `pval`. An additional variable named `significance_threshold` has been defined for you in **script.py**, which is equal to the significance threshold for the test. After the p-value calculation, add a line of code that uses `significance_threshold` to determine whether the p-value is 'significant' or 'not significant'. Save the result as `result` and print it out.

Stuck? Get a hint

Checkpoint 2 Passed

2.

Press “Run” a few times until you see both a 'significant' and a 'not significant' result. Note that it is possible to get different results every time you sample a new group of 100 recipients.

SOLUTIONS

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# pre-set values
significance_threshold = 0.05
sample_size = 100
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

# simulate a dataset
sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate,1-control_rate])
sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])

group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
outcome = list(sample_control) + list(sample_name)
sim_data = {"Email": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)

# run a chi-square test
ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
chi2, pval, dof, expected = chi2_contingency(ab_contingency, correction=False)
print("P Value:")
print(pval)

# determine significance here:
result = ('significant' if pval < significance_threshold else 'not significant')

print("Result:")
print(result)
```

Estimating Power

In the last exercise, we learned how to simulate a dataset for a Chi-Square test, run the test, and then output a result: 'significant' or 'not significant'. In this exercise, we'll repeat that process many times so that we can inspect the relative frequency of each outcome.

To do this, we'll start by creating an empty list to store the results of our repeated experiments. Next, we'll move all of our simulation code (to create a sample dataset, run a Chi-Square test, and determine a result) inside of a for-loop. In each iteration of the loop, we'll append the outcome to our results list so that we can inspect it later.

The outline of the code looks something like this:

```
Set the sample size and subscription probabilities
Create an empty list named `results`
```

```
Repeat 100 times in a for-loop:
    Simulate a dataset
    Run a Chi-Square test
    Use the p-value to determine significance
    Append the result ('significant' or 'not significant') to `results`
```

Finally, we can inspect `results` by calculating the proportion of simulated tests where the result was `'significant'`:

```
results = np.array(results)
```

```
print(np.sum(results == 'significant')/100)
```

1.

In **script.py**, we've copied over the code from the previous exercise and moved the simulation inside a for-loop as described in the narrative. We've also initialized an empty list named `results`.

Below the determination of `result`, but still inside the for-loop, add a line of code to append `result` onto `results`.

Hint

`results` is a list, so you can use the `.append()` method to append a value to it. Make sure to keep this inside the for-loop by indenting to keep it inline with the rest of the code in the for-loop.

Checkpoint 2 Passed

2.

Outside of the for-loop, create a variable named, `sig_prop` that calculates the proportion of `results` that are 'significant'. Make sure to divide it by the total number of simulations to make it a proportion!

Print the results, then press "Run" a few times (note: you'll see slightly different numbers each time because this is a random process). Approximately what proportion of the results were significant (would have led us to switch to the new, name email subject)?

Hint

To find the proportion of results that were significant, you can add the 1s and divide by the total number of simulations. The code is similar to what we used above:

```
results = np.array(results)
sig_prop = np.sum(results == 'significant')/100
print(sig_prop)
```

Copy to Clipboard

Don't forget to convert the list into an array before summing.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 100
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list here:
    results.append(result)

# calculate proportion of significant results here:
```

```
print("Proportion of significant results:")
results = np.array(results)
sig_prop = np.sum(results == 'significant')/100
print(sig_prop)
```

False Positives and True Positives

In the previous exercise, we simulated 1,000 datasets and ran a Chi-Square test for each one, recording whether the results were 'significant' or 'not significant'. This allowed us to estimate the proportion of simulated datasets that led to a 'significant' result.

In general, we hope that the test reflects reality. We therefore want the result to be 'significant' if there really **is** a significant difference in the probability of an open for the two email subjects ($\text{lift} > 0$). In that case, the proportion of significant results is the true positive rate, also called the *power* of the test. Most sample size calculators aim for a power of 80%.

On the other hand, if there's no difference in the probability of an email being opened for the two email subjects ($\text{lift} = 0$), a 'significant' result would be a false-positive (also called a type I error). This would lead us to invest time and resources into adding first names into email subjects when there's no real pay-off in the long run.

Instructions

Checkpoint 1 Enabled

1.

The simulation code from the previous exercises is loaded for you in **script.py**. We've included the code to print out the proportion of tests where a significant result was recorded. Currently, the simulation is set up so that there **is** a difference in the probability of a subscription for the two buttons.

Press "Run" a few times and inspect the proportion of significant tests (printed to the output terminal) each time. If we ran a test with the provided sample size (100), baseline conversion rate (50%) and lift (30%), approximately what percent of the time would we correctly observe a significant result? Note that this is the "power" of the test.

Hint

When we run this code, we see that approximately 28% of the tests result in a significant p-value. Every time we run the simulation, we'll get slightly different numbers, but they're all roughly between .20 and .36.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, change the value of `lift` so that the proportion of significant tests is equal to the **false positive rate** and press "Run" once more.

Note that the proportion of significant tests should be approximately equal to the significance threshold if you've done this correctly.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 100
lift = 0
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
```

```

sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
outcome = list(sample_control) + list(sample_name)
sim_data = {"Email": group, "Opened": outcome}
sim_data = pd.DataFrame(sim_data)

# run the test
ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
chi2, pval, dof, expected = chi2_contingency(ab_contingency)
result = ('significant' if pval < significance_threshold else 'not significant')

# append the result to our results list:
results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)

```

Trade Offs

At this point, let's return to the point of view of a product manager who is actually planning this A/B test. Suppose that the product manager wants to be able to accurately detect a lift of 30% (or higher), but also wants to avoid false positives (they don't want to change the email subjects unless there's actually a difference between them). To plan their test, the product manager needs to consider the following:

Increasing the sample size increases the power of the test (the probability of detecting a difference if there is one); however, larger sample sizes require more time and resources.

Increasing the significance threshold also increases the power of the test; however, it simultaneously increases the false positive rate (the probability of detecting a difference when there **isn't** one).

Finally, if the project manager chooses a larger minimum detectable effect/lift, then they'll be able to decrease the sample size without decreasing power. However, if they set up their test to detect a minimum lift of 30% (for example), they may not be able to detect smaller differences that are still meaningful.

Instructions

Checkpoint 1 Enabled

1.

The simulation code from the previous exercises is provided for you in **script.py**. Currently, the simulation is set up to use an open rate of 50% for the control email, and a lift of 30% for the name email subject. Set the sample size of 100 and press "Run" and make note of the proportion of significant results (which is the power of the test).

Hint

Make sure the sample size variable, `sample_size`, is set to 100.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now increase the sample size to 500 and press "Run" again. Note that the power of the test also increases. Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Next, increase the significance threshold to 0.10. Note that the power of the test increases even more.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Finally, increase the lift to 40%. Note that again, the power of the test increases.

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.10
sample_size = 500
lift = .4
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list:
    results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)
```

Review

Congratulations! You've now learned how to use simulation to investigate the trade-offs for an A/B test sample-size calculation. As a recap, this lesson covered the following:

- The significance threshold for a test is equal to the false positive rate
- The power of a test is the probability of correctly detecting a significant result
- Increasing sample size increases the power of a test
- Increasing the significance threshold increases power, but also increases the false positive rate
- Larger sample sizes are needed to detect smaller effect sizes

Two notes about the terminology in the sample size calculator:

- Baseline conversion rate** is equivalent to our `control_rate` in the code.
- Minimum detectable effect (MDE)** is the smallest effect size (or `lift`) that we want our test to be able to detect. If the MDE is *larger* than our true `lift`, power will decrease because our sample size might not be large enough to detect the difference between the two groups.

```

import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# preset values
significance_threshold = 0.05
sample_size = 180#change the sample size here
lift = .3
control_rate = .5
name_rate = (1 + lift) * control_rate

# initialize an empty list of results
results = []

# start the loop
for i in range(100):
    # simulate data:
    sample_control = np.random.choice(['yes', 'no'], size=int(sample_size/2),
p=[control_rate, 1-control_rate])
    sample_name = np.random.choice(['yes', 'no'], size=int(sample_size/2), p=[name_rate, 1-
name_rate])
    group = ['control']*int(sample_size/2) + ['name']*int(sample_size/2)
    outcome = list(sample_control) + list(sample_name)
    sim_data = {"Email": group, "Opened": outcome}
    sim_data = pd.DataFrame(sim_data)

    # run the test
    ab_contingency = pd.crosstab(np.array(sim_data.Email), np.array(sim_data.Opened))
    chi2, pval, dof, expected = chi2_contingency(ab_contingency)
    result = ('significant' if pval < significance_threshold else 'not significant')

    # append the result to our results list:
    results.append(result)

# calculate proportion of significant results:
print("Proportion of significant results:")
results = np.array(results)
print(np.sum(results == 'significant')/100)

```

Codecademy Live: Hypothesis Testing for an Association

Watch a live demo about hypothesis testing for an association!

This video is the seventh installment of a livestream series covering content from the Master Statistics Skill Path at Codecademy.

In this session, we'll turn our attention to hypothesis tests that can be used to evaluate an association between two variables among a population that we can't observe.

https://www.youtube.com/watch?v=w-7dSKL0UHs&t=332s&ab_channel=Codecademy

If you'd like to code along off-platform, you can download our code for this session from our [Github repository](#). The example code is provided in a Jupyter notebook. To open it on your computer, we recommend installing [Anaconda](#) (download links at the bottom of the page). You can also find a live demo of how to get started on your own computer during the first part of [this video](#).

GitHub: <https://github.com/Codecademy/Master-Statistics-Live-Series>

Introduction: Linear Algebra

Find out what you'll learn about linear algebra and why it's important.

Welcome to Linear Algebra

In this unit, we will cover fundamental linear algebra operations for vectors and matrices—including how to represent and solve linear systems of equations.

Why is this important?

Vectors and matrices are fundamental building blocks of data science as they are used to organize and manipulate data we feed through learning models. Whether you are just starting your journey in data science or have already played around with machine learning models, learning linear algebra fundamentals is key to your success in understanding advanced data science material.

There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

Efficient computation of linear transformation on large quantities of data. Computer programs and hardware like GPUs have been optimized around these linear algebra data structures to enable much of the high-performance computing advances.

Computer graphics, especially video games, rely heavily on linear algebra to render images quickly and efficiently.

Machine learning and advances in deep learning, whose foundations are matrix and vector multiplication and addition.

What will you learn?

At the end of this module, you will be able to:

Perform basic mathematical operations between scalars, vectors, and matrices.

Use vectors and matrices to represent and solve linear systems of equations.

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Linear Algebra?

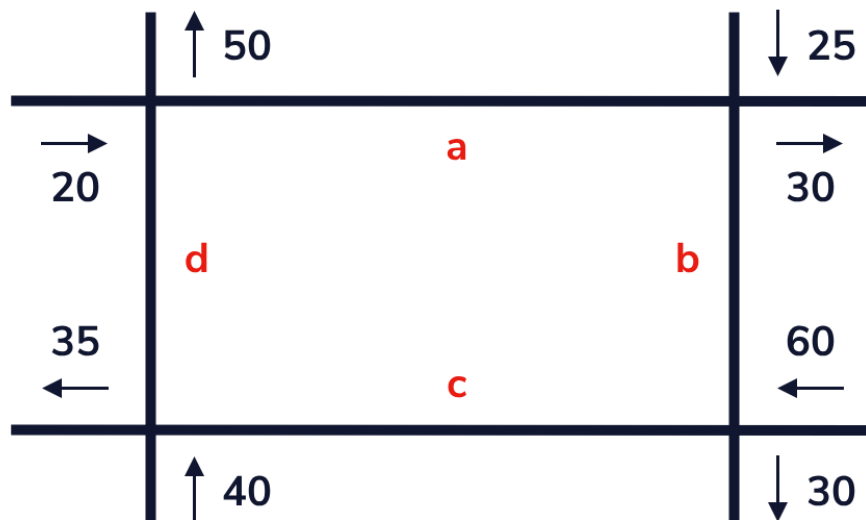
Learn about what linear algebra is and how it is relevant for data science!

Linear algebra focuses on the mathematics surrounding linear operations and solving systems of linear equations. While most of us learned about basic linear operations in algebra classes in high school and middle school, “linear algebra” extends these lessons to apply to multi-dimensional data.

The reality is that many engineering optimization problems can be reduced to systems of linear equations, albeit systems with many equations and many variables. By simplifying problems, we can solve for a solution or set of solutions.

Let’s imagine a problem where we can do this. We have a section of town containing four roads and four intersections. We are trying to find the number of cars along the roads in between the intersections and have data about the number of cars departing intersections outside this road system. How can we find out our missing car counts?

To solve the problem mathematically, we can realize that this system can be modeled as a system of linear equations since the number of cars entering an intersection must equal the number leaving an intersection. We can visualize the data on the diagram below.



This creates a straightforward system, written out below— one that we can solve without linear algebra techniques.

$$20+d=50+a \quad 25+a=30+b \quad 60+b=30+c \quad 40+c=35+d$$

2

Now, what if we extend this problem to the entire city of Manhattan? And now only provide inconsistent intersection counts, requiring us to solve for many unknown road segment car counts? In this case, we can still set the problem up as an extensive system of linear equations, but we quickly realize that solving for those unknown car counts by hand is impossible. This is where linear algebra will come in and allow us to solve these complex problems.

Coming Up

In the upcoming content, we will look at two important linear algebra data structures that can store high-dimensional data. This data can be actual numerical data like one might see on a spreadsheet or representative data like if we need to represent a large system of equations. We will learn how to perform various operations between these data structures and use these techniques to solve systems of linear equations.

As mentioned earlier, linear algebra is fundamental because it allows us to mathematically operate on large amounts of data, which is vital to modern data science techniques. There are many real-world applications of linear algebra, some of which we will cover in the coming exercises and lessons, including:

Solving systems of linear equations, especially those with many variables and cannot be reasonably solved by hand.

Efficient computation of linear transformation on large quantities of data. Throughout this lesson, we will go into great detail on data structures that hold large amounts of data (vectors and matrices) and their various operations that allow us to apply linear transforms. Computer programs and hardware like GPU's have been optimized around these linear algebra data structures to enable much of the high-performance computing advances.

Computer graphics, especially video games, rely heavily on linear algebra to render images quickly and efficiently.

Machine learning and advances in deep learning, whose foundations are matrix and vector multiplication and addition.

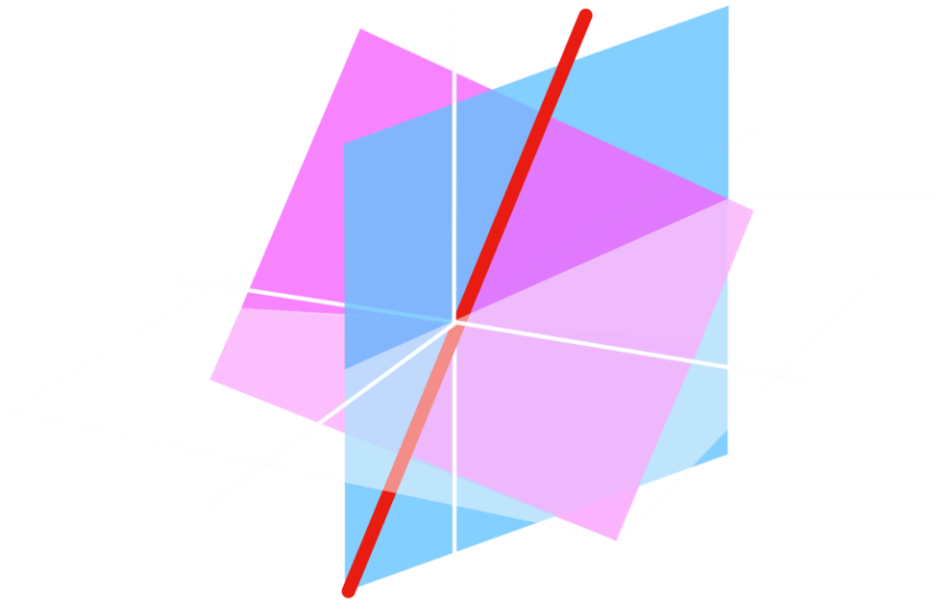
Introduction

Linear algebra is a branch of mathematics that studies linear transformations, including the use of vectors and matrices. Linear algebra enables us to solve problems like linear regression, as well as computational problems that concern large amounts of data (machine learning, computer graphics, etc.).

In this lesson, we will learn about the following topics:

- Vectors and their basic operations
- The dot product between vectors
- Matrices and their basic operations
- Matrix multiplication and special matrices
- Gauss-Jordan Elimination to solve systems of linear equations
- Gauss-Jordan Elimination to solve for inverse matrices

These topics will give us a solid foundation in linear algebra, providing a strong base to solve more complicated problems like linear regression in future content.



Vectors

The fundamental building blocks of linear algebra are *vectors*. Vectors are defined as quantities having both direction and magnitude, compared to *scalar* quantities that only have magnitude. In order to have direction and magnitude, vector quantities consist of two or more elements of data. The *dimensionality* of a vector is determined by the number of numerical elements in that vector. For example, a vector with four elements would have a dimensionality of four.

Let's take a look at examples of a scalar versus a vector. A car driving at a speed of 40mph is a scalar quantity. Describing the car driving 40mph to the east would represent a two-dimensional vector quantity since it has a magnitude in both the x and y directions.

Vectors can be represented as a series of numbers enclosed in parentheses, angle brackets, or square brackets. In this lesson, we will use square brackets for consistency. For example, a three-dimensional vector is written as:

$$\mathbf{v}$$

The magnitude (or length) of a vector, $\|\mathbf{v}\|$, can be calculated with the following formula:

$$\|\mathbf{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$$

This formula translates to the sum of each vector component squared, which can be also written out as:

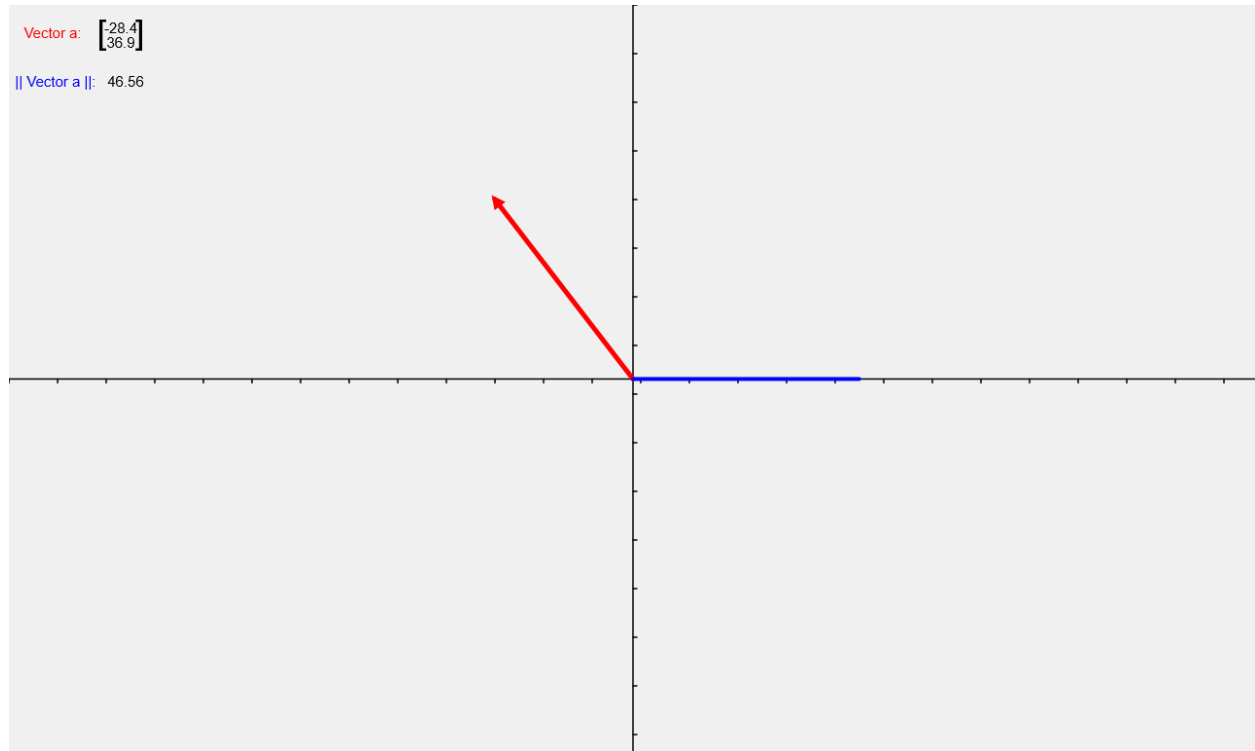
$$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

Let's look at an example. We are told that a ball is traveling through the air and given the velocities of that ball in its x, y, and z directions in a standard Cartesian coordinate system. The velocity component values are:

$$\begin{aligned} x &= -12 \\ y &= 8 \end{aligned}$$

$$z = -2$$

Convert the velocities into a vector, and find the total speed of the ball. (Hint: the speed of the ball is the magnitude of the velocity vector!)



Basic Vector Operations

Now that we know what a vector is and how to represent one, we can begin to perform various operations on vectors and between different vectors. As we've previously discussed, the basis of linear algebra is the linear combinations we can generate between vectors (and matrices).

Scalar Multiplication

Any vector can be multiplied by a scalar, which results in every element of that vector being multiplied by that scalar individually.

$$k \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kz \end{bmatrix}$$

Multiplying vectors by scalars is an *associative operation*, meaning that rearranging the parentheses in the expression does not change the result. For example, we can say $2(a3) = (2a)3$.

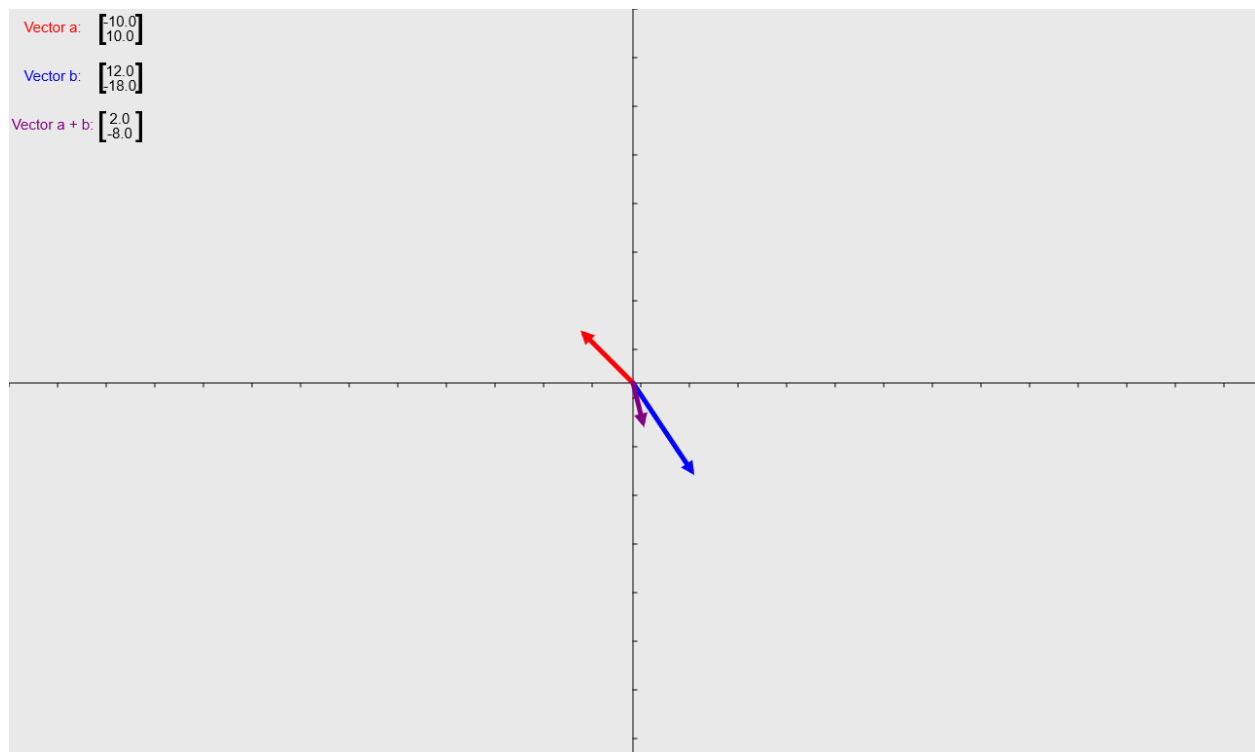
Vector Addition and Subtraction

Vectors can be added and subtracted from each other when they are of the same dimension (same number of components). Doing so adds or subtracts corresponding elements, resulting in a new vector of the same dimension

as the two being summed or subtracted. Below is an example of three-dimensional vectors being added and subtracted together.

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} + 2 \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix} - 3 \begin{bmatrix} x_3 \\ y_3 \\ z_3 \end{bmatrix} = \begin{bmatrix} x_1 + 2x_2 - 3x_3 \\ y_1 + 2y_2 - 3y_3 \\ z_1 + 2z_2 - 3z_3 \end{bmatrix}$$

Vector addition is *commutative*, meaning the order of the terms does not matter. For example, we can say $(a+b = b+a)$. Vector addition is also *associative*, meaning that $(a + (b+c) = (a+b) + c)$.



Vector Dot Products

An important vector operation in linear algebra is the *dot product*. A dot product takes two equal dimension vectors and returns a single scalar value by summing the products of the vectors' corresponding components. This can be written out formulaically as:

$$a \cdot b = \sum_{i=1}^n a_i b_i$$

The dot product operation is both commutative ($a \cdot b = b \cdot a$) and distributive ($a \cdot (b+c) = a \cdot b + a \cdot c$).

The resulting scalar value represents how much one vector "goes into" the other vector. If two vectors are perpendicular (or orthogonal), their dot product is equal to 0, as neither vector "goes into the other."

Let's take a look at an example dot product. Consider the following two vectors:

$$a = \begin{bmatrix} 3 \\ 2 \\ -3 \end{bmatrix}, b = \begin{bmatrix} 0 \\ -3 \\ -6 \end{bmatrix}$$

To find the dot product between these two vectors, we do the following:

$$a \cdot b = 3 * 0 + 2 * -3 + -3 * -6 = 12$$

The dot product can also be used to find the magnitude of a vector and the angle between two vectors. To find the magnitude, we can reference Exercise 2 and see that the magnitude of a vector is simply the square root of a vector's dot product with itself.

$$||a|| = \sqrt{a \cdot a}$$

$$||a|| = \sqrt{12}$$

To find the angle between two vectors, we rely on the dot product between the two vectors and use the following equation.

$$\theta = \arccos \frac{a \cdot b}{||a|| ||b||}$$

Let's look at the same two vectors from above:

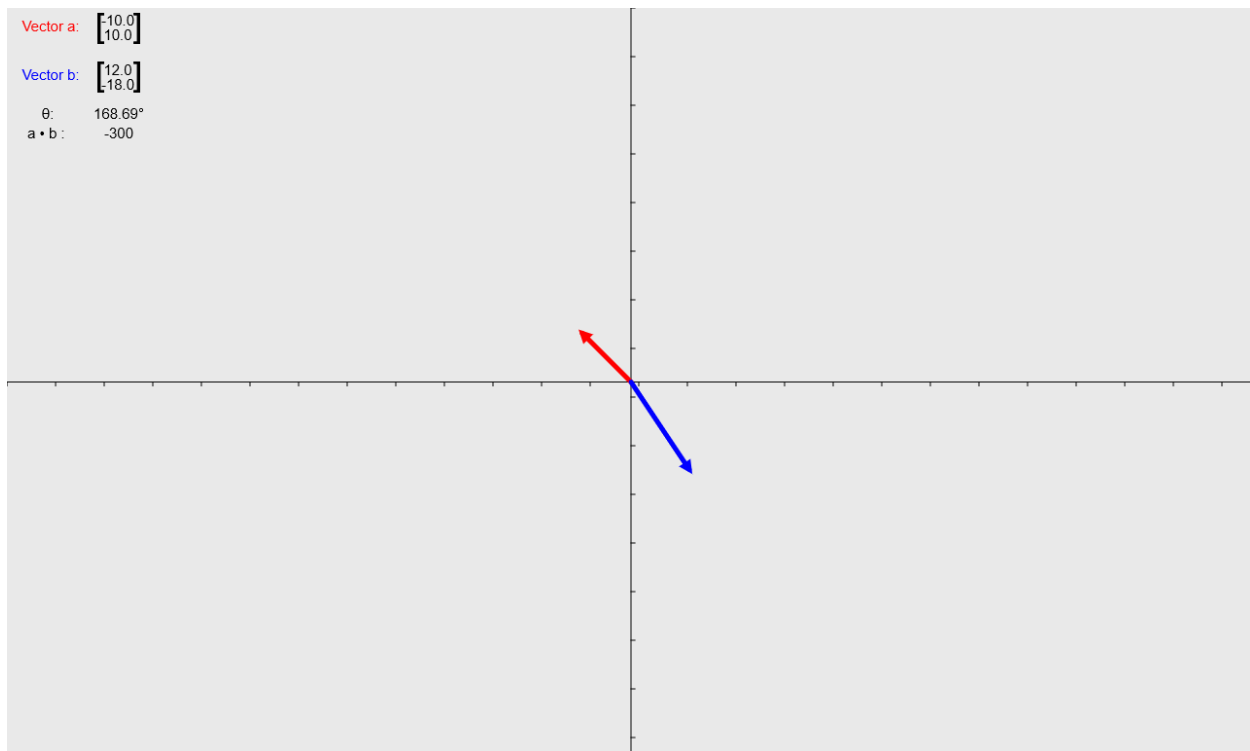
$$a = \begin{bmatrix} 3 \\ 2 \\ -3 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ -3 \\ -6 \end{bmatrix}$$

To find the angle between these two vectors, we do the following:

$$\theta = \arccos \frac{3 * 0 + 2 * -3 + -3 * -6}{\sqrt{(3)^2 + (2)^2 + (-3)^2} * \sqrt{(0)^2 + (-3)^2 + (-6)^2}}$$

Solving this, we end up with:

$$\theta = 67.58^\circ$$



Matrices

Now that we've learned about vector quantities, we can expand upon those lessons and focus on matrices. A *matrix* is a quantity with m rows and n columns of data. For example, we can combine multiple vectors into a matrix where each column of that matrix is one of the vectors.

To the right, you can see a comparison between scalars, vectors, and matrices for context.

We can also think of vectors as single-column matrices in their own right. Matrices are helpful because they allow us to perform operations on large amounts of data, such as representing entire systems of equations in a single matrix quantity.

Matrices can be represented by using square brackets that enclose the rows and columns of data (elements). The shape of a matrix is said to be $m \times n$, where there are m rows and n columns. When representing matrices as a variable, we denote the matrix with a capital letter and a particular matrix element as the matrix variable with an " m, n " determined by the element's location. Let's look at an example of this. Consider the matrix below.

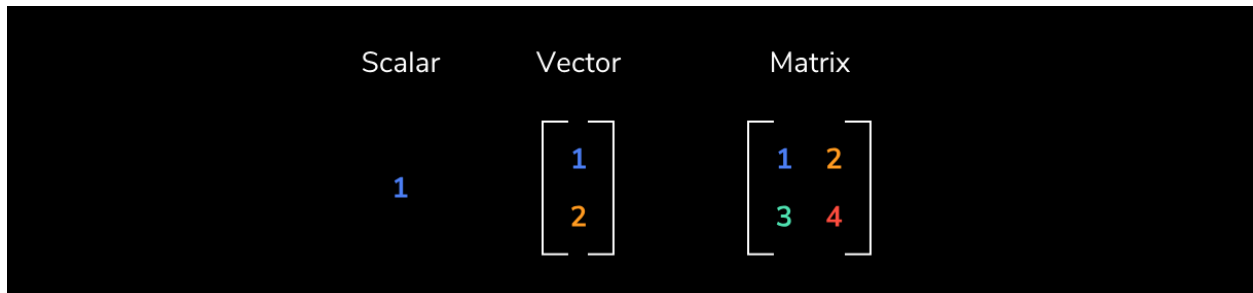
$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

The value corresponding to the first row and second column is b .

$$A_{1,2} = b$$

What is the value corresponding to the second row and third column?

$$A_{2,3} = f$$



Matrix Operations

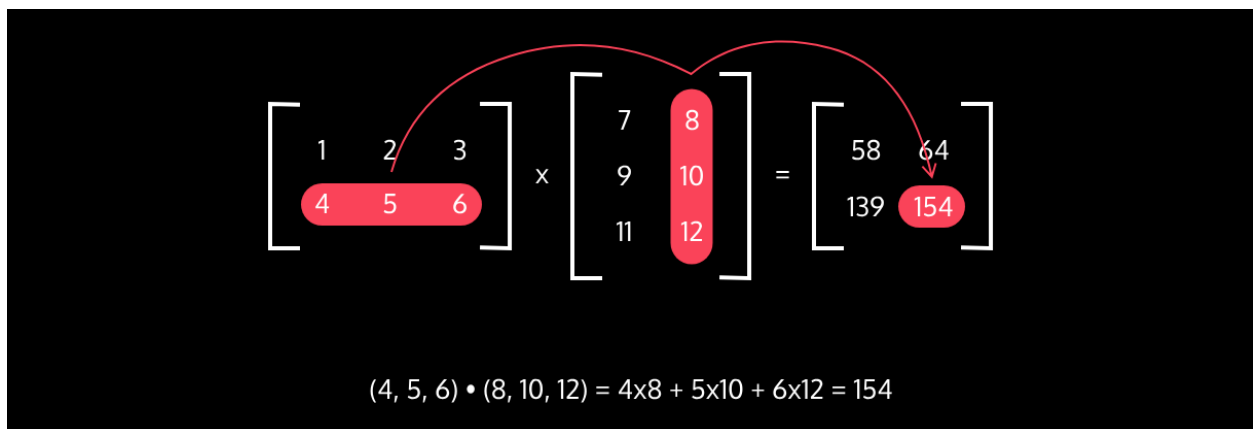
Like with vectors, there are fundamental operations we can perform on matrices that enable the linear transformations needed for linear algebra. We can again both multiply entire matrices by a scalar value, as well as add or subtract matrices with equal shapes.

$$2 \begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix} + 3 \begin{bmatrix} c_1 & d_1 \\ c_2 & d_2 \end{bmatrix} = \begin{bmatrix} 2a_1 + 3c_1 & 2b_1 + 3d_1 \\ 2a_2 + 3c_2 & 2b_2 + 3d_2 \end{bmatrix}$$

A new and important operation we can now perform is matrix multiplication. Matrix multiplication works by computing the dot product between each row of the first matrix and each column of the second matrix. For example, in computing $AB = C$, element (1,2) of the matrix product C will be the dot product of row 1 of matrix A and column 2 of matrix B . The animation to the right walks through the multiplication of two matrices showing the dot product between each row and column.

An important rule about matrix multiplication is that the shapes of the two matrices AB must be such that the number of columns in A is equal to the number of rows in B . With this condition satisfied, the resulting matrix product will be the shape of $rowsA \times columnsB$. For example, in the animation to the right, the product of a 2x3 matrix and a 3x2 matrix ends up being a 2x2 matrix.

Based on how we compute the product matrix with dot products and the importance of having correctly shaped matrices, we can see that matrix multiplication is not commutative, $AB \neq BA$. However, we can also see that matrix multiplication is associative, $A(BC) = (AB)C$.



Identity matrix

The *identity matrix* is a square matrix of elements equal to 0 except for the elements along the diagonal that are equal to 1. Any matrix multiplied by the identity matrix, either on the left or right side, will be equal to itself.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Transpose matrix

The *transpose* of a matrix is computed by swapping the rows and columns of a matrix. The transpose operation is denoted by a superscript uppercase "T" (A^T).

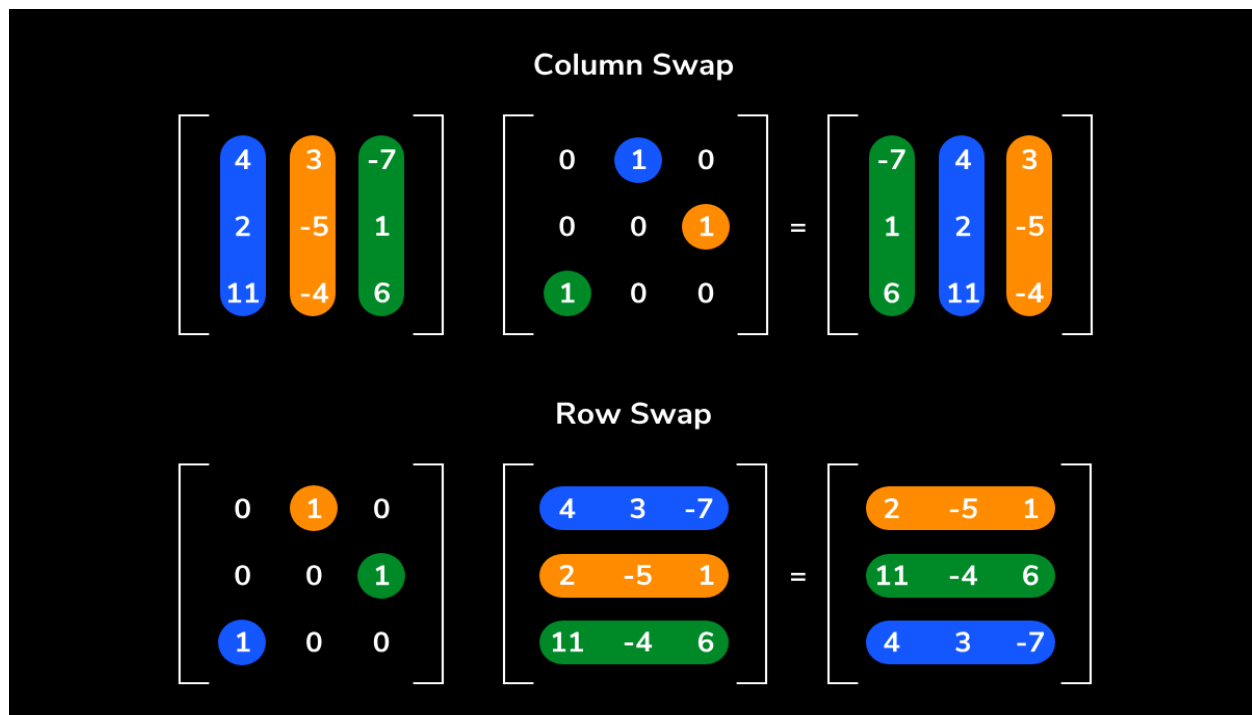
$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}^T = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}$$

Permutation Matrix

A *permutation matrix* is a square matrix that allows us to flip rows and columns of a separate matrix. Similar to the identity matrix, a permutation matrix is made of elements equal to 0, except for one element in each row or column that is equal to 1. In order to flip rows in matrix A , we multiply a permutation matrix P on the left (PA). To flip columns, we multiply a permutation matrix P on the right (AP).

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \begin{bmatrix} d & e & f \\ g & h & i \\ a & b & c \end{bmatrix}$$

$$AP = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} c & a & b \\ f & d & e \\ i & g & h \end{bmatrix}$$



Linear Systems in Matrix Form

An extremely useful application of matrices is for solving systems of linear equations. Consider the following system of equations in its algebraic form.

$$\begin{aligned} a_1x + b_1y + c_1z &= d_1 \\ a_2x + b_2y + c_2z &= d_2 \\ a_3x + b_3y + c_3z &= d_3 \end{aligned}$$

This system of equations can be represented using vectors and their linear combination operations that we discussed in the previous exercise. We combine the coefficients of the same unknown variables, as well as the equation solutions, into vectors. These vectors are then scalar multiplied by their unknown variable and summed.

$$x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

Our final goal is going to be to represent this system in form $Ax = b$, using matrices and vectors. As we learned earlier, we can combine vectors to create a matrix, which we'll do with the coefficient vectors to create matrix A . We can also convert the unknown variables into vector x . We end up with the following $Ax = b$ form:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

Click below if you would like to see how these two expressions are equivalent:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Click below if you would like to see how these two expressions are equivalent:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = x \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + y \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + z \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

► Click to see proof

We can also write $Ax = b$ in the following form:

$$Ax = b \rightarrow [A \mid b]$$

This means we can write our $Ax = b$ equation above as:

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix} \rightarrow \begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$$

This is what is known as an *augmented matrix*, and we will use these in the following exercises to solve for linear systems!

Gauss-Jordan Elimination

Now that we have our system of linear equations in augmented matrix form, we can solve for the unknown [variables](#) using a technique called *Gauss-Jordan Elimination*. In regular algebra, we may try to solve the system by combining equations to eliminate variables until we can solve for a single one. Having one variable solved for then allows us to solve for a second variable, and we can continue that process until all variables are solved for.

The same goal can be used for solving for the unknown variables when in the matrix representation. We start with forming our augmented matrix.

$$\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$$

To solve for the system, we want to put our augmented matrix into something called *row echelon form* where all elements below the diagonal are equal to zero. This looks like the following:

$$\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & b'_1 & c'_1 & d'_1 \\ 0 & 1 & c'_2 & d'_2 \\ 0 & 0 & 1 & d'_3 \end{bmatrix}$$

Note that the values with apostrophes in the *row echelon form* matrix mean that they have been changed in the process of updating the matrix. Once in this form we can rewrite our original equation as:

$$\begin{aligned} x + b'_1 y + c'_1 z &= d'_1 \\ y + c'_2 z &= d'_2 \\ z &= d'_3 \end{aligned}$$

This allows us to solve the equations directly using simple algebra. But how do we get to this form?

To get to row echelon form we swap rows and/or add or subtract rows against other rows. A typical strategy is to add or subtract row 1 against all rows below in order to make all elements in column 1 equal to 0 under the diagonal. Once this is achieved, we can do the same with row 2 and all rows below to make all elements below the diagonal in column 2 equal to 0.

Once all elements below the diagonal are equal to 0, we can simply solve for the variable values, starting at the bottom of the matrix and working our way up.

It's important to realize that not all systems of equations are solvable! For example, we can perform Gauss-Jordan Elimination and end up with the following augmented matrix.

Inverse Matrices

The *inverse of a matrix*, A^{-1} , is one where the following equation is true:

$$AA^{-1} = A^{-1}A = I$$

This means that the product of a matrix and its inverse (in either order) is equal to the identity matrix.

The inverse matrix is useful in many contexts, one of which includes solving equations with matrices. Imagine we have the following equation:

$$xA = BC$$

If we are trying to solve for x , we need to find a way to isolate that variable. Since we cannot divide matrices, we can multiply both right sides of the equation by the inverse of A , resulting in the following:

$$xAA^{-1} = BCA^{-1} \rightarrow x = BCA^{-1}$$

An important consideration to keep in mind is that not all matrices have inverses. Those matrices that do not have an inverse are referred to as *singular matrices*.

To find the inverse matrix, we can again use Gauss-Jordan elimination. Knowing that $AA^{-1} = I$, we can create the augmented matrix $[A \mid I]$, where we attempt to perform row operations such that $[A \mid I] \rightarrow [I \mid A^{-1}]$.

One method to find the necessary row operations to convert $A \rightarrow I$ is to start by creating an upper triangular matrix first, and then work on converting the elements above the diagonal afterward (starting from the bottom right side of the matrix). If the matrix is invertible, this should lead to a matrix with elements equal to 0 except along the diagonal. Each row can then be multiplied by a scalar that leads to the diagonal elements equaling 1 to create the identity matrix.

Cancel out the first entry in row 2

$R_2 - R_1 \rightarrow$

-1	1	2	0	0	1
0	-3	-2	0	1	-1
0	2	1	1	0	0

Using NumPy Arrays

The core of using NumPy effectively for linear algebra is using NumPy arrays. NumPy arrays are n-dimensional

Preview: Docs Loading link description

[array](#)

Preview: Docs Loading link description

[data structures](#)

that can be used to represent both vectors (1-dimensional array) and matrices (2-dimensional arrays).

A NumPy array is initialized using the `np.array()` function, and including a Python list

Preview: Docs Loading link description

[argument](#)

or Python nested list argument for arrays with more than one dimension.

For example, the following creates a NumPy array representation of a vector:

```
v = np.array([1, 2, 3, 4, 5, 6])
```

Copy to Clipboard

We can also create a matrix, which is the equivalent of a two-dimensional NumPy array, using a nested Python list:

```
A = np.array([[1,2],[3,4]])
```

Copy to Clipboard

If we print this NumPy array, it will look like the following:

```
[[1 2]
 [3 4]]
```

Copy to Clipboard

Matrices can also be created by combining existing vectors using the `np.column_stack()` function:

```
v = np.array([-2,-2,-2,-2])
u = np.array([0,0,0,0])
w = np.array([3,3,3,3])

A = np.column_stack((v, u, w))
print(A)
```

Copy to Clipboard

Output:

```
[[ -2  0  3]
 [ -2  0  3]
 [ -2  0  3]
 [ -2  0  3]]
```

Copy to Clipboard

To access the shape of a matrix or vector once it's been created as a NumPy array, we call the `.shape` attribute of the array variable:

```
A = np.array([[1,2],[3,4]])
print(A.shape)
```

Copy to Clipboard

Here, the output will be:

```
(2, 2)
```

Copy to Clipboard

since there are two rows and two columns.

To access individual elements in a NumPy array, we can

Preview: Docs Loading link description

[index](#)

the array using square brackets. Unlike regular Python lists, we can index into all dimensions in a single square bracket, separating the dimension indices with commas.

Thus in order to index the element equal to 2 in matrix A, we can do the following:

```
A = np.array([[1,2],[3,4]])
print(A[0,1])
```

Copy to Clipboard

The first index accesses the specific row, while the second index accesses the specific column. Note that rows and columns are zero-indexed. In this example, the output will be the following:

```
2
```

Copy to Clipboard

We can also select a subset or entire dimension of a NumPy array using a colon. For example, if we want the entire second column of a matrix, we can index the second column and use an empty colon to select every row as such:

```
A = np.array([[1,2],[3,4]])
print(A[:,1])
```

Copy to Clipboard

Output:

```
[2 4]
```

Copy to Clipboard

Note: `[2 4]` is a column vector, but it outputs in the

Preview: Docs A terminal is a command-line interface (CLI) that allows users to interact with a computer's operating system by executing text-based commands.

[terminal](#)

horizontally.

```
import numpy as np

# Given vectors
vector_1 = np.array([-2,-6,2,3])
vector_2 = np.array([4,1,-3,8])
vector_3 = np.array([5,-7,9,0])
```

```
import numpy as np

# Given vectors
vector_1 = np.array([-2,-6,2,3])
vector_2 = np.array([4,1,-3,8])
vector_3 = np.array([5,-7,9,0])

# Answer 1
matrix = np.column_stack((vector_1, vector_2, vector_3))
print(matrix)
```

```
# Answer 2
print(matrix.shape)

# Answer 3
second_column = matrix[:, 2]
print(second_column)
```

Using NumPy for Linear Algebra Operations

Now that we know how to use NumPy arrays to create vectors and matrices, we can learn how to perform various linear algebra operations using Python.

To multiply a vector or matrix by a scalar, we use inbuilt Python multiplication between the NumPy [array](#) and the scalar:

```
A = np.array([[1,2],[3,4]])
4 * A
```

```
import numpy as np

# Given
# 2 x 3 matrix
A = np.array([[2,3,-4], [-2, 1, -3]])
# 2 x 3 matrix
B = np.array([[1,-1,4], [3,-3,3]])
# 3 x 2 matrix
C = np.array([[1, 2], [3, 4], [5, 6]])

# Calculate D = 4A - 2B
D = 4*A - 2*B
print(D)

# Calculate E = AC
E = A@C
print(E)

# Calculate F = CA
F = C@A
print(F)
```

Special Matrices

In addition to having built-in support for many linear algebra-related operations, Let's see how NumPy can create special matrices, such as the identity matrix.

An identity matrix can be constructed using the `np.eye()` functions, which takes an integer

Preview: Docs An argument is the actual value of a variable passed into a function.
[argument](#)
that determines the $n \times n$ size of the square identity matrix.

```
# 4x4 identity matrix
identity = np.eye(4)
```

Copy to Clipboard

In the output

Preview: Docs A terminal is a command-line interface (CLI) that allows users to interact with a computer's operating system by executing text-based commands.

[terminal](#)

,identity renders as:

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
```

Copy to Clipboard

A matrix or vector of all zeros can be constructed using the `np.zeros()` function, which takes in a

Preview: Docs Loading link description

[tuple](#)

argument for the shape of the NumPy

Preview: Docs Loading link description

[array](#)

filled with zeros.

```
# 5-element vector of zeros
zero_vector = np.zeros((5))
```

```
import numpy as np

# Given
A = np.array([[1,-1,1], [0,1,0], [-1,2,1]])
B = np.array([[0.5,1.5,-0.5], [0,1,0], [0.5,-0.5,0.5]])

# Find the matrix product AB and BA, and determine what that product says about A and B.
print(np.matmul(A,B))
print(B@A)

# Find the transpose of matrix A and matrix B
print(A.T)
print(B.T)
```


Additional Linear Algebra Operations

Moving past special matrices, there are some more advanced linear algebra operations we can perform using NumPy. To start, we will be using the `numpy.linalg` sublibrary to perform the following operations:

The “norm” (or length/magnitude) of a vector can be found using `np.linalg.norm()`:

```
v = np.array([2,-4,1])
v_norm = np.linalg.norm(v)
```

Copy to Clipboard

This outputs `v_norm` as:

```
4.58257569495584
```

Copy to Clipboard

The inverse of a square matrix, if one exists, can be found using `np.linalg.inv()`:

```
A = np.array([[1,2],[3,4]])
print(np.linalg.inv(A))
```

Copy to Clipboard

This outputs:

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

Copy to Clipboard

Finally, we can actually solve for unknown

Preview: Docs Loading link description

[variables](#)

in a system on linear equations in $Ax=b$ form using `np.linalg.solve()`, which takes in the A and b parameters. Given:

```
x + 4y - z = -1
-x - 3y + 2z = 2
2x - y - 2z = -2
```

Copy to Clipboard

We convert to $Ax=b$ form and solve.

```
# each array in A is an equation from the above system of equations
A = np.array([[1,4,-1],[-1,-3,2],[2,-1,-2]])
# the solution to each equation
b = np.array([-1,2,-2])
# solve for x, y, and z
x,y,z = np.linalg.solve(A,b)
```

Copy to Clipboard

If we `print((x, y, z))`, we see the following output:

```
(0, 0, 1.0)
```

Copy to Clipboard

showing that the solution to the system of equations is:

```
x = 0
y = 0
z = 1
import numpy as np
```

```
# Represent the following system in NumPy matrix/vector form, then solve for x, y, and z

# Given
'''
4x + z = 2
-y + 2z - 3x = 0
.5y - x - 1.5z = -4
'''

# Answer 1
A = np.array([[4, 0, 1], [-3, -1, 2], [-1, 0.5, -1.5]])

# Answer 2
b = np.array([2, 0, -4])

# Answer 3
x,y,z = np.linalg.solve(A,b)
print((x,y,z))
```

```
(6.0, -62.0, -22.0)
```

Introduction: Differential Calculus

Find out what you'll learn about differential calculus and why it's important.

Welcome to Differential Calculus

In this unit, we will cover fundamentals of differential calculus and investigate how we can use limits and derivatives to analyze functions.

Why is this important?

Differential calculus deals with the study of rates of change. It allows us to quantify how one variable relates to another. In the field of data science, this is invaluable information to understand. Learning models are built upon quantifying how variables relate to each other to make predictions or understand patterns. Whether you are just starting on your data science journey or have worked with machine learning models, understanding the principles of calculus is an invaluable asset to add to your toolbelt!

What will you learn?

At the end of this module, you will be able to:

Where can you get help and discuss the content?

Learning is social. Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#) and on [Discord](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

What is Calculus?

Learn about what calculus is and how it is important in the field of data science.

The core task of a data scientist is to make data-driven recommendations on the best way to solve a problem. Along the way, data scientists build and fit statistical models to support their claims, make predictions, and more. In this article, we will look at what calculus is and why it is vital in the data science process.

Rate of Change

Differential calculus is the field that studies rates of change. By quantifying how variables relate to each other, we can better understand our statistical model or physical system. For example, as a data scientist at a t-shirt company, we may want to know how a change in the marketing budget impacts the company's total profit. Spend too little, and our product won't be well known; spend too much, and we might reach a point of diminishing returns where our money would be better spent elsewhere. How can we find the optimal size of our marketing budget?

Optimization

The last question above is the sort of questions asked and answered by the field of *mathematical optimization*, which intertwines closely with calculus. Optimization problems involve looking for the best solution to a problem. Perhaps that's finding the 'optimal' size of the marketing budget for our company, the point where we maximize our profits with a budget that allows us to reach a broad audience without paying too much. Perhaps that means finding the statistical model that 'best' fits our data according to some criteria that we define ourselves— mathematical optimization gives us a way to decide among all our possible options. It is a powerful field that underpins much of the modern machine learning revolution and it is built upon the fundamentals of calculus we will investigate.

Infinitesimal Analysis

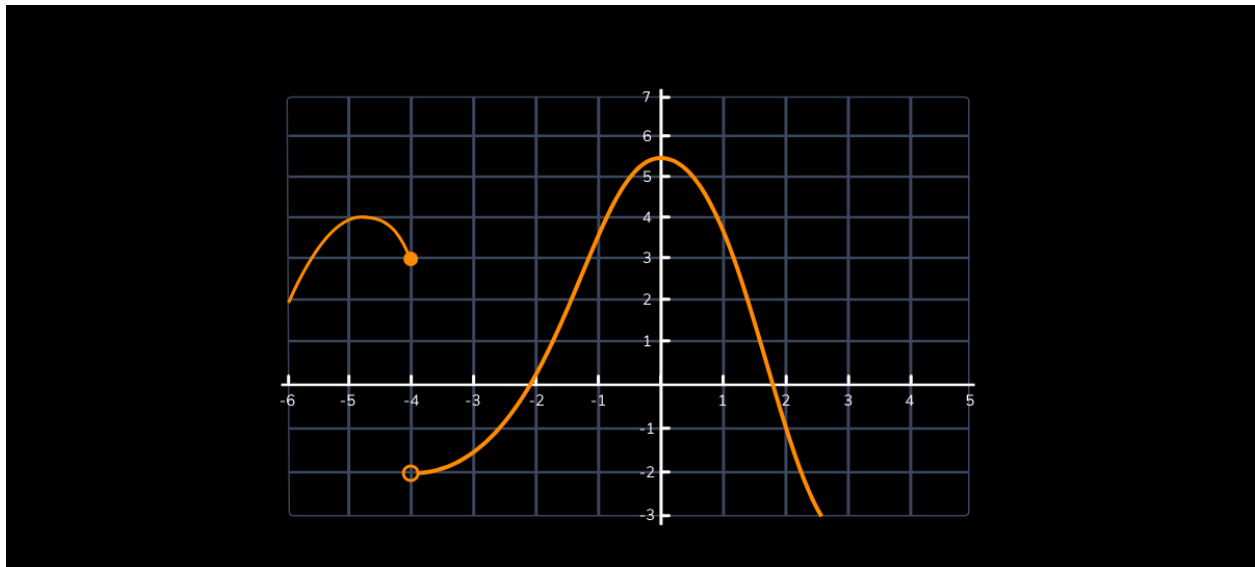
One of the fundamental paradigms in calculus is to think via *infinitesimals* (extremely small quantities) to say things about instantaneous rates of change. We will discuss this at length in the upcoming lesson. To give a rough overview, however, the idea is that by 'zooming in' enough and considering how a variable changes over an infinitesimal time frame, we can learn how that variable is changing instantaneously, at a specific moment in time.

HOW LONG CAN YOU WORK ON MAKING A ROUTINE TASK MORE
EFFICIENT BEFORE YOU'RE SPENDING MORE TIME THAN YOU SAVE?
(ACROSS FIVE YEARS)

		HOW OFTEN YOU DO THE TASK					
		50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
HOW MUCH TIME YOU SHAVE OFF	1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
	5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
	30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
	1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
	5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
	30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
	1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
	6 HOURS				2 MONTHS	2 WEEKS	1 DAY
	1 DAY					8 WEEKS	5 DAYS

Limits

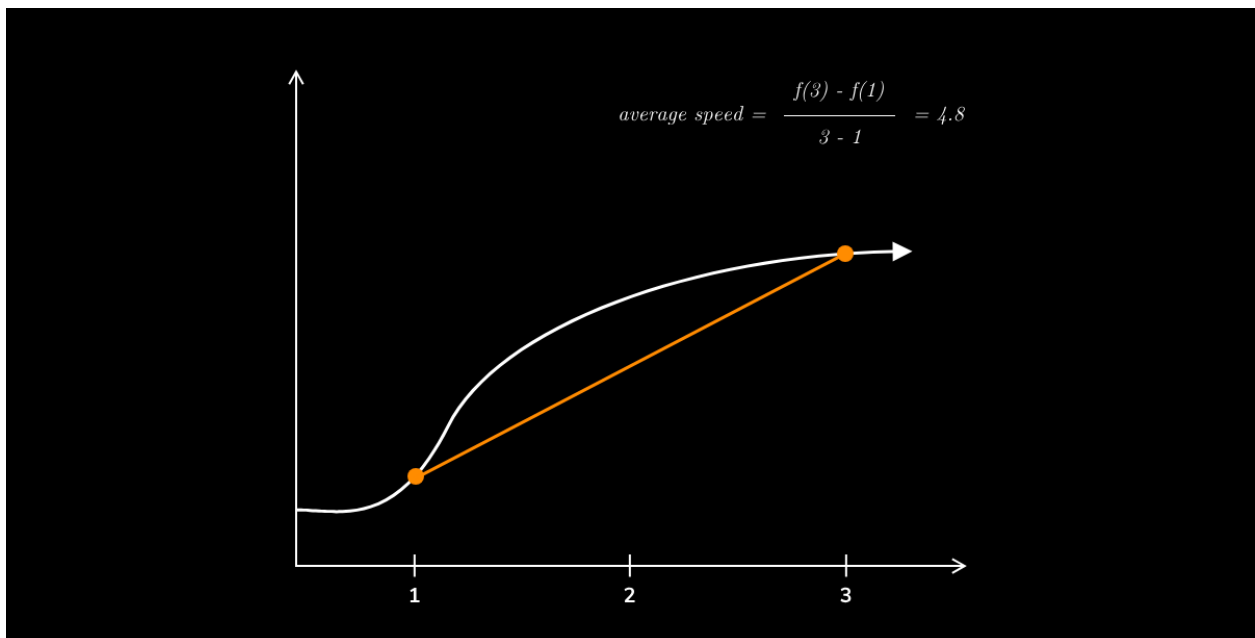
The first concept we will look at is something called a *limit*. Limits quantify what happens to the values of a function as we approach a given point. This can be defined notationally as:



Limit Definition of a Derivative

Suppose we wanted to measure a runner's instantaneous speed using a stopwatch. By instantaneous speed, we mean their speed at an exact moment in time.

Let's define $f(t)$ as the runner's distance from the start time at time t . How could we find the runner's instantaneous speed at 1 second after the race? We could record their positions at 1 second and some time after 1 second and then do some calculations. Let's say we measure their distance from the start at $f(1)$ and $f(3)$. The change in position is $f(3) - f(1)$, and the time between 1 second and 3 seconds is 2 seconds

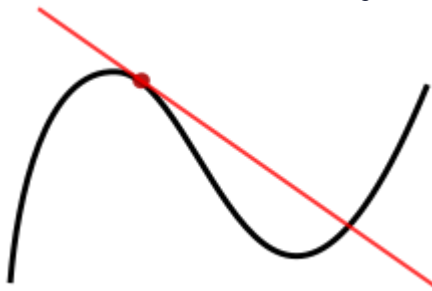


The Derivative Function

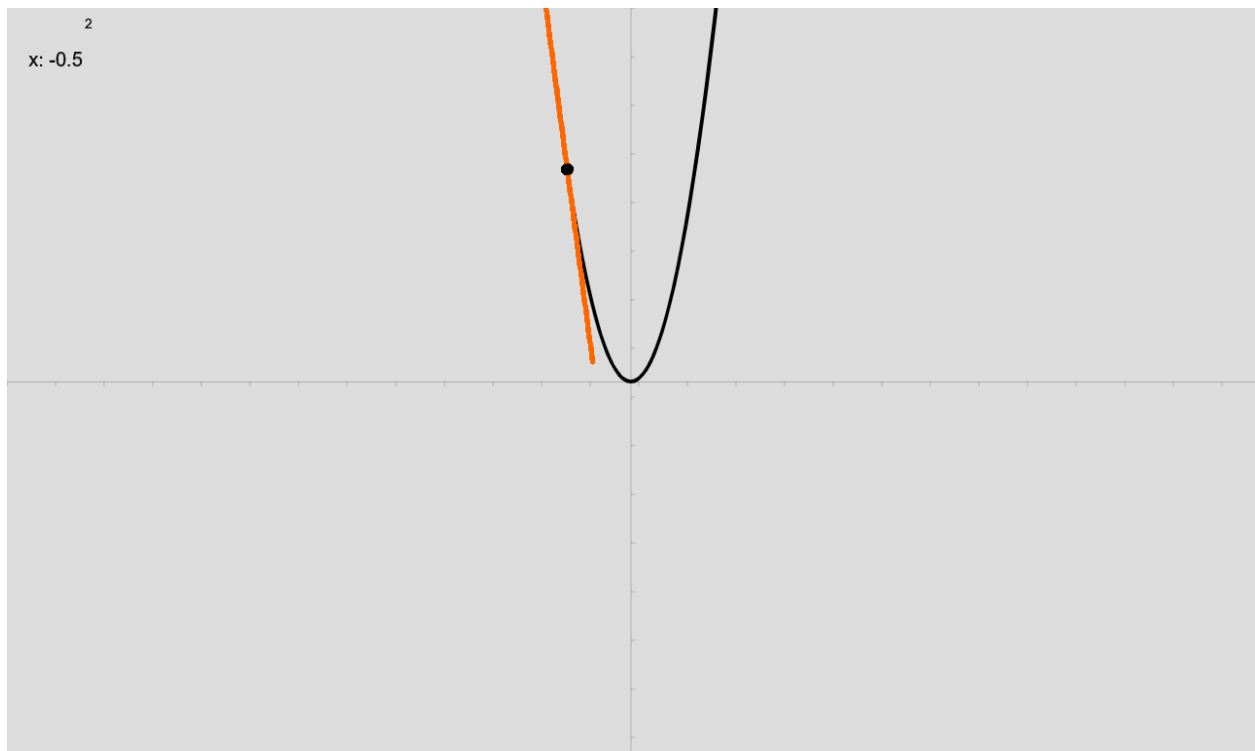
In the previous exercise, we discussed the idea of the derivative at a point as the instantaneous rate of change at that point. The idea behind the definition is that by finding how the function changes over increasingly small intervals starting at a point, we can understand how a function changes exactly at that point. The formal expression of this concept is written as:

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

An equivalent interpretation is that the derivative is the slope of the tangent line at a point. A *tangent line* at a point is the line that touches the function at that point. Just as the slope of a line describes how a line changes, the slope of the tangent line describes how a “curvier” function changes at a specific point. The visual below shows what a



tangent line looks like:



Properties of the Derivative Function

Derivative functions contain important information. Let's look at a couple of examples. Going back to our running example, $f(t)$ describes a runner's position as a function of time; therefore, the derivative function $f'(t)$ describes the runner's speed at any time t . Let's say we have a function $s(t)$ that describes the real-time sales of a particular product; the corresponding derivative function $s'(t)$ describes how those sales are changing in real-time.

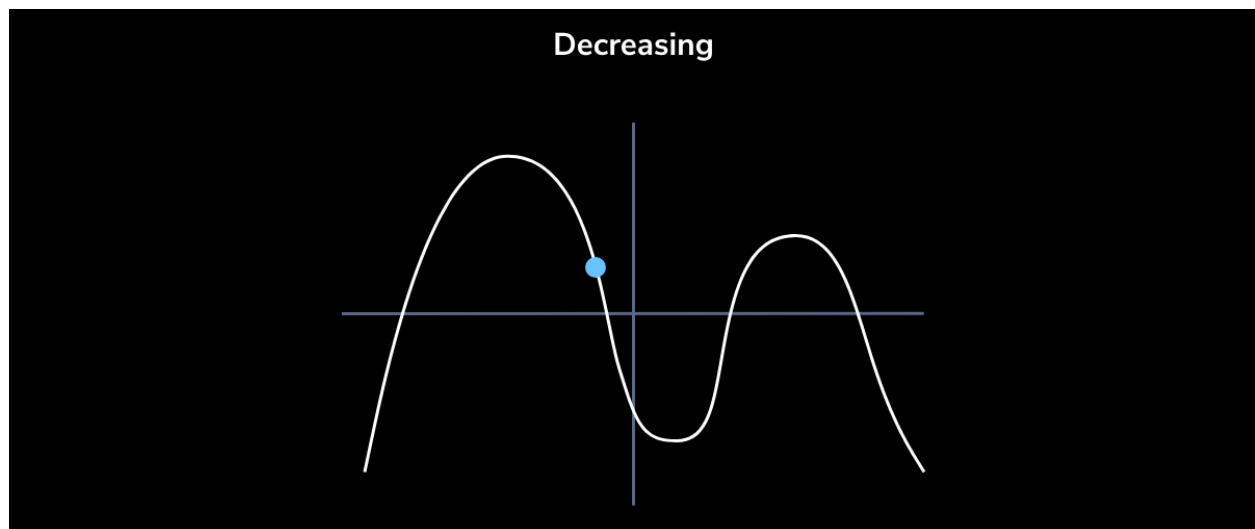
Let's look at the general information we can get as well. If $f'(x) > 0$, the corresponding function $f(x)$ is increasing. Keep in mind that $f(x)$ itself may be negative even though the derivative $f'(x)$ is positive. The derivative says nothing about the value of the original function, only about how it is changing. Similarly, if $f'(x) < 0$, then the original function is decreasing.

If $f'(x) = 0$, then the function is not changing. This can mean one of a few things.

It may mean that the function has reached a *local maximum* (or maxima). A local maximum is a value of x where $f'(x)$ changes from positive to negative and thus hits 0 along the way. In $f(x)$, the local maximum is higher than all the points around it.

It may also mean that the function has reached what is called a *local minimum*. A local minimum is lower than the points around it. When $f'(x)$ goes from negative values to 0 to positive values, a local minimum forms.

It may be an *inflection point*. This is a point where a function has a change in the direction of curvature. For example, the curve of the function goes from "facing down" to "facing up." Finding inflection points involves a second derivative test, which we will not get to in this lesson.



Calculating Derivatives

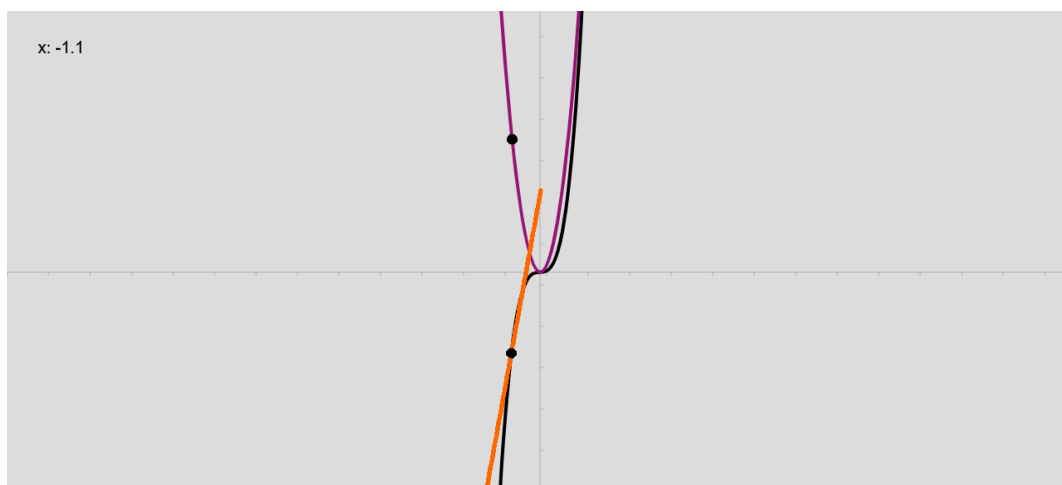
We have talked at length about derivatives abstractly. But how do we actually find derivative functions? We will outline this in this exercise and the next one.

The answer is through a series of "rules" that we'll introduce in this exercise and the next. These rules are building blocks, and by combining rules, we can find and plot the derivatives of many common functions.



More on Calculating Derivatives

So we now have an idea on where to start differentiating (that is, take the derivative of) things
lik



Calculating Derivatives in Python

We've discussed how to compute derivatives by hand. However, oftentimes when working on a computer, we have functions given to us in array form rather than an explicit equation. In this exercise, we will investigate how to calculate derivatives using Python.

For example, let's say we have an array that represents the following function:

$$f(x)=x^2+3$$

$$f'(x)=2x$$

$$2$$

$$+3$$

We can define this function in two sets of arrays using the following code.

```
from math import pow
# dx is the "step" between each x value
dx = 0.05
def f(x):
    # to calculate the y values of the function
    return pow(x, 2) + 3
# x values
f_array_x = [x for x in np.arange(0,4,dx)]
# y values
f_array_y = [f(x) for x in np.arange(0,4,dx)]
```

Copy to Clipboard

The `pow()` function within the `f(x)` function we defined allows us to calculate the values of x^2 , and we use a list comprehension to create a list of the y-values in a variable called `f_array_y`.

To compute the derivative of `f_array`, we use a NumPy function called `gradient()`.

```
f_array_deriv = np.gradient(f_array_y, dx)
```

Copy to Clipboard

`gradient()` takes in an array (in this case, a one-dimensional array) as its first argument. We also need to specify the distance between the x values, which is `dx`. For a one-dimensional array, `gradient()` cal

```
# Turning our function into an array
import numpy as np
from math import pow
from math import sin
import codecademylib3
import matplotlib.pyplot as plt

# our change in x value
dx = 0.01
def f(x):
    return sin(x)
sin_y = [f(x) for x in np.arange(0,20,dx)]
sin_x = [x for x in np.arange(0,20,dx)]

# Define your derivative here
sin_deriv = np.gradient(sin_y,dx)

plt.plot(sin_x, sin_y)
plt.plot(sin_x, sin_deriv)
plt.show()
```

SOLUTIONS:

```
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3

heart_img = np.array([[255,0,0,255,0,0,255],
                      [0,255/2,255/2,0,255/2,255/2,0],
                      [0,255/2,255/2,255/2,255/2,255/2,0],
                      [0,255/2,255/2,255/2,255/2,255/2,0],
                      [255,0,255/2,255/2,255/2,0,255],
                      [255,255,0,255/2,0,255,255],
                      [255,255,255,0,255,255,255]])

# This is a helper function that makes it easy for you to show images!
def show_image(image, name_identifier):
    plt.imshow(image, cmap="gray")
    plt.title(name_identifier)
    plt.show()

# Show heart image
show_image(heart_img, "Heart")

# Invert color
inverted_heart_img = 255 - heart_img
show_image(inverted_heart_img, "Inverted Heart")

# Rotate heart
rotated_heart_img = heart_img.T
show_image(rotated_heart_img, "Rotated Heart")

# Random Image
random_img = np.random.randint(0,255, (7,7))
show_image(random_img, "Random Image")

# Solve for heart image
x = np.linalg.solve(random_img, heart_img)
solved_heart_img = np.matmul(random_img, x)
show_image(x, "x")
show_image(solved_heart_img, "Solved Heart Image")
```

Introduction to Linear Regression

Welcome to Linear Regression

In this unit, we will cover the fundamentals of linear regression.

We'll introduce you to some of the math behind linear regression models and teach you how to fit and interpret your own model. Finally, we'll teach you to compare a few different models and choose one to address your research question(s).

Why is this important?

Linear regression allows us to model the relationship between a quantitative variable and one or more predictor variables. It is a foundational machine learning algorithm and is the basis for many other kinds of models, such as logistic and poisson regression. It is also used as a data analysis technique and crops up in many different fields.

For example, linear regression can be used to answer research questions such as:

Can we predict how much a customer will spend at a store based on attributes such as age, income, and location?

What is the relationship between a person's income and other attributes such as education level and years of experience?

Finally, learning about linear regression will prepare you to explore other kinds of machine learning models and will enable you to read and understand research papers in just about any field of study.

What will you learn?

At the end of this module, you will be able to:

- Fit and interpret a simple linear regression model
- Fit and interpret a multiple linear regression model
- Compare models using:
 - R-squared/adjusted R-squared
 - F test
 - Log Likelihood
 - AIC/BIC
 - Training and test sets

Multiple Linear Regression

Fitting a Multiple Regression in Python

To run a multiple linear regression in Python, we can use the function `OLS.from_formula()` from `statsmodels.api`. For example, if we want to run a regression to predict `score` using `hours_studied` and `breakfast` (contained in a dataset named `survey`), we can fit the model as follows:

```
import statsmodels.api as sm
model = sm.OLS.from_formula('score ~ hours_studied + breakfast', data=survey).fit()
```

[Copy to Clipboard](#)

To actually view the results, we can print a summary of them to the console using the following code.

```
print(model.summary())
```

Copy to Clipboard

Rather than printing the entire summary table, we can call the model coefficients directly using

`model.params`. We can even call a specific coefficient by order of appearance in the table. For instance:

```
print(model.params)
# Output:
# Intercept      32.665570
# hours_studied   8.540499
# breakfast      22.495615

print(model.params[0])
# Output:
# 32.66556979549575
```

Copy to Clipboard

From the coefficient table, we can see the intercept is approximately 32.7, the coefficient on `hours_studied` is 8.5, and the coefficient on `breakfast` is 22.5.

Instructions

Checkpoint 1 Enabled

1.

Using the `student` dataset, fit a multiple regression model for the response variable `port3` with quantitative predictor `math1` and binary predictor `address`. Save the results as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the intercept and coefficients from `model1` using `.params`. Are they listed in the order you thought they'd be?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Using `model1.params`, save the intercept as `b0`, the coefficient for `math1` as `b1`, and the coefficient for `address` as `b2`. If we added students' first semester Portuguese score (`port1`) as another predictor to the model, what index would it be in `model1.params`?

Multiple Linear Regression

Binary Categorical Variables in Multiple Regression

Binary categorical variables are variables with exactly two possible values. In a regression model, these two values are generally coded as 1 or 0. For example, a multiple regression equation from the `survey` dataset might look like this:

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * \text{breakfast}$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * \text{breakfast}$$

`breakfast` is a binary categorical predictor with two possible values: “ate breakfast,” which is coded as `1` in the model and “didn’t eat breakfast,” which is coded as `0`. If we substitute these values for `breakfast` in the regression equation, we end up with two equations: one for each group.

For breakfast eaters, we substitute 1 for `breakfast` and simplify:

$$\begin{aligned} \text{score} &= 32.7 + 8.5 * \text{hours_studied} + 22.5 * 1 \\ \text{score} &= 32.7 + 8.5 * \text{hours_studied} + 22.5 \\ \text{score} &= (32.7 + 22.5) + 8.5 * \text{hours_studied} \\ \text{score} &= 55.2 + 8.5 * \text{hours_studied} \end{aligned}$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * 1$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5$$

$$\text{score} = (32.7 + 22.5) + 8.5 * \text{hours_studied}$$

$$\text{score} = 55.2 + 8.5 * \text{hours_studied}$$

For the group that didn’t eat breakfast, we substitute 0 for `breakfast` and simplify:

$$\begin{aligned} \text{score} &= 32.7 + 8.5 * \text{hours_studied} + 22.5 * 0 \\ \text{score} &= 32.7 + 8.5 * \text{hours_studied} + 0 \\ \text{score} &= 32.7 + 8.5 * \text{hours_studied} \end{aligned}$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * 0$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 0$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied}$$

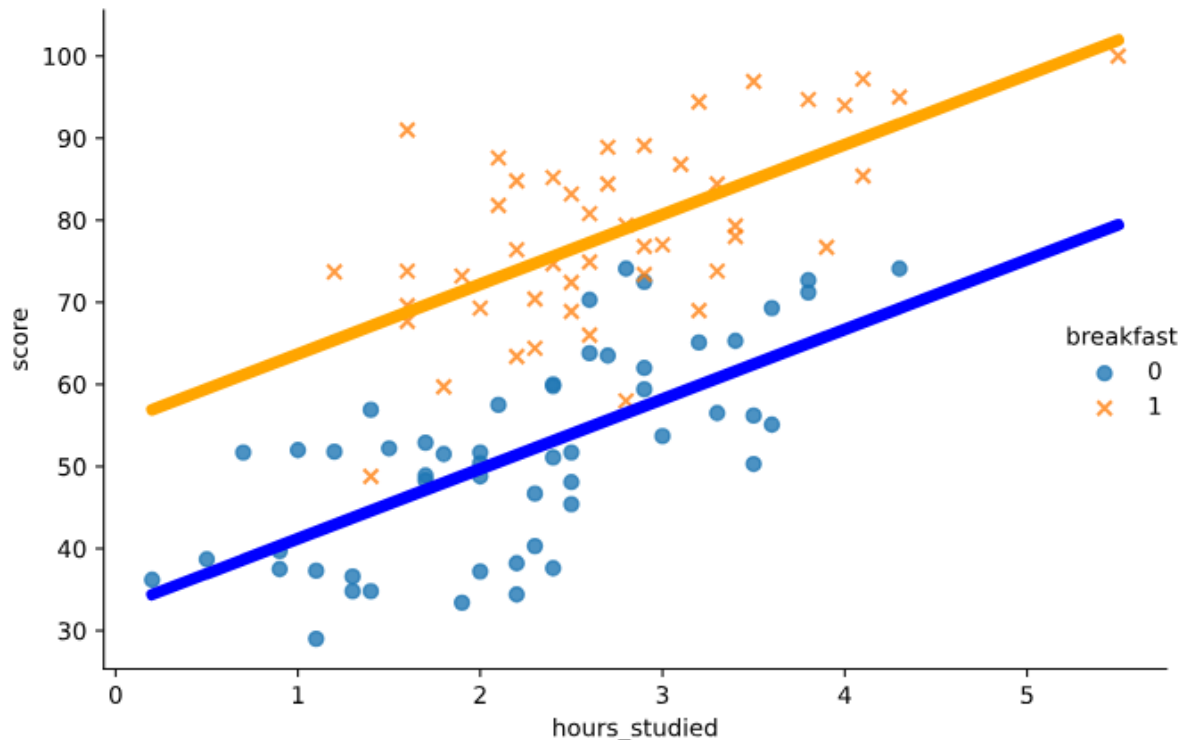
If we inspect these two equations, we see that the only difference is the larger intercept for the group that ate breakfast (55.2) compared to the group that didn’t eat breakfast (32.7). The coefficient on `hours_studied` is the same for both groups.

We can visualize this regression equation by adding both lines to the scatter plot of `score` and `hours_studied` with `plt.plot()` as follows:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.lmplot(x='hours_studied', y='score', hue='breakfast', markers=['o', 'x'],
           fit_reg=False, data=survey)
plt.plot(survey.hours_studied, 32.7+8.5*survey.hours_studied,
         color='blue', linewidth=5)
plt.plot(survey.hours_studied, 55.2+8.5*survey.hours_studied,
         color='orange', linewidth=5)
plt.show()
```

Copy to Clipboard



From the plot, we can see the regression lines have the same slope. The orange line for the breakfast-eaters starts higher, but increases at the same rate as the blue line for the group that didn't eat breakfast.

Instructions

Checkpoint 1 Enabled

1.

Code has been provided for you in `script.py` to fit a regression model predicting `port3` based on `math1` and `address`. The fitted model has been saved as `model1`. Use `.params` to print the intercept and coefficients from the results and inspect the coefficient for `address`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The variable `address` has two values: `R` for rural (coded as `address = 0` in the model) and `U` for urban (coded as `address = 1`). Because we've included this binary variable in our model, we've actually fit two separate regression lines: one for students who live at a rural address, and one for students who live at an urban address.

Using the output from the model, write out the regression equation for when `address` is equal to `R` and save the value of the intercept as `interceptR`. Then, write out the regression equation for when `address` is equal to `U` and save the value of the intercept as `interceptU`. Finally, since the slope on `math1` will be the same for both equations, save this value as `slope`. Round all final values to one decimal place (i.e., the tenth's place).

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

The code for the scatter plot of `port3` and `math1` has been provided for you in `script.py`. Using the regression equations you created in the last checkpoint, add a blue line to the scatter plot for rural addresses.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Using the regression equations with rounded values that you created in the second step, add an orange line to the scatter plot for urban addresses. What's similar about the two lines you just plotted? What's different?

```
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

student = pd.read_csv('student.csv')

modell = sm.OLS.from_formula('port3 ~ math1 + address', data=student).fit()

# Print model results here:
print(modell.params)
# Save intercepts and slope here:
interceptR = 3.2
interceptU = 3.8
slope = 0.5
# Add lines to complete scatter plot and display
sns.lmplot(x='math1', y='port3', hue='address', markers=['o', 'x'], fit_reg=False,
data=student)
# Line for rural addresses (R)
plt.plot(student.math1, 3.2+0.5*student.math1, color='blue',linewidth=5)
# Line for urban addresses (U)
plt.plot(student.math1, 3.8+0.5*student.math1, color='orange',linewidth=5)
plt.show()
```

Intercept

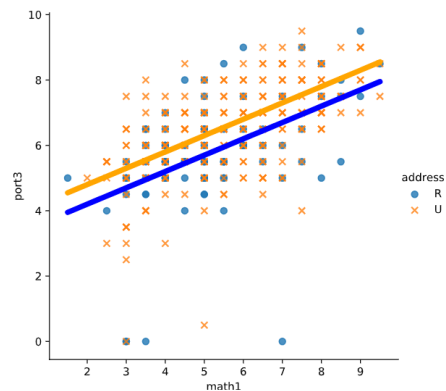
3.23407052724496

address[T.U]

0.5576311650908965

math1

0.47589205972410537



Multiple Linear Regression

Interpretation of Binary Categorical Variables

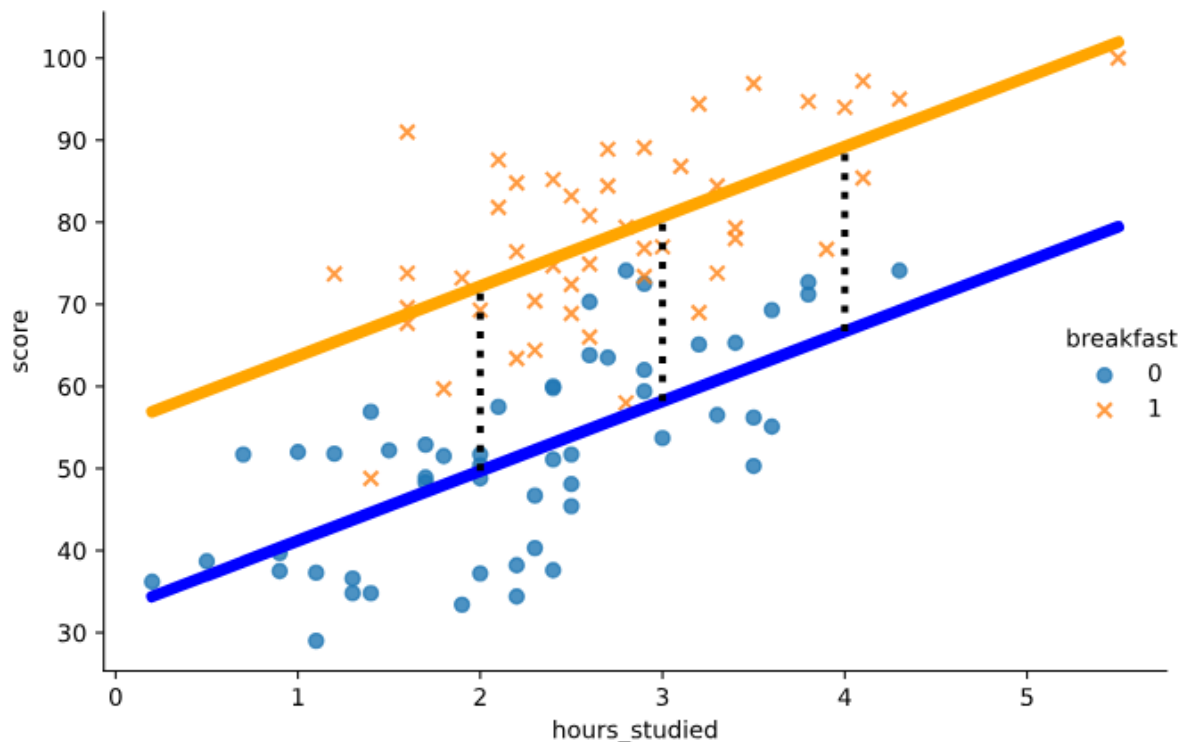
While we can view a binary categorical variable as a way of creating two new regression equations with different intercepts, we don't need to make these equations every time we want to interpret a binary predictor in a multiple regression equation.

In the `survey` dataset, `breakfast` is a binary variable that is equal to 1 for students who ate breakfast on test day and 0 for those who didn't. For predicting `score` based on `hours_studied` and `breakfast`, the multiple regression equation is:

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * \text{breakfast}$$

$$\text{score} = 32.7 + 8.5 * \text{hours_studied} + 22.5 * \text{breakfast}$$

Take a look at the scatter plot with regression lines on top:



We can interpret the regression coefficients as follows:

The `breakfast` variable has a coefficient of 22.5. The interpretation is: holding all other variables constant, students who ate breakfast scored 22.5 points higher than students who did not. "Holding all other variables constant" means that we're comparing breakfast groups among

students who studied the same number of hours. Visually, this means that the distance between the two regression lines is always 22.5 for any value of `hours_studied` (the dotted lines in the picture above are all the same length).

The intercept (32.7) is the average value of the response variable when all predictors in the equation are equal to 0. According to our full regression equation, this means that students who didn't study (`hours_studied = 0`) and didn't eat breakfast (`breakfast = 0`) earned an average score of 32.7 (the y-intercept for the blue line).

Instructions

Checkpoint 1 Enabled

1.

Suppose that we fit a model to predict `port3` (final Portuguese score) with predictors `math1` (first semester math score) and `address` (urban or rural residence). The coefficients are printed below.

```
# Output:
# Intercept      3.234071
# address[T.U]   0.557631
# math1          0.475892
```

Copy to Clipboard

In the file **interpretations.txt** write a one-sentence interpretation for the intercept. Does this interpretation make practical sense?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add a one-sentence interpretation to **interpretations.txt** for the coefficient on `address` in terms of the average Portuguese scores (`port3`) of students from rural areas (`r` or `address = 0`) and students from urban areas (`u` or `address = 1`). Check your solution against the sample solutions in **solutions.txt**.

Interpretation of the intercept

For students from rural areas whose first math scores were equal to zero, the average final Portuguese score was 3.23.

Interpretation of the coefficient on address

On average, students from urban areas scored 0.56 points higher on their final Portuguese scores than did students from rural areas, holding all other variables constant (in this case: holding `math1` scores constant).

Multiple Linear Regression

Quantitative Variables in Multiple Regression

In the previous exercises, we looked at regression models with one quantitative predictor and one binary predictor, but we can also have models with multiple quantitative predictors. For example, consider the following model using the `survey` dataset (`assignments` is the number of homework assignments the student has completed):

```
import statsmodels.api as sm
model = sm.OLS.from_formula('score ~ hours_studied + assignments', data=survey).fit()
print(model.params)
```

Output:

```
# Intercept      16.676498
# hours_studied   6.273886
# assignments     4.687796
```

Copy to Clipboard

From the coefficients above, our regression equation is:

$$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * \text{assignments}$$

$$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * \text{assignments}$$

We can still think of multiple regression as creating a new regression line for each value of a quantitative predictor. However, it is challenging to visualize this because we now have different regression lines for every possible value of `assignments`. To visualize the regression output, it is helpful to choose a few sample values: for example, 1, 5, and 10 assignments.

We can add these lines to our scatter plot of `score` vs. `hours_studied` as before:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create scatter plot of hours_studied and score
sns.lmplot(x='hours_studied', y='score', hue='assignments', palette='Blues',
fit_reg=False, data=survey)
```

Copy to Clipboard

This time we will directly put the model coefficients into each regression equation by calling them individually from `model.params`. The code for 1, 5, and 10 assignments is given below.

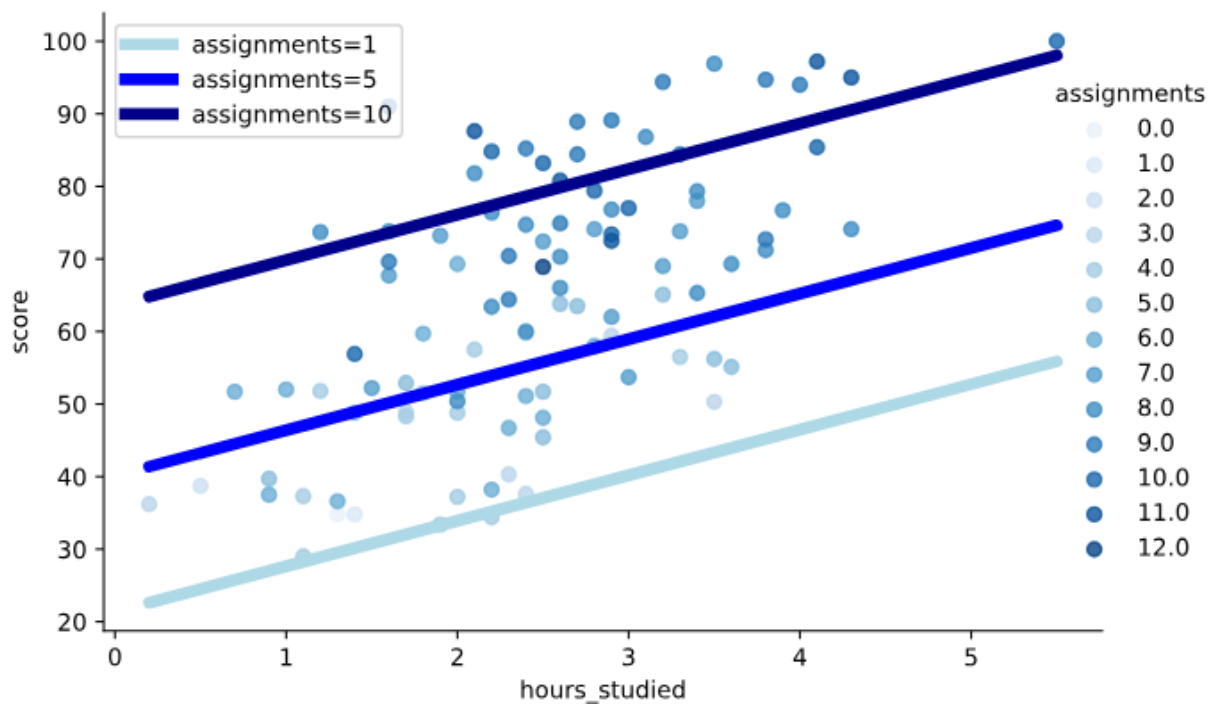
```
# Add regression line for 1 assignment
plt.plot(survey.hours_studied, model.params[0] + model.params[1]*survey.hours_studied
+ model.params[2]*1, color='lightblue',linewidth=5)

# Add regression line for 5 assignments
plt.plot(survey.hours_studied, model.params[0] + model.params[1]*survey.hours_studied
+ model.params[2]*5, color='blue',linewidth=5)

# Add regression line for 10 assignments
plt.plot(survey.hours_studied, model.params[0] + model.params[1]*survey.hours_studied
+ model.params[2]*10, color='darkblue',linewidth=5)

# Show plot with legend
plt.legend(['assignments=1','assignments=5', 'assignments=10'])
plt.show()
```

Copy to Clipboard



We can see in the plot that the slopes of all three lines are the same, but the intercepts differ. As the number of completed assignments increases, the intercept of the corresponding regression line also increases.

Instructions

Checkpoint 1 Enabled

1.

In **script.py** we've fit a regression predicting `port3` based on `math1` and `port1` and saved the fitted model as `model2`. Print `model2.params` and inspect the coefficients. What is the relationship between final Portuguese score (`port3`) and first Portuguese score (`port1`)?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We've already provided you with code to plot two regression lines for students with first semester Portuguese scores (`port1`) of 4 and 6. Using the results from the regression model, write a line of code to add a third regression line to the plot for `port1 = 8`. Make the color of the line `darkblue`. How do the lines on the plot match with the results from the model?

```
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

student = pd.read_csv('student.csv')

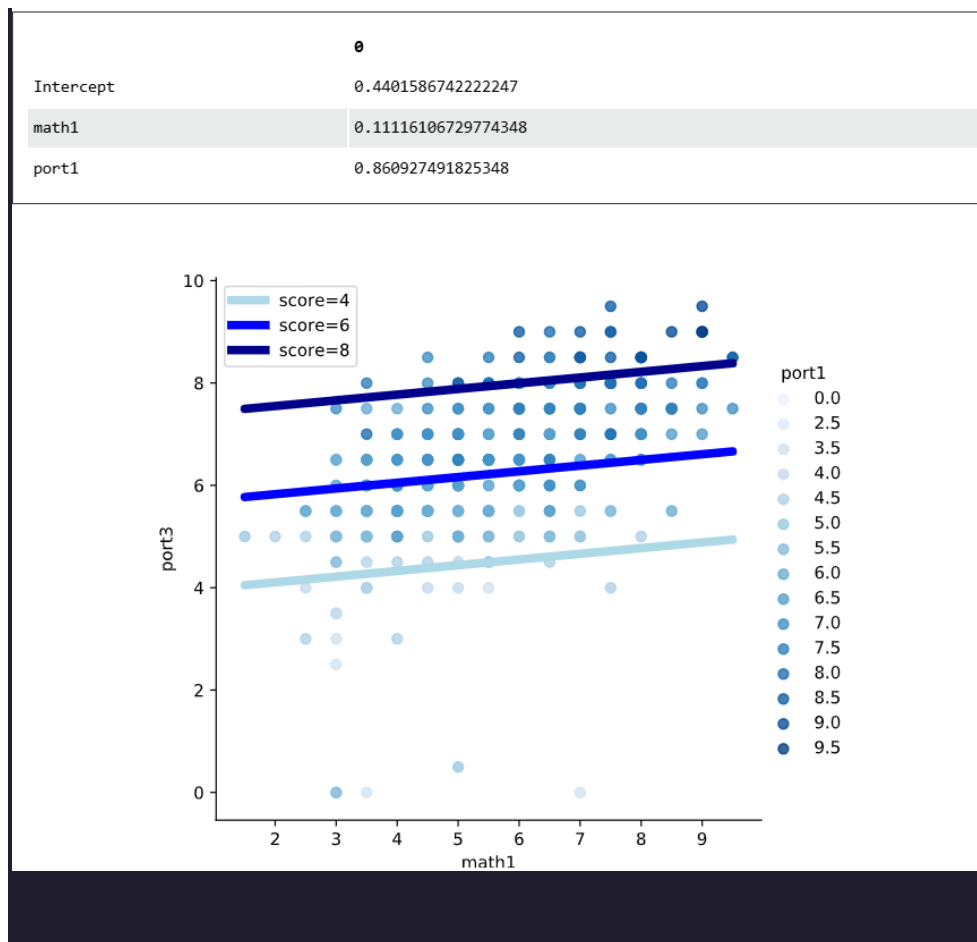
model2 = sm.OLS.from_formula('port3 ~ math1 + port1', data=student).fit()

# Print resulting coefficients here:
print(model2.params)

# Scatter plot code:
sns.lmplot(x='math1', y='port3', hue='port1', palette='Blues', fit_reg=False, data=student)
```

```
plt.plot(student.math1, model2.params[0]+model2.params[1]*student.math1+model2.params[2]*4,
color='lightblue', linewidth=5)
plt.plot(student.math1, model2.params[0]+model2.params[1]*student.math1+model2.params[2]*6,
color='blue', linewidth=5)
# Add third line to scatter plot here:
plt.plot(student.math1, model2.params[0]+model2.params[1]*student.math1+model2.params[2]*8,
color='darkblue', linewidth=5)

plt.legend(['score=4', 'score=6', 'score=8'])
plt.show()
```



Portuguese sc

Multiple Linear Regression

Interpretation of Quantitative Variables

In a multiple regression model, the coefficient on a quantitative predictor is the expected difference in the outcome variable for a one-unit increase of the predictor, holding all other predictors constant.

For the `survey` dataset, the multiple regression equation is:

$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * \text{assignments}$ The predictor `assignments` is a quantitative variable. Let's substitute a few different values for `assignments` into the regression equation to see how it changes:

For students who completed 0 assignments:

$$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * 0 \quad \text{score} = 16.7 + 6.3 * \text{hours_studied}$$

For students who completed 1 assignment:

$$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * 1 \quad \text{score} = 21.4 + 6.3 * \text{hours_studied}$$

For students who completed 2 assignments:

$$\text{score} = 16.7 + 6.3 * \text{hours_studied} + 4.7 * 2 \quad \text{score} = 26.1 + 6.3 * \text{hours_studied}$$

The only difference between the equations is that we add 4.7 points to the intercept for each additional completed assignment. Thus, among students who studied the same number of hours (i.e., holding all other variables constant), students who completed one more assignment earned a 4.7 point higher test score on average.

Instructions

Checkpoint 1 Enabled

1.

Suppose that we ran a model predicting Portuguese score (`port3`) based on first math score (`math1`) and first Portuguese score (`port1`). The code and coefficients are shown below:

```
import statsmodels.api as sm
model2 = sm.OLS.from_formula('port3 ~ math1 + port1',
data=student).fit()
print(model2.params)
# Output:
# Intercept      0.440159
# math1          0.111161
# port1          0.860927
```

Copy to Clipboard

In the file `interpretations.txt` write a one-sentence interpretation for the intercept. What does this value represent in the context of the dataset?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

In `interpretations.txt`, add a one-sentence interpretation for the coefficient on `math1` and another for the coefficient on `port1`. Check the sample solutions in the file `solutions.txt` and compare them to your own.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

In `interpretations.txt`, add a one-sentence interpretation for the coefficient on `port1` and another for the coefficient on `port1`. Check the sample solutions in the file `solutions.txt` and compare them to your own. Why is it important to hold the other predictors constant in our interpretation of a coefficient?

One-sentence interpretation of the intercept:

When students have a value of zero for both first math and first Portuguese score, their average final Portuguese score is 0.44.

One-sentence interpretation of the coefficient on `math1`:

Holding first Portuguese score constant, for every additional point on first math score, final Portuguese score increases by 0.11 points.

One-sentence interpretation of the coefficient on `port1`:

Holding first math score constant, for every additional point on first Portuguese score, final Portuguese score increases by 0.86 points.

Multiple Linear Regression

Changes in Predictor Coefficients

Sometimes we use regression to understand the relationship between two variables because we wish to control for potential confounders. For example, based on the `survey` dataset, we may be primarily interested in how studying (`hours_studied`) is related to test score (`score`); however, in order to understand this relationship, we may want to control for additional student attributes, such as whether the student ate breakfast (`breakfast`).

If we perform a simple linear regression predicting `score` from `hours_studied`, we get the following results:

```
import statsmodels.api as sm
model0 = sm.OLS.from_formula('score ~ hours_studied', data=survey).fit()
print(model0.params)

# Output:
# Intercept      34.990700
# hours_studied  11.881045
```

Copy to Clipboard

However, if we add `breakfast` to the model and inspect the new coefficients, we'll find that the intercept and slope on `hours_studied` have changed:

```
import statsmodels.api as sm
model1 = sm.OLS.from_formula('score ~ hours_studied + breakfast', data=survey).fit()
print(model1.params)

# Output:
# Intercept      32.665570
# hours_studied   8.540499
# breakfast      22.495615
```

Copy to Clipboard

Note that the coefficient on `hours_studied` changes from 11.9 to 8.5. Why does this happen? Perhaps people who eat breakfast are more likely to study longer and also more likely to score better on their exam. Without taking `breakfast` into account, some of the relationship between `score` and `breakfast` is attributed to `hours_studied` instead.

Instructions

Checkpoint 1 Enabled

1.

The `student` dataset has been loaded for you in `script.py`. Run a regression model predicting final Portuguese score (`port3`) from first math score (`math1`). Save the fitted model as `simple`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit a multiple regression model predicting final Portuguese scores (`port3`) from first math score (`math1`) AND first Portuguese score (`port1`). Save the fitted model as `multiple`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print the resulting intercept and coefficients from the `simple` model using `.params`. What is the apparent relationship between final Portuguese score and first math score?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print the resulting coefficients from the `multiple` model using `.params`. How did the coefficient on `math1` change when `port1` was added to the model?

```
import pandas as pd
import statsmodels.api as sm

student = pd.read_csv('student.csv')

# Run regression with only math1 here:
simple = sm.OLS.from_formula('port3 ~ math1', student).fit()
# Run regression with math1 and port1 here:
multiple = sm.OLS.from_formula('port3 ~ math1 + port1', student).fit()
# Print the results of simple here:
print(simple.params)
# Print the results of multiple here:
print(multiple.params)
```

```
Intercept    3.627031
math1        0.484442
dtype: float64
Intercept    0.440159
math1        0.111161
port1        0.860927
```

Simpson's Paradox

In the last exercise, we explored how a coefficient may change when additional predictors are added to a model. When adding predictors entirely reverses the sign of the coefficient, this is called Simpson's Paradox: one model says the direction of the relationship is positive, while the other says it is negative.

Let's look at a simple example of this with the fictional dataset `tv`. We'll start by fitting a simple regression predicting weekly hours of television watched (`tv_hours`) from the number of books in the home (`books`).

```
import statsmodels.api as sm

model0 = sm.OLS.from_formula('tv_hours ~ books', data=tv).fit()

print(model0.params)

# Output:

# Intercept    12.219967
# books        0.089078
```

Copy to Clipboard

The coefficient on `books` is positive: the model indicates that more books is associated with more weekly hours of television. Now, let's add a categorical variable to the model that specifies type of home (`hometype`): rented room, one-bedroom apartment, two-bedroom apartment, or house.

```
import statsmodels.api as sm

model1 = sm.OLS.from_formula('tv_hours ~ books + hometype', data=tv).fit()

print(model1.params)

# Output
```

```
# Intercept          22.952692
# hometype[T.2BR]     11.754848
# hometype[T.house]   5.878007
# hometype[T.room]    -5.880337
# books               -0.093223
```

Copy to Clipboard

This model leads to the opposite conclusion: the negative coefficient on `books` indicates that more books is associated with FEWER hours of television watched. How is this possible?

Let's look at a scatter plot of `tv_hours` (y-axis) against `books` (x-axis) colored by `hometype`. This time we'll allow `lmpplot` to fit regression lines for each type of home by NOT setting `fit_reg` to `False`. We'll also add the regression line for `model0`, which only has `books` as a predictor.

```
import seaborn as sns

import matplotlib.pyplot as plt

# Scatter plot

sns.lmplot(x='books', y='tv_hours', hue='hometype', palette='colorblind',
markers=['o', 'x', 'v', 's'], ci=False, legend=False, data=tv)

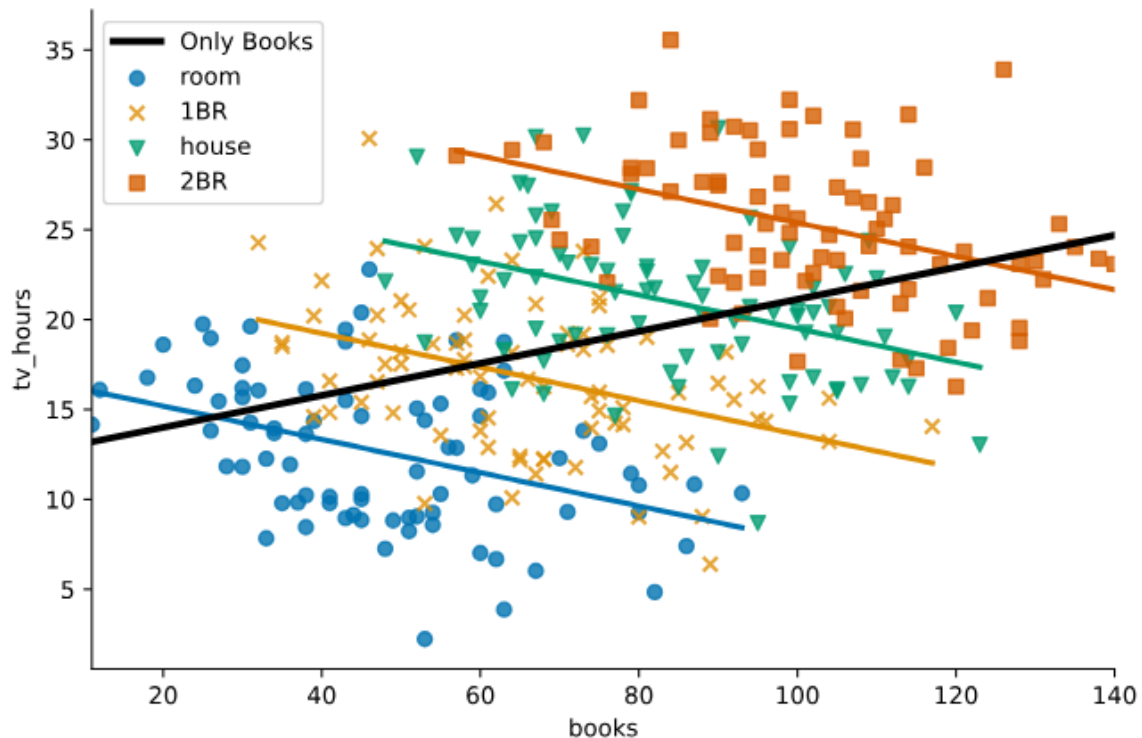
# Black line

plt.plot(tv.books, model0.params[0] + model0.params[1]*tv.books, color='black',
linewidth=3, label='Only Books')

plt.legend()

plt.show()
```

Copy to Clipboard



Broadly, the points slope upward (black line), but the points slope downward within each `hometype` group (colored lines).

This could be because bigger homes mean more room for a larger book collection. Or maybe certain home types are associated with higher incomes, so people have more time and money for television. We can examine the pattern of association from the model, and possibly make good predictions from new data with it, but we can't find out the cause from this type of analysis.

Instructions

Checkpoint 1 Failed, try again

1.

The dataset `sat` has been loaded for you in `script.py`. This dataset examines U.S. school expenditures and SAT scores by states from the 1994-1995 school year and includes the following variables:

`total`, average total of math and verbal scores on the SAT

`expend`, average expenditure per pupil in thousands of dollars

`takingR`, percentage of eligible students taking the SAT (we rounded this to the nearest 5 to simplify plotting)

In `script.py` we've provided code to fit a linear regression model of average total SAT score predicted from average expenditure per pupil. The fitted model is saved as `model0`. Print the regression coefficients from `model0` and inspect them.

What is the relationship between SAT score and expenditure per pupil? Is this what you expected?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit the same model but with `takingR` as an additional predictor and save the fitted model as `model1`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print the coefficients of `model1` using `.params` and inspect the coefficient on `expend`. How does it differ from the coefficient in `model0`? What does this model say about the relationship between average expenditure and SAT scores?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We have provided code in **script.py** to generate a scatter plot with `expend` on the x-axis and `total` on the y-axis. The points are colored by `takingR`, and we've plotted regression lines for each value of `takingR`. Add a line to the plot that shows the regression line for `model0` in black. Be sure to call the model coefficients directly from the output of step 1.

Not taking into account the percentage of students taking the test, how does the broad pattern of points reflect the results of `model0`? How do the regression lines within each group reflect the results of `model1`?

```
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

sat = pd.read_csv('sat.csv')
model0 = sm.OLS.from_formula('total ~ expend', data=sat).fit()

# Print simple regression coefficients:
print(model0.params)

# Run regression with takingR added:
model1 = sm.OLS.from_formula('total ~ expend + takingR', data=sat).fit()

# Print multiple regression coefficients:
print(model1.params)

# Code for scatter plot:
sns.lmplot(x='expend', y='total', hue='takingR', palette='colorblind', ci=None,
data=sat)
# Add regression line for model0 here:
plt.plot(sat.expend, model0.params[0] + model0.params[1]*sat.expend, color='black',
linewidth=3, label='Only Expenditures')
plt.show()
```

Intercept	1089.293718
expend	-20.892174
dtype: float64	
Intercept	996.329849
expend	11.737303
takingR	-2.809061

Assumptions of Multiple Regression

When doing any type of statistical analysis, we should always keep the assumptions in mind. Multiple linear regression requires some of the same assumptions as simple linear regression:

- Linear functional form, which can be assessed by plotting the outcome variable against the predictor variable and looking for a linear relationship

Normality, which can be assessed by plotting a histogram of the residuals and looking for an approximately normal distribution

Homoscedasticity, which can be assessed by plotting residuals against fitted values and confirming that there is no clear pattern

In addition, we also have to check that the predictors are not linearly related. This is referred to as *multicollinearity* and can lead to misleading results.

We can detect multicollinearity by checking the correlations between pairs of variables in our data. Correlations close to 1 or -1 may be considered too closely related to both be included in a model. The following code calculates the correlation pairs from dataset `df` and saves them as `corr_grid`.

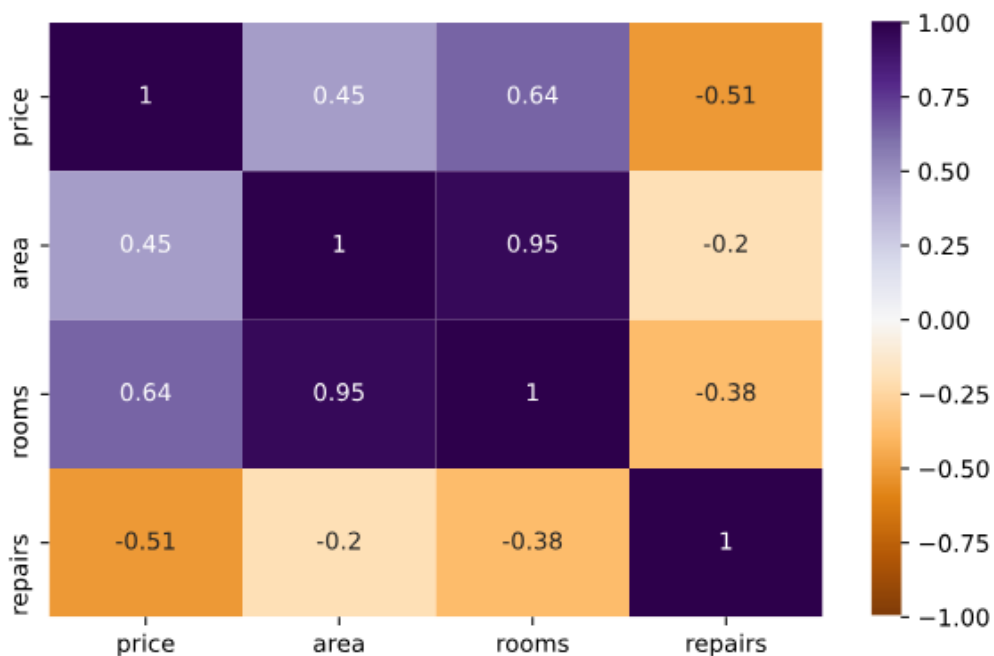
```
corr_grid = df.corr()
```

Copy to Clipboard

For easy visual detection, we can use Python's `heatmap()` function from `seaborn` to create a heat map of correlations between quantitative variables in a dataset. The code to produce a heat map from `corr_grid` is shown below.

```
sns.heatmap(corr_grid, xticklabels=corr_grid.columns, yticklabels=corr_grid.columns,
vmin=-1, center=0, vmax=1, cmap='PuOr', annot=True)
plt.show()
```

Copy to Clipboard



The heat map above is particularly dark purple (near 1) for the `area` and `rooms` variables, indicating a strong linear relationship (corr = 0.95). If we were running a multiple regression to predict `price`, we might decide to keep only one of those two variables in order to avoid multicollinearity.

Instructions

Checkpoint 1 Enabled

1.

The `student` dataset has been loaded for you in `script.py`. Get the correlations for pairs of quantitative variables in the `student` dataset and save them as `corrs`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add code to create a heat map of the correlations of quantitative variable pairs and inspect it. Why might the pairs near 1 or -1 be so similar to each other?

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

student = pd.read_csv('student.csv')

# Save correlations here:
corrs = student.corr()

# Plot heatmap here:
sns.heatmap(corrs, xticklabels=corrs.columns, yticklabels=corrs.columns, vmin=-1,
center=0, vmax=1, cmap='PuOr', annot=True)
plt.show()
```

Review

Congratulations! In this lesson you've learned to:

- Fit a multiple linear regression model in Python
- Write and interpret a multiple regression model
- Understand what binary and quantitative predictor coefficients mean visually and in context
- Check the assumption that multicollinearity isn't present

Instructions

1. A new dataset has been loaded for you called `family`. The data is modified from the Family Income and Expenditure Survey (FIES) of the Philippine Statistics Authority (PSA), a survey taken every three years on family income and expenditure in the Philippines. We'll work with the following variables from this dataset:
 - income (`income`)
 - total food expenditure (`food`)
 - total housing and water expenditure (`housing`)
 - source of income (`source`).
2. The income and expenditure variables are measured in thousands of Philippine pesos. Try practicing multiple regression in **script.py** using the following instructions. Sample solutions are provided in **solutions.py**.
 - Create a heat map of the quantitative variables in the `family` dataset. Do any pairs have high correlations?
 - Fit a model for `income` using `food`, `housing`, and `source` as predictors and inspect a summary of the results. The binary variable `source` has values `Entrepreneurial Activities` and `Wage/Salaries`. According to the summary, which value of `source` is coded as `1` and which is coded as `0`?

Write out the regression equation from the coefficients. Did you remember that you can print just the coefficients using `.params`?

Interpret the intercept of the equation. Is this interpretation practical?

Interpret the coefficient on the variable `source` in terms of expected income. How is the intercept different between groups?

Interpret the coefficient on `food`. Is there an increase or decrease in income associated with an increase in food expenditure?

Interpret the coefficient on `housing`. Is there an increase or decrease in income associated with an increase in housing expenditure?

Create a scatter plot of `housing` on the x-axis and `income` on the y-axis, colored by `source`.

Looking only at the `Wage/Salaries` group, use the regression equation for step 3 to add three lines to the plot for when food expenditure is 10,000, 100,000, and 200,000 pesos, giving each line a different color. Remember that `food` is measured in thousands of pesos, so 10,000 pesos is `food = 10`. Why did we have to look at only one value of `source` to produce these lines?

Matrix Representation of Linear Regression

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-regression-for-inference-data-science/modules/dsinf-multiple-linear-regression/articles/matrix-representation-of-linear-regression>

Interactions and Polynomial Terms

Why We Need Interaction and Polynomial Terms

In the real world, most relationships between variables are difficult to describe with one simple straight line. For example, consider the code and scatter plot below showing happiness level (y-axis) versus stress level (x-axis) colored by exercise.

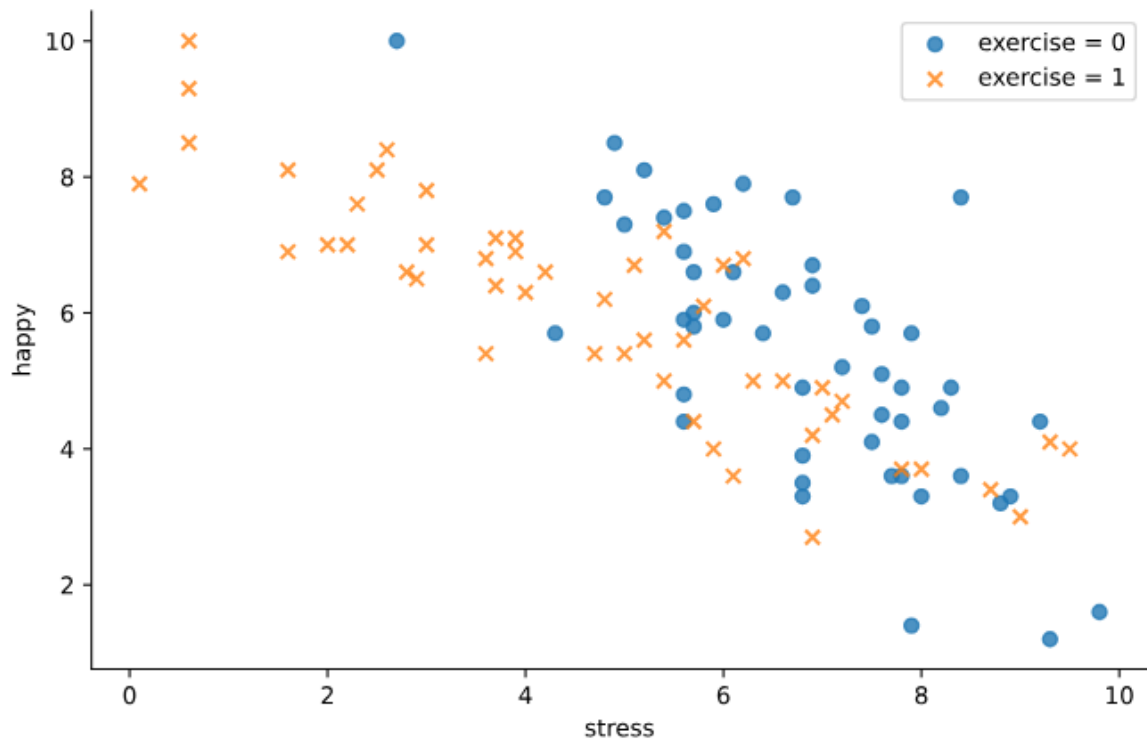
```
import seaborn as sns

import matplotlib.pyplot as plt

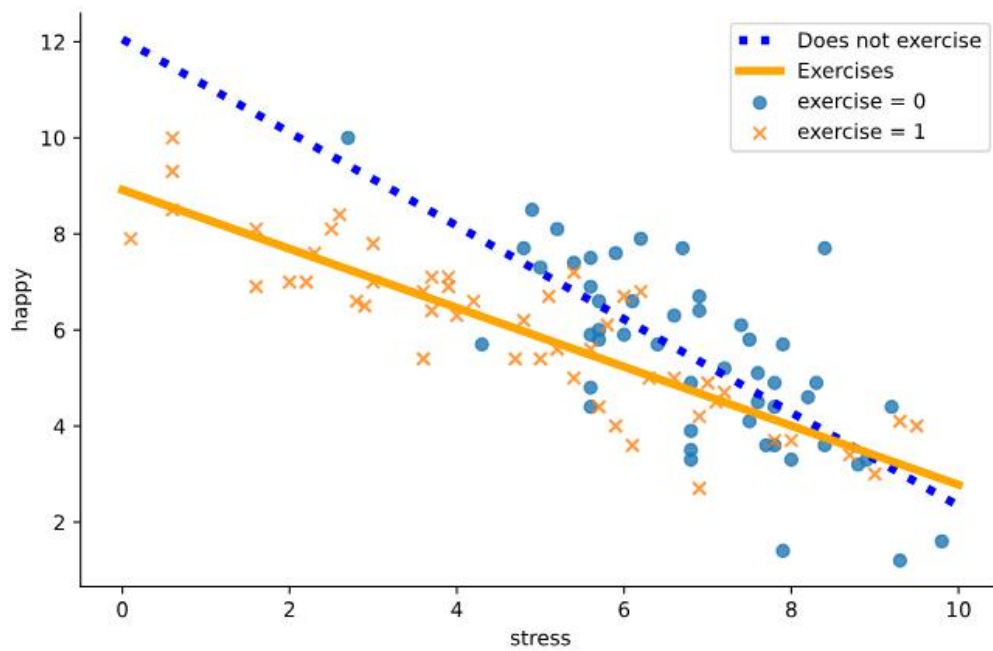
sns.lmplot(x='stress', y='happy', hue='exercise', markers=['o', 'x'],
           fit_reg=False, data=happiness)

plt.show()
```

Copy to Clipboard

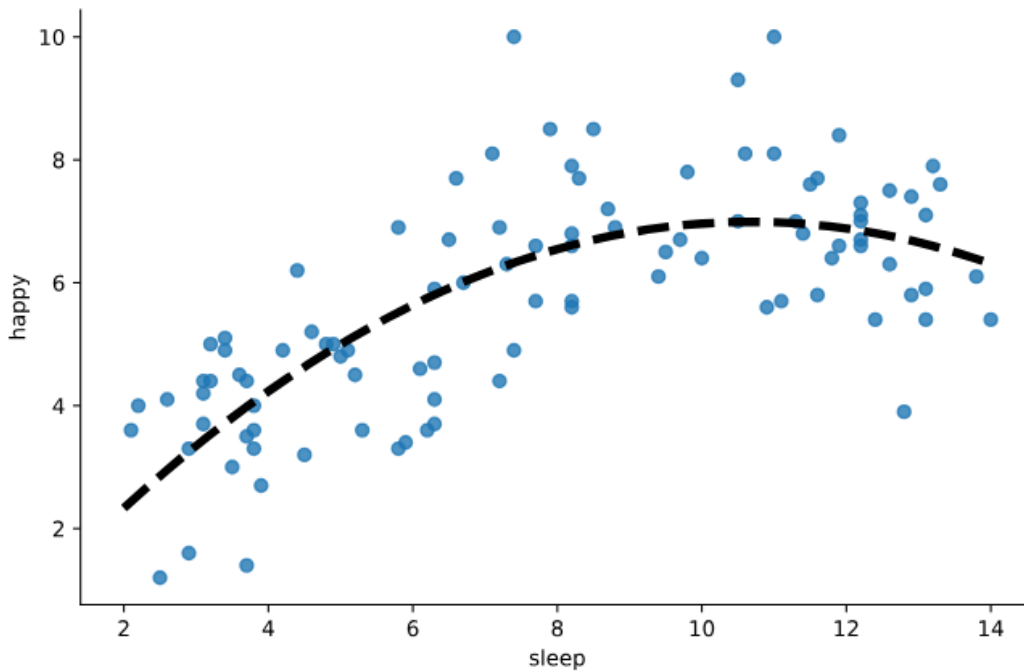


Imagine drawing two lines through the points: one for the orange crosses of the exercise group and one for the blue circles of the non-exercise group. Your lines might look something like this:



Note that the lines have both different intercepts AND different slopes. This means that exercise may modify the relationship between happiness and stress.

Other times, the relationship between two variables appears more CURVILINEAR, or curved in shape, than straight.



When we are using multiple regression to investigate the relationship between more than two variables, we may use interaction and polynomial terms to capture more complex relationships among the variables. To do this in Python, we modify our regression model formula to include extra terms. As a result, we also have to adjust our interpretations to match the new complexity of the model.

Instructions

Checkpoint 1 Enabled

1.

A fictional dataset called `plants` has been loaded for you in `script.py`. Add code to create a scatter plot of plant height in centimeters (`height`) on the y-axis and plant weight in kilograms (`weight`) on the x-axis. Color points by plant species (`species`). Note that you should set `fit_reg` equal to `False` within `lmplot` so that the function will not automatically compute and plot regression lines.

Does the relationship between height and weight appear the same for species A as for species B?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Next, create a scatter plot of the number of dead leaves on the plant (`dead`) on the y-axis and the amount of light the plant received (`light`) on the x-axis. Is the pattern in the plot straight or curved?

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

plants = pd.read_csv('plants.csv')

# Scatter plot of height and weight here:
sns.lmplot(x='weight', y='height', hue='species', markers=['o', 'x'],
fit_reg=False, data=plants)
plt.show()

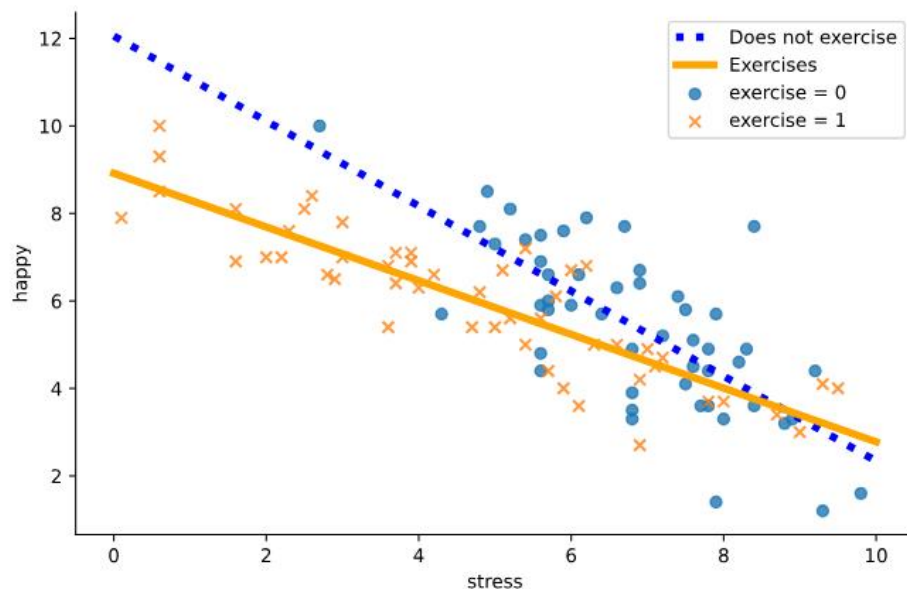
# Scatter plot of dead and light here:
```

```
sns.lmplot(x='light', y='dead', fit_reg=False, data=plants)
plt.show()
```

Interactions and Polynomial Terms

Visualizing Interactions: Binary and Quantitative

Let's return to a plot we saw in Exercise #1.



The data for this plot is from a fictional study on happiness that measures the following variables about its participants:

`happy` – their happiness level on a quantitative scale of 1 to 10
`stress` – their stress level on a quantitative scale of 1-10
`exercise` – whether they exercise regularly, where `1` = yes and `0` = no

We have drawn in a line estimating the relationship between stress and happiness for each exercise group. The line for the group that exercises appears flatter than that for the non-exercise group.

This indicates that `exercise` might modify the relationship between stress and happiness. Perhaps regular exercise buffers the effects of stress on happiness. Or perhaps people who exercise are also likely to do stress-reducing activities like meditation. While we don't know the exact reason, we do see a potential difference when we examine the exercise groups separately.

If we fit a regression modeling `happy` from the quantitative predictor `stress` and the binary predictor `exercise`, we get the following results:

```
import statsmodels.api as sm

model = sm.OLS.from_formula('happy ~ stress + exercise',
data=happiness).fit()
```



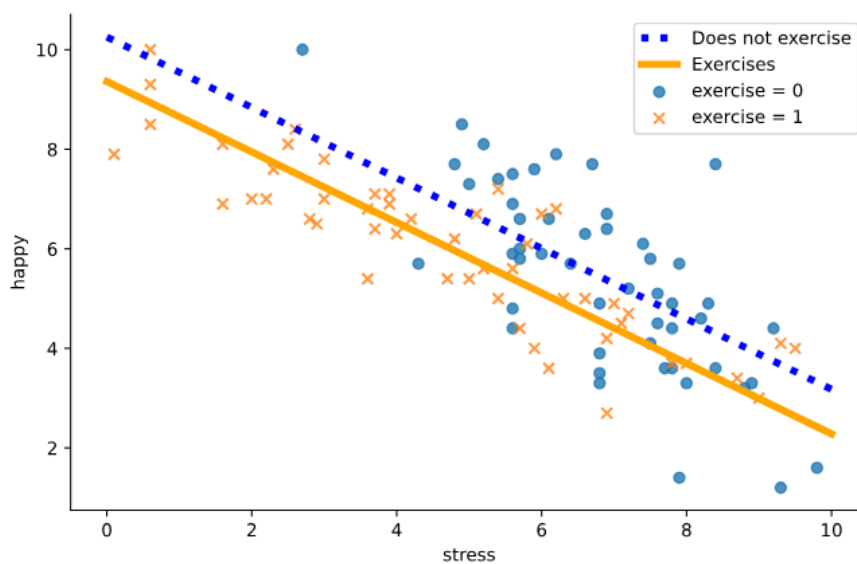
```
print(model.params)

# Output:

# Intercept      10.256296
# stress         -0.707925
# exercise       -0.894058
```

Copy to Clipboard

Using these coefficients, we can plot two lines with differing intercepts for each exercise group.



Our lines have different intercepts, but seem to be missing the steeper slope of the points for the non-exercise group. Since a model for `happy` with just `stress` and `exercise` as predictors only allows for the intercepts to differ, we must add an interaction term to our model to capture the difference in slopes.

Instructions

Checkpoint 1 Enabled

1.

The `plants` dataset has been loaded for you in `script.py`. Fit a multiple regression predicting `height` with `weight` and `species` as predictors and save the results as `model`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the intercept and coefficients of `model`. What do the coefficients tell us about the relationships between the variables?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Remove the `#` symbols to uncomment the code and run the plot of `height` and `weight` colored by `species`. Does the coefficient on `weight` seem to describe the slope for both species of plant?

```
import pandas as pd
```

```
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

plants = pd.read_csv('plants.csv')

# Fit regression model here:
model = sm.OLS.from_formula('height ~ weight + species', data=plants).fit()
# Print coefficients here:
print(model.params)
# Uncomment the scatter plot below:
sns.lmplot(x='weight', y='height', hue='species', markers=['o', 'x'],
fit_reg=False, data=plants)
plt.show()
```

Interactions and Polynomial Terms

Interactions in Python: Binary and Quantitative

In the last exercise, we ran a regression predicting happiness scores from stress scores and exercise participation without an interaction term. We got the following model coefficients:

```
# Output:
# Intercept      10.256296
# stress         -0.707925
# exercise       -0.894058
```

Copy to Clipboard

Using these coefficients, our regression equation is:

$$\text{happy} = 10.3 - 0.7 * \text{stress} - 0.9 * \text{exercise}$$

In the Python library `statsmodels.api`, we can easily add an interaction term to the model formula by adding a third predictor that combines `stress` and `exercise` with a colon (`stress:exercise`). The code to run the updated model and print the coefficients is shown below.

```
import statsmodels.api as sm
model = sm.OLS.from_formula('happy ~ stress + exercise + stress:exercise',
data=happiness).fit()
print(model.params)

# Output:
# Intercept      12.053583
# stress         -0.971225
# exercise       -3.135705
# stress:exercise  0.357365
```

Copy to Clipboard

In addition to the expected coefficients, when we add the interaction term, the coefficient table shows a new term with a coefficient: `stress:exercise`. The coefficient on `stress:exercise` is really a coefficient on a whole new variable formed by multiplying `stress` by `exercise`. Thus, our regression equation for this model looks like this:

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 * \text{exercise} + 0.4 * \text{stress} * \text{exercise}$$

Note that our other coefficients changed slightly with the additional predictor. This is because we have explicitly pulled out more of the relationship between stress and exercise, causing the other coefficients to adjust to take this into account.

Instructions

Checkpoint 1 Passed

1.

The `plants` dataset has been loaded for you in `script.py` and the regression predicting `height` from the predictors `weight` and `species` has been saved as `model1`. Run a regression predicting `height` with predictors `weight`, `species`, and an interaction term for these two predictors. Save the results of this model as `model2`.

Checkpoint 2 Passed

2.

Print the coefficients from `model1`. What do we learn from the coefficients about each species' regression line?

Checkpoint 3 Passed

3.

Print the coefficients from `model2`. How did the coefficients and regression equation change when the interaction term was added?

```
import pandas as pd
import statsmodels.api as sm

plants = pd.read_csv('plants.csv')
model1 = sm.OLS.from_formula('height ~ weight + species', data=plants).fit()

# Fit model2 regression here:
model2 = sm.OLS.from_formula('height ~ weight + species + weight:species',
data=plants).fit()
# Print model1 results here:
print(model1.params)
# Print model2 results here:
print(model2.params)
```

Intercept	6.953395
species[T.B]	0.714273
weight	1.992474
dtype: float64	
Intercept	8.168619
species[T.B]	-3.580515
weight	1.658621
weight:species[T.B]	1.115071

Interactions and Polynomial Terms

Interpreting Interactions: Binary and Quantitative

By adding an interaction term for our binary predictor, we have made our model more complex and have therefore also added complexity to its interpretation.

Returning to our multiple regression equation with an interaction term from the last exercise, we have:

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 * \text{exercise} + 0.4 * \text{stress} * \text{exercise}$$

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 * \text{exercise} + 0.4 * \text{stress} * \text{exercise}$$

We can rewrite this equation for the group that doesn't exercise regularly (`exercise = 0`) and for the one that does (`exercise = 1`).

When `exercise = 0`, the last two terms become zero and go away:

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 * 0 + 0.4 * \text{stress} * 0 \quad \text{happy} = 12.1 - 1.0 * \text{stress} - 0 + 0 \quad \text{happy} = 12.1 - 1.0 * \text{stress}$$

$$\text{happy}$$

When `exercise = 1`, the intercept goes down by 3.1 and the coefficient on `stress` increases by 0.4:

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 * 1 + 0.4 * \text{stress} * 1$$

$$\text{happy} = 12.1 - 1.0 * \text{stress} - 3.1 + 0.4 * \text{stress}$$

$$\text{happy} = (12.1 - 3.1) + (-1.0 + 0.4) * \text{stress}$$

$$y = 9.0 - 0.6 * \text{stress}$$

happy

We can see the coefficient on `exercise` tells us the difference in INTERCEPTS between the two exercise groups. In this case, the intercept of the regression line for the group that exercises (9.0) is 3.1 units lower than that of the group that doesn't exercise (12.1).

On the other hand, the coefficient on the interaction term tells us the difference in SLOPES between the two regression lines. The slope on `stress` for the group that exercises (-0.6) is 0.4 units greater than that of the group that doesn't exercise (-1.0). This would lead us to conclude that the happiness level of the group who exercises is less negatively impacted by stress.

Instructions

Checkpoint 1 Enabled

1.

The output of the regression predicting `height` from predictors `weight` and `species` with an interaction term from the last exercise is shown below.

```
# Output:
# Intercept           8.168619
# species[T.B]       -3.580515
# weight             1.658621
# weight:species[T.B] 1.115071
```

Copy to Clipboard

Write out and simplify the regression equation for species A. In **script.py**, save the value of the coefficient on `weight` as a variable named `slopeA`. What do we learn about the relationship between `height` and `weight` for species A?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Write out and simplify the regression equation for species B. In **script.py**, save the value of the coefficient on `weight` as `slopeB`. Why might plants of different species show different relationships between their heights and weights?

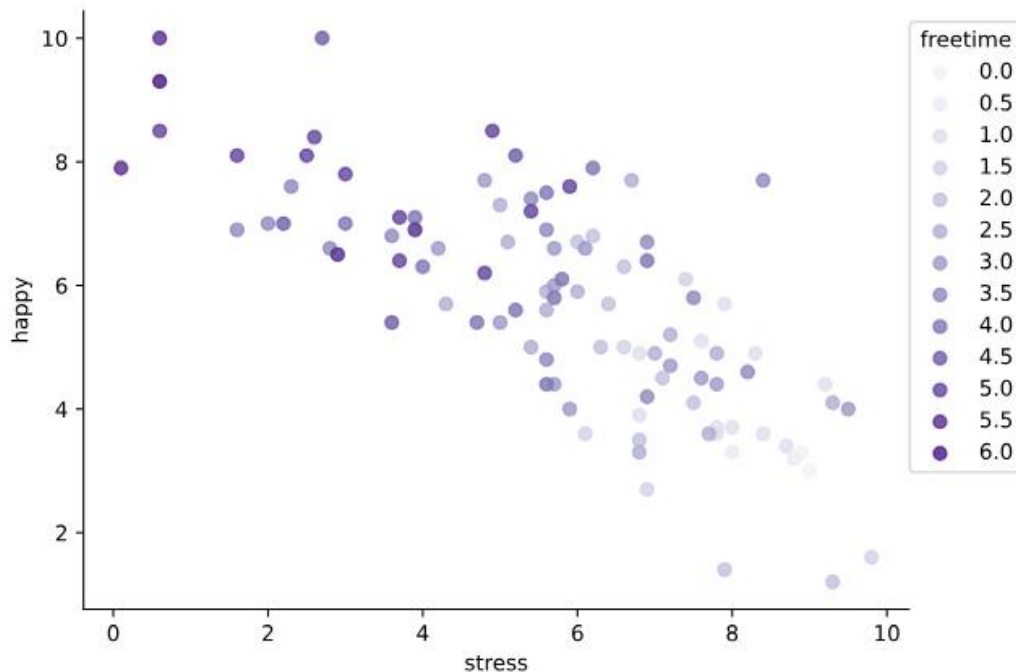
```
# Save slope for species A here:
slopeA = 1.658621

# Save slope for species B here:
slopeB = 1.658621 + 1.115071
```

Interactions and Polynomial Terms

Interactions: Two Quantitative

In the last few exercises we examined interactions between a quantitative predictor and a binary predictor, but we may also wish to use an interaction term for two quantitative variables. Consider the scatter plot of `happy` versus `stress`, this time colored by the quantitative variable `freetime`, which represents the number of hours of free time a participant has on average each day.



If we divided the points into groups based on their `freetime` value and fit a regression line for each group, would all the lines have the same slope?

If we wanted to fit a line amongst the darker points (`freetime` between 5 and 6), the line might have a flat but negative slope.

In contrast, a line for lighter points (`freetime` between 0 and 2) might be steeper in slope.

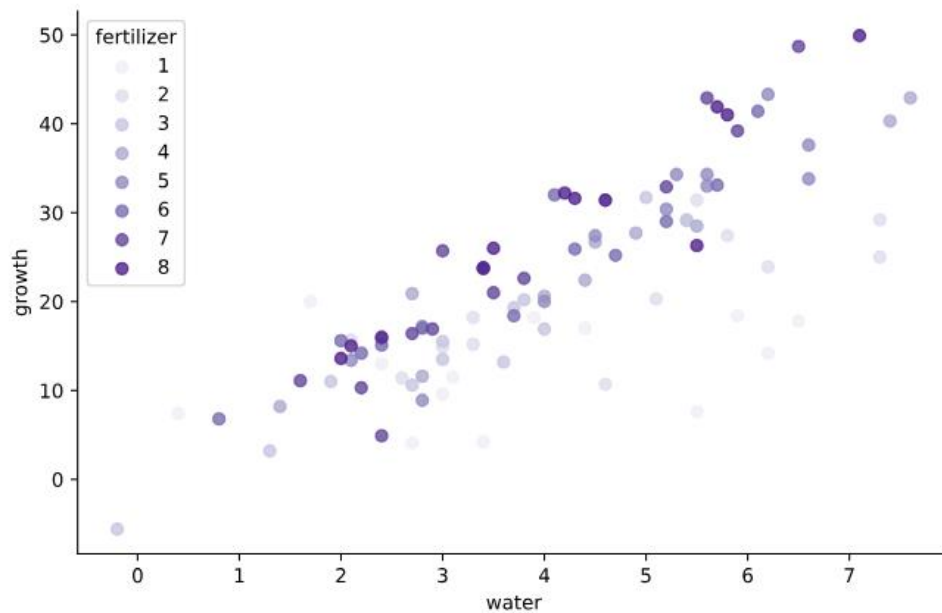
Thus, if we wanted to fit a regression for this data, we might consider fitting several lines for different values of `freetime` rather than a single one across all points. Much like in the previous exercises, we can achieve this by adding a term to the model for the interaction of `stress` and `freetime`.

Instructions

Checkpoint 1 Enabled

1.

The scatter plot below shows data from the `plants` dataset. The plot shows the amount of new growth in centimeters (`growth`) versus the number of times a plant was watered monthly (`water`), colored by amount of fertilizer the plant received (`fertilizer`).



Imagine drawing in a regression line for each of three different groups: when the amount of fertilizer is 1 (very light points), 4 (medium), or 8 (dark). Just by inspecting the graph, estimate the slopes of these imaginary lines and save these in **script.py** as `slope1`, `slope4`, and `slope8`, respectively. Do the slopes indicate that we may need to include an interaction term in our model?

```
# Saves slopes here:
slope1 = 10
slope4 = 12
slope8 = 15
```

Interactions and Polynomial Terms

Interactions in Python: Two Quantitative

Building on the last exercise, let's run a model predicting `happy` from `stress` and `freetime` with an interaction term for the predictors.

```
import statsmodels.api as sm
modelQ = sm.OLS.from_formula('happy ~ stress + freetime + stress:freetime',
data=happiness).fit()
print(modelQ.params)

# Output:
# Intercept          7.731785
# stress             -0.551098
# freetime           0.187882
# stress:freetime     0.040401
```

We form the regression equation from the coefficients just as we did for an interaction term with a binary variable.

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * \text{freetime} + 0.04 * \text{stress} * \text{freetime}$$

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * \text{freetime} + 0.04 * \text{stress} * \text{freetime}$$

We can write a new equation for participants with differing amounts of daily free time.

For participants with 0 hours of free time, the equation is:

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * 0 + 0.04 * \text{stress} * 0$$

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0 + 0$$

$$\text{happy} = 7.73 - 0.55 * \text{stress}$$

For participants with 1 hour of free time, the equation is:

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * 1 + 0.04 * \text{stress} * 1$$

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 + 0.04 * \text{stress}$$

$$\text{happy} = (7.73 + 0.19) + (-0.55 + 0.04) * \text{stress}$$

When we simplify and combine terms, we see the intercept increases by 0.19 and the slope increases by 0.04 compared to the participants with 0 hours of free time. An additional 0.19 and 0.04 get added to the intercept and slope, respectively, when we increase `freetime` to 2 hours:

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * 2 + 0.04 * \text{stress} * 2$$

$$\text{happy} = 7.73 - 0.55 * \text{stress} + 0.19 * 2 + 0.04 * \text{stress} * 2$$

$$\text{happy} = (7.73 + 0.19 * 2) + (-0.55 + 0.04 * 2) * \text{stress}$$

The 0.19 that gets added to the intercept with each increase in `freetime` is the coefficient on `freetime` from our regression. The 0.04 that gets added to the slope of `stress` with each increase in `freetime` is the coefficient on `stress:freetime` from our regression.

Instructions

Checkpoint 1 Enabled

1.

The `plants` dataset has been loaded for you in `script.py`. Fit a model predicting `growth` with `water`, `fertilizer`, and an interaction between them as predictors, and save this model as `model3`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the intercept and coefficients from `model3`. Are they what you expected?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Save the value of the coefficient that represents the difference in the slope on `water` as `fertilizer` increases by one as `slopeDiff`. What do we learn about the relationship between plant growth and number of times watered as amount of fertilizer increases?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Using the model coefficients, write the regression equation for when `fertilizer = 3`. In `script.py`, save the intercept of the equation as `intercept3` and the coefficient on `water` as `slope3`. Do the same for when `fertilizer = 5`, and save the intercept as `intercept5` and the coefficient on `water` as `slope5`. Why are these slopes farther apart than 0.774034?

```
import pandas as pd
import statsmodels.api as sm

plants = pd.read_csv('plants.csv')

# Save model3 here:
model3 = sm.OLS.from_formula('growth ~ water + fertilizer +
water:fertilizer', data=plants).fit()
# Print model3 coefficients here:
print(model3.params)
# Save slopeDiff here:
#slopeDiff = 0.774034
slopeDiff = model3.params[3]
# Save intercept3 and slope3 here:
#intercept3 = 0.774034 + -1.196669*3
intercept3 = model3.params[0] + model3.params[2]*3
#slope3 = 1.860867 + 0.774034*3
slope3 = model3.params[1] + model3.params[3]*3
# Save intercept5 and slope5 here:
#intercept5 = 0.774034 + -1.196669*5
intercept5 = model3.params[0] + model3.params[2]*5
#slope5 = 1.860867 + 0.774034*5
slope5 = model3.params[1] + model3.params[3]*5
```

Intercept 5.904379

water	1.860867
fertilizer	-1.196669
water:fertilizer	0.774034

Interactions and Polynomial Terms

Interpreting and Visualizing Interactions: Two Quantitative

Now that you've seen that an interaction term for two quantitative variables creates regression lines for each value of one of the interacting variables, let's visualize this in a plot. We start with code for a scatter plot of `happy` against `stress` colored by `freetime`:

```
import seaborn as sns

import matplotlib.pyplot as plt

sns.lmplot(x='stress', y='happy', hue='freetime', palette='Purples', fit_reg=False,
data=happiness)
```

Copy to Clipboard

Next, we'll add lines to the plot for a few sample values of `freetime`: 0, 3, and 6 hours. Rather than write out each model coefficient, we can call them directly from `modelQ`, where we stored our regression results in the last exercise. Here is the code to add lines for 3 and 6 hours of free time:

```
plt.plot(happiness.stress,
modelQ.params[0]+modelQ.params[1]*happiness.stress+modelQ.params[2]*3+modelQ.params[3]
*happiness.stress*3, color='mediumpurple', linewidth=3)

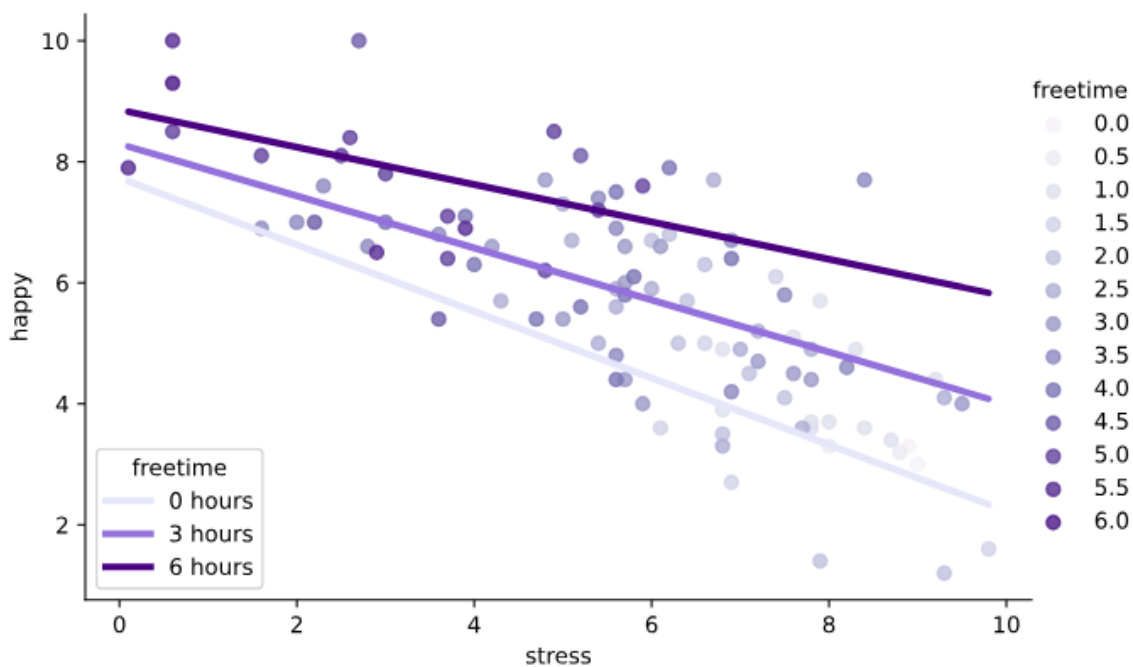
plt.plot(happiness.stress,
modelQ.params[0]+modelQ.params[1]*happiness.stress+modelQ.params[2]*6+modelQ.params[3]
*happiness.stress*6, color='indigo', linewidth=3)

# Add legend and show plot

plt.legend(['0 hours', '3 hours', '6 hours'])

plt.show()
```

Copy to Clipboard



Just as we saw in the regression equations in the last exercise, the intercepts and slopes both increase by 0.19 and 0.04, respectively, for each additional hour of free time. In context, the relationship between stress and happiness appears to be negative. However, this flattening of the slope with increasing amounts of free time might be interpreted as stress impacting happiness less negatively as people have more free time.

This is NOT to say that the data shows that stress CAUSES unhappiness, or that free time CAUSES stress to be less impactful on happiness, just that the association between happiness and stress looks different for people with different amounts of free time.

Instructions

Checkpoint 1 Enabled

1.

In `script.py`, create a scatter plot of `growth` on the y-axis and `water` on the x-axis. Color the points by `fertilizer`. Note that you'll need to uncomment `plt.show()` a few lines down in order to display the scatter plot.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment the rest of the `plt.plot()` objects, which are regression lines for when `fertilizer` is equal to 2 and 4. Using these as a guide, along with the regression coefficients from the model results, add one more line to the plot for when `fertilizer` is equal to 6 and set its `color` argument as `indigo`. Make sure to uncomment the code for the legend as well as `plt.show()`.

Are the slopes of the lines what you expected based on the model coefficients you obtained?

```
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

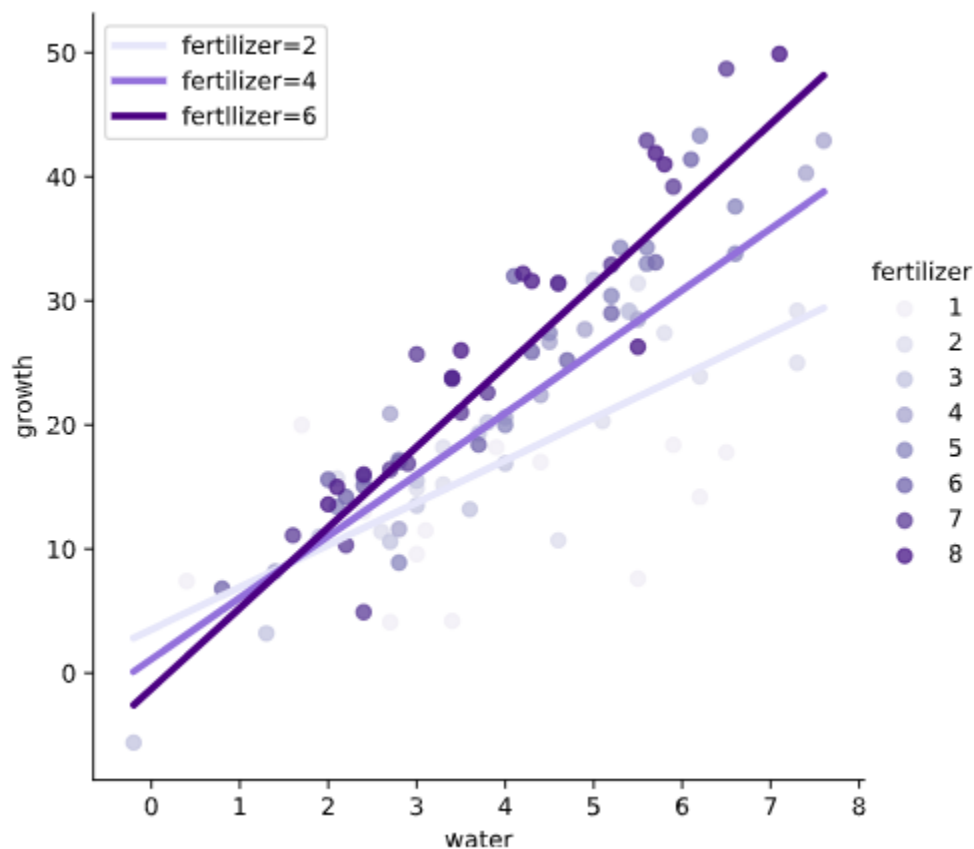
plants = pd.read_csv('plants.csv')
model = sm.OLS.from_formula('growth ~ water + fertilizer + water:fertilizer',
data=plants).fit()

# Create scatter plot here:
```

```

sns.lmplot(x='water', y='growth', hue='fertilizer', palette='Purples',
fit_reg=False, data=plants)
# Uncomment regression lines here:
plt.plot(plants.water, model.params[0] + model.params[1]*plants.water+
model.params[2]*2 + model.params[3]*plants.water*2, color='lavender', linewidth=3)
plt.plot(plants.water, model.params[0] + model.params[1]*plants.water+
model.params[2]*4 + model.params[3]*plants.water*4, color='mediumpurple',
linewidth=3, label='fertilizer=4')
# Add third line here:
plt.plot(plants.water, model.params[0] + model.params[1]*plants.water+
model.params[2]*6 + model.params[3]*plants.water*6, color='indigo', linewidth=3,
label='fertilizer=6')
# Uncomment legend and display code here:
plt.legend(['fertilizer=2','fertilizer=4','fertilizer=6'])
plt.show()

```

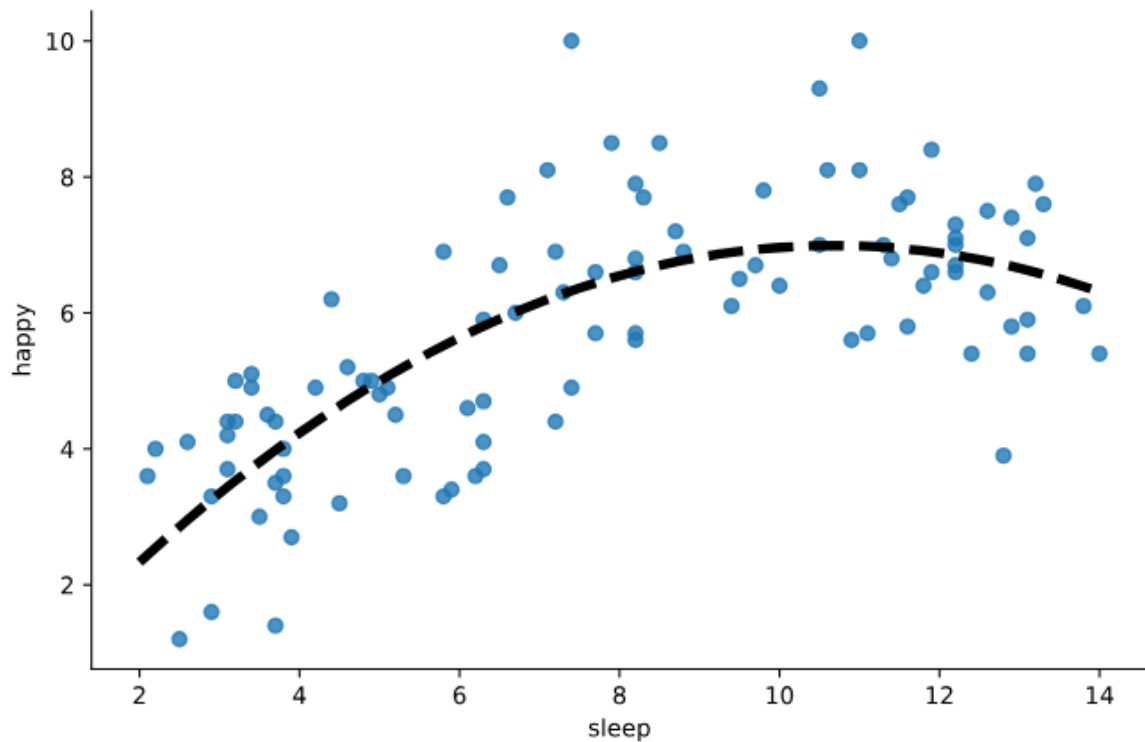


Interactions and Polynomial Terms

Fitting Polynomial Terms in Python

At the beginning of this lesson, we mentioned that the relationship between variables often does not appear to be a straight line and may instead be somewhat curved. In this case, we may produce a better fitting regression model by adding a polynomial term that raises our predictor variable to a higher exponent to better account for the curving.

In the Python library `statsmodels.api`, polynomial terms can be added to a multiple regression model formula by adding a term with the predictor of interest raised to a higher power. This may be done using the NumPy function `np.power` and specifying the predictor name and degree.



For example, the plot above suggests a curvilinear relationship between happiness (`happy`) and hours slept on average (`sleep`). We can add a quadratic (squared) term for the variable `sleep` by doing the following:

```
import statsmodels.api as sm

import numpy as np

modelP = sm.OLS.from_formula('happy ~ sleep + np.power(sleep,2)',
data=happiness).fit()

print(modelP.params)

# Output:
# Intercept          -0.058995
# sleep              1.320429
# np.power(sleep, 2) -0.061827
```

Copy to Clipboard

This creates a second predictor term in the model with an additional coefficient. Correspondingly, the new term shows up in our model equation as `sleep` squared.

$$\text{happy} = -.06 + 1.32*\text{sleep} - .06*\text{sleep}^2$$

We can check happiness scores by substituting in different values of sleep.

For 2 hours of sleep:

$$\text{happy} = -.06 + 1.32*2 - .06*2^2 = 2.34$$

For 10 hours of sleep:

$$\text{happy} = -.06 + 1.32*10 - .06*10^2 = 7.14$$

For 14 hours of sleep:

$$\text{happy} = -.06 + 1.32*14 - .06*14^2 = 6.66$$

Our curved model picks up on the pattern that beyond about 10 hours of sleep, more sleep is not associated with greater happiness. Perhaps the people who sleep a lot are ill so they need more sleep and are less happy, or perhaps sleeping too much causes problems in other parts of their lives that takes away from their happiness. We can't know the exact cause from our model, just the association.

Note that we can use Python and our model results to perform the above computations for us. Below is code to compute `happy` when `sleep` is 10 and 14. Since we use exact rather than rounded numbers here, the results are slightly different but more accurate than our original work.

```
print(modelP.params[0] + modelP.params[1]*10 + modelP.params[2]*np.power(10,2))

# Output:
# 6.9626120786831445

print(modelP.params[0] + modelP.params[1]*14 + modelP.params[2]*np.power(14,2))

# Output:
# 6.308953414816589
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

The `plants` dataset has been loaded for you in `script.py`. Run a regression predicting number of dead leaves (`dead`) from amount of light (`light`) with an additional squared term for `light`. Save the results as `model`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the coefficients of `model`. Are you surprised that the terms with `light` in them have differing signs?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the model coefficients to find the predicted number of dead leaves for `light = 10`. Instead of using rounded versions of the coefficients from the model output, call the coefficients directly using the index numbers of `model.params` and save this computation in `script.py` as `numDead10`. Repeat this process for when `light = 18` and save the computation as `numDead18`.

Here is the code to do this for when `light = 5` and save the computation as `numDead5`:

```
numDead5 = model.params[0] + model.params[1]*5 +  
model.params[2]*np.power(5,2)
```

Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

We can print these values to inspect the pattern of dead leaves as light increases. Here's the code to print

```
numDead5:  
print(numDead5)
```

Copy to Clipboard

Print `numDead10` and `numDead18`. Based on these three predicted values of `dead`, what does the pattern of outcomes say about what might be the best amount of light to keep leaves from dying?

```
import pandas as pd  
import numpy as np  
import statsmodels.api as sm  
  
plants = pd.read_csv('plants.csv')  
  
# Fit regression here:  
model = sm.OLS.from_formula('dead ~ light + np.power(light,2)', data=plants).fit()  
# Print coefficients here:  
print(model.params)  
# Compute and save numDead10 and numDead18:  
numDead10 = model.params[0] + model.params[1]*10 + model.params[2]*np.power(10,2)  
numDead18 = model.params[0] + model.params[1]*18 + model.params[2]*np.power(18,2)  
# Print numDead10 and numDead18:  
#print(numDead10, numDead18)  
print(numDead10)  
print(numDead18)
```

Interactions and Polynomial Terms

Interpreting Polynomial Terms

The results of the regression from the previous exercise, saved as `modelP`, modeled happiness level from hours of sleep with a polynomial term. The model coefficients are given below:

```
import statsmodels.api as sm  
import numpy as np  
modelP = sm.OLS.from_formula('happy ~ sleep + np.power(sleep,2)',  
data=happiness).fit()  
print(modelP.params)  
  
# Output:  
# Intercept          -0.058995  
# sleep              1.320429  
# np.power(sleep, 2) -0.061827
```

Copy to Clipboard

It is generally difficult to interpret the coefficients on polynomial terms directly, but we can interpret the overall relationship if we visualize our regression line on the scatter plot. We can plot the regression line for `happy` predicted by `sleep` WITHOUT a polynomial term by excluding the `fit_reg=False` argument in `lmpplot()`. However, note that we've added `ci=None` to prevent the function from plotting a confidence interval.

```
import seaborn as sns  
import matplotlib.pyplot as plt  
import numpy as np
```

```
sns.lmplot(x='sleep', y='happy', ci=None, data=happiness)
```

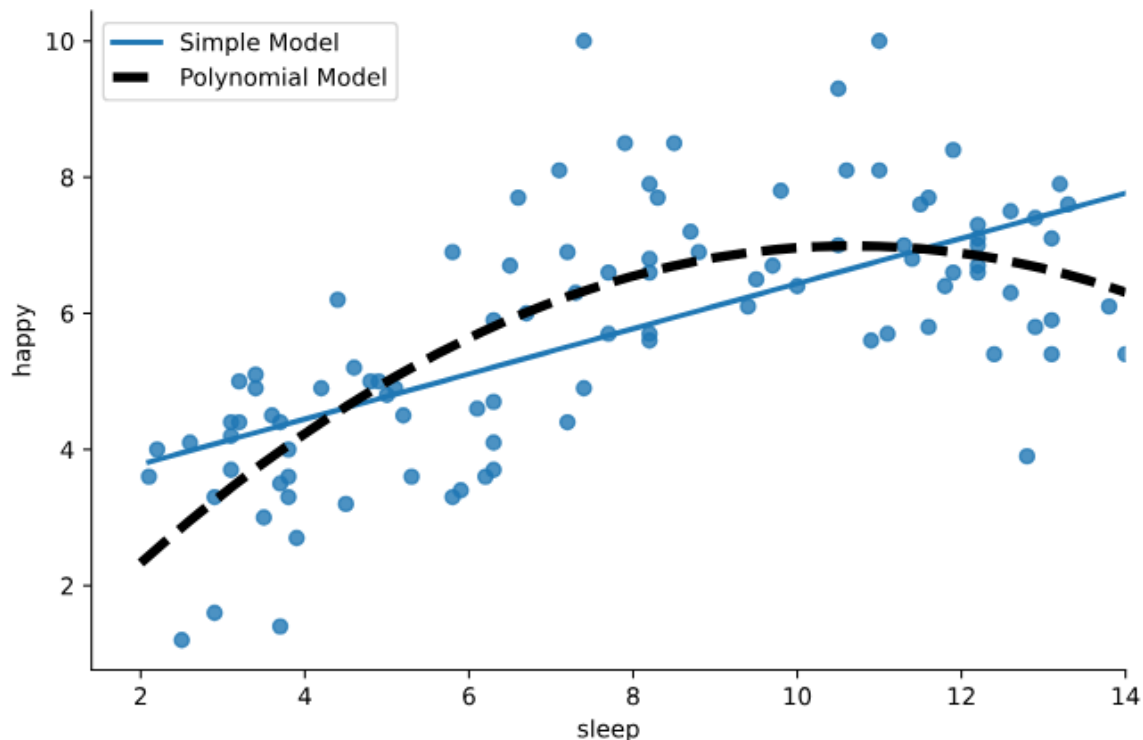
Copy to Clipboard

To add our curved regression line to the scatter plot, we first create a dataset of 100 values of `sleep` ranging from 2 to 14 (saved as `x`) and 100 predicted values of `happy` from our regression equation (saved as `y`). Then we plot our `x` and `y` values using `plt.plot()` and add a legend.

```
x=np.linspace(2,14,100)
y=modelP.params[0]+modelP.params[1]*x+modelP.params[2]*np.power(x,2)

plt.plot(x, y, linestyle='dashed', linewidth=4, color='black')
plt.legend(['Simple Model', 'Polynomial Model'])
plt.show()
```

Copy to Clipboard



From the plot of the polynomial regression line, we see that happiness increases as sleep increases, but that the increase slows as sleep reaches around 10 hours and then begins to decrease with further hours of sleep.

The simple regression line misses these details and describes the relationship as a steady increase in happiness for any additional hour of sleep. Thus, the polynomial model captures a more detailed relationship that will make our predictions better.

Instructions

Checkpoint 1 Enabled

1.

The `plants` dataset has been loaded for you in `script.py`. A simple linear regression predicting `dead` with predictor `light` has been saved for you as `simple`. Print the regression coefficients and inspect them. According to this model, what do we learn about the relationship between number of dead leaves and amount of light a plant receives?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

A new regression model predicting `dead` from `light` but with an additional squared term for `light` has been saved for you as `polynomial`. Print the resulting coefficients. Why can't we interpret the coefficient on `light` directly anymore?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Uncomment the code for the scatter plot that automatically plots the simple regression line for you. Be sure to uncomment `plt.show()` a few lines down as well to display the plot.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

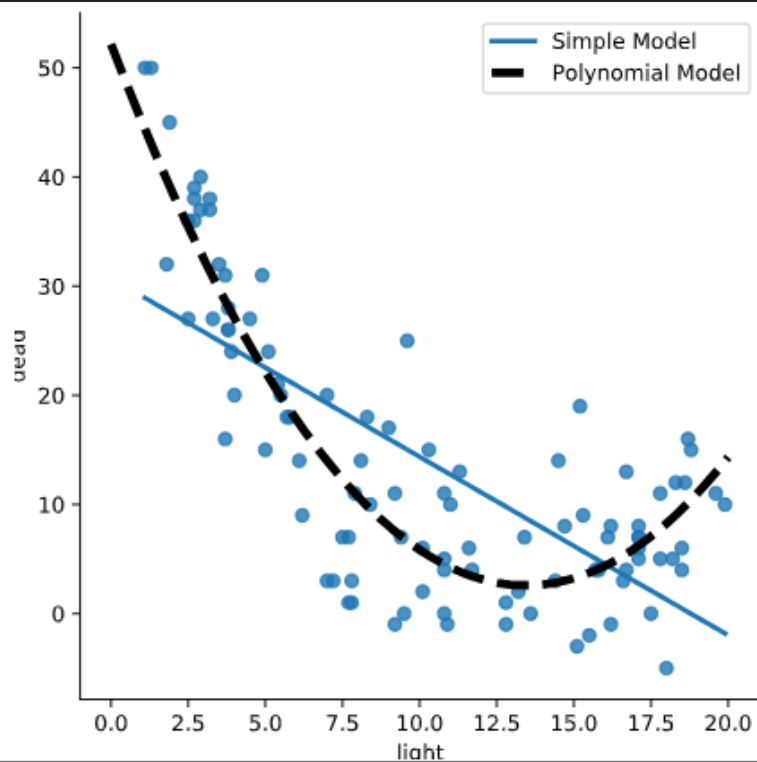
A set of 100 values of `light` has been saved for you as `x` and 100 values of `dead` predicted from those `light` values substituted into the regression equation of `polynomial` has been saved for you as `y`. Use `x` and `y` to add the curved polynomial line in black and dashed. What features of the relationship between `dead` and `light` does the polynomial model capture that the straight line of the simple model misses?

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
import codecademylib3

plants = pd.read_csv('plants.csv')

simple = sm.OLS.from_formula('dead ~ light', data=plants).fit()
polynomial = sm.OLS.from_formula('dead ~ light + np.power(light,2)', data=plants).fit()

# Print simple coefficients here:
print(simple.params)
# Print polynomial coefficients here:
print(polynomial.params)
# Uncomment scatter plot here:
sns.lmplot(x='light', y='dead', ci=None, data=plants)
x=np.linspace(0,20,100)
y=polynomial.params[0]+polynomial.params[1]*x+polynomial.params[2]*np.power(x,2)
# Add polynomial line here:
plt.plot(x, y, linestyle='dashed', linewidth=4, color='black')
# Uncomment legend and display:
plt.legend(['Simple Model', 'Polynomial Model'])
plt.show()
```



Interactions and Polynomial Terms

Review

Congratulations! You have now learned:

- We need interaction and polynomial terms for more complex situations.
- How to fit and interpret interaction terms for a binary predictor with a quantitative predictor.
- How to fit and interpret interaction terms for two quantitative predictors.
- How to fit and interpret polynomial terms.

One final note: You may be wondering how we can be adding multiplied and squared terms and still consider our models to be linear in nature. Although we can add interaction and polynomial terms to a multiple regression model, the model is still considered a multiple LINEAR regression model because the COEFFICIENTS themselves are not raised to higher powers or multiplied by one another.

In other words, the model does not consider the polynomial or interaction terms any differently than any other variable; when we add an interaction or polynomial term, it's like we're just adding another predictor to the model that happens to be a composite of some of the other predictors.

Instructions

- The dataset `hp` has been loaded for you in `script.py`. This dataset was obtained from [Kaggle](https://www.kaggle.com/datasets/robertogallego/harry-potter-fan-fiction) and is a subset of a much larger dataset of Harry Potter fan fiction that had been scraped from the site <https://www.fanfiction.net/book/Harry-Potter/>. Fan fiction stories are written by fans who use the characters and world of Harry Potter to create their own stories for other fans to read.
This dataset has been cleaned, and some variables have been modified, for ease of analysis. The dataset contains many variables, including:
Quantitative Variables:
 - `words` – number of words in the story
 - `reviews` – number of reviews the story received
 - `favorites` – number of readers who favorited the story
 - `follows` – number of readers who follow the story
- Binary Categorical Variables:** *These variables are indicators that are 1 if the following is true and 0 otherwise*
 - `harry` – Harry is a character in the story
 - `hermione` – Hermione is a character in the story
 - `multiple` – the story has multiple chapters
 - `english` – the story is in English
 - `humor` – the story's genre is humor

As this is real data, it is messier than our example datasets, but it is good practice to see how interactions and polynomials might work out on more realistic patterns of variables. Try making some scatter plots to look for potential patterns. Then run some models with interactions or polynomials and check out the resulting coefficients. Do they match what you thought you saw in the scatter plots?

Feel free to check out some sample code in `samples.py` to get ideas if you're stuck!

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import numpy as np
import codecademylib3

hp = pd.read_csv('hp.csv')

# Try a scatter plot of two quantitative variables

# Try the same plot colored by a binary variable
```



```
# Try a model predicting a quantitative variable from two predictors

# Try the same model but with an interaction term

# Try a polynomial model
```

Log Transformations

Log Transformations (And More)

Learn when to use a log transformation of the dependent variable of your linear regression and how to interpret the resulting regression equation.

Introduction

When fitting a linear regression model, we use interaction and polynomial terms to capture complex relationships and improve predictive accuracy. We create these new terms by multiplying predictors together or by raising them to higher exponential powers and then add our new predictors to our model. These are examples of transformations of predictor variables, but sometimes we may want to transform the response (dependent) variable instead. This article will specifically explore when it might make sense to perform a log transformation of the response variable to improve a multiple linear regression model and how to interpret the resulting regression equation.

When to use a log transform

Using a logarithm to transform the response variable may make sense if we notice either or both of the following when checking the assumptions for linear regression:

Sometimes, violated regression assumptions (as described above) indicate that we simply should not use a linear regression model; but if transforming the response variable appears to correct these violations, we may be justified in (carefully) proceeding!

Example dataset

As an example, we'll use a dataset called `countries` which is a cleaned subset of larger dataset from [Kaggle](#). This dataset contains variables for 221 countries for the years 1970-2017, including the following:

```
birth_rate
phones
```

Though the concepts in this article certainly apply to multiple linear regression, we'll use a simple linear regression as an example. Let's say we are interested in predicting `phones` from `birth_rate` using a linear model. First, let's [read in the CSV dataset](#), examine the first few observations, and look at a scatter plot of number of phones versus birth rate.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

countries = pd.read_csv('countries.csv')
print(countries.head())

# Scatter plot with regression line
sns.lmplot(x='birth_rate', y='phones', ci=None, data=countries)
plt.title('Number of Phones vs Birth Rate', fontsize=16, weight='bold')
plt.show()
```

`countries.head()` output:

	country	birth_rate	phones
0	Afghanistan	46.60	3.2
1	Albania	15.11	71.2
2	Algeria	17.14	78.1
3	AmericanSamoa	22.46	259.5
4	Andorra	8.71	497.2

Plot output:

The scatter plot shows a negative correlation between `phones` and `birth_rate`. However, there are some indications that a simple linear regression may not be appropriate for this data:

```
phones    birth_rate
phones    birth_rate
```

To highlight this, we've circled some countries in the plot and have drawn arrows from the points down to the regression line – these are the residuals for these points. We can see a lot of variability in the size of residuals for low birth rates, with very minimal variability for higher birth rates.

To better check our regression assumptions, we can fit the regression in [Python](#) using the following code and save both the residuals and predicted response values as the objects `residuals1` and `fitted_values1`, respectively.

```
import statsmodels.api as sm

# Fit regression model
modell = sm.OLS.from_formula('phones ~ birth_rate', data=countries).fit()

# Save fitted values and residuals
'fitted_values1' = modell.predict(countries)
'residuals1' = countries.phones - fitted_values1
```

Now we'll produce some plots to check the modeling assumptions of normality and homoscedasticity of the residuals.

```
# Check normality of residuals
plt.hist(residuals1)
plt.title('Model 1: Histogram of Residuals', fontsize=16, weight='bold')
plt.show()

# Check variance of residuals
plt.scatter(fitted_values1, residuals1)
plt.axhline(y=0, color='black', linestyle='-', linewidth=3)
plt.title('Model 1: Residuals vs Fitted Values', fontsize=16, weight='bold')
plt.show()
```

Histogram of residuals:

Residuals vs fitted values:

In the histogram, we see some right skewing caused by the few very high residuals for countries like Bermuda, indicating we may not be meeting the normality assumption. Perhaps more concerning, the scatter plot of residuals against fitted values shows a wave-like pattern from narrow to wide, rather than the constant spread we look for to indicate that homoscedasticity has been met. We've additionally highlighted the same countries in the scatter plot again so we can see how their residuals map out in this plot compared to where we saw them in the original.

Log transformation in Python

Since we see two potential assumption violations, we are going to try a log transformation of the `phones` variable and check if it improves our concerns. In Python, we can easily take the log of `phones` using the [NumPy](#) function `.log()`. Let's add this new variable to our dataset and see how it looks compared to `phones`. Note that, generally, when we see *log* with no specified base in a statistics equation, we can assume the base is *e* (the mathematical constant 2.718...). In other words, *log* with no base means we are taking the *natural log*, or *ln*. Also, note that we can only take the log of a variable with values greater than zero; the log of values less than or equal to zero are undefined.

```
import numpy as np
# Save log_phones to dataset
countries['log_phones'] = np.log(countries.phones)
print(countries.head())
```

Output:

	country	birth_rate	phones	log_phones
0	Afghanistan	46.60	3.2	1.163151
1	Albania	15.11	71.2	4.265493
2	Algeria	17.14	78.1	4.357990
3	AmericanSamoa	22.46	259.5	5.558757
4	Andorra	8.71	497.2	6.208992

We can see that this transformation has drastically reduced the range of values for our dependent variable. Let's run a second model predicting `log_phones` from `birth_rate` and see what else has changed.

```
# Fit regression model
model2 = sm.OLS.from_formula('log_phones ~ birth_rate', data=countries).fit()
# Save fitted values and residuals
'fitted_values2' = model2.predict(countries)
'residuals2' = countries.log_phones - fitted_values2
```

If we examine the scatter plot of `log_phones` against `birth_rate`, we can see a big change in the appearance of our data:

While there's some crowding in the upper lefthand corner, the pattern now appears much more linear and more evenly spaced about the regression line. Specifically, countries that had larger residuals earlier (like Bermuda and Australia) are now much closer to the line and each other vertically. Likewise, countries that had small residuals earlier (like Mayotte and Angola) are now further from the line and each other vertically. This change is reflected in both the histogram of the residuals (now much less skewed) and the scatter plot of the residuals versus the fitted values (now much more evenly spaced across the line $y = 0$).

Histogram of residuals:

Scatter plot of residuals:

Interpretation

While it's great that our new variable seems to be better meeting our model assumptions, how do we interpret the coefficients in our model now that logs are involved? First, let's look at the output of the model predicting `log_phones` from `birth_rate` and write out the regression equation:

```
print(model2.params)
# Output:
# Intercept      7.511024
# birth_rate     -0.130456
```

$$\log(\text{phones}) = 7.51 - 0.13 * \text{birth_rate}$$

$$\log(\text{phones}) = 7.51 - 0.13 * \text{birth_rate}$$

We can always interpret the coefficient on `birth_rate` in the traditional way: for every increase of one birth per 1000 people, the natural log of `phones` decreases by 0.13 phones per 1000 people. While this is accurate, it's not very informative about the relationship between `phones` and `birth_rate`. To examine this relationship, we need to do a little math with logs and exponentiation.

To get a more direct relationship between `phones` and `birth_rate`, we first have to *exponentiate* the coefficient on `birth_rate`. This means we raise *e* to the power of the coefficient on `birth_rate`. We may write this as *e*-0.13, or more simply as *exp*(-0.13), and we can use NumPy to compute this in Python. In short, we're doing this because exponentiating both sides of the regression equation cancels out the log on `phones`, but we'll save the more thorough explanation for the bonus section at the end of this article.

```
import numpy as np
np.exp(-0.13)
# Output
# 0.8780954309205613
```

Then we also subtract 1 to change our coefficient into an easily readable percentage change:

```
np.exp(-0.13) - 1
# Output:
# -0.1219045690794387
```

We are now ready to interpret this coefficient: for every additional birth per 1000 people, the number of phones per 1000 people decreases by about 12.2 PERCENT. Our interpretation changes from the traditional *additive* relationship, where increases in the predictor are associated with differences in UNITS of the outcome, to a *multiplicative* relationship, where increases in the predictor are associated with differences in the PERCENTAGE of the outcome.

We also see this change in the interpretation of the intercept: rather than the *arithmetic* mean, the exponentiated intercept *exp*(7.51) is the *geometric* mean number of phones for countries with a birth rate of 0. The arithmetic mean is computed by SUMMING values, while the geometric mean is computed by MULTIPLYING values.

Conclusion

Log transformations of the dependent variable are a way to overcome issues with meeting the requirements of normality and homoscedasticity of the residuals for multiple linear regression. Unfortunately, a log transformation won't fix these issues in every case (it may even make things worse!), so it's important to reassess normality and homoscedasticity after making the transformation and running the new model. Log transformations can also be performed on predictors, and there are other dependent variable transformations available as well (e.g., square-rooting). To learn more about some of these transformations, check out the [Penn State Statistics Department's website](#).

Bonus: Logs in more detail

Why did taking the log of the dependent variable help?

As we recall from the scatter plot of `phones` versus `birth_rate`, there were a lot of large positive residuals for lower birth rates and a lot of smaller residuals for higher birth rates. Taking the log of `phones` brought the large residuals lower and the small residuals higher, which gave us a more even spread with less extremes. But why did this happen? Let's take a quick look at what happens as *e* is raised to higher exponents. Note that we use 2.718 as an approximation of *e* here.

power	e^{power}	multiplied	output	difference
1	e_1	2.718	2.718	—
2	e_2	2.718*2.718	7.388	4.670
3	e_3	2.718*2.718*2.718	20.079	15.409
4	e_4	2.718*2.718*2.718*2.718	54.576	34.497
5	e_5	2.718*2.718*2.718*2.718*2.718	148.336	93.760
6	e_6	2.718*2.718*2.718*2.718*2.718*2.718	403.178	254.842

As we can see from the table, every time the power e is raised to increases, the output nearly triples. This means the difference in the outputs between low powers is smaller than the difference in outputs between larger powers. Taking the log of the output column “undoes” this process, returning the corresponding value in the power column (e.g., $\log(2.718) = 1$, $\log(7.388) = 2$, etc.).

In terms of our dataset, the output column is like the raw `phones` values, and the power column is the new `log_phones` variable. Big differences in the upper values of `phones` translate to the same size jump on the `log_phones` scale as small differences in the lower values of `phones`. Thus, translated to the log scale, the large values of `phones` (like those of Bermuda and Australia) pull in, while the small values of `phones` (like those of Mayotte and Angola) spread out.

Why do we interpret the exponentiated coefficients on the predictors as percentage differences of the dependent variable?

Let's say `birth_rate0` is a value of `birth_rate` and `phones0` is the value of `phones` at `birth_rate0` such that:

$$\log(\text{phones0}) = 7.51 - 0.13 * \text{birth_rate0}$$

Let's also say `phones1` is the value of `phones` when `birth_rate` is increased by 1 birth from `birth_rate0`. Then,

$$\log(\text{phones1}) = 7.51 - 0.13 * (\text{birth_rate0} + 1)$$

Next, we distribute the -0.13 and substitute $\log(\text{phones0})$ for $7.51 - 0.13 * \text{birth_rate0}$. Then we subtract $\log(\text{phones0})$ from both sides to isolate the `birth_rate` coefficient of -0.13.

Finally, under the quotient rule, we find that our coefficient on `birth_rate` is equal to a single log. We exponentiate both sides to find our exponentiated coefficient on `birth_rate` is equal to a simple quotient that gives the percentage change in the `phones` variable between `phones0` and `phones1`.

$$\log(\text{phones1} / \text{phones0}) = -0.13$$

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-regression-for-inference-data-science/modules/dsinf-multiple-linear-regression/articles/data-transformations-for-multiple-linear-regression>

Logistic Regression

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-regression-for-inference-data-science/modules/dsinf-logistic-regression/lessons/logistic-regression/exercises/review>

Choosing a Linear Regression Model

Introduction

In this lesson, we'll discuss some of the ways we can compare and choose linear regression models using a variety of different methods.

For example, suppose that we work at a bike rental company and have a dataset where each row represents a unique day of business (including info about the number of bikes that were rented, the weather, the day of the week, etc.). We might want to use this dataset to predict how many bikes will be rented tomorrow (so we can plan). Alternatively, we might want to use the dataset to understand which factors are most predictive of bike usage.

For either goal, we can use a linear regression model. The problem is: there are many different models that we could create. How do we know which one to use?

This lesson will focus on some common ways to compare and choose a linear model, both for prediction and data analysis.

Instructions

Checkpoint 1 Enabled

1.

Run the code in **script.py** to create and fit two different linear regression models. **IMPORTANT NOTE:** In order to view the output (in this exercise and the rest of the lesson), you'll need to expand the width of the output terminal by clicking and dragging the left edge further left. For both models, the outcome variable is the number of bike rentals on a particular day. Inspect the output for each model. Do you have any ideas about how you might compare these models and choose one? Some things you might think about:

What are the benefits of a complex model (with many predictors, interaction terms, polynomial terms, etc.)?
What are the benefits of a simple model (with just a few predictors)?
Are some predictors more important to include than others?

```
# Load libraries
import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + atemp + hum + windspeed + weathersit +
season + mnth + workingday + holiday + weekday + temp:mnth + temp:windspeed +
temp:hum', data=bikes).fit()

# Print model1 summary
print(model1.summary())

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ atemp + windspeed + mnth + holiday',
data=bikes).fit()

# Print model2 summary
print(model2.summary())
```

Choosing a Linear Regression Model

R-Squared

R-squared is one of the most common metrics to evaluate linear regression models. We can interpret R-squared as the proportion of variation in an outcome variable that is explained by a linear regression model. More explained variation is generally better.

For example, suppose we have a dataset containing information about apartment rentals for NYC apartments. We can build two different models to predict rental price and print out the R-Squared for each model as follows:

```
# Create and fit the first model to predict rent

model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + min_to_subway',
data=rentals).fit()

# Create and fit the second model
```



```
model2 = sm.OLS.from_formula('rent ~ bathrooms + building_age_yrs + borough',
data=rentals).fit()
```

```
# Print out R-squared for both models
```

```
print(model1.rsquared) #Output: 0.664
```

```
print(model2.rsquared) #Output: 0.596
```

Copy to Clipboard

This tells us that the first model (using bedrooms, square-footage, and minutes to the subway) explains about 66.4% of the variation in rental prices, whereas the second model only explains about 59.6% of the variation. This would lead us to choose the first model over the second.

Instructions

Checkpoint 1 Enabled

1.

Using the `bikes` dataset, fit a model to predict `cnt` (the number of bike rentals) based on the temperature (`temp`), windspeed (`windspeed`), and whether or not it is a holiday (`holiday`). Save the fitted model as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using the `bikes` dataset, fit a second model to predict `cnt` (the number of bike rentals) based on humidity (`hum`), season (`season`), and the day of the week (`weekday`). Save the fitted model as `model2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out the R-squared for both models.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Based on the R-squared values, which model would you choose? Indicate your answer by setting a variable named `which_model` equal to `1` if you would choose `model1` and equal to `2` if you would choose `model2`.

```
# Load libraries
import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + windspeed + holiday', data=bikes).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ hum + season + weekday', data=bikes).fit()

# Print R-squared for both models
print(model1.rsquared)
print(model2.rsquared)

# Choose a model
which_model = 1
```

R-Squared and the Dangers of Overfitting

Let's again suppose that we want to use the StreetEasy data to predict rental prices in NYC. We have the following two models that we want to compare:

```
# Fit model 1

model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + min_to_subway',
                             data=rentals).fit()

# Fit model 2

model2 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + min_to_subway +
                             borough', data=rentals).fit()

# Print out R-squared for both models

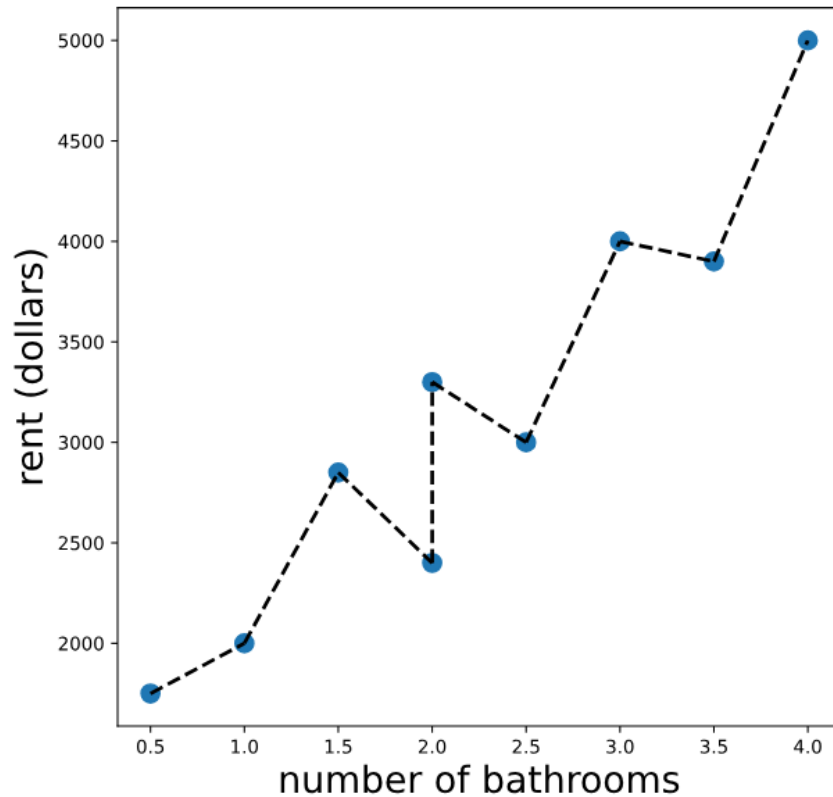
print(model1.rsquared) # Output: 0.664

print(model2.rsquared) # Output: 0.728
```

Copy to Clipboard

Note that these models both use `bedrooms`, `size_sqft`, and `min_to_subway` as predictors; but `model2` uses `borough` as well. Because all of the predictors in `model1` are also contained in `model2`, these are called *nested models*.

It turns out that larger nested models will ALWAYS have higher R-squared than their smaller counterparts. However, adding a lot of additional predictors can lead to a different issue: over-fitting. To understand the intuition behind why overfitting is problematic, consider the following plot of rental prices vs. number of bathrooms. We can perfectly predict each datapoint if we fit the zig-zagging line shown below:



However, imagine that we collect a new sample of apartments in NYC and record the number of bathrooms in each. Then, suppose we want to use our model to predict rental prices. Even if the overall relationship between bathrooms and rent is the same in our new data, the exact values will be slightly different. Predictions based on the zig-zag line may be less accurate because the model was so heavily influenced by the quirks of the data we originally collected. A straight line through the middle of the points is actually more useful.

Instructions

Checkpoint 1 Enabled

1.

Using the `bikes` dataset, fit a model to predict `cnt` (the number of bike rentals) based on the temperature (`temp`), windspeed (`windspeed`), and whether or not it is a holiday (`holiday`). Save the fitted model as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit a second model with `cnt` as the outcome variable and all the same predictors as in `model1` plus humidity (`hum`). Save the fitted model as `model2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now fit a third model with `cnt` as the outcome variable and all the same predictors as in `model2` plus the day of the week (`weekday`). Save the fitted model as `model3`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print out the R-squared for all three models. Notice that the R-squared increases from `model1` to `model2` and from `model2` to `model3`, even though the increase is very small.

This means that adding more predictors allows us to explain only a TINY bit more variation in the number of bike rentals. This may mean that we are overfitting the model to the data.

How could we decide which model to use? Are these extra predictors helping or hurting the model? Let's keep exploring this in the next exercise!

```
# Load libraries
```

```

import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + windspeed + holiday', data=bikes).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ temp + windspeed + holiday + hum',
data=bikes).fit()

# Fit model3
model3 = sm.OLS.from_formula('cnt ~ temp + windspeed + holiday + hum + weekday',
data=bikes).fit()

# Print R-squared for all models
print(model1.rsquared)
print(model2.rsquared)
print(model3.rsquared)

```

Adjusted R-Squared

While R-squared is useful for comparing models with different sets of predictors, we saw that it could lead to overfitting when choosing between nested models.

To address this issue, we can instead use adjusted R-squared, which gives a small penalty for each additional predictor in a model. For example:

```

model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + borough',
data=rentals).fit()

model2 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + borough +
has_doorman', data=rentals).fit()

print(model1.rsquared) #Output: 0.72761
print(model2.rsquared) #Output: 0.72765

print(model1.rsquared_adj) #Output: 0.72739
print(model2.rsquared_adj) #Output: 0.72738

```

Copy to Clipboard

Note that the second model (with an additional predictor) has a slightly larger R-squared, but a slightly smaller adjusted R-squared, compared to the first model. Based on the adjusted R-squared, we would choose the smaller model.

Instructions

Checkpoint 1 Enabled

1.

Using the `bikes` dataset, fit a model to predict the number of bike rentals (`cnt`) with the following predictors: `temp`, `hum`, `windspeed`, `season`, `holiday`, and `weekday`. Save the fitted model as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit a second model with `cnt` as the outcome variable and all the same predictors as in `model1` plus the "feels like" temperature (`atemp`). Save the fitted model as `model2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out the R-squared for both models.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print out the adjusted R-squared for both models.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Based on the R-squared values, which model would you choose? Indicate your answer by setting a variable named `which_model_rsqr` equal to 1 if you would choose `model1` and equal to 2 if you would choose `model2`.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Based on the adjusted R-squared values, which model would you choose? Indicate your answer by setting a variable named `which_model_adj_rsqr` equal to 1 if you would choose `model1` and equal to 2 if you would choose `model2`.

```
# Load libraries
import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed + season + holiday + weekday', data=bikes).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed + season + holiday + weekday + atemp', data=bikes).fit()

# Print R-squared for both models
print(model1.rsquared)
print(model2.rsquared)

# Print adjusted R-squared for both models
print(model1.rsquared_adj)
print(model2.rsquared_adj)

# Which model based on R-squared?
which_model_rsqr = 2
```

```
# Which model based on adjusted R-squared?  
which model adj_rsqu = 1
```

F-Tests

In the previous exercises, we compared nested models based on adjusted R-squared.

Another way to compare nested models is by using a hypothesis test called an F-test. Suppose we want to compare the following two models:

```
model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft',  
data=rentals).fit()  
  
model2 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + building_age_yrs  
+ has_elevator', data=rentals).fit()
```

Copy to Clipboard

Note that the second model has two more predictors than the first (`building_age_yrs` and `has_elevator`). For an F-test comparing these two models:

The *null hypothesis* is that the coefficients on `building_age_yrs` and `has_elevator` are equal to zero (they are not useful in explaining the observed variation in rent).

The *alternative hypothesis* is that least one of the coefficients is non-zero.

We can run the test in Python as follows:

```
from statsmodels.stats.anova import anova_lm  
  
anova_results = anova_lm(model1, model2)  
  
print(anova_results)
```

Copy to Clipboard

Output:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	4997.0	1.4e+10	0.0	NaN	NaN	NaN

1	4995.0	1.4e+10	2.0	9.2e+08	170.9	1.6e-72
---	--------	---------	-----	---------	-------	---------

The p-value (1.6e-72, which is equal to .00000...[72 total zeros]..16) is located in the bottom right corner of this table. The column name `Pr(>F)` means “the probability of observing an F statistic greater than observed (170.9) if the null hypothesis is true”.

Using a significance threshold of 0.05, the p-value is below the threshold. Therefore, we would conclude that either (or both) of the coefficients on `building_age_yrs` and `has_elevator` is non-zero. Thus, including at least one of these two predictors significantly improves the model.

This would lead us to choose `model2` over `model1`. After running this test, we might also want to separately compare a model with `building_age_yrs` and a model with `has_elevator` to see whether both are necessary. We could do this with separate F-tests or adjusted R-squared.

Instructions

Checkpoint 1 Failed, try again

1.

Using the `bikes` data, fit a model to predict the number of bike rentals (`cnt`) with the following predictors: `temp`, `hum`, and `windspeed`. Save the fitted model as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit a second model with all the same predictors plus the day of the week (`weekday`) and “feels like” temperature (`atemp`). Save this fitted model as `model2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Run an F-test to compare the two models and print out the results.

NOTE: you may want to round the results using `.round(2)` so that they fit in the output terminal. Otherwise, you’ll need to scroll to the right in the output table to see the p-value.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Based on the F-test and a significance threshold of 0.05, which model would you choose? Indicate your answer by setting a variable named `which_model` equal to 1 if you would choose `model1` and equal to 2 if you would choose `model2`.

```
# Load libraries
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.anova import anova_lm
import codecademylib3

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed', data=bikes).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed + weekday + atemp',
data=bikes).fit()

# Run the F-test and print results
anova_results = anova_lm(model1, model2)
print(anova_results.round(2))
```

```
# Which model is better?  
which_model = 2
```

Log-Likelihood

So far, we've used R-squared, adjusted R-squared, and an F-test to compare models. These criteria are most useful for finding a model that best fits an observed set of data. They are often used when our goal is interpreting a model to understand relationships between variables.

If our goal is to choose the best model for making predictions for new/unobserved data, we may want to use a likelihood based criteria instead.

Log-likelihood of a linear regression model essentially measures the probability of observing our data given a particular model. Higher log-likelihood is better.

For example, we can compare two models based on log likelihood as follows:

```
model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + min_to_subway',  
data=rentals).fit()  
  
model2 = sm.OLS.from_formula('rent ~ bathrooms + building_age_yrs + borough',  
data=rentals).fit()  
  
print(model1.llf) #Output: -44282.327  
print(model2.llf) #Output: -44740.623
```

Copy to Clipboard

Because model 1 has a higher log-likelihood (a smaller negative number is larger), we would choose model 1 over model 2.

Instructions

Checkpoint 1 Enabled

1.

Using the `bikes` dataset, fit a model to predict the number of bike rentals (`cnt`) with the following predictors: temperature (`temp`), windspeed (`windspeed`), and whether or not it is a holiday (`holiday`). Save the fitted model as `model1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now fit a second model to predict `cnt` using the following predictors: humidity (`hum`), season (`season`), and the day of the week (`weekday`).

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out the log-likelihood for both models.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Based on the log likelihood values, which model would you choose? Indicate your answer by setting a variable named `which_model` equal to 1 if you would choose `model1` and equal to 2 if you would choose `model2`.

```
# Load libraries
import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + windspeed + holiday', data=bikes).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ hum + season + weekday', data=bikes).fit()

# Print log likelihood for both models
print(model1.llf)
print(model2.llf)

# Which model based on log likelihood?
which_model = 1
```

AIC and BIC

Similarly to R-squared, log-likelihood only increases as we add more predictors to a model. In the same way that adjusted R-squared penalizes R-squared for more predictors, there are criteria that penalize the log-likelihood for more predictors.

The two most commonly used are *Akaike information criterion (AIC)* and *Bayesian information criterion (BIC)*. Both AIC and BIC use negative log-likelihood, so we actually want the model with the LOWEST AIC or BIC.

AIC and BIC are similar, but BIC gives a bigger penalty for each additional predictor, so it is used for finding the best “simple” model. This is useful because it makes the model more interpretable. For example:

```
model1 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + borough',
data=rentals).fit()

model2 = sm.OLS.from_formula('rent ~ bedrooms + size_sqft + borough +
has_doorman', data=rentals).fit()

print(model1.llf) #Output: -43756.418
```

```

print(model2.llf) #Output: -43756.017

print(model1.aic) #Output: 87522.837

print(model2.aic) #Output: 87524.034

print(model1.bic) #Output: 87555.423

print(model2.bic) #Output: 87563.137

```

Copy to Clipboard

We see that the log-likelihood for model 2 is slightly larger (better), but the AIC for model 2 is slightly larger (worse), and BIC even more so. Both AIC and BIC would lead us to choose model 1, whereas log-likelihood would lead us to choose model 2.

Instructions

Checkpoint 1 Enabled

1.

Two different models have been fit for you in **script.py** and saved as `model1` and `model2`, respectively. Print out the log-likelihood for both models.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Based on the log-likelihood values, which model would you choose? Indicate your answer by setting a variable named `which_model_loglik` equal to `1` if you would choose `model1` and equal to `2` if you would choose `model2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out the AIC for `model1` and `model2`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Based on the AIC values, which model would you choose? Indicate your answer by setting a variable named `which_model_aic` equal to `1` if you would choose `model1` and equal to `2` if you would choose `model2`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Print out the BIC for `model1` and `model2`.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Based on the BIC values, which model would you choose? Indicate your answer by setting a variable named `which_model_bic` equal to `1` if you would choose `model1` and equal to `2` if you would choose `model2`.

```

# Load libraries
import pandas as pd
import statsmodels.api as sm

# Import data
bikes = pd.read_csv('bikes.csv')

# Fit model1

```

```

modell1 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed + season + holiday +
weekday', data=bikes).fit()

# Fit model2
modell2 = sm.OLS.from_formula('cnt ~ temp + hum + windspeed + season + holiday +
weekday + atemp', data=bikes).fit()

# Print log likelihood for both models
print(modell1.llf)
print(modell2.llf)

# Which model based on log likelihood?
which_model_loglik = 2

# Print AIC for both models
print(modell1.aic)
print(modell2.aic)

# Which model based on AIC?
which_model_aic = 1

# Print BIC for both models
print(modell1.bic)
print(modell2.bic)

# Which model based on BIC?
which_model_bic = 1

```

Training and Test Sets

Another way of choosing a model to make predictions for new data (also called out-of-sample prediction) is by using *training* and *test* datasets. The idea is that we only use PART of our data to fit the model, then we see how well the model performs in predicting the outcome of interest for the rest of our data. The process is as follows:

First, we split our data into two subsets: a training set and a test set. Often, the training set is a larger proportion of the data.

In other Python libraries, there are built-in functions to split a dataframe, but for the sake of understanding, we'll do it explicitly here by randomly sampling from a list of row indices:

```

import numpy as np

# Create a list of indices
indices = range(len(rentals))

# Determine the size of the training set (s)
s = int(0.8*len(indices))

# Randomly select 80% of the indices
train_ind = np.random.choice(indices, size = s, replace = False)

# Create a list of the remaining 20% of indices
test_ind = list(set(indices) - set(train_ind))

```

```
# Split the data into the training and test sets
rentals_train = rentals.iloc[train_ind]
rentals_test = rentals.iloc[test_ind]
```

Copy to Clipboard

Next, we fit the models we want to compare using the training set data only:

```
model1 = sm.OLS.from_formula('rent ~ bedrooms + bathrooms + size_sqft +
min_to_subway + floor', data=rentals_train).fit()

model2 = sm.OLS.from_formula('rent ~ bedrooms + bathrooms + size_sqft +
min_to_subway + floor + borough', data=rentals_train).fit()
```

Copy to Clipboard

Then, we use those models to predict the rental price for the apartments in the test set:

```
fitted1 = model1.predict(rentals_test)
fitted2 = model2.predict(rentals_test)
```

Copy to Clipboard

Finally, we can compare the predicted rents to the true rents in the test set and use a metric to determine how well each model performed.

In this example, we'll use a metric called *predictive root mean squared error (PRMSE)*, which is exactly what the name sounds like: the square root of the mean squared difference between predicted and true values of the outcome variable. A smaller PRMSE means that the model performed better (the predicted values were more similar to the true values):

```
true = rentals_test.rent
prmse1 = np.mean((true-fitted1)**2)**.5
prmse2 = np.mean((true-fitted2)**2)**.5
print(prmse1) #output: 1326.258
print(prmse2) #output: 1224.269
```

Copy to Clipboard

Based on this metric, we would choose the second model over the first one because it has a smaller PRMSE.

Instructions

Checkpoint 1 Enabled

1.

In **script.py**, we've provided you with code to split the `bikes` dataset into training and test sets (`bikes_train` and `bikes_test`). We've then fit two different models with the training set and saved those models as `model1` and `model2`, respectively.

Use `model1` to predict the number of bikes rented (`cnt`) for each day in the test dataset and save the result as `fitted1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use `model2` to predict the number of bikes rented (`cnt`) for each day in the test dataset and save the result as `fitted2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Calculate the PRMSE for `model1` and save it as `prmse1`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Calculate the PRMSE for `model2` and save it as `prmse2`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Print out the PRMSE for both models.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Which model would you choose based on PRMSE? Indicate your answer by setting a variable named `which_model` equal to 1 if you would choose `model1` and equal to 2 if you would choose `model2`.

```
# Load libraries
import pandas as pd
import numpy as np
import statsmodels.api as sm

# Set seed (don't change this)
np.random.seed(123)

# Import data
bikes = pd.read_csv('bikes.csv')

# Split bikes data
indices = range(len(bikes))
s = int(0.8*len(indices))
train_ind = np.random.choice(indices, size = s, replace = False)
test_ind = list(set(indices) - set(train_ind))
bikes_train = bikes.iloc[train_ind]
bikes_test = bikes.iloc[test_ind]

# Fit model1
model1 = sm.OLS.from_formula('cnt ~ temp + atemp + hum', data=bikes_train).fit()

# Fit model2
model2 = sm.OLS.from_formula('cnt ~ season + windspeed + weekday',
data=bikes_train).fit()

# Calculate predicted cnt based on model1
fitted1 = model1.predict(bikes_test)

# Calculate predicted cnt based on model2
fitted2 = model2.predict(bikes_test)

# Calculate PRMSE for model1
true = bikes_test.cnt
prmse1 = np.mean((true-fitted1)**2)**.5

# Calculate PRMSE for model2
prmse2 = np.mean((true-fitted2)**2)**.5

# Print PRMSE for both models
print(prmse1)
print(prmse2)

# Which model performed better based on PRMSE?
which_model = 1
```

Review

Congratulations! In this lesson, you've learned a number of different methods for model comparison:

For choosing a model that best represents the data we have:

- R-squared
- Adjusted R-squared
- F-test

For choosing a model for accurate out-of-sample prediction:

- Log likelihood
- AIC/BIC
- Training/test sets

Note that we've covered many different methods for choosing a model and they don't always agree. In order to choose a method, it's important to consider your ultimate goal (analysis vs. prediction) and what you want to prioritize (simplicity and interpretability vs. accuracy)

Instructions

1. In this final workspace, we've loaded the StreetEasy dataset for you to investigate further. The dataset contains the following columns:
 - `rent`: the monthly rental price in dollars
 - `bedrooms`: the number of bedrooms
 - `bathrooms`: the number of bathrooms
 - `size_sqft`: the area in square feet
 - `min_to_subway`: minutes walking distance to the nearest subway station
 - `building_age_yrs`: age of the building in years
 - `no_fee`: whether or not there is a broker fee
 - `has_roofdeck`: whether or not there is a roofdeck
 - `has_washer_dryer`: whether or not there is a washer and dryer
 - `has_doorman`: whether or not there is a doorman
 - `has_elevator`: whether or not there is an elevator
 - `has_dishwasher`: whether or not there is a dishwasher
 - `has_patio`: whether or not there is a patio
 - `has_gym`: whether or not there is a gym
 - `neighborhood`: neighborhood where the apartment is located
 - `borough`: borough where the apartment is located
2. Which predictors do you think will be most important in predicting the rental price of an apartment in NYC? Using the predictors you think are most relevant:
 - Fit a few different models**
 - Compare the models based on adjusted R-squared. Which would you choose?**
 - Compare the models using an F-test. Which would you choose?**
 - Compare the models using AIC/BIC. Which would you choose?**
 - Overall, think about which model you would choose based on your analysis. Did these comparison methods agree or disagree in terms of what was considered "best"?

Basic Syntax in R

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-r-for-programmers/modules/dsinf-basic-data-analysis/articles/regression-models-in-r>

This article will give a basic overview of some of the basic operations of R programming, including commenting, object assignment methods, naming rules, and math operations.

Comments

Before we talk about calculations and other functions in R, let's talk about *comments*. Comments are initiated by a `#` and include everything in the line following. You can use multiple comment markers in a line. This can be helpful to make comments more readable. You can even write a comment in the middle of a line as long as there is nothing after the `#` needed for the operation. For example:

```
## Using 2 or more markers might indicate separate sections of code
> print("Hello World")  #example of an inline comment
```

Output

```
"Hello World"
```

Math operations

R uses the `+`, `-`, `*`, and `/` signs to perform addition, subtraction, multiplication, and division, respectively. R follows the order of operations (parenthesis, exponents, multiplication, division, addition, then subtraction), so be sure to include parentheses when they are needed. For example:

```
25+7/4
```

```
(25+7)/4
```

Output:

```
26.75
8
```

We can see here that using parentheses around 25 and 7 told R to add these values together before dividing, rather than dividing 7 by 4 and then adding 25.

You can calculate exponents using `^`. For example:

```
5^2
2^3
```

Output:

```
25
8
```

Logical operators

Logical operators in R look like this:

```
> <      =
>= <=
==      !=
```

For example:

```
5 > 3
5 < 3
```

```
5 == 3
5 != 3
```

Output

```
TRUE
FALSE
FALSE
TRUE
```

We can join logical operators together using:

```
&      &&
|      ||
```

For example:

```
5 > 3 && 3 > 4
5 > 3 || 3 > 4
```

Output

```
FALSE
TRUE
```

The shorthand operators (`&` and `|`) and longhand operators (`&&` and `||`) function the same when evaluating individual values. However, when evaluating a vector or list of values, they behave differently. The shorthand operator returns `TRUE` or `FALSE` for **each** item in the vector (the output is a vector of `TRUE`s and `FALSE`s). The longhand operator returns a single `TRUE` or `FALSE` based on the **first** item in the list. For example, if `examp` is a vector with numbers 1 through 10:

```
examp < 5 & examp > 3 # checks if any values are less than 5 and greater than 3
examp < 5 && examp > 3 # checks if the first value (the number 1) is less than 5 and greater than 3
```

Output

```
FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE
FALSE
```

Multiple choice

Questions

Which of the following logical statements checks that every element in vector `vec1` is either less than or equal to 7 or greater than 13?

Answer Choices

```
vec1 <= 7 || vec1 > 13
vec1 < 7 | vec1 > 13
vec1 <= 7 | vec1 > 13
vec1 < 7 & vec1 > 13
```

Object assignment

In R, we *assign*, or name, an object using the assignment operator, an arrow sign (`<-`) made with a carat and a dash. For example, to create an object that was the number of minutes in a year, we would do:


```
minutes_in_year <- 525600
print(minutes_in_year)
```

Output:

```
525600
```

We can also use a single equals sign (=) to assign an object. This is used to define the parameters of a function. For example, `rnorm(1, mean = 5, sd = 2)` draws 1 random value from the normal distribution with mean 5 and standard deviation 2.

Both methods work, but it is best practice to use `<-` when naming an object and `=` when defining a parameter within a function.

You can directly assign a name to a math operation. For example, to both calculate and name the seconds in a year, you can write:

```
minutes_in_year <- 60*24*365
print(minutes_in_year)
```

Output:

```
525600
```

Coding question

Questions

Currently, the object `foot_to_centimeters` is assigned the value 0. Change the assignment of `foot_to_centimeters` from 0 to an operation that computes the number of centimeters in a foot (note: there are 2.54 centimeters in an inch and 12 inches in a foot).

Code

```
1
2
3
4
```

Run

Run your code to check your answer

Naming rules

The names of objects in R follow three rules:

```
minutes in year      minutes.in.year      minutes_in_year      minutesinyear
.      _
2times5              prod_2_5
```

```
days_in_week <- 7
five.div.seven <- 5/7
product 25 4 <- 25*4
```

Variable Types in R

This article will explain the different types of objects and variables used in R programming.

When an object is created, its contents will determine what type of object it is. Different object types have different functions and limitations.

Numeric

A *numeric* object is a number, with or without a decimal point. For example, `5`, `17.9`, and also an empty null value (`NA`) can all be numeric objects.

A non-numeric object can be made a numeric object with the `as.numeric()` function (assuming the object is only numbers). This function can be used on both individual elements and lists.

Character

A *character* object can be one or more letters, or a combination of letters, numbers, or punctuation, etc. When these objects are created, they need to be wrapped in either single quotes (`' '`) or double-quotes (`" "`). For example: `'A'`, `"ABC"`, and `"ABC3"` are all character objects.

Numeric objects can be turned into character objects with the `as.character()` function. This function can be used on both individual elements or lists.

Logical

A *logical* object can either be `TRUE` or `FALSE` with all capital letters and no quotation marks. Logical objects can also be just `T` or `F`, but it is best practice to use the full `TRUE` and `FALSE`.

Logical objects can be summarized like numeric objects. In a list of logical objects, you can calculate the total number of `TRUE` objects using `sum()` and the proportion of `TRUE` objects using `mean()`.

NA

An object with a name and no contents is filled with an `NA`. By default, these are logical objects, however they can be made into a numeric or character objects with the `as.numeric()` or `as.character()` functions.

These are case-sensitive, meaning null values are written as `NA` (`Na` will not work).

Vectors

Vectors are lists of objects that are all the same type. Vectors can be lists of numbers, characters, or logical objects. Vectors are created using `c()` with each object separated by a comma. For example, making a vector of whole numbers between 1 and 5 might look like:

```
one_five <- c(1, 2, 3, 4, 5)
one_five
class(one_five)
```

Output:

```
1 2 3 4 5
"numeric"
```

You can use `:` as shorthand for creating a vector that is a sequence of whole numbers. For example:

```
one_five_short <- 1:5

one_five_short
```

Output:

```
1 2 3 4 5
```

If you were to mix numeric and character objects in a single vector, the entire vector would default to a character vector. For example:

```
one_five <- c(1, 2, "three", 4, 5)

one_five
class(one_five)
```

Output:

```
"1"      "2"      "three" "4"      "5"
"character"
```

The quotation marks around each printed value means that these values are characters regardless of whether they are numbers or not. If you were to remove the `"three"` from this vector, the other values would still be characters. If `"three"` were removed, you could use the `as.numeric()` function to change the variable type.

Coding question

Questions

There is a soccer player who is taking 12 penalty kicks. Let `goals` be a logical vector that represents the result of each kick, where `TRUE` means a goal is scored and `FALSE` means there wasn't a goal.

Calculate the total number of goals scored, save it to the object `total_goals`, and then print `total_goals`.

Then calculate the proportion of goals scored, save it to the object `prop_goals`, and then print `prop_goals`.

Check variable types

The function `class()` is helpful to check the variable type of a given object. You can use this function on both individual elements or vectors. For example:

```
# individual elements
class(2000) #numeric
class(NA)  #logical
```

```
class("apple") #character

# vectors
results <- c(F, T, T, F)
class(results) #logical
class(c(19, 21, 25, 30)) #numeric
class(c('Hello', 'world')) #character
```

One way you can use the outcome of the `class()` function is in a logical statement.

Vectors in R

Learn how to initiate, view the properties of, subset, and replace elements of vectors in R programming.

A vector contains objects that are all the same variable type. Vectors are the most basic data structure in R.

Initiating a vector

Vectors can be lists of numbers, character objects, or logical objects. Vectors are initialized using `c()` with each object separated by a comma. For example, making a vector of the number of customers on each day of a given week might look like:

```
customers <- c(47, 40, 38, 42, 56, 72, 70)
customers
```

Output

```
47 40 38 42 56 72 70
```

The following functions return different properties of a vector:

```
length()
class()
min()    max()
range()
```

```
length(customers)
class(customers)
range(customers)
```

Output:

```
7 #number of elements in the vector
"numeric" #type of elements
38 72 #minimum and maximum
```

Subsetting within a vector

We can select a particular section of a vector using square brackets (`[]`) directly after the vector name. For example:

```
customers[2]
customers[2:4]
customers[c(2,4)]
customers[3:length(customers)]
```

The `which()` function returns the index (or indexes) of which elements meet the given criteria. We can call the `which()` function within the brackets (`[ ]`) to select the elements that meet the criteria. For example, using the same `customers` vector, we can get the indexes of which days had more than 45 customers:

```
# Print original customers vector
customers <- c(47, 40, 38, 42, 56, 72, 70)
customers

# Print indexes of days with more than 45 customers
which(customers > 45)

# Print the value of each day with more than 45 customers
customers[which(customers > 45)]
```

Output:

```
47 40 38 42 56 72 70
1 5 6 7
47 56 72 70
```

Replacing elements of a vector

We can change a specific element in a vector by assigning a new value to that selected element. For example, if we wanted to update the third element in `customers` to 25:

```
customers <- c(47, 40, 38, 42, 56, 72, 70)
customers
customers[3] <- 25
customers
```

Output:

```
47 40 38 42 56 72 70
47 40 25 42 56 72 70
```

We can use the `which()` function to select specific elements to replace based on specific criteria. For example, if we wanted to replace all the values under 50 in `customers` with 0:

```
# Print customers
customers

# Change elements less than 50 to 0
customers[which(customers < 50)] <- 0
customers
```

Output:

```
47 40 25 42 56 72 70
0 0 0 0 56 72 70
```

Practice space

Coding question

Questions

Here are a few vectors (`vec1`, `vec2`, and `vec3`) for us to practice some of these methods. Some things we can try are:

Feel free to try out making new vectors as well. Try to get comfortable working with vectors!

Matrices in R

Learn how to initiate, subset, and perform linear algebra operations on matrices in R programming.

A *matrix* is a two-dimensional structure that holds elements of the same type. A matrix can be thought of as a collection of rows and columns. In R, we index matrices as rows then columns.

Initiating a matrix

We can initiate a matrix using the `matrix()` function with parameters:

```
data      NA      1      1
nrow
ncol
```

If `data` is a single value, the matrix will simply be a matrix of that value. For example:

```
ones <- matrix(data = 1, nrow = 2, ncol = 3)
ones
```

Output:

```
      [,1] [,2] [,3]
[1,]    1    1    1
[2,]    1    1    1
```

If `data` is a vector, the default process for matrices is to fill down columns. For example:

```
a_b <- matrix(data = c("a", "b"), nrow = 2, ncol = 3)
a_b
```

Output

```
      [,1] [,2] [,3]
[1,] "a"  "a"  "a"
[2,] "b"  "b"  "b"
```

We can see here that the first column has the vector values of `a` then `b`. This vector repeats in the next open space, which in this case is the first value of the second column. If the vector in `data` does not evenly fit down a column, the data will repeat itself until all spots are filled. For example:

```
abc <- matrix(data = c("a", "b", "c"), nrow = 4, ncol = 3)
abc
```

Output

```
      [,1] [,2] [,3]
[1,] "a"  "b"  "c"
[2,] "b"  "c"  "a"
[3,] "c"  "a"  "b"
[4,] "a"  "b"  "c"
```

If we want our matrix to fill across rows rather than down columns, we can set `byrow = TRUE`. For example:

```
abc_row <- matrix(data = c("a", "b", "c"), nrow = 4, ncol = 3, byrow = TRUE)
abc_row
```

Output

```
      [,1] [,2] [,3]
[1,] "a"  "b"  "c"
[2,] "a"  "b"  "c"
[3,] "a"  "b"  "c"
[4,] "a"  "b"  "c"
```

See here that `"a"`, `"b"`, and `"c"` fill across rows rather than down columns.

We can check the dimensions of a matrix using `dim()`, which returns the number of rows followed by the number of columns.

```
dim(abc_row)
```

Output:

```
[1] 4 3
```

Identity/diagonal matrices

We can create an identity matrix using the `diag(x)` function where `x` is the number of elements along the diagonal. `x` could also be a vector, and the resulting matrix would contain the values of the vector along the diagonal.

```
# Identity matrix of size 3
diag(3)
```

```
# Diagonal matrix
diag(c(4,3,2))
```

Output

```
      [,1] [,2] [,3]
[1,] 1    0    0
[2,] 0    1    0
```

```
[3,]    0    0    1

      [,1] [,2] [,3]
[1,]    4    0    0
[2,]    0    3    0
[3,]    0    0    2
```

Subsetting within a matrix

Matrices are indexed using brackets with a comma `[x,y]`, where `x` refers to the row(s) selected and `y` refers to the column(s) selected. For example, `test[4,6]` will call the value that is in the 4th row and the 6th column of the `test` matrix.

We can select multiple rows or columns using the following:

```
# select the 4th through 6th rows and the 1st through 3rd columns
test[4:6,1:3]

# select the 4th through 6th rows and ALL of the columns of the matrix
test[4:6,]

# select the 4th AND 6th rows (skipping the 5th) and the 1st through 3rd columns
test[c(4,6),1:3]

# select all rows of the 2nd AND 5th columns
test[,c(2,5)]
```

Linear algebra operations

Let's say we had a 3-by-3 numeric matrix `mat`. The following code demonstrates performing different linear algebra operations on `mat`:

```
## Transpose
t(mat)

## Inverse
solve(mat)

## Matrix multiplication
mat %*% t(mat)

## Eigenvalues and Eigenvectors
eig <- eigen(mat)
# eig$val are the eigenvalues
# eig$vec are the eigenvectors
```

Practice space

Coding question

Questions

Here are two matrices (`mat1` and `mat2`) for us to practice some of these methods. Some things we can try are:

- print a matrix
- check the dimensions of the matrix
- subset to certain rows or columns
- replace parts of a matrix
- logical operations
- mathematical operations

Feel free to try out making new matrices as well. Try to get comfortable working with matrices!

Lists in R

Learn how to work with lists in R including how to initiate a list, how to add or remove objects, and how to modify objects.

Lists are special data structures in R because lists can be filled with objects of different variable types. Objects in lists can be individual objects, vectors, matrices, data frames (we will describe shortly), and even other lists.

Initializing a list

Lists are initiated using the `list(x, y, z)` function where `x`, `y`, and `z` are the different objects in the list. We can see the structure of a list using `str()`

```
> ## Initiate list1
> list1 <- list(5, "b", c(0.5, 0.25), list(1,1,0))
> str(list1)
List of 4
 $ : num 5
 $ : chr "b"
 $ : num [1:2] 0.5 0.25
 $ :List of 3
  ..$ : num 1
  ..$ : num 1
  ..$ : num 0
```

We can get the contents of an object in a list using double-brackets `[[a]]` where `a` is the index of the desired object. If we were to use single brackets `[a]`, R would return the whole object at index `a` rather than the contents of the object.

```
> ## Call object 3 with single brackets
> list1[3]
[[1]]
[1] 0.50 0.25

> ## Call just the contents of object 3 with double-brackets
> list1[[3]]
[1] 0.50 0.25
```

We can also give names to each object in the list. This would allow the contents of an object to be called more easily by just using `$`.

```
> ## Initiate list using names for objects
> list2 <- list(a = 5, b = "b", c = c(0.5, 0.25), d = list(day1 = 1,
                                                         day2 = 1,
                                                         day3 = 0))

> ## View structure of list2
> str(list2)
List of 4
 $ a: num 5
 $ b: chr "b"
 $ c: num [1:2] 0.5 0.25
 $ d:List of 3
  ..$ day1: num 1
  ..$ day2: num 1
  ..$ day3: num 0

> ## Print object named c from list2
```

```
> list2$c
[1] 0.50 0.25
```

Naming objects in a list can be tedious for lists with lots of objects. But sometimes it can be a useful tool.

Note that the fourth object in `list1` and `list2` is a list within a list or a *nested list*. Also, each object in nested list `d` is named. Accessing data in nested lists follows the pattern of accessing data in a regular list.

```
> ## Call unnamed object from a nested list by index numbers
> list1[[4]][[2]]
[1] 1

> ## Call named object from a nested list by names
> list2$d$day2
[1] 1
```

This is when naming objects in a list may be convenient; having multiple sets of double-brackets can be hard to interpret sometimes.

Adding and removing from a list

To add a 5th object to an existing list of 4 objects, we can do:

```
> ## Add an unnamed object
> list1[[5]] <- 45

> ## Add a named object
> list2$e <- 45
> print(list2)
$a
[1] 5

$b
[1] "b"

$c
[1] 0.50 0.25

$d
$d$day1
[1] 1

$d$day2
[1] 1

$d$day3
[1] 0

$e
[1] 45
```

Removing an object is as simple as assigning the value `NULL` to an object:

```
> ## Remove an unnamed object
> list1[[5]] <- NULL

> ## Remove a named object
> list2$e <- NULL
> print(list2)
```

```
$a
[1] 5

$b
[1] "b"

$c
[1] 0.50 0.25

$d
$d$day1
[1] 1

$d$day2
[1] 1

$d$day3
[1] 0
```

Changing objects in a list

We can make a change to an object in a list by calling the specific object and using the assignment sign `<-`. The following code shows how to make changes to either named or unnamed objects in a list:

```
> ## Change unnamed objects
> list1[[2]] <- "q"

> ## Change named objects
> list2$b <- "q"
> print(list2)
$a
[1] 5

$b
[1] "q"

$c
[1] 0.50 0.25

$d
$d$day1
[1] 1

$d$day2
[1] 1

$d$day3
[1] 0
```

We can also change an element of an object in a list (e.g., change one value of a vector that is an object in a list). For example, if we wanted to change the second value of the third object (and do nothing to the other values), we could use the following code:

```
> ## Unnamed object
> list1[[3]][2] <- 1

> ## Named object
> list2$c[2] <- 1
> print(list2)
$a
[1] 5
```

```
$b
[1] "q"

$c
[1] 0.5 1.0

$d
$d$day1
[1] 1

$d$day2
[1] 1

$d$day3
[1] 0
```

Data Frames in R

Learn how to work with data frames in R including how to initiate a data frame, subset data, add and remove rows or columns, and modify specific values.

A data frame is like a spreadsheet: a table made up of rows and named columns. Each column can be thought of as a vector because it contains data of one variable type. Data frames are great in R because they are easy to manipulate and extract information from.

Initiating a data frame

Data frames are the default data structure when you read in a csv file using `read.csv()`. This function will automatically use the first row of data as the column names unless otherwise specified.

We can initiate data frames for data we create using the `data.frame()` function:

```
> # Initiate data frame, dat
> dat <- data.frame(col1 = c(1:4),
                    col2 = c("a", "b", "c", "d"),
                    col3 = 1)

> dat
  col1 col2 col3
1     1    a     1
2     2    b     1
3     3    c     1
4     4    d     1
```

```
colnames()
```

```
> colnames(dat)
[1] "col1" "col2" "col3"

> colnames(dat) <- c("newname1", "newname2", "newname3")
> colnames(dat)
[1] "newname1" "newname2" "newname3"
```

Subsetting a data frame

Individual rows, columns, and cells

Data frames are structured as rows and columns, so one way to access information in a data frame is by using the syntax `dat[x,y]`, where `x` refers to a row index and `y` refers to a column index.

```
dat[x,y] calls the cell that is in the x-th row and y-th column
dat[,y] calls the entire y-th column
dat[x,] calls the entire x-th row
```

Columns can also be called using the column name: `dat$col1`.

```
> # Call the 2nd row
> dat[2,]
  col1 col2 col3
2     2     b     1

> # Call the 3rd column
> dat[,3]
[1] 1 1 1 1

> dat$col3
[1] 1 1 1 1
```

Note that when we call entire rows, each object still has the name of the column it came from. This property means that there are multiple ways we can call specific cells:

```
> # Use only index
> dat[2,3]
[1] 1

> # Use column name and item index
> dat$col3[2]
[1] 1

> dat[2,]$col3
[1] 1
```

Multiple rows or columns

When using the `dat[x,y]` method, either `x` or `y` could be a vector or series of values. Doing so would call multiple rows or columns. We can also use the column names to select multiple columns. However, this method can become tedious if we are calling many columns.

```
> # Call 1st through 3rd rows
> dat[1:3,]
  col1 col2 col3
1     1     a     1
2     2     b     1
3     3     c     1

> # Call 1st and 3rd columns
> dat[,c(1,3)]
  col1 col3
1     1     1
2     2     1
3     3     1
4     4     1
```

```
> # Call 1st and 3rd columns using names
> dat[,c("col1", "col3")]
  col1 col3
1     1     1
2     2     1
3     3     1
4     4     1
```

Conditional selection

We can use the `which()` function to call specific rows that meet some logical criteria.

```
> # Call rows that have col1 > 2
> dat[which(dat$col1 > 2),]
  col1 col2 col3
3     3     c     1
4     4     d     1
>
> # Call rows that have col2 == "b" or "d"
> dat[which(dat$col2 == "b" | dat$col2 == "d"),]
  col1 col2 col3
2     2     b     1
4     4     d     1
```

Adding data to a data frame

Adding columns

Once a data frame is created, we can add a column of data by calling the name of the new column using the `$` method. Then we assign the contents of the column. The contents of the new column must be in one of the following shapes:

- a single value that will flood through the whole column
- a vector whose length is the same as the number of rows in the data frame
- a vector whose length is a factor of the number of rows in the data frame. This vector will repeat through the column.

```
> # Single value
> dat$col4 <- "b"

> # Vector whose length equals the number of rows
> dat$col5 <- c("e", "f", "g", "h")

> # Vector whose length is a factor of the number of rows
> dat$col6 <- c(1, 2)

> # Print data frame
> dat
  col1 col2 col3 col4 col5 col6
1     1     a     1     b     e     1
2     2     b     1     b     f     2
3     3     c     1     b     g     1
4     4     d     1     b     h     2
```

Adding rows

We can manually append rows to a data frame using `rbind(data_frame, new_rows)`. Similar to adding columns to a data frame, the contents must be in one of the following shapes:

a single value that will fill across the entire row
 a vector whose length is the same as the number of columns in the data frame
 a vector whose length is a factor of the number of columns in the data frame. This vector will repeat itself through the row.

```
> # Single value that will fill across the entire row
> dat <- rbind(dat, 1)

> # Vector whose length equals the number of columns
> dat <- rbind(dat, c(5, "a", 3, "e", 1, "i"))

> # Vector whose length is a factor of the number of columns
> dat <- rbind(dat, c("x", "z"))

> # Print data frame
> dat
```

	col1	col2	col3	col4	col5	col6
1	1	a	1	b	e	1
2	2	b	1	b	f	2
3	3	c	1	b	g	1
4	4	d	1	b	h	2
5	1	1	1	1	1	1
6	5	a	3	e	1	i
7	x	z	x	z	x	z

Modifying specific values

We can modify existing data the same way we can add data in data frames: we call the row(s), column(s), or cell(s) we want to change and assign a new value.

```
> # Change the whole 3rd column to equal "!"
> dat[,3] <- "!"

> # Change the 4th and 5th rows to equal "?"
> dat[4:5,] <- "?"

> # Change the bottom right value to equal "%"
> dat$col6[7] <- "%"

> # Print dat
> dat
```

	col1	col2	col3	col4	col5	col6
1	1	a	!	b	e	1
2	2	b	!	b	f	2
3	3	c	!	b	g	1
4	?	?	?	?	?	?
5	?	?	?	?	?	?
6	5	a	!	e	1	i
7	x	z	!	z	x	%

```
which()
```

```
> # Change any rows that have "a" in col2 to "+"
> dat[which(dat$col2 == "a"),] <- "+"

> # Print dat
> dat
```

	col1	col2	col3	col4	col5	col6
1	+	+	+	+	+	+

```
2      2      b      !      b      f      2
3      3      c      !      b      g      1
4      ?      ?      ?      ?      ?      ?
5      ?      ?      ?      ?      ?      ?
6      +      +      +      +      +      +
7      x      z      !      z      x      %
```

Removing data

There are a few methods we can use to remove rows or columns from data sets. The basic idea is that we are reassigning the data frame using a selected subset of the data.

Specify rows or columns to keep

This method is convenient when there are not many rows or columns to keep, or if the rows or columns we want to remove are together in a sequence.

```
dat <- dat[1:3,] only keeps rows 1 through 3.
dat <- dat[,c("col2", "col4")] only keeps columns 2 and 4. Alternatively, dat <- dat[,c(2,4)]
```

Specify rows or columns to remove

This method is convenient when there are not many rows or columns to remove, or if the rows or columns we want to remove are together in a sequence.

```
dat <- dat[-c(1,3,6),] removes rows 1, 3, and 6.
dat <- dat[, -c(2:5)] removes columns 2 through 5.
```

Specify rows using a logical statement

This method is convenient when the rows we want to remove share a common characteristic.

```
dat <- dat[which(dat$col3 == "!"),] keeps only rows with "!" in col3.
dat <- dat[-which(dat$col3 != "!"),] does the same, but by REMOVING the rows without a "!" in col3.
```

Practice space

Coding question

Questions

Here is a data frame ([dat](#)) with data about a certain class for us to practice some of these methods. Some things we can try are:

- check the number of rows and columns
- check and/or change the names of columns
- subset to certain rows, columns, or cells
- use logical criteria to select certain rows
- add or remove rows or columns to the data frame

Feel free to try out making new data frames as well. Try to get comfortable working with data frames!

Control Flow and Loops in R

Learn the rules and syntax for manipulating sections of R scripts using `if` statements, `else` statements, `for` loops, and `while` loops.

Control Flow

In R, scripts are read line-by-line from top to bottom. We can control which lines are run or skipped by implementing conditional statements throughout the script.

If, else if, and else statements

The syntax for a basic `if` statement is as follows:

```
if (logical_statement) {  
  code to execute  
}
```

The `logical_statement` should either be an object that has been assigned the value `TRUE` or `FALSE`, or it should be a statement that returns `TRUE` or `FALSE`. Statements that return `TRUE` or `FALSE` use logical operators such as `<`, `>`, `<=`, `>=`, `==`, and `!=`. We can combine multiple operators using `|` as “or” and `&` as “and”:

```
a <- 10  
  
# print a if it is either less than 5 or greater than 9  
if (a < 5 | a > 9) {  
  print(a)  
}
```

Output

```
10
```

We can see that because `a` is 10 which is greater than 9, `a` is printed.

If we want certain code to run if a condition is met and other code to run if it is not met, we can add an `else` statement after `if`:

```
a <- 10  
  
if (a >= 8) {  
  print("greater than or equal to 8")  
} else {  
  print("less than 8")  
}
```

Output

```
"greater than or equal to 8"
```

Multiple choice

Questions

Take a moment to read through this code that includes `if` and `else`.

`amount_per_student` will equal the `funding` number divided by `number_of_students`. If that value is at least 500, then we print one statement. If not, then we print another.

Do the division on your own to find the value of `amount_per_student`. Which statement will be printed in the end?

Code

```
number_of_students <- 88
funding <- 30000
amount_per_student <- funding / number_of_students

if (amount_per_student >= 500) {
  print("We have enough funding for each student!")
} else {
  print("We're lacking funds this year.")
}
```

Answer Choices

"We're lacking funds this year."

"We have enough funding for each student!"

If we have more than 2 cases, we can add `else if` statements:

```
a <- 15

if (a <= 5) {
  print("small")
} else if (a > 5 & a <=10) {
  print("medium")
} else if (a > 10 & a <= 15) {
  print("large")
} else {
  print("extra large")
}
```

Output:

"large"

Nested statements

If there are sub-conditions within a conditional statement, we can nest an `if` statement within another `if` statement:

```
a <- 10
# first condition
if (a >= 5) {
```

```

# second condition
if (a <= 8) {
  # code to run if first and second conditions are TRUE
  print("a is between 5 and 8")
# third condition
} else if (a <= 11) {
  # code to run if first and third conditions are TRUE
  print("a is between 9 and 11")
} else {
  # code to run if only first condition is TRUE
  print("a is greater than 11")
}
}

```

Output

```
"a is between 9 and 11"
```

While this would have the same logic as a list of `else if` statements, it can sometimes simplify syntax and be easier to follow code if we nest statements.

Fill in the blank

Questions

Building on the same example, let's say we again first want to check if `amount_per_student` is at least 500. If it is, we want it to print "We have enough funding for each student!" and calculate a `leftover` value. If `leftover` is at least 2000, we want the code to print the statement "We can update the AV equipment this year!". If not, we'll print "Let's save for AV equipment next year".

Pick the right keywords for each conditional!

Code

```

number_of_students <- 88
funding <- 30000
amount_per_student <- funding / number_of_students

```

blank 1

```

(amount_per_student >= 500) {
  print("We have enough funding for each student!")
  leftover = funding - (number_of_students * 500)
}

```

blank 2

```

(leftover >= 2000) {
  print("We can also update the AV equipment this year.")
}

```

blank 3

```

{
  print("Let's save for AV equipment next year.")
}
} else {
  print("We're lacking funds this year.")
}

```

```
}
```

Answer Choices

check
else if
else
check
if
for
if

Click or drag and drop to fill in the blank

Single-line if-else statement

The shorthand function for simple if-else statements is `ifelse()`. This function is particularly handy when the conditions are straightforward and the resulting output is simple. For example:

```
a <- 8

# ifelse(logical_statement, if_true, if_false)
group_size <- ifelse(a >= 6, "large", "small")
group_size
```

Output:

```
"large"
```

If the logical statement of a single-line statement uses a vector, the function will run element-wise and return an object of the same length of the vector:

```
# Vector of scores
score <- c(2, 6, 10, 12, 4, 8)

# Return 1 if greater than 5, 0 otherwise
greater_than_5 <- ifelse(score > 5, 1, 0)

print(greater_than_5)
```

Output

```
0 1 1 1 0 1
```

Loops

Like other programming languages, loops will repeat through a section of code. The amount of times the code is repeated is either predetermined — a *for loop* — or until a condition is met — a *while loop*.

For loops

The syntax for a `for` loop includes the *iterator*, which we will call `value`, and a vector to count through. The number of objects in the vector is how many times the loop will repeat. For example, if we wanted to print a statement 5 times with the numbers increasing by 1 each time, we could use:

```
for (value in 1:5) {  
  # code to execute  
  print(paste(value, "is greater than", value-1))  
}
```

Output

```
[1] "1 is greater than 0"  
[1] "2 is greater than 1"  
[1] "3 is greater than 2"  
[1] "4 is greater than 3"  
[1] "5 is greater than 4"
```

We can see here that `value` started at 1 and increased to 5, as set in the code. Note that `value` automatically increased by 1 at the end of the code to execute.

Coding question

Questions

Let's say we want to print "Hello" 10 times without having 10 lines of `print("Hello")`.

Run the code below. Here, the variable `i` is the placeholder for whatever the current count is between 1 and 10. On the first iteration the variable `i` is equal to 1. Then it's equal to 2. At the last iteration, `i` is equal to 10. The print statement is the same at each iteration.

In the code editor, change the number of iterations from 10 to 20. Click Run to see the result this time.

Tibbles and dplyr

Learn about the data structure of the tidyverse — tibbles — and dplyr data handling operations.

The *tidyverse* is a collection of packages in R that aid with different aspects of data science. There are many operations that are easier in the tidyverse than in base R, and vice versa. Being comfortable in both base R and the tidyverse will drastically expand our ability to handle data.

Tibbles

The main data structure in the tidyverse is what's called a *tibble*. Tibbles are like data frames: a collection of rows and named columns. Tibbles and data frames are relatively interchangeable; there are very few functions that will only work for data frames or only tibbles. A data frame is converted into a tibble by using the `as_tibble()` function, and a tibble can be converted into a data frame using the `as.data.frame()` function. A data frame will also be converted to a tibble if a function from the dplyr package is used on it.

The dplyr package

We can think of dplyr as a grammar of data manipulation. The dplyr package is part of the tidyverse and allows us to handle data using five main functions. We will use an example tibble called `tib` to explore how these functions work:

```
names      test1 test2 quiz1 quiz2  
<chr>      <dbl> <dbl> <dbl> <dbl>
```

```

1 Michael      87      85      10      8
2 Jae-jin      92      87       9      9
3 Molly        73      82      10     10
4 Harvey       85      90       8      8
5 Mia          95      92      10      8
6 Kylee        86      90       7      8
7 Dhee         89      82       9      9
8 Peter        80      84       8     10
9 Cate         75      86       9      9
10 Lauren      91      88       8      9

```

Mutate

The `mutate()` function creates a new column in the tibble. The data in this new column can be a set vector of data or some function of an existing variable. A helpful function to use with `mutate()` is `case_when()`, which acts like an if-else statement.

```

mutate(tib,
  # create age column
  age = c(20, 21, 22, 19, 20, 22, 19, 18, 21, 20),
  # create test1_letter column
  test1_letter = case_when(
    # assign letter based on test1 score
    test1 < 80 ~ "C",
    test1 >= 80 & test1 < 90 ~ "B",
    test1 >= 90 ~ "A"))

```

Output:

```

A tibble: 10 x 7
  names      test1 test2 quiz1 quiz2   age test1_letter
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 Michael      87      85      10      8     20 B
2 Jae-jin      92      87       9      9     21 A
3 Molly        73      82      10     10     22 C
4 Harvey       85      90       8      8     19 B
5 Mia          95      92      10      8     20 A
6 Kylee        86      90       7      8     22 B
7 Dhee         89      82       9      9     19 B
8 Peter        80      84       8     10     18 B
9 Cate         75      86       9      9     21 C
10 Lauren      91      88       8      9     20 A

```

The `mutate()` operation runs line-by-line. This means that if we created a column in the `mutate()` operation, we could immediately use the data in that column to create another column.

Select

The `select()` function subsets to specific columns by name. It also allows you to easily remove columns from a tibble.

```

# select only quiz1 and quiz2 columns from tib
select(tib, quiz1, quiz2)

```

Output:

```

A tibble: 10 x 2
  quiz1 quiz2
  <dbl> <dbl>
1     10     8
2      9     9
3     10    10
4      8     8
5     10     8

```

```
6      7      8
7      9      9
8      8     10
9      9      9
10     8      9
```

Select everything except `names` and `quiz1` from `tib`:

```
select(tib, -names, -quiz1)
```

Output:

```
A tibble: 10 x 3
  test1 test2 quiz2
  <dbl> <dbl> <dbl>
1     87    85     8
2     92    87     9
3     73    82    10
4     85    90     8
5     95    92     8
6     86    90     8
7     89    82     9
8     80    84    10
9     75    86     9
10    91    88     9
```

Filter

The `filter()` function subsets to specific rows by the values in a column. For example, to filter to only students who scored over 90 on test 1:

```
# filter tib to only rows where test1 > 90
filter(tib, test1 > 90)
```

Output:

```
A tibble: 3 x 5
  names test1 test2 quiz1 quiz2
  <chr> <dbl> <dbl> <dbl> <dbl>
1 Jae-jin 92    87     9     9
2 Mia     95    92    10     8
3 Lauren  91    88     8     9
```

Summarise

The `summarise()` function reduces one or more variables to a summary value. Common summary values are mean, median, standard deviation, minimum, maximum, etc.

```
# create a new tibble that has a column for the average of all of
# quiz1 and the standard deviation of all of quiz1
summarise(tib, quiz1_avg = mean(quiz1), quiz1_sd = sd(quiz1))
```

Output:

```
A tibble: 1 x 2
  quiz1_avg quiz1_sd
  <dbl>    <dbl>
1     8.8     1.03
```

Coding question

Questions

We have filtered our store items down to only expensive items in a tibble called `store`. We have also created a new tibble called `store_summary` that has summarized the number of expensive items.

Add to the existing code to collect more summary statistics about `store`. Specifically:

```
    price
price
price
```

After adding these summary statistics, print `store_summary`.

Code

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Run

Run your code to check your answer

Arrange

The `arrange()` function changes the order of the rows in a tibble based on the row's value in a selected column. Default is in ascending order (smallest to largest value), but this can be changed using `desc(column_name)`.

```
# arrange tib from highest to lowest scores of test1
arrange(tib, desc(test1))
```

Output:

```
A tibble: 10 x 5
  names    test1 test2 quiz1 quiz2
  <chr>    <dbl> <dbl> <dbl> <dbl>
1 Mia      95     92     10     8
```


2	Jae-jin	92	87	9	9
3	Lauren	91	88	8	9
4	Dhee	89	82	9	9
5	Michael	87	85	10	8
6	Kylee	86	90	7	8
7	Harvey	85	90	8	8
8	Peter	80	84	8	10
9	Cate	75	86	9	9
10	Molly	73	82	10	10

Piping

The tidyverse utilizes the pipe operator `%>%` to string multiple functions together. The pipe operator takes the tibble output from one function and inputs it as the first argument in the next function. This allows us to create more intricate operations without having to save intermittent steps.

For example, if we wanted to get the average scores for tests 1 and 2 and then get the letter value for both test averages, we could pipe a `summarize()` operation and a `mutate()` operation. Then if we wanted to put the columns together in a specific order—test 1 average, test 1 letter, test 2 average, test 2 letter—we could pipe an additional `select()` operation:

```
tib %>%
  # make a new tibble that has a column for the
  # averages of test1 and test2
  summarize(avg_test1 = mean(test1), avg_test2 = mean(test2)) %>%
  # add new columns test1_letter and test2_letter
  mutate(
    # assign test1_letter based on test1 average score
    test1_letter = case_when(
      avg_test1 < 80 ~ "C",
      avg_test1 >= 80 & avg_test1 < 90 ~ "B",
      avg_test1 >= 90 ~ "A"),
    # assign test2_letter based on test2 average score
    test2_letter = case_when(
      avg_test2 < 80 ~ "C",
      avg_test2 >= 80 & avg_test2 < 90 ~ "B",
      avg_test2 >= 90 ~ "A")) %>%
  # reorder the columns to be by test1 then test2
  select(avg_test1, test1_letter, avg_test2, test2_letter)
```

Output:

```
A tibble: 1 x 4
  avg_test1 test1_letter avg_test2 test2_letter
  <dbl> <chr>          <dbl> <chr>
1    85.3 B              86.6 B
```

Note that for each operation — `summarize()`, `mutate()`, and `select()` — we did not call the tibble `tib` as the first parameter. This is because when we use pipes, we call the tibble initially in the first line and then do not have to after that.

Coding question

Questions

We're going to combine multiple tasks to start with `store` and end with `store_summary` using piping `%>%` to string tasks together.

1 Use `mutate()` to create a new variable `category` that is determined by each price:

```
less than $30 is "cheap"
between $30 and $50 is "low moderate"
between $50 and $75 is "high moderate"
greater than $75 is "expensive"
```

2 Use `filter()` to select only items that are “expensive”

3 Use `summarize()` to create some summary statistics. Specifically:

items: the number of rows in the dataset
minimum: the smallest value in price
mean: the mean of price
maximum: the largest value in price

Functions in R programming

Learn how to create and use functions in R.

Global environment functions

In R programming, we can define custom functions in the global environment using the `function()` command. The following is generic code for a function called `our_function`:

```
our_function <- function(optional_parameters){code_to_execute}
```

`optional_parameters` can be anything we would want to input to the function. Let's say we might use the same function with different datasets `dat1` and `dat2`. We can make a parameter of the function to be `input_data` and then when we call the function we specify a dataset `dat1`:

```
our_function(input_data = dat1)
```

Doing this will replace `input_data` with `dat1` at all occurrences in the `code_to_execute`.

`code_to_execute` is as it sounds: it is the code that will execute when the function is called. Everything between the opening brace `{` and the closing brace `}` will execute.

Let's look at a full function. Say we have a vector of temperatures in degrees Celsius called `test_temp`:

```
test_temp
```

Output:

```
[1] 16.550540 26.942005 32.591088 8.580116 28.003873  
[6] 28.542391 20.976820 21.173577 21.048836 18.769735
```

If we wanted a function that would convert `test_temp` from degrees Celsius to Fahrenheit, we could write the following function:

```
celsius_to_fahrenheit <- function(dat){  
  temp_far <- dat*(9/5) + 32  
  
  return(temp_far)  
}
```

In this function, `dat` is the parameter required to make the function run. We use the `return()` statement to specify the object we would like to get from the function. We can then call `celsius_to_fahrenheit()` on our data `test_temp` to convert the data to degrees Fahrenheit:

```
dat_far <- celsius_to_fahrenheit(test_temp)  
dat_far
```

Output:

```
[1] 61.79097 80.49561 90.66396 47.44421 82.40697 83.37630  
[7] 69.75828 70.11244 69.88790 65.78552
```

In R, custom functions can only return one object. If we wanted to return multiple objects, we would have to combine them into a single object. This single object could be a single dataframe or tibble if the different objects have compatible dimensions. Another option could be to save multiple objects as separate items in a single list.

Fill in the blank

Questions

Now let's say we want a function called `meters_to_feet` that takes a number of meters and converts it to the equivalent number of feet. Then we want to call the function to convert 80 meters to feet.

Code

```
blank 1
<-
blank 2
(meters) {
  feet <-
blank 3
  * 3.28084

blank 4
(feet)
}

meters_to_feet(
blank 5
)
```

Answer Choices

print
define
{
function
output
meters_to_feet
}
80
feet_to_meters
meters
return

Click or drag and drop to fill in the blank

apply()

The `apply()` function is a kind of shortcut to custom functions. As given in its name, it applies a function to either each row or column of a dataframe or matrix.

The syntax of `apply()` is:

```
apply(X, MARGIN, FUN)
```

The `MARGIN` specification determines whether the function `FUN` is applied across the rows (1) or down the columns (2) of the object `X`.

Let's try this on the following matrix:

```
test_mat
```

Output:

```
      [,1] [,2] [,3]
[1,]    1    1    1
[2,]    2    2    2
```

If we were to sum across the rows, we would get 3 and 6. If we were to sum down the columns, we would get 3, 3, and 3. Let's try this using `apply()`.

Apply the `sum()` function over each row (margin value 1):

```
apply(X = test_mat, MARGIN = 1, FUN = sum)
```

Output:

```
[1] 3 6
```

Apply the `sum()` function down each column (margin value 2):

```
apply(X = test_mat, MARGIN = 2, FUN = sum)
```

Output:

```
[1] 3 3 3
```

The functions we can use in `apply()` are `mean`, `median`, `sum`, `min`, or `max` as well as other existing or user-defined functions. Let's say we had a dataframe `dat` where each column represents a test or quiz and each row represents a student in our class.

`dat`

Output:

	test1	test2	quiz1	quiz2
1	87	85	10	8
2	92	87	9	9
3	73	82	10	10
4	85	90	8	8
5	95	92	10	8
6	86	90	7	8
7	89	82	9	9
8	80	84	8	10
9	75	86	9	9
10	91	88	8	9

Let's say we want to know the maximum score of each test or quiz. Without using `apply()`, this could be tedious because we would have to call each column separately and check the maximum value. `apply()` can streamline this process, and we can even drop the names of the arguments as long as we enter them in order.

Use `apply()` to get the maximum of each column:

```
apply(dat, 2, max)
```

Output:

test1	test2	quiz1	quiz2
95	92	10	10

See here that we were able to extract the maximum value of each column in one command rather than 4 separate lines of code.

Summary Statistics in R

[Learn how to run summary statistics in R.](#)

In R programming, there are many built-in functions that provide useful summary statistics.

Let's say we have a vector `sales` with 30 values that represent the number of sales we made on each day over a month at our pest control company.

```
> sales
[1] 2 6 5 6 7 6 1 3 6 5
[11] 6 5 4 8 4 7 4 4 3 3
[21] 4 4 3 1 3 7 5 8 7 2
```

We want to understand our data better, so we will calculate some summary statistics on the data.

Averages

Some commonly used averages we can calculate for a set of values are mean and median. R makes it easy to calculate these averages.

```
> # Calculate mean
> mean(sales)
[1] 4.633333

> # Calculate median
> median(sales)
[1] 4.5
```

Note for both of these functions, we simply entered the name of the numeric vector into the function. If we had a dataframe instead of a vector, we would have to specify which column we would like to run the function on.

Coding question

Questions

Let's say we have a dataframe `soccer` of data about our soccer team. We have columns `goals_scored` and `goals_allowed` representing the score of each game.

We want to know the mean and median number of goals our team scored during the season. Create the objects `mean_scored` and `median_scored` and assign them the mean and median of the number of goals scored. Then print the objects.

Code

1
2
3
4
5
6
7
8

Run

Run your code to check your answer

Range and spread

Maybe we wanted to see the minimum or maximum values in our data or understand how the values are spread. R has built-in `min()` and `max()` functions as well as a `range()` function that performs both.

```
> # Minimum value
> min(sales)
[1] 1

> # Maximum value
> max(sales)
[1] 8

> # Range of values
> range(sales)
[1] 1 8
```

To understand the spread of the data, we might want to calculate the standard deviation using `sd()` or the variance using `var()`.

```
> # Standard deviation
> sd(sales)
[1] 1.956128

> # Variance
> var(sales)
[1] 3.826437
```

Coding question

Questions

Let's say we have a dataframe `soccer` of data about our soccer team. We have columns `goals_scored` and `goals_allowed` representing the score of each game.

We want to know the minimum and maximum values of goals allowed from the season as well as the variance of goals allowed.

Create an object `allowed_range` and use `range()` to calculate the minimum and maximum values. Then print `allowed_range`.

Then create an object `allowed_var` and calculate the variance of the number of goals allowed and then print `allowed_var`.

Code

1
2
3
4
5
6

Run

Run your code to check your answer

Tables

R has a couple of very useful built-in tables that can be used as summary statistics. `summary()` returns the minimum, maximum, mean, and median values as well as the 1st and 3rd quartile values.

`table()` returns a table with the count of how many times each value occurred in the dataset. This can be helpful to see how the data are distributed within the dataset.

```
> # Print summary table of the sales dataframe
> summary(sales)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
1.000   3.000   4.500   4.633   6.000   8.000

> # Print frequency table
> table(sales)
sales
1 2 3 4 5 6 7 8
2 2 5 6 4 5 4 2
```

We can include multiple datasets into `table()`, which will create a cross-table. For example, say we had another vector `weekend` that represents whether or not a given day was on a weekend. We can make a cross-table with `sales` to see how sales differed between weekdays and weekends.

```
> # Weekend & sales cross-table
> table(weekend, sales)
      sales
weekend 1 2 3 4 5 6 7 8
      0 2 0 3 4 4 3 2 2
      1 0 2 2 2 0 2 2 0
```

Coding question

Questions

Let's say we have a dataframe `soccer` of data about our soccer team. We have columns `goals_scored`, `goals_allowed`, and `weekend` representing the score of each game and whether or not the game was played on a weekend.

Create an object `table_weekend` that is a table of how many games were played on weekends and how many were not. Then print `table_weekend`.

Then create an object `table_weekend_scored` that is a cross-table such that the scores go across the columns and the weekends are different rows. Then print `table_weekend_scored`.

Visualizations in R

Learn how to make visualizations in R using the `ggplot2` package.

There are two main ways to visualize data in R: using base R visualizations and using `ggplot()` and related functions from the `ggplot2` package, which is also part of the tidyverse. While both have their own strengths and limitations, we will be focusing on learning to use `ggplot()`.

One way to think about how `ggplot()` works is to think of layers of visualizations. The following image depicts the different layers that can go into a single plot.

We start almost every plot with:

```
ggplot(data = dat, mapping = aes())
```

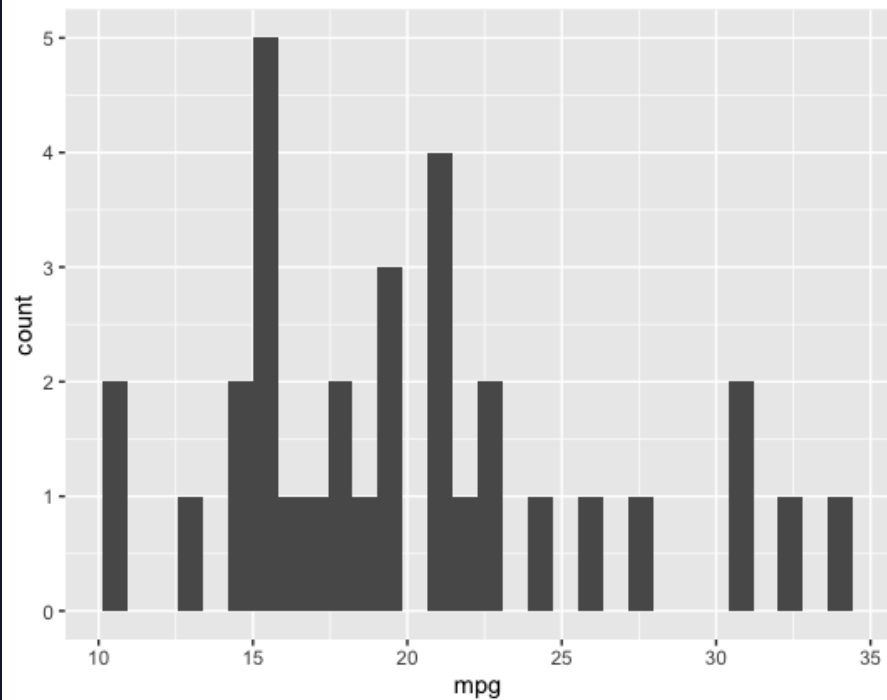
`dat` tells us what dataframe or tibble to use, and `aes()` is a general description of the axes. From there, we use `+` to add additional layers to this plot. Data layers are called *geometries* and are indicated by the title `geom`.

The following sections will give a basic outline of how to create the main types of data visualizations. We will be using the `mtcars` dataset.

Histograms

To create a histogram, we start with the basic `ggplot()` function and then add `geom_histogram()`. Let's say we wanted to create a histogram of the different values of `mpg` in our dataset:

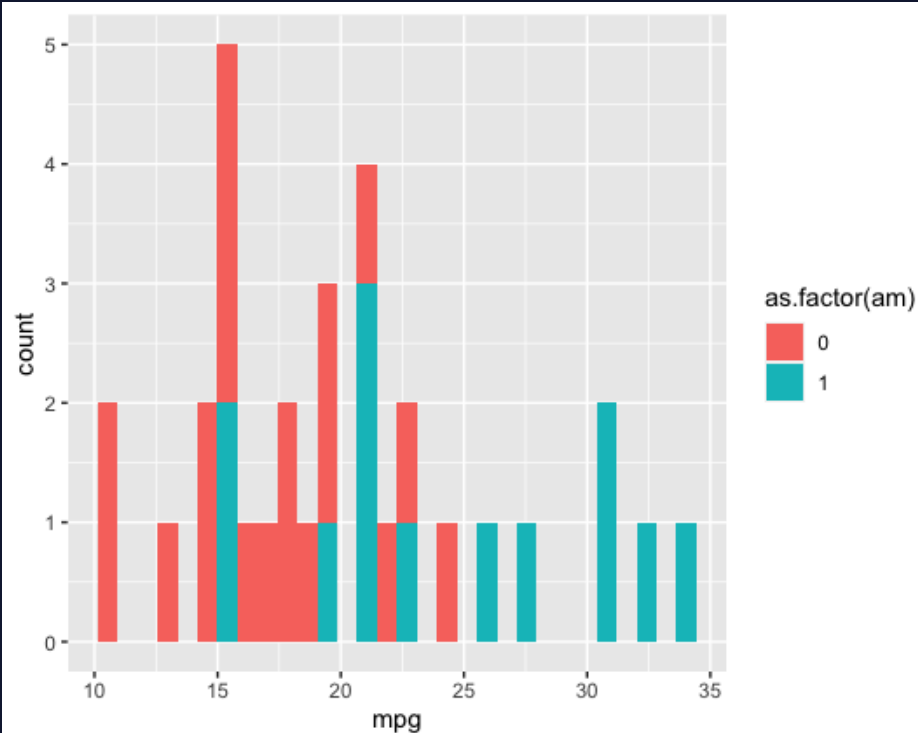
```
ggplot(data = mtcars, aes(x = mpg)) +  
  geom_histogram()
```



Note that we did not need to include a `y` variable in `aes()`. This is because `geom_histogram()` automatically generates a count variable given the variable of interest, in this case `mpg`.

If we wanted to depict a grouping variable within each bar in this histogram, we could do that by including a `fill` variable. In this case, we will use `am` for automatic or manual transmission:

```
ggplot(data = mtcars, aes(x = mpg, fill = as.factor(am))) +  
  geom_histogram()
```



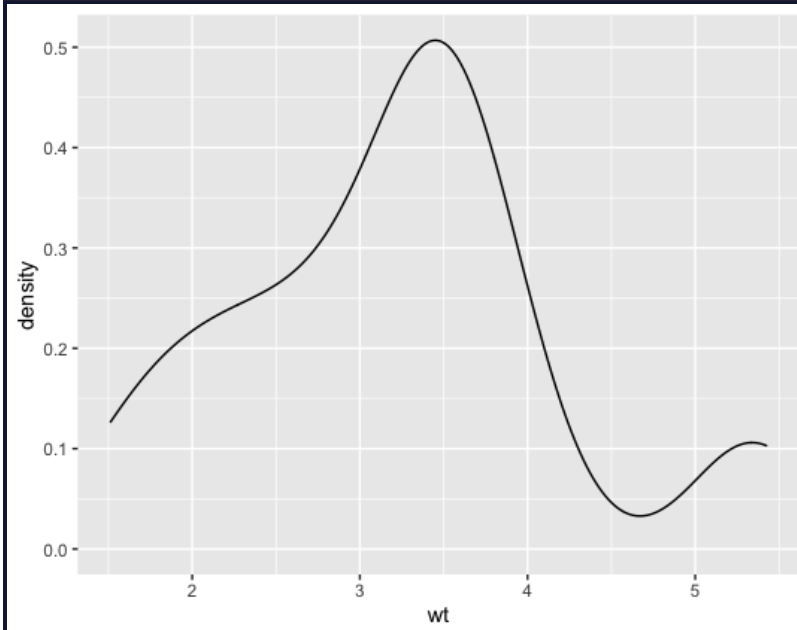
We used `as.factor(am)` in order to make distinct groups. Unless otherwise specified, `ggplot()` will assume a numeric variable is continuous.

Note that these bars are stacked, not overlapped. So at `mpg = 21`, we can see a total height of 4, with 3 from `am = 1` and 1 from `am = 0`.

Density plots

If we wanted to make a density plot to see the distribution of car weights `wt`, we could use `geom_density()`:

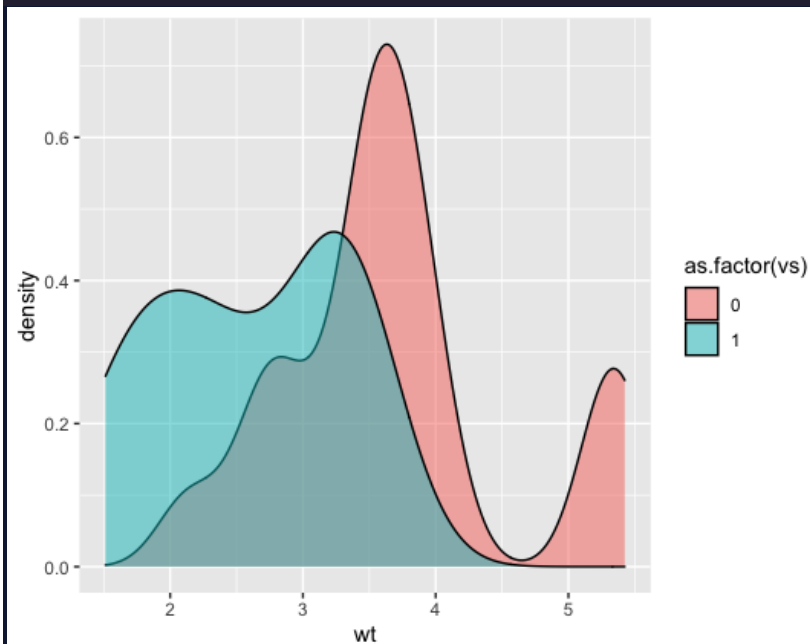
```
ggplot(data = mtcars, aes(x = wt)) +  
  geom_density()
```



Similar to histograms, we did not have to specify a y-variable because `geom_density()` generates the density for us.

Let's say we wanted to create two overlapping density plots grouped by another variable—let's say engine shape `vs` (v-shape or inline). We could do this the same way we did for histograms:

```
ggplot(data = mtcars, aes(x = wt, fill = as.factor(vs))) +  
  geom_density(alpha = 0.5)
```

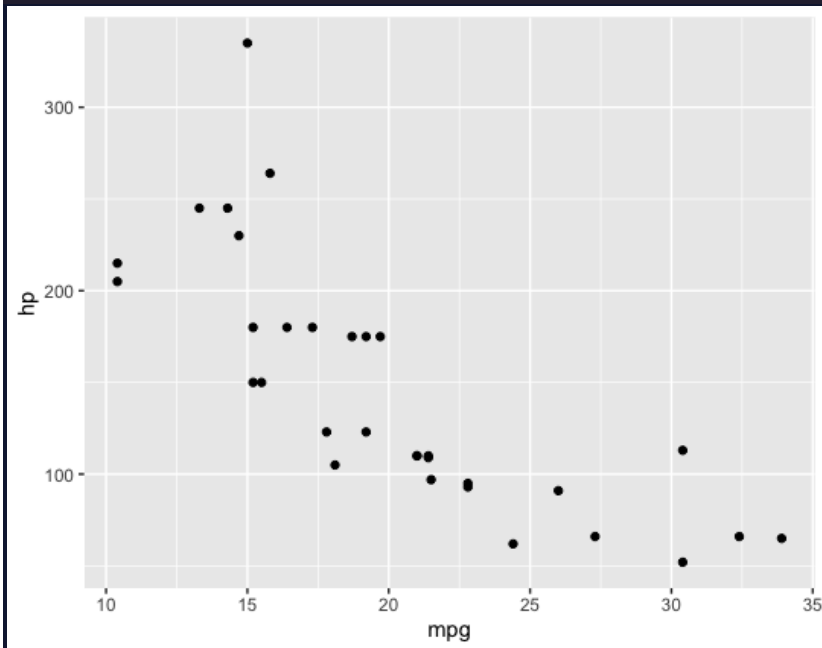


Note that here, we added `alpha = 0.5` into our `geom_density()` layer. We did this so that the densities are somewhat transparent, and we can fully see both densities on our plot.

Scatterplots and lines

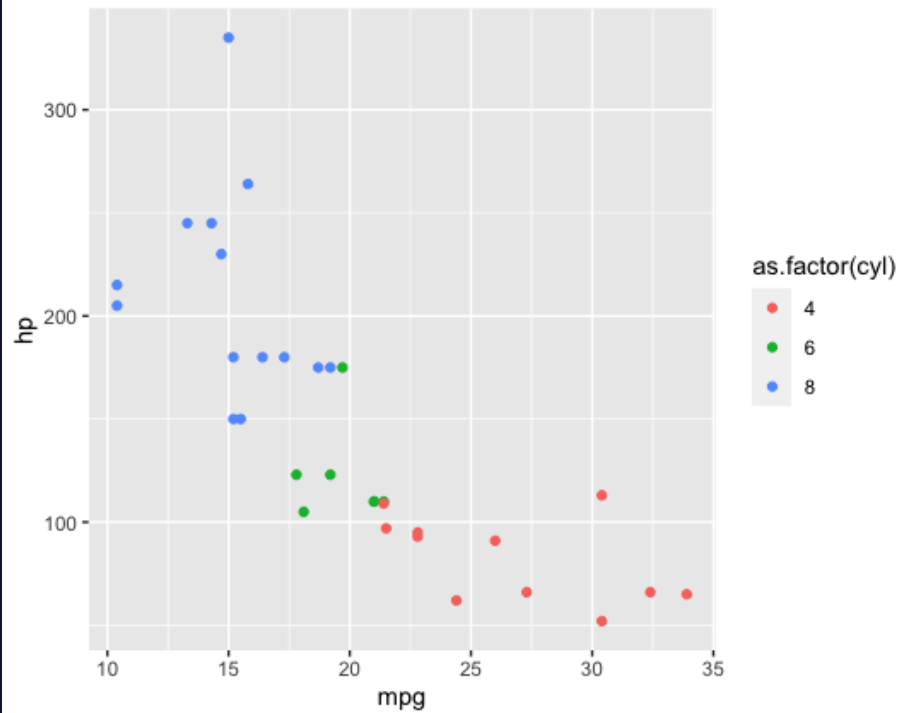
Let's say we wanted to look at the relationship between miles per gallon `mpg` and horsepower `hp`. We would likely want to use a scatterplot, which can be added using `geom_point()`:

```
ggplot(data = mtcars, aes(x = mpg, y = hp)) +  
  geom_point()
```



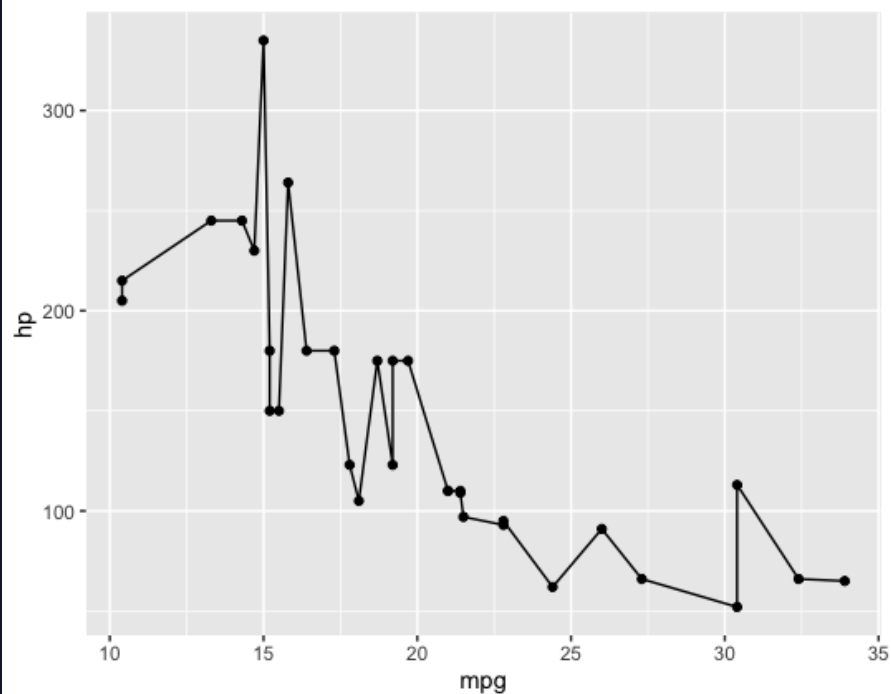
Let's say we wanted to color our data points based on the number of cylinders `cyl`:

```
ggplot(data = mtcars, aes(x = mpg, y = hp,  
                           color = as.factor(cyl))) +  
  geom_point()
```



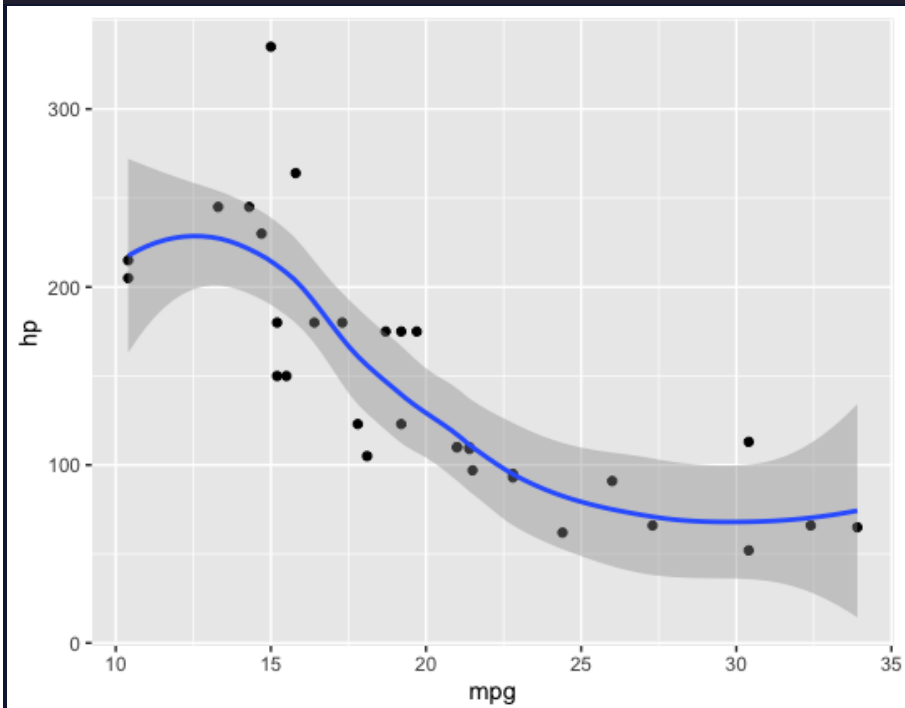
If we wanted to connect our datapoints moving across the x-axis, we could add `geom_line()` as another layer in our plot:

```
ggplot(data = mtcars, aes(x = mpg, y = hp)) +
  geom_point() +
  geom_line()
```



This line is very jagged and doesn't necessarily show us what we would see if we averaged out the relationship between `mpg` and `hp`. If we wanted to add a more smoothed regression line, we could use `geom_smooth()` instead.

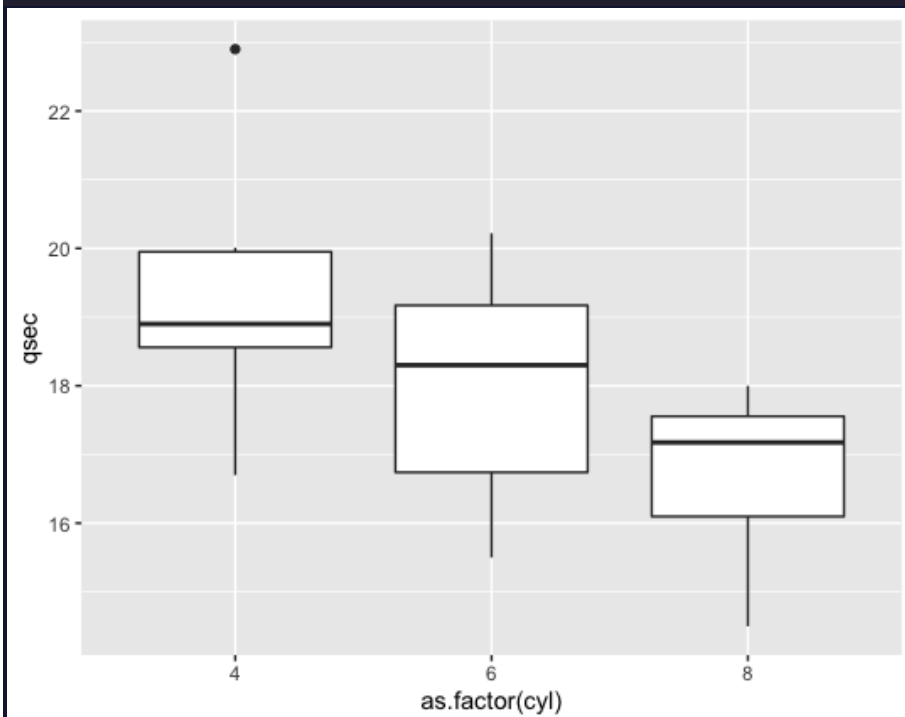
```
ggplot(data = mtcars, aes(x = mpg, y = hp)) +
  geom_point() +
  geom_smooth()
```



Boxplots

Let's say we wanted to evaluate the quarter-mile time `qsec` with the number of cylinders `cyl`:

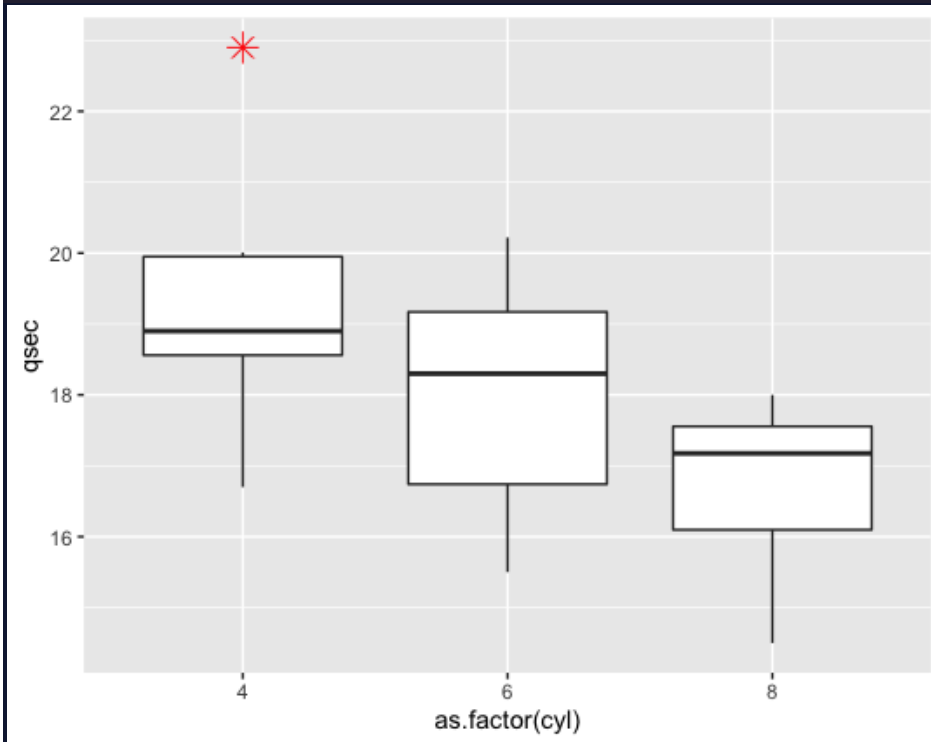
```
ggplot(data = mtcars, aes(x = as.factor(cyl), y = qsec)) +  
  geom_boxplot()
```



We can further add special features to our boxplots. For example, we can change the shape, color, and size of our outliers if we wanted to draw special attention to them:

```
ggplot(data = mtcars, aes(x = as.factor(cyl), y = qsec)) +
```

```
geom_boxplot(outlier.colour="red", outlier.shape=8,
             outlier.size=4)
```



Last thoughts on ggplot()

`ggplot()` can be very useful and flexible to create the specific visualizations we want from our data. It can, however, take some practice to get used to its conventions. Some other helpful tips:

```
+ labs(x="X axis", y = "Y axis", title = "Title")
```

```
ggplot()
  aes()
  geom_()
  geom_()
  geom_()
  cheatsheets
  ggplot()
```

Coding question

Questions

Here we have uploaded the `mtcars` dataset. Try making different plots using the methods explained above. Some possible plots:

Regression Models in R

Learn to make regression models in R.

In R programming, there are many built-in functions to easily create regression models. We will cover the basics of how to create a linear regression model and a logistic regression model.

Linear regression

Linear regression models are created using the `lm()` function. The required parameters of this function are `formula` and `data`.

`formula` generally follows the form of `outcome ~ variable1 + variable2....` This is the regression equation using the outcome variable, then a `~`, and then the predictor variable names separated by `+` signs. We can add an interaction term for predictor variables by multiplying them together, for example `variable1*variable2`.

`data` is simply the name of the dataset. If we are using variables from multiple datasets, omit this parameter and refer to each variable in `formula` using the form `dataset$variable`.

A useful way to view the output of the linear model is to save the model to an object and then use `summary(model)`.

Let's say we want to run a regression on a dataset `df` that has an outcome variable `y` and predictors `x1`, `x2`, and `x3`. We could make the linear model:

```
# Create the model
lm1 <- lm(y ~ x1 + x2 + x3, data = df)

# Print summary
summary(lm1)
```

Output

Call:

```
lm(formula = y ~ x1 + x2 + x3, data = df)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-15.1000	-2.9321	0.2195	3.1720	10.8278

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	7.0578	0.7699	9.167	9.18e-15	***
x1	0.1502	0.1048	1.432	0.1553	
x2	0.3229	0.1532	2.108	0.0377	*
x3	1.0097	0.0472	21.394	< 2e-16	***

Signif. codes:

0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.219 on 96 degrees of freedom

Multiple R-squared: 0.5021, Adjusted R-squared: 0.4866

F-statistic: 32.28 on 3 and 96 DF, p-value: 1.63e-14

The `Estimate` column returns the regression coefficients for each variable. We can call `summary(lm1)$coefficients` to extract just the estimates as a vector.

The `Pr(>|t|)` column returns the associated p-value for each variable. The symbols next to this column are a visual representation of the p-values. Any amount of `*` next to a p-value means there is a p-value of 0.05 or less.

Coding question

Questions

Here we have a dataset `scores` with information about 100 students, specifically how many hours each student studied, whether they had breakfast before this exam, and ultimately how they scored on an exam.

Create a linear model called `scores_lm` that evaluates the relationship between test scores to hours studied and breakfast before the exam. Then print the summary of `scores_lm`.

Logistic regression

The function for a logistic regression model is similar to linear regression except we use `glm()`, and we have to specify `family`. `family` indicates the type of regression we want to perform. Generally logistic regression will use `family = binomial()`.

Let's say we have a dataset `df` with a binary outcome variable `y` and predictors `x1`, `x2`, and `x3`. We could make the logistic model:

```
# Create the model
glm1 <- glm(y ~ x1 + x2 + x3, data = df, family = binomial())

# Print the summary
summary(glm1)
```

Output:

```
Call:
glm(formula = y ~ x1 + x2 + x3, family = binomial, data = df)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-1.84303	-0.22670	0.00767	0.40926	2.30363

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-6.22522	1.66860	-3.731	0.000191	***
x1	0.04197	0.11500	0.365	0.715178	
x2	0.06070	0.16837	0.361	0.718468	
x3	0.72997	0.16441	4.440	8.99e-06	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 138.469 on 99 degrees of freedom
Residual deviance: 56.264 on 96 degrees of freedom
AIC: 64.264

Number of Fisher Scoring iterations: 7

Coding question

Questions

We have a dataset `measurements` with height (cm) and weight (kg) measurements for 1000 men and 1000 women. Create an object `measurements_lm` that is a logistic regression evaluating the relationship of sex (`male`) to `height` and `weight`. Then print the summary of this model.

Predictions

To make predictions of the outcome variable using a regression model, we need a dataset whose column names match the names of the coefficients in the model. Once the column names match, we can use the `predict()` function to generate predictions. The model will produce 1 predicted outcome for each observation in this new dataset.

```
pred_data <- data.frame(
  x1 = c(0, 1, -1),
  x2 = c(1, 6, 5),
  x3 = c(10, -4, 9)
)
```

```
predict(lm1, pred_data)
```

Output:

```
      1      2      3  
17.477906  5.106418 17.609560
```

The trickiest part about making predictions is matching the name and order of variables in the new dataset when there are a lot of variables in our model.

Coding question

Questions

We have a dataset `scores` with information about 100 students, specifically how many hours each student studied, whether they had breakfast before this exam, and ultimately how they scored on an exam. We also have a regression model `scores_lm` that evaluates the relationship between test scores to hours studied and breakfast before the exam.

Create an object `pred_score` and use `predict()` to predict the expected exam score for a student who studied for 3.5 hours and ate breakfast before the exam. Then print `pred_score`.

Introduction to Causal Inference

Introduction to Causal Inference

Begin your journey into causal inference!

Welcome

Welcome to Causal Inference with R! The goal of this unit is to understand the conceptual foundations of causal thinking and learn how to use R to implement a few basic causal inference methods.

Why Learn Causal Inference?

Most questions that ask why one thing causes another are answered through experiments, but experiments are often difficult or impossible to perform. We also have a tremendous amount of observational data available to us outside of experiments, but data science frequently focuses on using that data to answer predictive questions. Causal inference gives us methods to leverage observational data to answer causal questions without the use of experiments.

What Will We Cover in This Course?

First, we will cover the conceptual foundations of causal inference. Then, we will cover specific methods using R, including:

- Propensity score weighting
- Regression discontinuity design
- Instrumental variables
- Difference-in-differences

After this unit, you will be able to:

- Explain the basis of causal thinking through the Potential Outcomes Framework
- Identify and visualize situations where a causal inference method could be used
- Apply several basic causal inference methods in R

Learning is Social

Whatever you're working on, be sure to connect with the Codecademy community in the [forums](#). Remember to check in with the community regularly, including for things like asking for code reviews on your project work and providing code reviews to others in the [projects category](#), which can help to reinforce what you've learned.

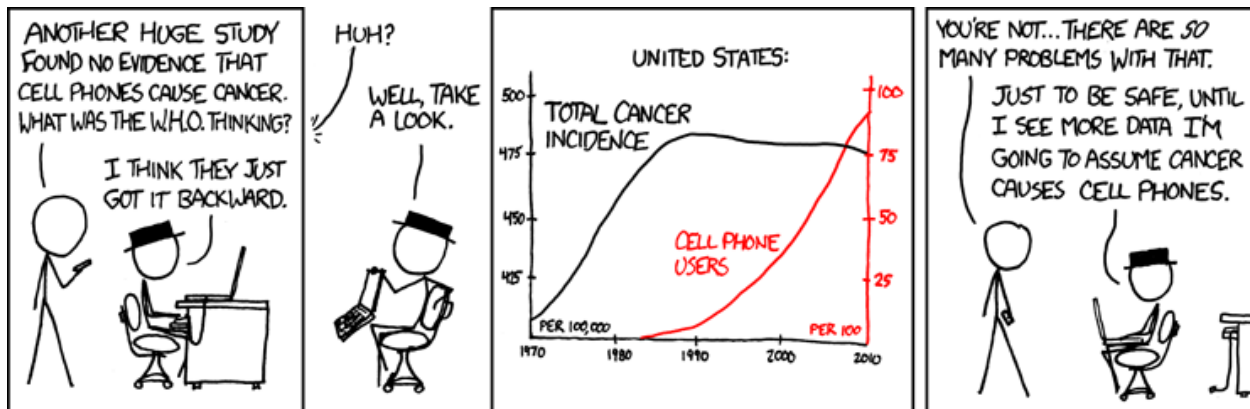
Potential Outcomes Framework

Causal Inference: An Introduction

<https://docs.google.com/document/d/1qtdzXVhAtofQQdh4WSBQ5ZDrFkQ79FREndN1WNyuxg8/edit?tab=t.0>

As humans, we are hardwired to look for patterns and identify relationships between things we observe in the world around us. Our brains naturally tend to fill in details and come up with explanations for these relationships. Unfortunately, jumping to conclusions often leads us to see nonrandom in the random and to blur the lines between association and causation.

Before we get some hands-on experience using causal inference methods, we need to build up some intuition about what exactly causal inference is, when causal inference is appropriate to use, and what causal inference can (and cannot) do. In this lesson, we will gradually introduce you to the key ideas, statistical frameworks, and required assumptions that are fundamental to causal inference.



Correlation is Not Causation

Chances are, you or someone you know is superstitious to some extent. Whether it's wearing a lucky t-shirt to a sporting event or using a favorite pencil and eraser on exams, we believe in superstitions because we think our actions will lead to some desired—but usually unrelated—result. Superstitions are, in fact, extreme examples of assuming an associational relationship is actually causal in nature.

One of the most important concepts in causal inference is the distinction between association and causation. Let's formally define these two terms:

Association is a general term to describe a relationship between variables. Association can describe the strength or pattern of a relationship, but it does not explain the mechanism behind the relationship.

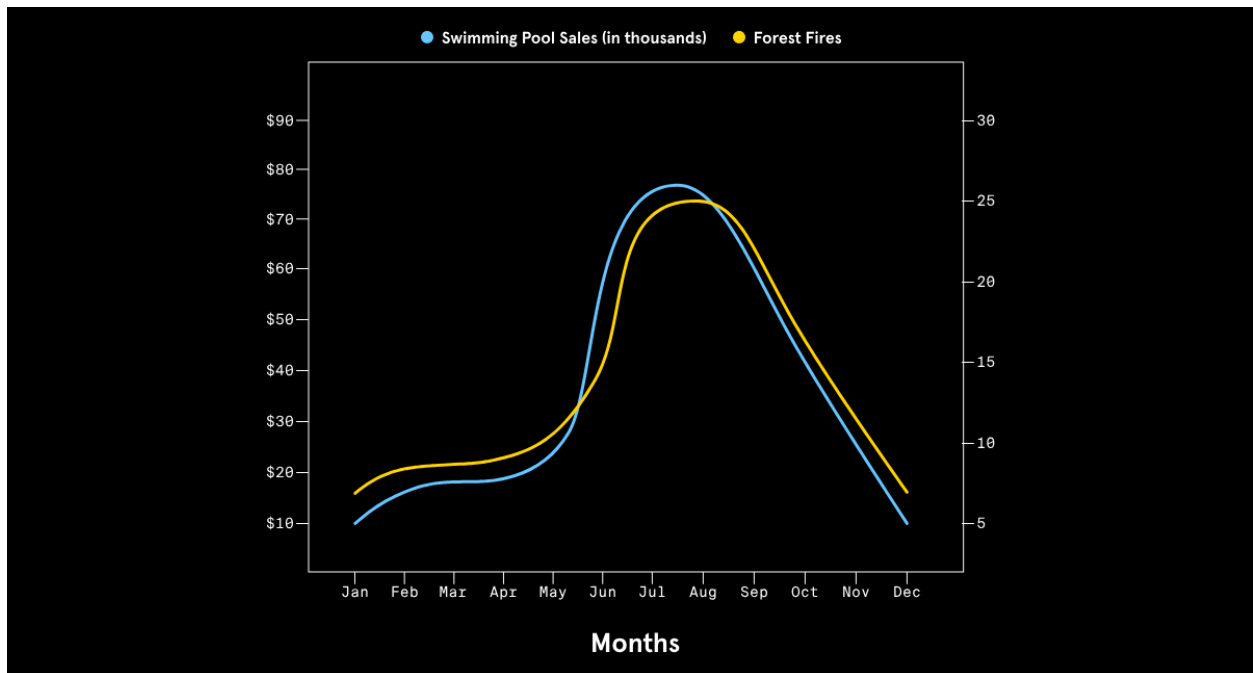
One frequently used statistical measure of association is *correlation*. Correlation is typically used to describe the association between two variables with a linear pattern. The animation below shows what variables with different degrees of correlation look like.

Causation describes not only the strength or pattern of a relationship but also the **MECHANISM** of a relationship. In a causal relationship, variable X **CAUSES** a change in variable Y; we know that X must happen before Y.

Instructions

Take a look at the plot in the learning environment. This plot shows the relationship between monthly swimming pool sales and monthly forest fires. It might seem like the plot suggests that swimming pool sales cause more forest fires to occur! Obviously, our intuition tells us that, no, swimming pools do not **CAUSE** forest fires. A more sensible explanation for this relationship is that swimming pool sales and forest fires both peak in the hot, dry, summer months.

This is a silly but effective illustration of the adage “correlation is not causation.” Keep this example in your mind as you move throughout this lesson to help you think more critically about what factors might be at play in causal relationships.



Thinking Counterfactually

The second important concept we must learn is *counterfactual thinking*. Counterfactual thinking is the process of asking, “What **WOULD** have happened if circumstances were different?” Let’s illustrate counterfactual thinking using the following example:

Dogs are often called “human’s best friend,” but did you know there may be a biological explanation behind this saying? Research has shown that just a few minutes of interacting with dogs or cats can reduce levels of cortisol, a hormone linked to stress that may lead to weight gain or a weakened immune system. Let’s say we’re interested in learning whether interacting with a trained therapy animal leads to decreased levels of cortisol in hospital patients.

Approaching this using counterfactual thinking, we must consider what each patient’s cortisol level would be in two different “universes”:

In one universe, a patient interacts with a therapy animal—we will call this “Universe T.”

In the other universe, referred to as “Universe C,” the same patient does **NOT** interact with a therapy animal.

The cortisol levels in these two parallel universes are called *potential outcomes* because either could potentially be observed. But in reality, we can only observe one cortisol level for a particular patient at one specific moment in time.

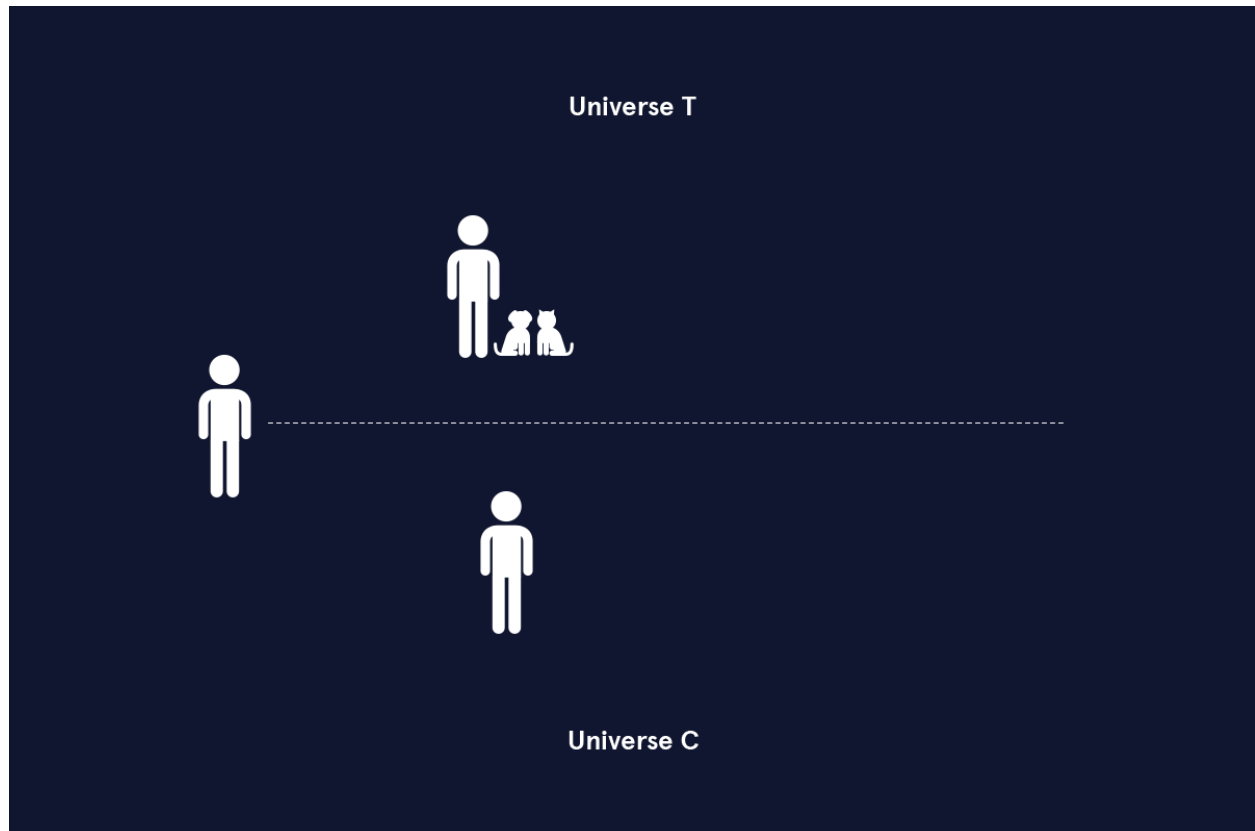
Assume that a patient exists in Universe T and **ACTUALLY** interacted with a therapy animal. The cortisol level observed in this scenario is the *observed or factual outcome* because it is the outcome that was observed. The cortisol level that would have been observed if the patient existed in Universe C and did not interact with the therapy animal would be the *counterfactual outcome*. We can never actually observe the counterfactual outcome.

Using counterfactual thinking allows us to compare the exact same person at the exact same time under two different circumstances. Because the only difference between the two universes is that the patient received the

treatment in Universe T and the control in Universe C, we could compare the cortisol levels from each universe to get an estimate of the effect of interacting with therapy animals.

Instructions

The animation to the right illustrates which outcome is the factual outcome and which is the counterfactual outcome when universe T and universe C are true. When the individual is actually in universe T, the cortisol level observed in universe T is the factual outcome and the cortisol level observed in universe C is the counterfactual outcome. If the individual is actually in universe C, the reverse is true.



Potential Outcomes Notation

In order to generalize our understanding of the potential outcomes framework, we will now introduce some notation that will be used throughout the rest of this course.

Z represents the treatment or exposure condition. When Z is binary, there are only two possible treatment conditions: treatment ($Z = 1$) or control ($Z = 0$).

(Z could also be a continuous variable, such as medication dosage or number of days in the hospital, but our example assumes that there are only two treatment conditions for simplicity.)

Y represents the observed value of the outcome variable.

In the potential outcomes framework, we consider how the Y values we observe would change if the treatment were different:

When Z is binary, the two potential outcomes are represented with Y_1 and Y_0 :

Y_1 is the potential outcome under the treatment condition ($Z = 1$).

Y_0 is the potential outcome under the control condition ($Z = 0$).

In real life, an individual can only be in one treatment group at any specific point in time. Thus, we can never know the value of both potential outcomes for an individual. We will discuss how to deal with this problem in future exercises. For now, assume that we could know the values of both Y_0 and Y_1 .

Instructions

The interactive table in the learning environment shows the potential outcomes of five hospital patients under the two treatment conditions. The Y_1 column shows what each patient's potential cortisol level would be if they interacted with a therapy animal. The Y_0 column shows what each patient's potential cortisol level would be if they did NOT interact with a therapy animal. The Y column shows the observed (factual) outcome. Play around with entering values in the Z column to see how the observed value in the Y column changes (remember: $Z = 1$ is the treatment condition, and $Z = 0$ is the control condition). When Z is 1, the value in the Y_1 column should appear in the Y column. When Z is 0, the value from the Y_0 column appears.

A Missing Data Problem

If we knew both potential observations for every individual, we could use them to estimate several different statistics that summarize the effect of the treatment:

The *individual treatment effect* (ITE) is computed as $Y_1 - Y_0$. This statistic directly compares the two potential outcomes for each individual.

The *Average Treatment Effect* (ATE) is the average of all individual treatment effects, which can be calculated as the difference between the average of Y_1 and the average of Y_0 .

Hold up! You may be wondering, "How are we supposed to calculate the individual treatment effect or true ATE if we can never observe the counterfactual outcome?" This question gets at the fundamental problem of causal inference. Causal inference is essentially a missing data problem: since we can only observe the outcome that actually happened, we are always missing the counterfactual outcome.

Instructions

The table at the right shows data for 12 hospital patients who either interacted with a therapy animal ($Z = 1$) or did not interact with a therapy animal ($Z = 0$). Select "Theoretical" or "Reality" to toggle between the theoretical data (which is unobservable in real life) and what we could actually observe from these 12 patients in reality.

The theoretical data contains both potential outcomes. Because we have both potential outcomes, we can compute the true ATE by subtracting the potential outcome averages. The true ATE is -5.8, which means that interaction with therapy animals results in an average decrease in cortisol levels of 5.8 units.

The real data contains only one potential outcome for each individual. Notice now that the Y_1 values are missing for individuals in the control group ($Z = 0$), and the Y_0 values are missing for individuals in the treatment group ($Z = 1$). In real life, we will always be missing half of the data we need to calculate the true ATE.

Estimating the ATE

Because we can never know both potential outcomes for an individual, we need to use a different [method](#) to estimate causal effects. The most accurate way to do this is to use *randomization*.

Randomization is a method of treatment group assignment that is essentially a coin flip to determine whether an individual receives the treatment or the control. This ensures that, for a large enough sample size, the treatment groups will be similar on average with respect to all factors EXCEPT for the treatment condition. While we still won't know the counterfactual outcome for any individual, we can be reasonably confident that similar individuals received each treatment. This allows us to estimate the potential outcomes we weren't able to observe.

When randomization is possible, we can estimate the ATE by taking the difference of the average observed outcome values in the treatment and control groups. In the example in the learning environment, the estimated ATE equals -5.4, which is very close to the true ATE of -5.8 that was calculated in the previous exercise.

1. Here are some ATE calculation practice questions!

True ATE Calculation

Calculate the true ATE using the following data.

	Z	Y1	Y0	Y1 - Y0
	1	15	8	7
	1	23	19	4
	0	26	21	5
	0	31	22	9
	1	20	21	-1
	0	23	17	6

- 2.

Expand for Solution

Estimated ATE Calculation

Now calculate the estimated ATE when treatment assignment is randomized.

	Z	Y1	Y0	Y
	1	23	?	23
	1	31	?	31
	0	?	33	33
	0	?	22	22

1	30	?	30
0	?	17	17

confounders and Selection Bias

Often, randomization of treatment is unethical. For example, it would be unethical to force hospital patients who don't like animals to receive therapy animal services. Randomized experiments can also be expensive and demand a lot of resources. It is often more practical to use data that is already available. In these scenarios, the treatment group is not random but is based on other factors, such as personal preference.

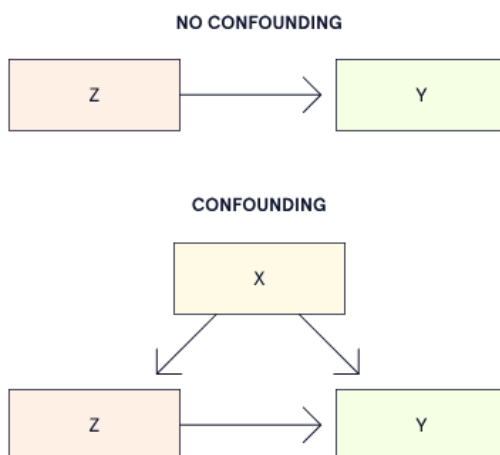
When randomization is not possible or plausible, several sources of bias can impact the accuracy of estimated causal effects. One source of bias is *selection bias*. Selection bias is bias that happens because of how individuals were put into the treatment or control groups.

In terms of our therapy animal example, selection bias might arise if:

- individuals choose whether they want the therapy (dog-lovers might be over-represented in this case)
- individuals in the control group come from a different hospital
- individuals are only able to receive the therapy based on another variable, such as insurance coverage

If any of these [variables](#) that are associated with treatment assignment are also related to the outcome variable (cortisol level), they are considered *confounders*, or *confounding variables*. These variables may lead us to incorrect conclusions about the impact of the treatment on the outcome.

The following is a graphical representation of how the confounder X is associated with both the treatment Z and outcome Y:



Instructions

To illustrate the concept of confounding variables, consider the following example. Suppose we are interested in learning about the effect of coffee on blood pressure. If we assume that there are no confounding variables impacting coffee intake or blood pressure, the causal relationship would look something like the first diagram in the learning environment where drinking coffee and high blood pressure are linked by a single arrow.

Assuming that there are no confounders that are related to both coffee intake and blood pressure is unrealistic from a scientific perspective. We know that people who smoke cigarettes tend to drink more coffee than people who do

not. We also know that nicotine in cigarettes is associated with higher blood pressure. Therefore, it is reasonable to conclude that cigarette smoking is a potential confounder in the relationship between coffee intake and blood pressure. In our diagram, we add smoking cigarettes as a confounder that points to both drinking coffee and high blood pressure.

Estimating the ATE with Confounders

So how do we deal with confounders and estimate the ATE when the treatment assignment is not randomized? Let's return to the therapy animal example once more.

Suppose that instead of randomizing patients to receive therapy animal services, we allow the twelve hospital patients to CHOOSE whether or not they want therapy. Imagine that we also have data for a new confounding variable X that represents whether an individual does ($X = 1$) or does not ($X = 0$) have a diagnosis of anxiety disorder. Here, X is a confounder and impacts both the treatment and the outcome: patients who have anxiety might be more likely to choose to receive therapy animal services AND have higher cortisol levels generally.

This confounding is problematic because it means there may be more people with anxiety in the treatment group than in the control group. More people with anxiety means the treatment group may have a higher average cortisol level compared to that of the control group before therapy animal services even occur!

When the treatment groups are unbalanced with respect to confounders, the treatment groups are not *exchangeable*: we would observe different outcomes if the treatment groups swapped treatment conditions.

To avoid making poor comparisons between potentially imbalanced treatment groups, we have to be able to assume *conditional exchangeability*:

Conditional exchangeability means that the treatment groups are exchangeable if we take into account confounding variables

Preview: Docs Loading link description

This is also called *ignorability* or *unconfoundedness*.

By taking anxiety diagnosis (variable X) into account, we avoid getting a biased estimate of the cortisol levels produced by those receiving therapy animal services in comparison to those who do not.

Instructions

Take a look at the slides in the learning environment to see how the anxiety confounder might bias our estimate of the treatment effect in our therapy animal analysis.

Slide 1

In our table, we have the data for our twelve hospital patients who chose whether or not to receive therapy animal services. The variables include anxiety diagnosis ($X=1$ for anxiety and $X=0$ for no anxiety) and treatment assignment ($Z=1$ for therapy and $Z=0$ for no therapy). Because this is reality, we cannot see both potential outcomes but rather just the observed outcome Y (observed cortisol level).

If we compute the estimated ATE as though treatment was randomized and don't take anxiety (X) into account, we get an estimated ATE of -2.9. We conclude that therapy animal services reduce cortisol levels by 2.9 units on average.

Slide 2

In order to account for the anxiety variable, we first compute the ATE for just those patients diagnosed with anxiety disorder ($X=1$). We do this as we always do: by subtracting the average cortisol level for the control patients from the average for the treated patients. Because we are only comparing patients with anxiety, the groups should theoretically have similar cortisol levels before receiving therapy. We find an estimated ATE of -5.5 for individuals with anxiety—much greater in magnitude than our initial estimate of -2.9.

Slide 3

Now we compute the ATE for just those patients NOT diagnosed with anxiety disorder ($X=0$). We find an estimated ATE of -5.5 for individuals WITHOUT anxiety.

Slide 4

Averaging the ATEs of our two groups gives us an estimated ATE of -5.5 when we take X into account. If we recall from Exercise 5, the true ATE was -5.8: therapy animal services reduced cortisol levels by 5.8 units on average. Our estimate when we took confounding into account was much closer to the true treatment effect.

Why did this happen? Anxiety correlated with higher cortisol, and more people with anxiety were in the treatment group. This made the treatment group's cortisol levels tend toward higher levels, making it look as though therapy animal services were not bringing down cortisol levels by much.

Assumptions

The last exercise brought up one assumption used throughout causal inference: conditional exchangeability. In this lesson, we will learn a few more assumptions.

A second assumption made in causal inference is *Stable Unit Treatment Value Assumption* (SUTVA). The name is a mouthful, but it's a pretty simple assumption that can be broken down into two components:

An individual's treatment assignment doesn't impact the outcome of other individuals. Using the example of the hospital patients, this would mean that one individual getting therapy animal services doesn't impact the stress level of other individuals in the hospital.

The treatment (or control) is applied exactly the same way to all patients. For example, the patients receiving therapy animal services should receive therapy for the same amount of time and ideally from the same exact animal to ensure consistent treatment.

The next assumption we need to familiarize ourselves with is *overlap*. The assumption of overlap means that all subgroups of patients divided by their characteristics have a positive, non-zero probability of getting either treatment assignment. Overlap is also referred to as the *common support* or *positivity* assumption.

Instructions

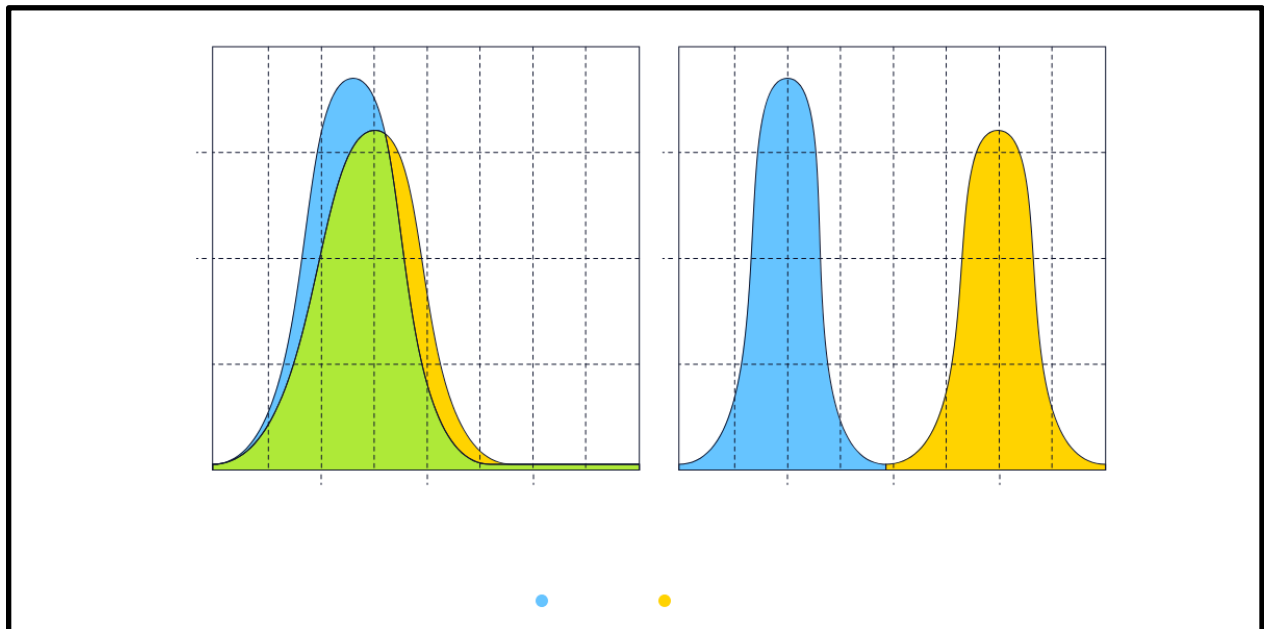
1. Take a look at the plot in the learning environment. The left half of the plot shows an example of when the assumption of overlap is met. For individuals of all ages, there is a positive probability of being assigned to either the treatment or control group. No age has a 100% probability of being assigned to treatment and a 0% probability of being in the control group or vice versa. The area of overlap is called the *region of common support*.

In contrast, the right half of the plot shows when the assumption of overlap is NOT met.

Patients under the age of about 38 have zero probability of being assigned to the control group.

Patients above the age of 38 have no probability of being assigned to the treatment group.

If we try to make inferences based on the groups with no overlap, we are comparing a group of treated younger individuals to a group of untreated older individuals. We cannot reasonably assume age does not also affect our outcome.



Other Causal Estimands

So far, we've only focused on the ATE, but there are many other useful estimands we can use to summarize the average causal effect of some intervention or exposure. In many situations, it is not realistic to meet the assumptions of conditional exchangeability, SUTVA, AND overlap. Other causal estimands allow us to relax some of these assumptions.

The ATE summarizes the causal effect of treatment across ALL treatment conditions. But, we may only be interested in the causal effect of treatment in a particular group. In these cases, we can use a different estimand:

Average Treatment Effect of the Treated (ATT) is the average of $Y_1 - Y_0$ for all individuals assigned to the treatment condition ($Z = 1$).

Average Treatment Effect of the Control (ATC) is the average of $Y_1 - Y_0$ for all individuals assigned to the control condition ($Z = 0$).

There are other estimands we may encounter in causal methods, but the ATE, ATT, and ATC are three that we see in a variety of situations.

Instructions

View the table in the learning environment. It shows theoretical data as well as which subsets of that data we use to calculate the ATT and ATC. Here, because the data is theoretical, we have both the factual and counterfactual outcomes for each individual, so we can compute both the ATT and the ATC.

Causal Inference Process

Now that you have some experience with the assumptions needed for causal inference as well as familiarity with a few causal estimands, we need to set up a structured way to apply what you've learned when approaching causal inference problems.

We can think of causal inference as a two-step process.

Identification: During this stage we determine which causal estimand we will estimate based both on what we want to know and on what we are able to compute. We must also determine whether we will be able to meet the three assumptions in order to infer that the relationship is causal in nature.

Estimation: Now that we've reasoned what can compute and that this measure will reflect a causal relationship between [variables](#)

Preview: Docs Loading link description

, we must carry out the statistical model to compute the treatment effect. We used some simple computations throughout this lesson, but we will soon use more complicated methods like multiple linear regression to obtain our estimand.

Instructions

1. The flowchart in the learning environment summarizes the two steps in the causal inference process: Identification and Estimation. Think about these steps as we would have applied them in our therapy animal example.

Identification

Estimand: Are we interested in computing the ATE for all patients, or do we only care about the effect of therapy animals on the patients who were treated (ATT)? Are we able to compute the ATE in this case? When might we NOT be able to compute the ATE?

Assumptions:

Conditional Exchangeability: Are the treatment and control groups exchangeable? Or do we need to account for other measures of confounding beyond just anxiety diagnosis?

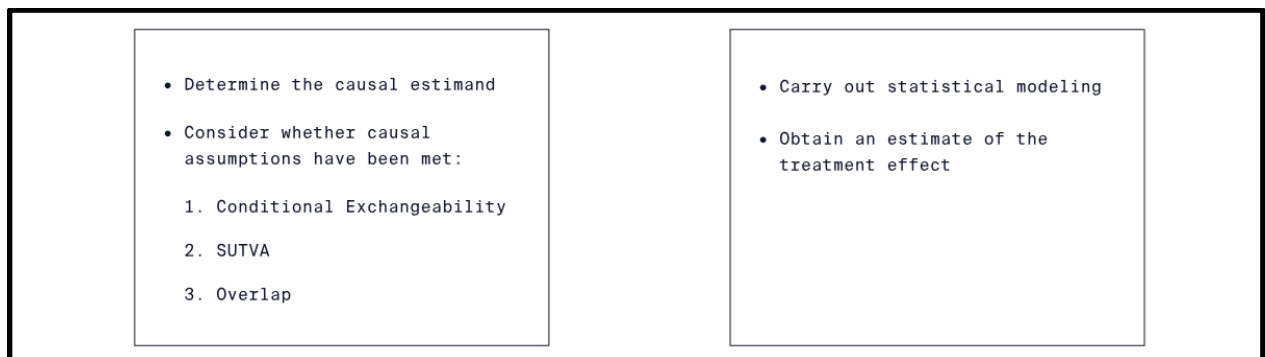
SUTVA: Does one patient receiving the therapy influence other patients to choose to receive the therapy? Is every patient receiving the same services (same frequency, same amount of time, same kind of animal)?

Overlap: Does every patient have some probability of being in either therapy group? Or do some types of patients have no probability of being treated, such as those with allergies or those under age 18?

2. **Estimation**

Which statistical model will we use to estimate the treatment effect?

Are there any additional assumptions that need to be met to use this technique?



Review

Congratulations! You've completed the Potential Outcomes lesson and now have a solid conceptual foundation for causal inference!

In this lesson, you learned:

Causation is different from association in that it implies a relationship where a change in one variable leads to a change in another variable.

If we could view both potential outcomes, we could accurately compute the true effect of a treatment.

The fundamental problem of causal inference is that we only get to view the observed outcome and not its counterfactual.

When randomization is not available to produce estimates of treatment effects, we must use other strategies to predict counterfactuals and find our estimand of interest.

The three main assumptions for causal inference are:

- Conditional exchangeability

- SUTVA

- Overlap

The causal inference process can be thought of as two steps: identification and estimation.

Finding Causal Effects

Explore how causal inference helps us identify the effects of treatment.

In 2004, the Journal of Applied Psychology published a study showing that height was strongly correlated with income even after controlling for sex, age, and weight. It was an amazing result that has been talked about almost constantly since. But how did the researchers figure this out? Clearly, they didn't randomly assign people to the "tall" and "not tall" groups. The answer lies in causal inference.

In this example, height (tall and not tall) is the treatment condition and income is the outcome.

To find the true causal effect of being tall, we would need to compare how much money someone made if they were tall and how much money that same person made if they were NOT tall. Since both of those things cannot exist, we have to compare what really happened to a prediction of what would have happened if a tall person was short or if a short person was tall.

To find the true causal effect of being tall (the treatment), we would need to compare the incomes (outcomes) for what would have happened had the person been tall (the treatment was received) to the income (the outcome) for what would have happened had they been not tall (the treatment was NOT received). Since we can't observe both of these *potential outcomes*, we must compare our *observed outcomes* to a prediction of their *counterfactuals*.

Randomization

In the case of our earners, we also had gender, age, and weight as additional traits.

If our treatment and control groups look similar across the additional traits, we can use them as counterfactuals for each other. So, if our short and tall groups are similar in terms of gender, age, and weight, we can use them as counterfactuals of each other.

Randomization is considered the best technique to accomplish this. We can think of randomization into one of two groups like the result of a coin toss: heads for the treatment group and tails for the control group. Since no specific trait determines which group an individual is assigned, we should end up with two fairly similar groups.



While randomization is the ideal technique, we frequently are unable to use it (traits like gender and height are inextricably related). As such, much of the field of causal inference is dedicated to establishing techniques to estimate causal effects for nonrandomized studies.

When Randomization Isn't Possible

Randomized experiments are not always feasible because:

Observational Studies and Natural Experiments

Since randomizing subjects into treatment and control conditions is not always possible, we often have to work with observational data. When doing an observational study, the researcher does not manipulate who gets the treatment. The outcomes are merely observed and recorded.

Examples of observational data include:



Some observational studies take advantage of events that result in some randomization of treatment. These types of studies are called natural experiments, or quasi-experimental studies.

Examples of events that might introduce a natural experiment include the following:

Causal Inference Techniques

Through study design and statistical estimation, causal inference methods frequently aim to create situations that look like the product of randomization: two groups with similar characteristics other than the treatment they received.

How does causal inference achieve this? Generally, possibilities include:

These are broad generalizations — the field of causal inference is filled with innovative techniques and statistical algorithms that are constantly evolving to be more accurate and to cover even more situations where experimentation isn't possible.

Introduction to Matching and Weighting Methods

Learn about matching and weighting methods for causal inference.

One of the challenges in using observational data for causal inference is that the treatment and the control groups are not similar at baseline (before there is any treatment). In other words, treatment aside, the two groups may differ from one another in ways that affect their outcomes.

We know randomized experiments are the gold standard for creating balanced treatment and control groups because, in randomized experiments, the two groups only differ by treatment status. However, we can mimic this balancing effect when working with nonrandomized data. Two popular methods for mimicking randomization are matching and weighting. New and more complex variations of these techniques are constantly being developed and employed, but this article will introduce you to the fundamental ideas behind them.

Unbalanced Observational Data

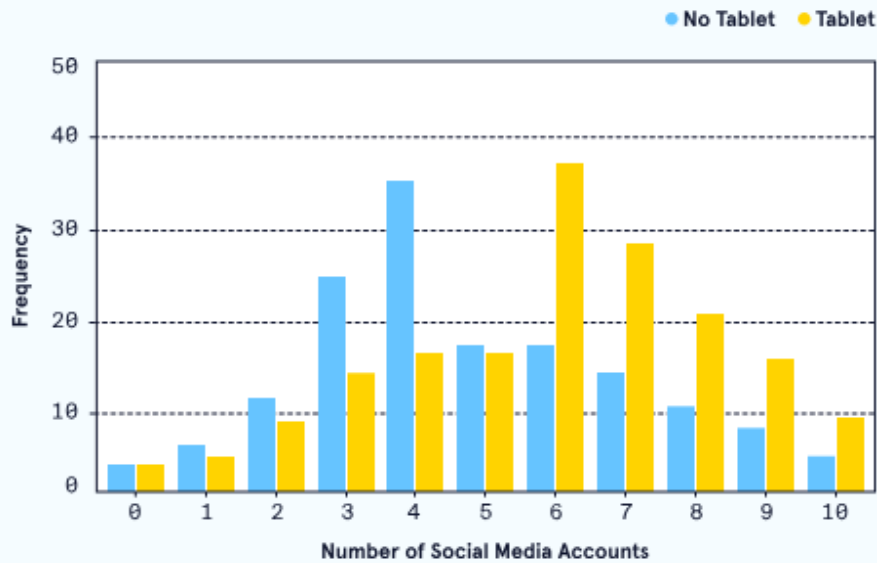
Let's consider an example of nonrandomized data. Imagine we're trying to estimate the causal effect of tablet ownership on weekly screentime. We don't have the funding to run an experiment where we give out tablets to participants, so we survey people to ask them whether they own a tablet and how many hours per week they use devices with screens.

Let's assume a binary treatment status: the value is 1 if the person has a tablet and is 0 otherwise. We notice that those who have tablets have higher weekly screentime on average.

We might already guess that this difference does not necessarily imply a causal effect of tablet ownership. Because this is an observational study, other factors could be involved. For example, suppose that the group that owns a tablet also has a higher number of social media accounts. Because social media use may also affect weekly screentime, we don't know whether the observed difference in screentime is due to social media use or tablet ownership (or something else entirely!).

Suppose the distribution of the number of social media accounts for each group is shown in the plot that follows.

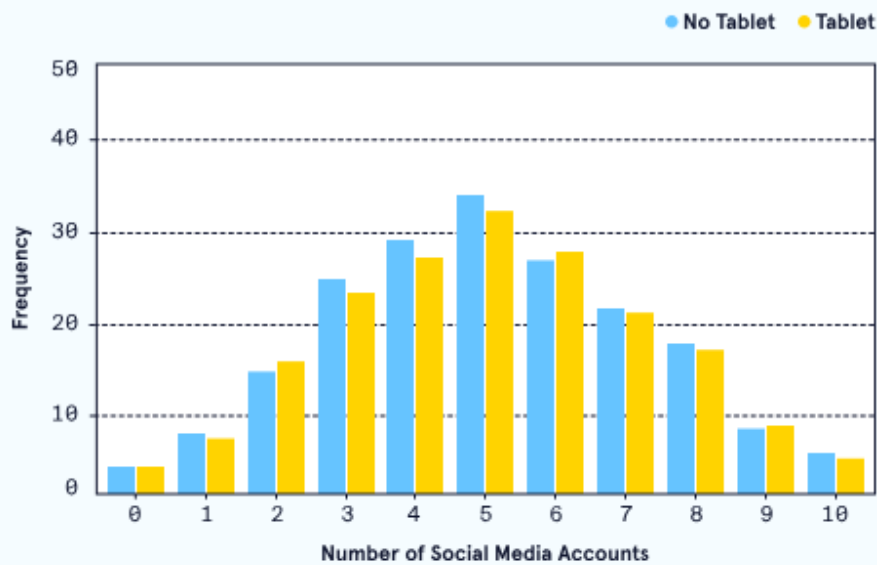
Observational Data



As we can see, the two distributions are different: the treatment group (tablet) appears to have more social media accounts than the control group (no tablet). These groups are different beyond just their treatment status!

In a randomized experiment, we would expect something more balanced like in the following plot.

Experimental Data



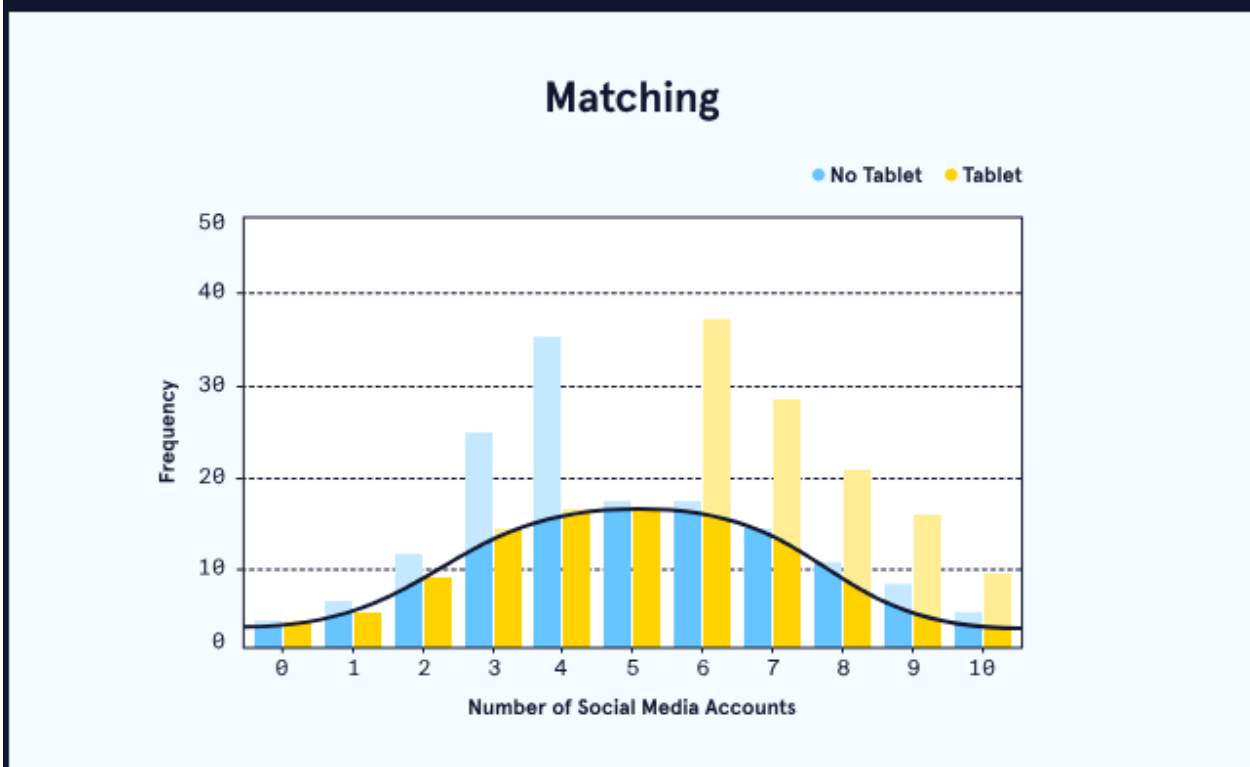
Matching and weighting methods make the two groups distributionally similar in terms of *observed variables*, or the additional attributes that we can measure and record. In the example above, the number of social media accounts is an observed variable. We can use matching or weighting methods to make the distribution of social media accounts similar for the tablet and no-tablet groups.

Matching

In matching, we match observations in the control group to observations in the treatment group based on the observed variables. Matching usually ensures that the two groups are similar in terms of those observed variables. Any unmatched observations are removed.

Let's consider a simple example where we match observations only on the social media accounts variable. We match individuals from the treatment group with individuals in the control group based on how many social media accounts they have. Then we discard any individuals without a match.

We can imagine this as though we are drawing a line across our distribution that cuts every taller bar to the size of the shorter bar in the pair, as illustrated in the following diagram.



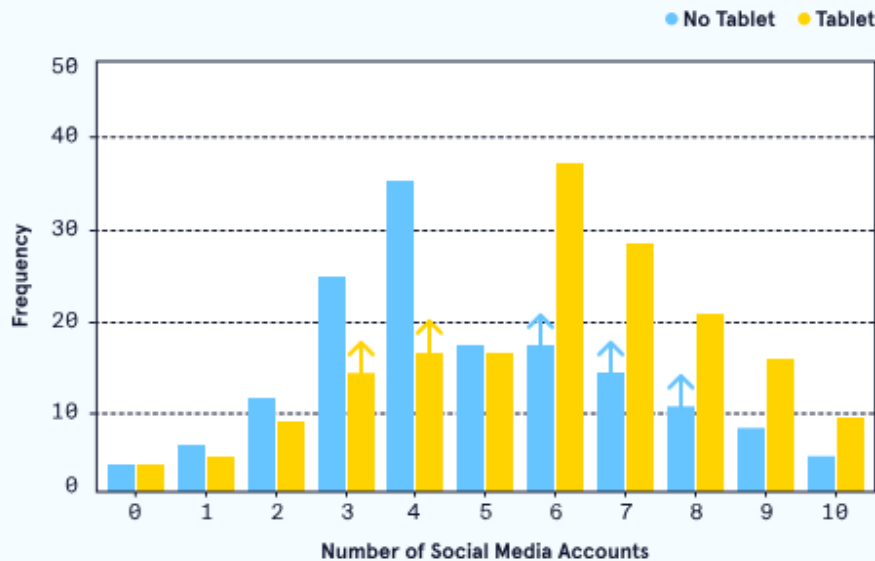
The unmatched observations above our line, such as some people with tablets and 3 accounts or some people without tablets and 7 accounts, will be discarded. We are left with distributions of treatment groups that are centered on 5 accounts and have similar spreads in a bell shape. This new sample looks a lot like the hypothetical randomized experiment data, only with fewer observations in total. This is a very simple example using only one variable, but we can extend this concept to match across multiple variables, or even combinations of variables, to form groups that mimic what we might expect from randomization.

Weighting

One of the drawbacks of matching is that we discarded some of our observations, making our sample size smaller. Losing information like that, especially with a small study, may mean losing *power*, or the ability to detect an existing effect. This is where weighting has an advantage.

The idea behind weighting is similar to matching: to find control observations that are similar to the treatment group. However, instead of keeping or dropping observations, we can give them weights. We can think of a weight as a number that indicates how much we want an observation to be represented in our group. Let's look at the illustration again, this time with weighting employed.

Weighting



Instead of discarding unmatched individuals, we give larger weights to the individuals that are fewer in frequency at each number of accounts. For example, we give more importance to people who have a tablet and only 4 accounts along with people who don't have a tablet and have 8 accounts. This is the same as giving more importance to the outcomes of individuals who look more like people in the opposite treatment group. By manipulating how much each individual's outcome counts for, we are able to make the two groups look more alike and still keep all of our observations!

We should note that, while not used here, we could have also *down-weighted* the outcomes of those that already look a lot like their own treatment group. This means giving these outcomes smaller weights so that they have less impact on our computation of the treatment effect.

Estimands

Because we use matching and weighting methods before estimating a treatment effect, these techniques are part of the identification phase of causal inference. We are using these methods to create groups that are balanced based on observed traits. Assuming the groups don't differ in some unobserved way, this means the only difference between them is the treatment. This way we can justify inferring a causal relationship between the treatment and the outcome variable.

The way we are able to compose these groups determines which estimand we're trying to compute. For example, if we have equal numbers of observations in each treatment group, and we match each observation to a counterpart in the other group, we may estimate the average treatment effect (ATE) because the groups stand as counterfactuals for each other.

Real data is often not so tidy, however. For this and other reasons, researchers frequently use the average treatment effect on the treated (ATT).

When estimating the ATT using matching:

- All of the treated observations are kept.
- Controls are matched to the treated individuals based on variable values.
- Any unmatched controls are removed from the dataset.

Similarly, to estimate the ATT using weighting:

- The treated observations are given a weight of 1.

The control group is made similar to the treatment group by up-weighting some controls (weights greater than 1) and down-weighting others (weights less than 1).

Review

To recap, in randomized experiments, the distribution of variables in the treatment and control groups are similar on average. This is a major advantage of randomization.

We learned that matching and weighting methods are ways of manipulating treatment and control groups in an observational study to make them similar in terms of observed traits.

So, why bother with randomized experiments in the first place? Why not use observational studies all the time? After all, they're cheaper and easier to do. The reason is as follows: While matching and weighting create balance with respect to traits that can be observed, randomization creates balance in terms of ALL traits, observed and unobserved. Randomization may be the top choice for estimating causal effects, but matching and weighting methods are very helpful when that's not possible!

Introduction to Stratification

[Learn about stratification for causal inference.](#)

Congratulations!

You have just been hired by the local health officer to estimate the effect of having high cholesterol on blood pressure. You have a dataset from a hospital and are eager to apply your new knowledge of causal inference.

The dataset includes the following variables on 100 patients:

High cholesterol — whether or not the patient has high cholesterol

Systolic blood pressure — measured in mmHg

Diastolic blood pressure — measured in mmHg

Age — the age of the patient in years

Smoking status — whether or not the patient smokes tobacco products

A primer on blood pressure:

Systolic blood pressure is the force the heart exerts on the walls of arteries DURING each beat

Diastolic blood pressure is the force the heart exerts on the walls of the arteries BETWEEN each beat

Normal blood pressure is generally defined as having a systolic pressure less than or equal to 120 mmHg or a diastolic pressure less than or equal to 80 mmHg.

Your first task is to estimate the effect of having high cholesterol on systolic blood pressure.

Cholesterol is the treatment variable. Blood pressure is the outcome variable. Age and smoking status are *covariates* (variables that are not the variables of interest).

There are many risk factors for high blood pressure, including obesity, diabetes, age, and family history. This is a problem because you will have trouble studying cholesterol (the treatment) independently from these other factors.

Because we cannot randomize who has high cholesterol, there will likely be imbalances in other variables, between patients with high cholesterol (treatment group) and patients who do not have high cholesterol (control group). For example, overall the high cholesterol group might be older or more people might be smokers.

But you have some tools to address this problem. The first is Randomization.

Randomization makes the estimation of causal effects easier. In randomized studies, the covariates in the treatment and control groups look similar to each other (i.e., there just as many smokers are in both groups), so treatment status is the only difference between them. By contrast, in non-randomized studies, the variables of the treatment and control groups may look very different (i.e., more smokers in the high cholesterol group).

You can balance the covariates with matching and weighting methods.

One matching method is to first separate the data into subgroups, or *strata* (*stratification*), based on some variable such as smoking.

First, start by dividing the group into the smokers and non-smokers. Looking at just the smokers, compare the incidence of high cholesterol and high blood pressure. Now do the same for the non-smokers. This article dives more into the specifics, but that is the basic idea: divide the data into groups based on some variable and compare within that group.

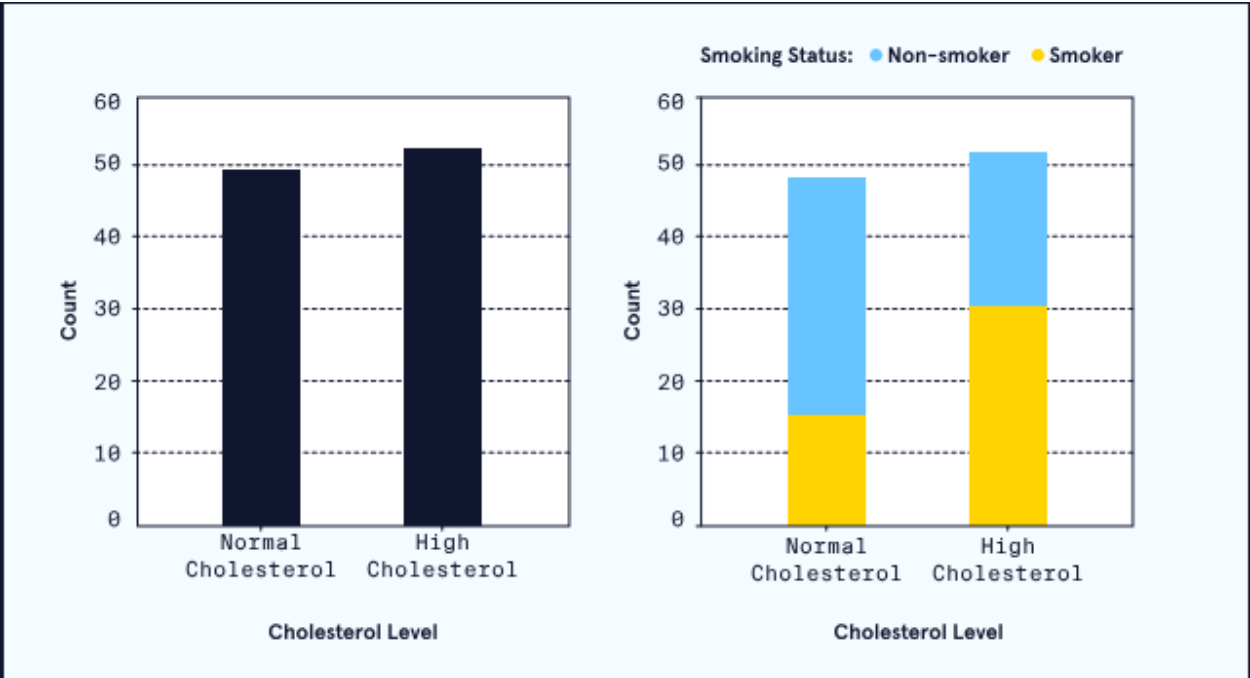
Because the individuals *within the subgroups* are similar to one another, you have satisfied the *conditional exchangeability assumption* of causal inference: if treatment assignments were swapped, the outcomes would still be the same within each subgroup, on average.

Either categorical or numeric variables can be used to split the data into subgroups, but the process differs slightly depending on which type of variable is used.

Stratification on Categorical Variables

Nicotine (the chemical in tobacco products) causes physical changes that cause both high blood pressure and high cholesterol. Because smoking tobacco is related to both high blood pressure (outcome) and high cholesterol (treatment), you may not be able to estimate the effect of high cholesterol on blood pressure if you do not take smoking into account.

The following two plots show what stratifying the blood pressure data according to tobacco use looks like.



In the first plot, we see that the number of individuals with high cholesterol ($n = 52$) is approximately the same as the number who have normal cholesterol ($n = 48$). However, the second plot shows that the ratio of smokers to non-smokers in each group is quite different. Among individuals with normal cholesterol, 33 (69%) are non-smokers; among those with high cholesterol, only 22 (42%) are non-smokers.

If we don't take smoking status into account when comparing the high and normal cholesterol groups, we would be comparing two very different groups. In other words, we would be violating the conditional exchangeability assumption of causal inference. This could lead to a biased estimate of the treatment effect because we would be ignoring the fact that smoking is related to blood pressure.

Estimation of Causal Effects

By separating the observations by smoking status, we have stratified our data into two subgroups. When using stratification, causal effects are not estimated directly from the entire sample, but rather within each subgroup. We can view this process as three steps:

The table below shows the calculation of the average blood pressure in the treatment and control groups for smokers and non-smokers.

Average Blood Pressure

Non-smoker

Smoker

High Cholesterol

139

155

Normal Cholesterol

110

117

Let's follow the three steps above to estimate the treatment effect of tobacco use when we stratify on smoking status.

First, we calculate the average treatment effect within each subgroup:

$$ATE_{\text{Non-smokers}} = 139 - 110 = 29 \text{ mmHg}$$

ATE

Non-smokers

$$= 139 - 110 = 29 \text{ mmHg}$$

$$ATE_{\text{Smokers}} = 155 - 117 = 38 \text{ mmHg}$$

ATE

Smokers

$$= 155 - 117 = 38 \text{ mmHg}$$

Next, we multiply the subgroup ATEs by the proportions of each subgroup. In this case, out of 100 patients, 55 were non-smokers and 45 were smokers.

$$\text{Non-smokers} 29 * \frac{55}{100} = 15.9 \text{ mmHg}$$

Non-smokers 29 *

100

55

$$= 15.9 \text{ mmHg}$$

$$\text{Smokers} 38 * \frac{45}{100} = 17.1 \text{ mmHg}$$

Smokers 38 *

100

45

$$= 17.1 \text{ mmHg}$$

Finally, to estimate a causal effect for ALL patients, we sum the weighted subgroup ATEs.

$$\text{Weighted ATE} = 15.9 + 17.1 = 33 \text{ mmHg}$$

$$\text{Weighted ATE} = 15.9 + 17.1 = 33 \text{ mmHg}$$

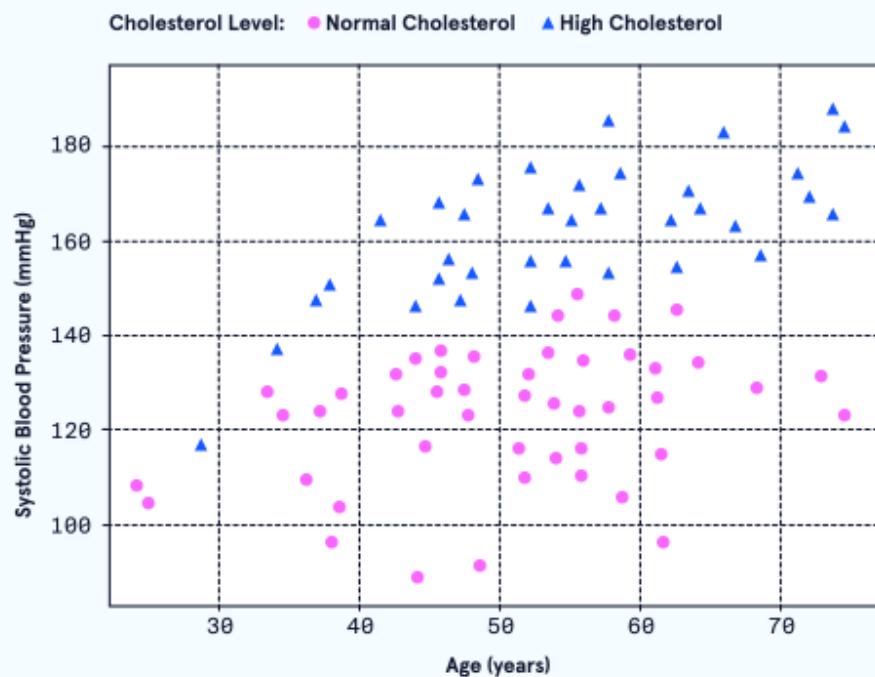
The weighted ATE of 33 mmHg suggests that, when accounting for smoking status, individuals with high cholesterol have on average a systolic blood pressure 33 mmHg higher than that of individuals with normal cholesterol.

Stratification on Numeric Variables

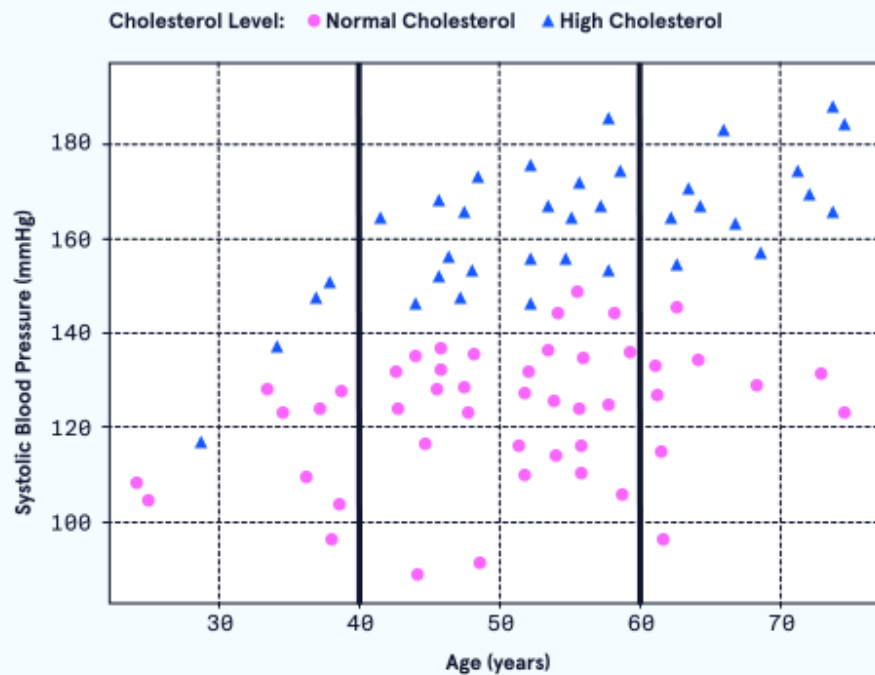
Stratification by numeric variables requires a little more effort. Unlike categorical variables, which have just a few subgroups, numeric variables can vary greatly. This means that without some kind of manipulation of the data, we could end up with a huge number of subgroups each containing very few observations.

Let's think about age first. If we have 100 people age 20-80 (a 60 year span), we will have 60 different ages to work with.

Rather than treating every unique age as its own subgroup, we could lump them together into ranges or *bins*. Let's start with three age groups: younger than 40, between 40 and 60, and older than 60.



Now that we have defined subgroups for age, we can estimate the average treatment effect of tobacco use. We can follow the same three steps we did when stratifying on smoking status.



We look at each subgroup separately and subtract the average blood pressure of the control group from the average blood pressure of the treatment group to get each subgroup's ATE.

$$\text{Age} < 40 \text{ ATE} = 138 - 116 = 22 \text{ mmHg}$$

$$\text{Age} < 40 \text{ ATE} = 138 - 116 = 22 \text{ mmHg}$$

$$40 \leq \text{Age} \leq 60 \text{ ATE} = 154 - 121 = 33 \text{ mmHg}$$

$$40 \leq \text{Age} \leq 60 \text{ ATE} = 154 - 121 = 33 \text{ mmHg}$$

$$\text{Age} > 60 \text{ ATE} = 169 - 127 = 42 \text{ mmHg}$$

$$\text{Age} > 60 \text{ ATE} = 169 - 127 = 42 \text{ mmHg}$$

Then, we weight each subgroup's ATE by the proportion of observations in that subgroup.

$$\text{Age} < 40 \text{ } 22 * (14/100) = 3.08 \text{ mmHg}$$

$$\text{Age} < 40 \text{ } 22 * (14/100) = 3.08 \text{ mmHg}$$

$$40 \leq \text{Age} \leq 60 \text{ } 33 * (56/100) = 18.5 \text{ mmHg}$$

$$40 \leq \text{Age} \leq 60 \text{ } 33 * (56/100) = 18.5 \text{ mmHg}$$

$$\text{Age} > 60 \text{ } 42 * (30/100) = 12.6 \text{ mmHg}$$

$$\text{Age} > 60 \text{ } 42 * (30/100) = 12.6 \text{ mmHg}$$

Finally, we add the weighted averages together.

$$\text{Weighted ATE} = 3.08 + 18.5 + 12.6 = 34.1 \text{ mmHg}$$

$$\text{Weighted ATE} = 3.08 + 18.5 + 12.6 = 34.1 \text{ mmHg}$$

The weighted average treatment effect of 34.1 mmHg suggests that, when accounting for age, individuals with high cholesterol have on average a systolic blood pressure 34.1 mmHg higher than that of individuals with normal cholesterol.

Limitations

You may notice that we got different estimates for the ATE depending on which variable we used to stratify the data, which leads to an important question: which estimate should we trust?

The estimated ATE is just that — an estimate. We could refine our estimate of the ATE by stratifying on both smoking status AND age, which would require us to compute a weighted ATE estimate for each of the six strata. However, stratifying on too many variables comes with its own limitations.

Stratification is useful primarily in *low-dimensional* data, which is data where there are very few observed variables. Stratification on multiple variables can lead to very small subgroups, making estimation of causal effects less reliable. Further, stratification is not a particularly efficient use of data. If the overlap assumption is not met for a subgroup, we cannot estimate a causal effect using that data.

Assumptions

The three main assumptions for causal inference apply for stratification.

Conditional exchangeability: This assumption states that as long as we account for confounders, we would obtain the same outcomes if the groups swapped treatment assignments.

Overlap: This assumption requires that individuals in each subgroup formed by the stratification process must have a positive probability of being in either treatment group. This assumption is necessary because we cannot calculate an average causal effect within a subgroup where 100% of the observations are in either the treatment or control group.

Stable unit treatment value assumption (SUTVA): SUTVA states that the potential outcomes of an individual are not affected by the treatment assignment of any other individuals in the same strata and that the treatment is exactly the same across all individuals.

Review

In this article, we learned that:

Stratification is a matching method for causal inference that is best used with low-dimensional data where treatment and control groups are unbalanced across variables.

Stratification rebalances observations by creating subgroups or “strata” based on variable values. Causal effects can then be estimated by comparing treatment and control groups within these subgroups.

After estimating causal effects within strata, the overall average treatment effect may be estimated by taking a weighted average.



Ggplot2, rdd, dplyr

Propensity Scores

What are Propensity Scores?

One of the main assumptions in causal inference is known as the assumption of *conditional exchangeability*. This assumption states that, so long as we account for confounders (the non-treatment, non-outcome variables), we would observe the same outcomes if the treatment and non-treatment groups were swapped. Conditional exchangeability is achieved via randomization since it balances both observed AND unobserved

[variables](#)

Preview: Docs Loading link description

between treatment groups. However, conditional exchangeability can be difficult to achieve in non-randomized situations.

Propensity score methods are widely used in causal inference because they can help reach conditional exchangeability even when randomization is not possible. So what are propensity scores, and how can we apply propensity score methods to our own questions?

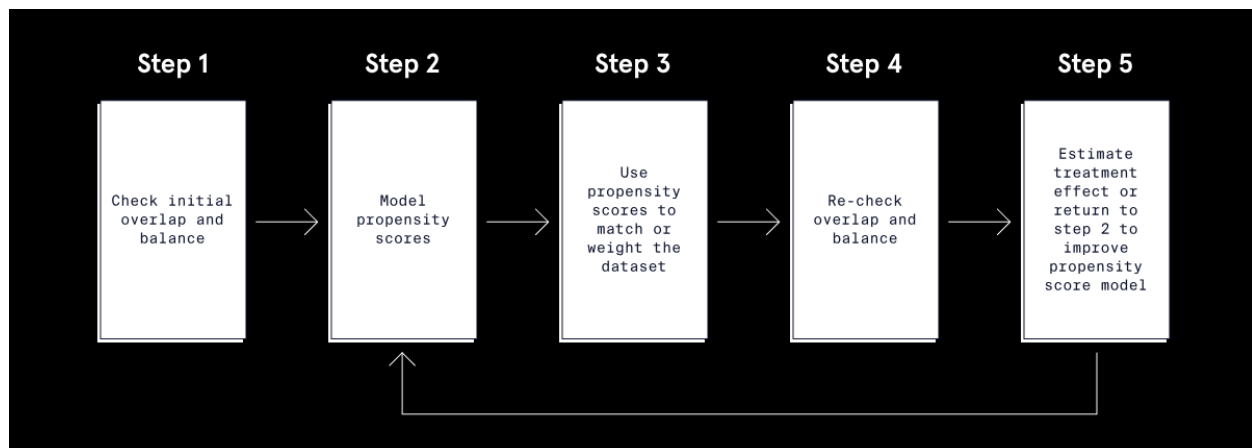
A *propensity score* is essentially the probability of being in a particular treatment group given a set of observed variables. Typically we will think of propensity scores as the probability of being in the treatment group as opposed to the control group. In a sense, propensity scores summarize all the traits of an observation to a single score, which can be an advantage when there are lots of observed variables.

Propensity score analyses can be broken down into five ordered steps:

- Check initial overlap and balance.
- Model propensity scores.
- Use propensity scores to weight the dataset.
- Re-check overlap and balance.
- Estimate the treatment effect, or return to step two to improve the propensity score model.

Instructions

Take a look at the flowchart in the learning environment. This flowchart describes the five general steps that we can use as a template for applying propensity score methods in analysis. Keep this process in mind as we move through the rest of this lesson.



Propensity Scores

Variable Overlap and Balance

The first step in a propensity score analysis is to check how similar the treatment and control groups are at baseline, before using propensity score methods. There are two measures that are commonly used to describe the degree of similarity between treatment groups: *overlap* and *balance*.

Overlap is the range of values of a variable that the treatment and control groups have in common.

Overlap can also be thought of as the range of values of a variable where the probability of being in the treatment group is greater than 0 but less than 1.

We already know that overlap is an important assumption of causal inference!

Balance describes how similar the treatment and control groups are with respect to the entire distribution of each of the other

[variables](#)

Preview: Docs Loading link description

.

Balance is expressed as a statistic that summarizes the entire distribution of a variable. Two

[statistics](#)

Preview: Docs Loading link description

are commonly used to measure balance.

Standardized mean difference (SMD). The SMD of a variable in a sample is defined as the difference in the average value of the variable between groups divided by the standard deviation of the variable in both groups.

Variance ratio. The variance ratio of a variable in a sample is the variance of the variable in one treatment group divided by the variance of the variable in the other treatment group.

So how is “good” or “bad” balance defined?

An SMD close to zero indicates good balance. This means the average value (and thus the center of the distribution) of the variable is similar between the treatment and control groups.

A variance ratio close to one is another indicator of good balance. This means that the variability, or spread, of the variable is the same in both groups.

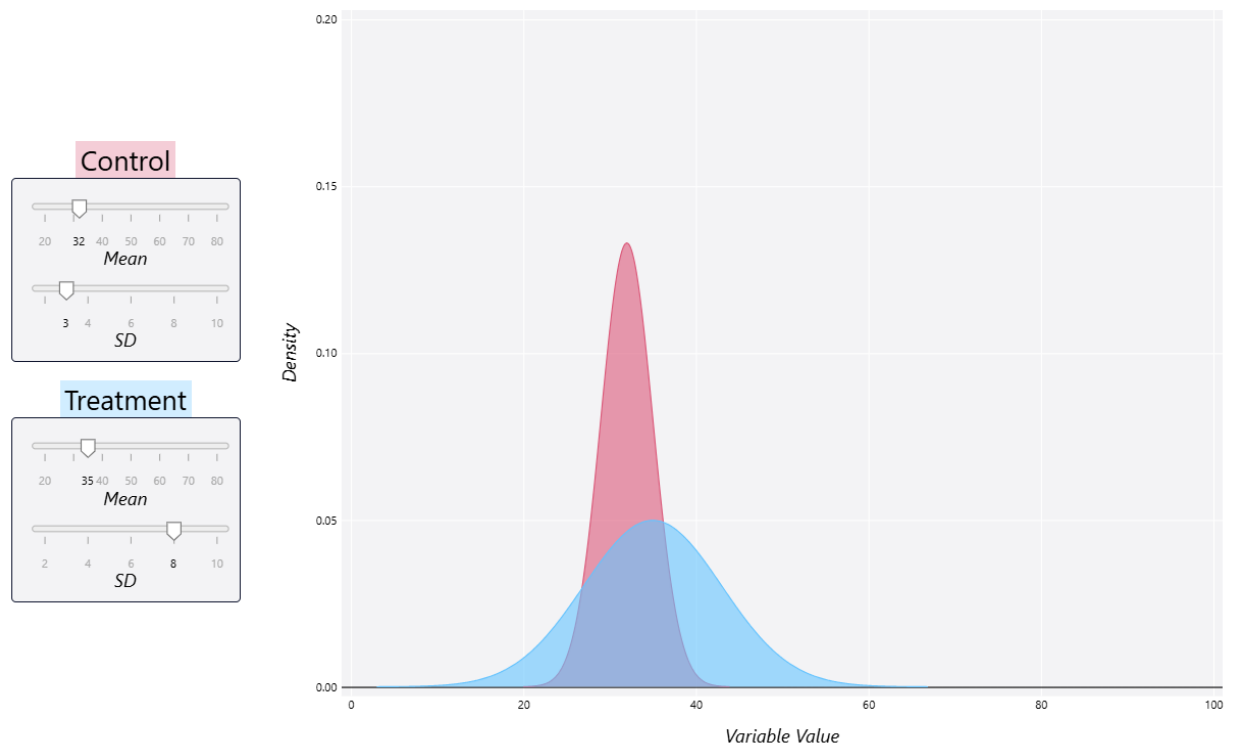
Instructions

1. The interactive visualization to the right illustrates how the overlap and balance of a variable are related to the characteristics of a variable's distribution. Adjust the means and standard deviations of the distributions to see how their shapes change. Then note how the adjustments cause the distributions to cover more or less of one another.

Try making the mean 40 for both distributions. How do you think having the same mean affects the SMD value?

Keeping both means at 40, try making one standard deviation 2 and the other 10. The standard deviation is the square root of the variance. How do you think changes in the standard deviation will impact the variance ratio?

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-matching-and-weighting-methods/lessons/propensity-scores/exercises/variable-overlap-and-balance>



Propensity Scores

Visual Check of Numeric Variables

Suppose we are interested in determining whether the practice of meditation increases the amount of sleep that university students get per night. To gather more information, we surveyed 250 students about their sleeping and meditation habits over the previous year. Note that students were not randomly assigned to treatment groups; students were simply asked about their actions. Their responses were recorded in a dataset with these variables:

`sleep` — average hours of sleep per night (outcome variable)

`meditate` — indicates whether or not the individual reports consistent use of meditation (treatment group variable)
`stress` — a self-reported measure of stress on a 1-to-100 scale, with 1 representing no stress and 100 representing extremely high stress
`graduate` — indicates whether or not a student is in a graduate program versus an undergraduate program (0 = undergraduate; 1 = graduate).

We can use the `bal.plot()` function from the R package, `cobalt`, to visually check if any

[variables](#)

Preview: Docs Loading link description

are severely imbalanced and whether propensity score methods might be useful. The function takes a formula where the left side specifies the treatment group indicator and the right side includes a variable we want to view. To check the `stress` variable, the formula would be `meditate ~ stress`. Then we specify the dataset name and the variable of interest in quotes as additional arguments.

```
# import library
library(cobalt)
# plot distributions for stress variable
bal.plot(
  x = meditate ~ stress, #formula
  data = sleep_data, #dataset
  var.name = "stress" #variable
  colors = c("#E69F00", "#009E73") #set fill colors
)
```

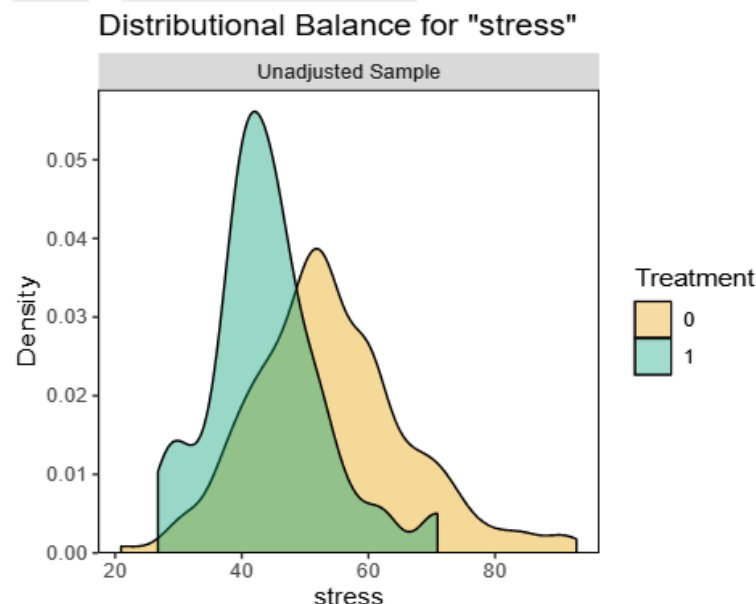
Copy to Clipboard

Note that we also set the optional

[argument](#)

Preview: Docs Loading link description

`colors` to `c("#E69F00", "#009E73")` for better contrast.



The distributions in the plot appear to differ pretty substantially between the treatment groups, potentially indicating poor balance. The meditation group (green) is centered around a score of 40 with a narrow spread, while the non-meditation group (orange) is centered around 55 with a much wider spread of stress scores.

Instructions

Checkpoint 1 Enabled

1.

It's time to start getting some hands-on experience with propensity score methods! Imagine that you are working for a hospital and you are tasked with figuring out why some patients stay longer than others. You decide to investigate heart health first, and will look at ejection fraction. Ejection fraction (EF) is a measure of how much blood the heart pumps with each beat. A low EF may indicate poor heart health.

You have access to a dataset called `los_data`, which contains the following variables related to the health of 1,000 hospital patients who were admitted to the cardiology unit:

- `age` — Age (years)
- `cholesterol` — Indicates whether the patient has high cholesterol (0 = normal cholesterol; 1 = high cholesterol)
- `heart_attack` — History of heart attack (0 = no heart attack history; 1 = previous heart attack)
- `low_ef` — Low ejection fraction (0 = normal ejection fraction; 1 = low ejection fraction)
- `hospital_los` — Length of hospital stay (days)

With this dataset, we will use propensity score methods to determine the causal effect of having a low ejection fraction on how long patients stay in the hospital.

The dataset `los_data` has been loaded for you in **notebook.Rmd**. Use the `bal.plot()` function to create a balance plot for the `age` variable. Save this plot as an object named `age_plot`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print `age_plot` and view the plot of the distributions. Think about whether `age` appears balanced based on your plot.

```
---
title: "Visual Check of Numeric Variables"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(cobalt)

import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r}
Create a balance plot for age here
age_plot <- bal.plot(
 x = low_ef ~ age, #formula
 data = los_data, #dataset
 var.name = "age", #variable
 colors = c("#E69F00", "#009E73") #set fill colors
)
```

# Checkpoint 2
```{r}
Print age_plot here
print(age_plot)
```
```

Propensity Scores

Visual Check of Categorical Variables

Distribution plots are great for numeric

[variables](#)

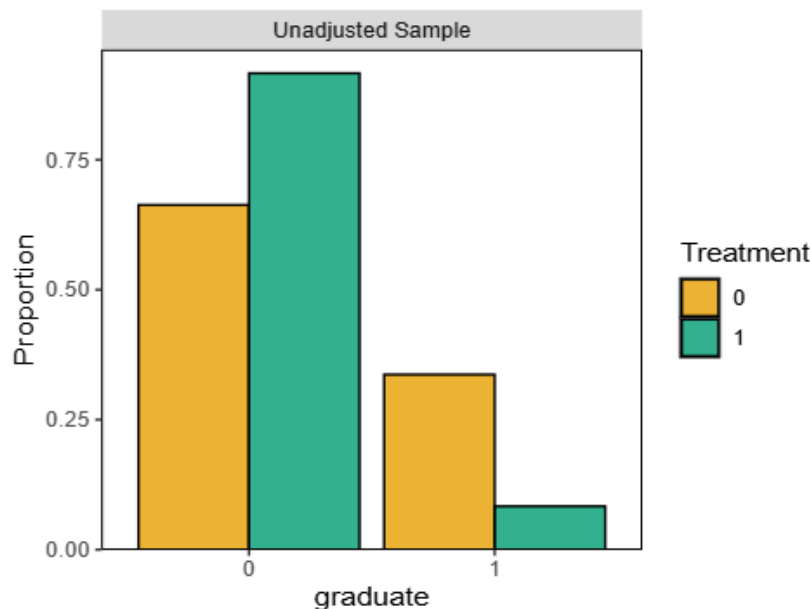
Preview: Docs Loading link description

, but we need a different type of plot for categorical variables. Fortunately, we can use the exact same `bal.plot()` function from `cobalt` with no need to specify the variable type. By updating the arguments for `x` and `var.name` to use `graduate`, we will get a bar plot to examine balance for the categorical variable `graduate`.

```
# import library
library(cobalt)
# plot distributions for stress variable
bal.plot(
  x = meditate ~ graduate, #new formula
  data = sleep_data, #dataset
  var.name = "graduate", #new variable
  colors = c("#E69F00", "#009E73") #set fill colors
)
```

Copy to Clipboard

Distributional Balance for "graduate"



From this plot, we see that the ratio of undergraduates to graduates is much larger for the meditation group (green) than for the non-meditation group (orange).

Both plots so far suggest that there are differences between the treatment and control groups with respect to the `stress` and `graduate` variables. However, balance plots don't precisely quantify the degree of imbalance in the dataset. To get a more detailed picture, we can check balance numerically.

Instructions

Checkpoint 1 Enabled

1.

Let's return to the heart health dataset.

Use the `bal.plot()` function to create a balance plot for the `heart_attack` variable. Save this plot as an object named `heart_plot`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print `heart_plot` to view the plot. Does heart attack history appear balanced across treatment groups?

```
---
title: "Visual Check of Categorical Variables"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(cobalt)

import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r}
Create a balance plot for heart_attack
heart_plot <- bal.plot(
 x = low_ef ~ heart_attack, #formula
 data = los_data, #dataset
 var.name = "heart_attack", #variable
 colors = c("#E69F00", "#009E73") #set fill colors
)
```

# Checkpoint 2
```{r}
Print heart_plot here
print(heart_plot)
```
```

Propensity Scores

Checking Balance Numerically

While visual assessments of balance are definitely helpful, we can also assess overlap and balance numerically using the standardized mean difference (SMD) and variance ratio for each variable.

Observing an SMD of exactly zero or a variance ratio of exactly one is pretty uncommon. Therefore, the following guidelines can be used to indicate good balance:

SMD between -0.1 and 0.1

Variance ratio between 0.5 and 2.0

The `bal.tab()` function from the cobalt package is a complement to the `bal.plot()` function that quantifies the balance of

[variables](#)

Preview: Docs Loading link description

in a dataset. The `bal.tab()` function has similar arguments and syntax to the `bal.plot()` function. We need to update our formula to include both variables of interest. Then we can show SMDs for all variables and variance ratios for all continuous variables in the sleep dataset by specifying `binary = "std"` and `disp.v.ratio = TRUE`, respectively:

```
# import library
library(cobalt)
# print table of SMDs and variance ratios
bal.tab(
  x = meditate ~ stress + graduate, #formula
  data = sleep_data, #dataset
  disp.v.ratio = TRUE, #display variance ratio
  binary = "std" #SMDs for binary variables
)
```

Copy to Clipboard

The output of the `bal.tab()` that follows shows that the `stress` variable has an SMD of -0.9132 and a variance ratio of 0.5461 between the treatment and control groups. The `graduate` variable has an SMD of -0.6548.

```
Balance Measures
      Type   Diff.Un  V.Ratio.Un
stress  Contin. -0.9132    0.5461
graduate Binary -0.6548
```

```
Sample Sizes
      Control Treated
All       190      60
```

Copy to Clipboard

The SMDs clearly fall outside the range of -0.1 to 0.1, which suggests there is an imbalance between the treatment and control groups. The variance ratio for the `stress` variable is only just within the acceptable range. Time to put propensity score methods to the test to see if we can reduce this imbalance!

Instructions

Checkpoint 1 Enabled

1.

Again working with our heart health dataset, use the `bal.tab()` function to get balance metrics for `age`, `cholesterol`, and `heart_attack` in the `los_data` dataset. Make sure you set `disp.v.ratio` to `TRUE` and `binary` to `"std"`. Save the table as `bt`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print `bt` to view the results. Do any variables appear to have poor balance? Do these results make sense based on the balance plots you made in the previous two exercises?

```
---
title: "Checking Balance Numerically"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(cobalt)

import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r, message=FALSE}
Create a balance table here
bt <- bal.tab(
 x = low_ef ~ age + cholesterol + heart_attack, #formula
 data = los_data, #dataset
 disp.v.ratio = TRUE, #print variance ratios
 binary = "std" #SMDs for binary variables
)
```

# Checkpoint 2
```{r}
Print balance table here
print(bt)
```
```

Propensity Scores

Modeling Propensity Scores

Returning to our student sleep data, we are interested in the effect of meditation on sleep. It seems intuitive that people with high levels of stress might struggle with sleep AND might be less likely to engage in coping mechanisms such as meditation. We can use propensity scores to model this interaction.

Propensity scores reflect the probability of being in the treatment group, as opposed to the control group, given a set of characteristics. Because this probability corresponds to a

[binary](#)

Preview: Docs Loading link description

outcome—either being in the treatment group or the control group—we can model the propensity scores using logistic regression. The outcome of the regression will predict whether or not an individual is in the treatment group. It will use potential confounding

[variables](#)

Preview: Docs Loading link description
as predictors.

With regards to our sleep data, the propensity score should model the probability of practicing meditation based on the other characteristics in the data. Let's start with a propensity score model with the `meditate` variable as the outcome and the `stress` variable as a predictor.

The `glm()` function in R makes modeling propensity scores via logistic regression simple. By default, the `glm()` function fits a linear regression model, so we need to modify the `family`

[argument](#)

Preview: Docs Loading link description

to specify that the treatment group variable is binary. To do this, we set the `family` argument to `"binomial"`:

```
prop_model <- glm(
  formula = meditate ~ stress, #formula
  data = sleep_data, #dataset
  family = "binomial" #specify logistic regression
)
```

Copy to Clipboard

To get a sense of what the propensity scores produced from a logistic regression look like, let's take a look at a histogram of the propensity scores from `prop_model`.

The plot indicates that our model produced a lot of low probabilities of being treated for our observations. This makes sense given that the `bal.tab()` output in the last exercise showed us we had 190 students in the non-meditation group and only 60 in the meditation group.

Instructions

Checkpoint 1 Enabled

1.

Since, in our heart health example, we are trying to estimate the causal effect of low ejection fraction on length of hospital stay, the propensity score model should model the probability of having low ejection fraction given other characteristics.

The dataset `los_data` has been loaded for you in **notebook.Rmd**. Use the `glm()` function to run a logistic regression that models the probability of having low ejection fraction given age and cholesterol level. Save the results as `ef_model`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment and run the code to create a histogram (named `ef_hist`) of the propensity scores predicted from the `ef_model` logistic regression. The histogram will print automatically. Do you have a lot of low or high scores, or is the distribution more symmetric?

```
---
title: "Modeling Propensity Scores"
output: html-notebook
---
```

```

```{r}
import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r}
Predict ejection fraction from age and cholesterol
ef_model <- glm(
 formula = low_ef ~ age + cholesterol, #predict low EF from age and cholesterol
 data = los_data, #dataset
 family = "binomial" #specifies logistic regression model
)
```

# Checkpoint 2
```{r}
Uncomment the code to view the histogram
ef_ps <- predict(ef_model, type = "response")
ef_hist <- hist(ef_ps)
```

```

Propensity Scores

Propensity Score Weighting

Now that you know about the basics of propensity scores, let's talk about some possible applications.

Propensity scores often show up in matching and stratification. However, we will focus on *propensity score weighting* because it is a popular choice with strong performance.

Propensity score weighting transforms estimated propensity scores into *weights* that emphasize or diminish certain observations in our dataset. One widely used type of propensity score weighting is known as inverse probability of treatment weighting (IPTW).

IPTW weights are calculated differently depending on whether we want to estimate the average treatment effect (ATE) or the average treatment effect on the treated (ATT). Note that with ATE we are looking at the effect across the entire population, both the treated and control groups. ATT is just on the treated group.

The formulas for the treatment and control group weights for each estimand are below. p represents the propensity score for a particular individual.

| | ATE | ATT |
|-------------------------|-----------|-----------|
| Treatment Weight | $1/p$ | 1 |
| Control Weight | $1/(1-p)$ | $p/(1-p)$ |

It may not be immediately obvious, but these formulas tell us that IPTW makes the treatment and control groups appear more similar in two ways:

By giving MORE weight to individuals who look like those in the **opposite** treatment assignment.
By giving LESS weight to individuals who look like those in their own treatment assignment.

Check out the following table that shows some example weights for treatment and control observations using IPTW for the ATE.

| treatment group | propensity score | weight |
|-----------------|------------------|--------|
| treatment | 0.1 | 10.0 |
| treatment | 0.5 | 2.0 |
| treatment | 0.9 | 1.1 |
| control | 0.1 | 1.1 |
| control | 0.5 | 2.0 |
| control | 0.9 | 10.0 |

Note the pattern:

The treated individual with the low propensity score of 0.1 (looks more like a control) is given a high weight of 10.

The treated individual with the high propensity score of 0.9 (looks like the other treated individuals) is given a low weight of 1.1.

The weighting goes in the opposite direction for the controls.

The justification for this

[method](#)

Preview: Docs Loading link description

is that someone who looks more like the individuals in the other treatment group is a better counterfactual for them, so we count these individuals as more important. This helps balance our treatment groups without discarding any observations.

Instructions

Checkpoint 1 Enabled

1.

Let's look at a sample of the data from our heart health example.

A small dataset called `los_sample` has been loaded for you in **notebook.Rmd**. This dataset contains a sample of observations from `los_data` for two variables:

`low_ef` — indicates the control (`low_ef = 0`) or treatment (`low_ef = 1`) group

`ps` — indicates the propensity score for each observation

Print `los_sample` using the `print()` function. Note that the dataset is arranged by treatment group with propensity scores in ascending order within each group.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment the code to make a variable called `ATE_weight` with the weights when calculating the ATE. Then print `los_sample` again to view the ATE weights. Why do you think the range of ATE weights is not as large for `low_ef = 0` in this sample?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Uncomment the code to make a variable called `ATT_weight`. This time the weights are for the ATT. Print `los_sample` again to view the ATT weights in comparison to the ATE weights. How does the weighting compare? Think about what's similar and what's different and why that might be.

```
---
title: "Propensity Score Weighting"
output: html-notebook
---

```{r}
import dataset
los_sample <- read.csv("los_sample.csv")
```

# Checkpoint 1
```{r}
Print the dataset
print(los_sample)
```

# Checkpoint 2
```{r}
Uncomment for IPTW weights for the ATE
los_sample$ATE_weight <- ifelse(
 los_sample$low_ef == 0,
 1 / (1 - los_sample$ps),
 1 / los_sample$ps
)
Print the dataset
print(los_sample)
```

# Checkpoint 3
```{r}
Uncomment for IPTW weights for the ATT
los_sample$ATT_weight <- ifelse(
 los_sample$low_ef == 0,
 los_sample$ps / (1 - los_sample$ps),
 1
)
Print the dataset
print(los_sample)
```
```

Propensity Scores

Performing IPTW in R

If this seems like a lot of work, don't worry! The `WeightIt` package in R has functions to model the propensity scores and simultaneously perform propensity score weighting. We don't need to make a separate logistic regression or compute the weights manually using a formula.

IPTW can be performed in R with the `weightit()` function from the `WeightIt` package. There are several key arguments to this function that allow us to tweak how weighting is performed.

`formula`—represents the propensity score model to use.

`method`

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

—determines the weighting method that will be used. While there are various options, we will use “ps” to perform IPTW using logistic regression.

`estimand`—specifies the desired treatment effect estimand: “ATE” for average treatment effect, “ATC” for average treatment effect on the controls, or “ATT” for average treatment effect on the treated.

To perform IPTW for the ATT on the student sleep data, we fill in these arguments accordingly. Remember that our propensity score model includes `meditate` as the outcome and only `stress` as the predictor.

```
# import library
library(WeightIt)
# model propensity scores and IPTW weights
iptw_sleep <- weightit(
  formula = meditate ~ stress, #propensity model
  data = sleep_data, #dataset
  method = "ps", #use IPTW
  estimand = "ATT" #estimand
)
```

Copy to Clipboard

The `weightit()` function models the propensity scores and creates the IPTW weights in one step. We save these outputs in a `weightit` object we name `iptw_sleep`, which we will use in our next step.

Instructions

Checkpoint 1 Enabled

1.

Great job so far!

Now you will perform IPTW using the `los_data` dataset, which has been loaded for you in **notebook.Rmd**. Use the `weightit()` function to model propensity scores and perform IPTW for the ATT. Remember we are modeling the probability of having low ejection fraction given age and cholesterol level.

Save the results as `iptw_ef`.

```
---
title: "Performing IPTW in R"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(WeightIt)
import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r}
Create a weightit object called iptw_ef
iptw_ef <- weightit(
 #propensity model
```



```

formula = low_ef ~ age + cholesterol,
data = los_data, #dataset
method = "ps", #propensity score weighting
estimand = "ATT" #estimand
)
...

```

## Propensity Scores

### Re-checking Overlap and Balance

If propensity score weighting is successful, we expect the distribution of propensity scores in the treatment group to be similar to that of the control group.

To check the overall balance of propensity scores, we can again use the `bal.plot()` function from the `cobalt` package. This time we pass the `weightit` object to the `x`

#### [argument](#)

Preview: Docs An argument is the actual value of a variable passed into a function.

and `"prop.score"` to the `var.name` argument, with no need for the `data` argument. Lastly, we set `which` equal to `both` so we can view the propensity scores before ("unadjusted") and after ("adjusted") weighting is performed.

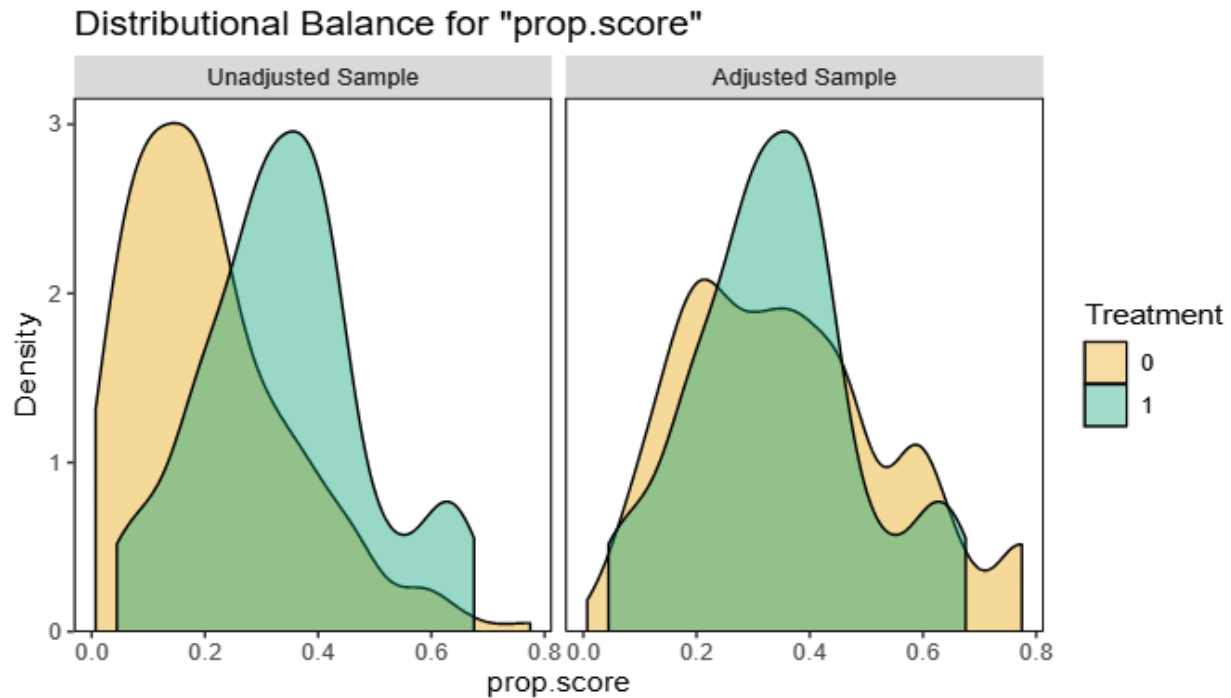
```

import library
library(cobalt)

create balance plot of propensity scores
bal.plot(
 x = iptw_sleep, #weightit object
 var.name = "prop.score", #propensity scores
 which = "both", #before and after
 colors = c("#E69F00", "#009E73") #sets fill colors
)

```

Copy to Clipboard



The distributions of propensity scores look similar after IPTW, but we should check the balance of individual

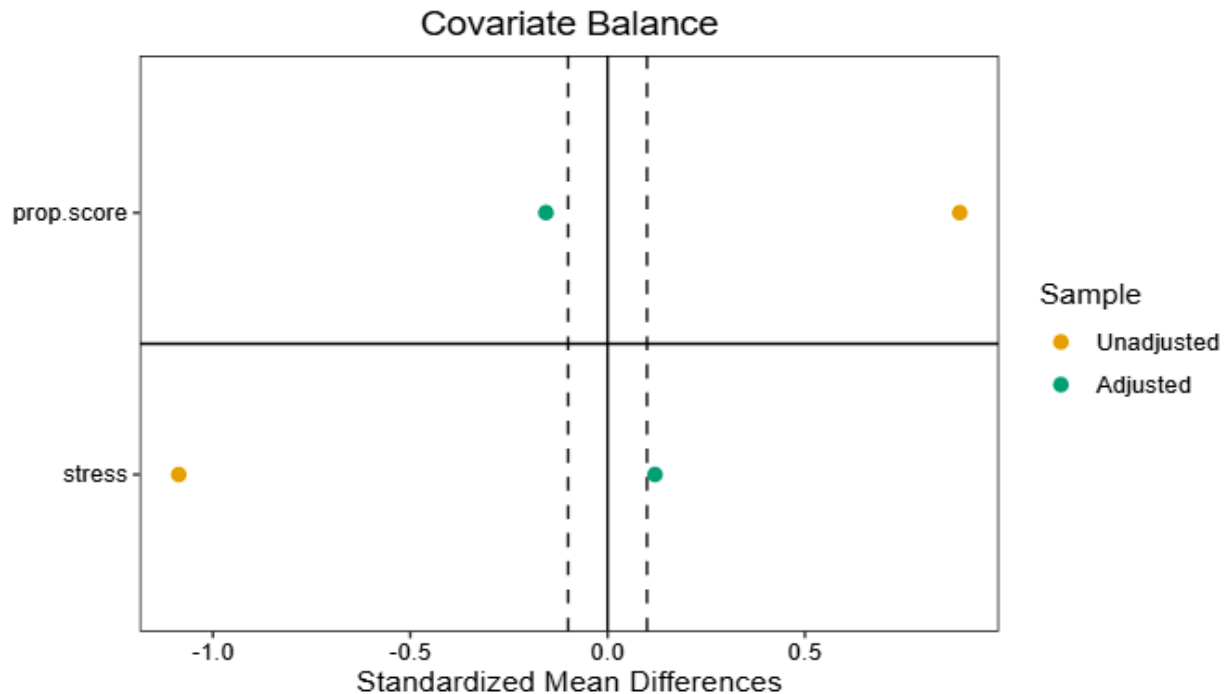
[variables](#)

Preview: Docs Loading link description

, too. The `love.plot()` function in cobalt visually checks the standard mean differences (SMD) between treatment groups for all variables before and after weighting. We can also show guidelines at  $\pm 0.1$  SMD by setting `thresholds = c(m = 0.1)`.

```
love.plot(
 x = iptw_sleep, #weightit object
 binary = "std", #use SMD
 thresholds = c(m = 0.1), #guidelines
 colors = c("#E69F00", "#009E73") #sets fill colors
)
```

Copy to Clipboard



Oh no! Based on this plot, it appears as if the propensity score weighting was unsuccessful: the SMDs between groups are outside the  $\pm 0.1$  cutoffs for the `stress` variable and for the propensity scores themselves. Since there is still some residual imbalance between treatment groups, we should backtrack to step 2 and refine the propensity score model.

#### Instructions

Checkpoint 1 Enabled

1.

Time to assess the quality of the propensity score weighting you completed in the previous exercises! The `los_data` dataset has been loaded for you, and code has been provided for the `weightit` model, which is saved as `iptw_ef`.

Use the `bal.plot()` function to show the distribution of propensity scores before and after the IPTW procedure and save the plot as `ps_bal`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print and inspect `ps_bal`. Do you think an adequate balance has been achieved?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the `love.plot()` function to plot the balance of variables after the propensity score weighting/IPTW.

Save this plot as `ps_love`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print and inspect `ps_love`. Take note of the standardized mean differences. Do the values match what you expected from the balance plot?

```

title: "Re-checking Overlap and Balance"
output: html-notebook

```{r, message=FALSE}
# import libraries
library(WeightIt)
library(cobalt)
# import dataset
```

```

los_data <- read.csv("los_data.csv")
# create IPTW weights
iptw_ef <- weightit(
  formula = low_ef ~ age + cholesterol,
  data = los_data,
  estimand = "ATT",
  method = "ps"
)
...
# Checkpoint 1
```{r}
Create balance plot here
ps_bal <- bal.plot(
 x = iptw_ef, #weightit object
 var.name = "prop.score", #propensity scores
 which = "both", #before and after
 colors = c("#E69F00", "#009E73") #sets fill colors
)
...
Checkpoint 2
```{r}
# Print ps_bal
print(ps_bal)
...
# Checkpoint 3
```{r}
Create a Love plot here
ps_love <- love.plot(
 x = iptw_ef, #weightit object
 binary = "std", #use SMD
 thresholds = c(m = 0.1), #guidelines
 colors = c("#E69F00", "#009E73") #sets fill colors
)
...
Checkpoint 4
```{r}
# Print ps_love
print(ps_love)
...

```

Propensity Scores

Refining the Propensity Score Model

As you may have noticed, propensity score methods are an iterative process: we check variable balance, model propensity scores, perform weighting, then check balance again. If imbalance still exists, we can change the propensity score model. A main assumption of propensity score weighting is that we've modeled the propensity scores correctly – a poor model could lead to biased estimates of the treatment effect!

Let's update our propensity score model from the student sleep data to see if imbalances between groups can be reduced further.

The initial propensity score model only included the `stress` variable as a predictor of meditation, but what happens if we add the `graduate` variable as a second predictor? We need to update the formula in the `weightit()` function:

```
# import library
library(WeightIt)
# update weightit object
iptw_sleep_update <- weightit(
  #new formula
  formula = meditate ~ stress + graduate,
  data = sleep_data,
  estimand = "ATT",
  method = "ps"
)
```

Copy to Clipboard

We re-check balance again to see if the new propensity score model produces a better balance between groups.

```
# import library
library(cobalt)
# create Love plot of updated model
love.plot(
  x = iptw_sleep_update, #updated model,
  binary = "std", #show SMD
  thresholds = c(m = 0.1), #guidelines
  colors = c("#E69F00", "#009E73") #fill colors
)
```

Copy to Clipboard

Success! Both plots show that this new propensity score model produces improved balance. The SMDs now fall between -0.1 and 0.1.

Note that it can take multiple tries to find a good propensity score model. If we needed to further refine our model, we might add more complex terms to the equation, such as polynomial terms or interactions.

Instructions

Checkpoint 1 Enabled

1.

Update the `formula` argument of the `weightit` model to add the `heart_attack` variable to the propensity score model as a predictor of low ejection fraction. Then uncomment and run the code to assign the output to `iptw_ef2`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a new Love plot showing the standardized mean differences and thresholds of ± 0.1 using the updated `weightit` model. Save your plot as `love_update`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print `love_update` and inspect the plot. Did your new model improve the balance?

```
---
title: "Refining the Propensity Score Model"
output: html-notebook
```

```

---

```{r, message=FALSE}
import libraries
library(WeightIt)
library(cobalt)
import dataset
los_data <- read.csv("los_data.csv")
```

# Checkpoint 1
```{r}
Update formula and uncomment
iptw_ef2 <- weightit(
 formula = low_ef ~ age + cholesterol + heart_attack,
 data = los_data,
 estimand = "ATT",
 method = "ps"
)
```

# Checkpoint 2
```{r}
Create Love plot here
love_update <- love.plot(
 x = iptw_ef2, #updated model,
 binary = "std", #show SMD
 thresholds = c(m = 0.1), #guidelines
 colors = c("#E69F00", "#009E73") #fill colors
)
```

# Checkpoint 3
```{r}
Print love_update
print(love_update)
```

```

Propensity Scores

Estimating Causal Effects

Now that we have a good balance, we can proceed to the last step of a propensity score analysis: estimating the causal treatment effect.

If we think back to the beginning of our lesson, the motivation for studying student sleep was to estimate the effect of meditation on average sleep in university students. So, to estimate the causal treatment effect of meditation, we need to fit a regression model for the outcome variable (hours of sleep) and incorporate the propensity score weights from our `weightit` model.

The final regression model should include hours of sleep as the outcome variable, use of meditation as the treatment group variable, and stress level and graduate status as the other predictor

[variables](#)

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

To use the propensity score weights from IPTW, we set the `weights`

[argument](#)

Preview: Docs An argument is the actual value of a variable passed into a function.

of the `glm()` function equal to the estimated IPTW weights. These are stored in our updated `weightit` model that we called `iptw_sleep_update`.

```
outcome_mod_weight <- glm(  
  #outcome model  
  formula = sleep ~ meditate + stress + graduate,  
  #dataset  
  data = sleep_data,  
  #IPTW weights  
  weights = iptw_sleep_update$weights  
)
```

Copy to Clipboard

To get the estimated treatment effect, we use the `coeftest()` function from the `lmtest` package. Weighting can cause our standard errors to be inaccurate. To get the best estimate of the treatment effect, we need a more *robust* calculation of the standard errors, so we add the argument `vcov. = vcovHC` made available by the `sandwich` package. We won't cover this in detail here, but this adjustment means we are using a *heteroscedasticity-consistent estimation of the covariance matrix* for estimates of the coefficients.

```
# import library  
library(lmtest)  
library(sandwich)  
# perform tests of regression coefficients  
coeftest(  
  outcome_mod_weight, #weighted outcome model  
  vcov. = vcovHC #robust standard errors  
)
```

Copy to Clipboard

As we can see in the following output, the coefficient for the meditation variable is 1.02. If we have met the assumptions of IPTW, this means that we can conclude that a typical student who practiced meditation got an additional 1.02 hours of sleep because of meditation.

```
z test of coefficients:  
  
              Estimate Std. Error z value Pr(>|z|)  
(Intercept)  8.971964   0.669241  13.4062 < 2.2e-16 ***  
meditate      1.024871   0.215333   4.7595 1.941e-06 ***  
stress       -0.045191   0.013664  -3.3072 0.0009422 ***  
graduate     -0.770913   0.280460  -2.7487 0.0059823 **  
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Copy to Clipboard

NOTE: You may need to adjust the width of this section of the screen to make it large enough to view the output properly.

Instructions

Checkpoint 1 Enabled

1.

A `weightit` model using the `los_data` dataset has been created for you in `notebook.Rmd` and saved as `iptw_ef2`.

Fit a regression model with length of hospital stay as the outcome and age, cholesterol level, and heart attack history as predictors. Use the IPTW weights from `iptw_ef2`. Save your model as `outcome_mod`.
Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `coeftest()` function to estimate the treatment effect from the model you just fit and assign the result to `att_robust`. Be sure to use robust standard error estimates.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print the output of `att_robust` to view the results of your analysis. Do your results match the conclusion that follows?

Based on the regression coefficient for the `low_ef` variable from the final outcome model, we can conclude that low ejection fraction causes a typical cardiology patient to stay in the hospital about 1.2 days longer than if they did not have a low ejection fraction.

```
---
title: "Estimating Causal Effects"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(WeightIt)
library(lmtest)
library(sandwich)
import dataset
los_data <- read.csv("los_data.csv")

create IPTW weights
iptw_ef2 <- weightit(
 formula = low_ef ~ age + cholesterol + heart_attack, #propensity model
 data = los_data, #dataset
 estimand = "ATT", #effect
 method = "ps", #use IPTW
)
...

Checkpoint 1
```{r}
# Create outcome model
outcome_mod <- glm(
  #outcome model
  hospital_los ~ low_ef + age + cholesterol + heart_attack,
  #dataset
  data = los_data,
  #IPTW weights
  weights = iptw_ef2$weights)
...

# Checkpoint 2
```{r}
Get coefficients with robust standard errors
att_robust <- coeftest(
 outcome_mod, #weighted outcome model
 vcov. = vcovHC #robust standard errors
)
...

Checkpoint 3
```



```
```{r}
# View the estimated coefficients
print(att_robust)
```
```

# Propensity Scores

## Conclusion

Congratulations! You have learned a lot about propensity score methods in this lesson and are quickly becoming a master of causation (not just correlation).

In this lesson, you learned:

There are five stages of using propensity scores in causal inference.

A propensity score is computed by predicting the probability of treatment from the other observed [variables](#)

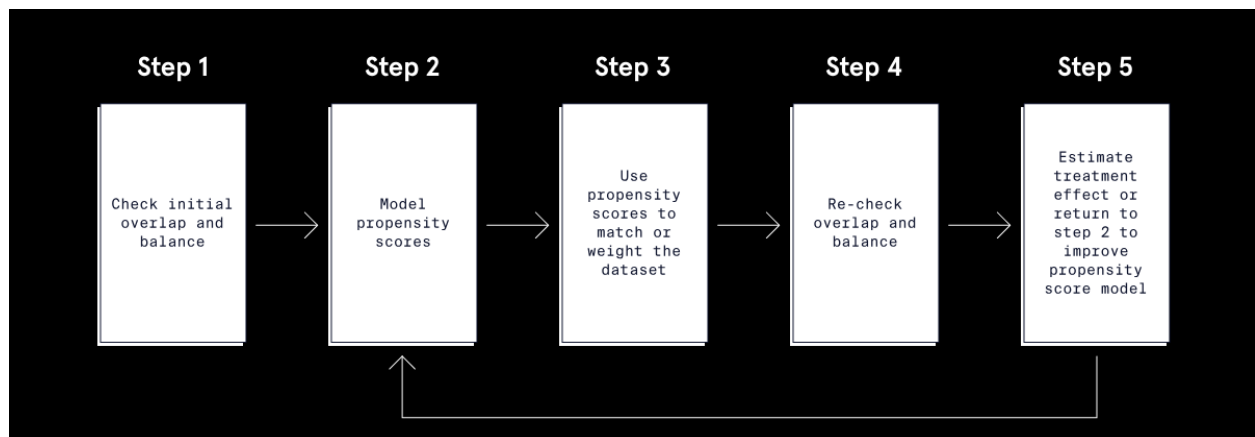
Preview: Docs Loading link description

We use propensity scores in Inverse Probability of Treatment Weighting (IPTW) to create balance across observed variables.

We can check balance between treatment and control groups across variables using the cobalt package functions `bal.tab()`, `bal.plot()`, and `love.plot()`.

A poor propensity score model may lead to biased estimates of the treatment effect, so it is very important that we find the best model possible.

We get an estimate of the treatment effect by creating a regression model with IPTW weights.



<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-matching-and-weighting-methods/quizzes/matching-and-weighting-quiz>

# Regression Discontinuity Design

## The Case for RDD

Research suggests that individuals put more into retirement accounts or pension plans if their employer matches a portion of their contributions.

Imagine that a country's legislature passes a law that requires employers with more than 300 employees to provide a retirement contribution matching program. Using tax data, lawmakers compile a dataset with the following information from each of 200 different companies:

`size`: Number of employees

`group`: Contribution Matching Program Group ("No Program" vs. "Program")

`contribution`: Average monthly employee contributions (in dollars)

Number of employees — acting as the forcing variable with a cutoff at 300 employees — dictates whether or not a company has a contribution matching program.

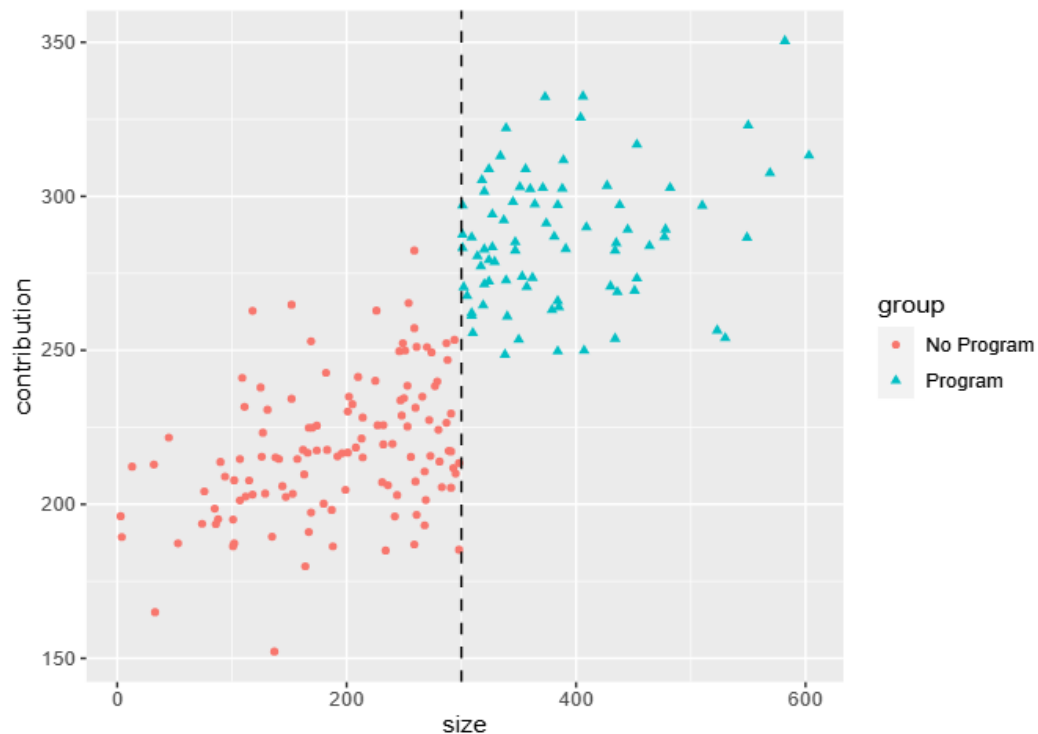
We want to assess whether the policy caused an increase in average monthly retirement contributions by employees. This example is a perfect candidate for a new technique, called regression discontinuity design.

RDD works by only focusing on the points near the cutoff. Companies with close to 300 employees are probably very similar to one another. So the ONLY difference, on average, between companies with 299 employees and those with 301 employees should be whether they have the contribution matching program. This means we should have a treatment and control group that look a lot like those of a randomized experiment!

### Instructions

Take a look at the plot in the learning environment, which shows the contribution matching program data. The number of employees (forcing variable) is on the x-axis, and the average monthly contribution (outcome variable) is on the y-axis.

Do you think that companies with 5 employees are similar to companies with 600? What about companies with 200 employees versus 400 employees?



## Regression Discontinuity Design

### Forcing Variables in RDD

We need a little more vocabulary before we can dive into more details about Regression Discontinuity Design.

Sometimes treatment group assignment is dictated by one continuous variable known as a *forcing variable* (the company size in our example). The forcing variable, also referred to as a *rating variable* or *running variable*, has a cutoff value such that:

Individuals with a value smaller than the cutoff are in one treatment group.

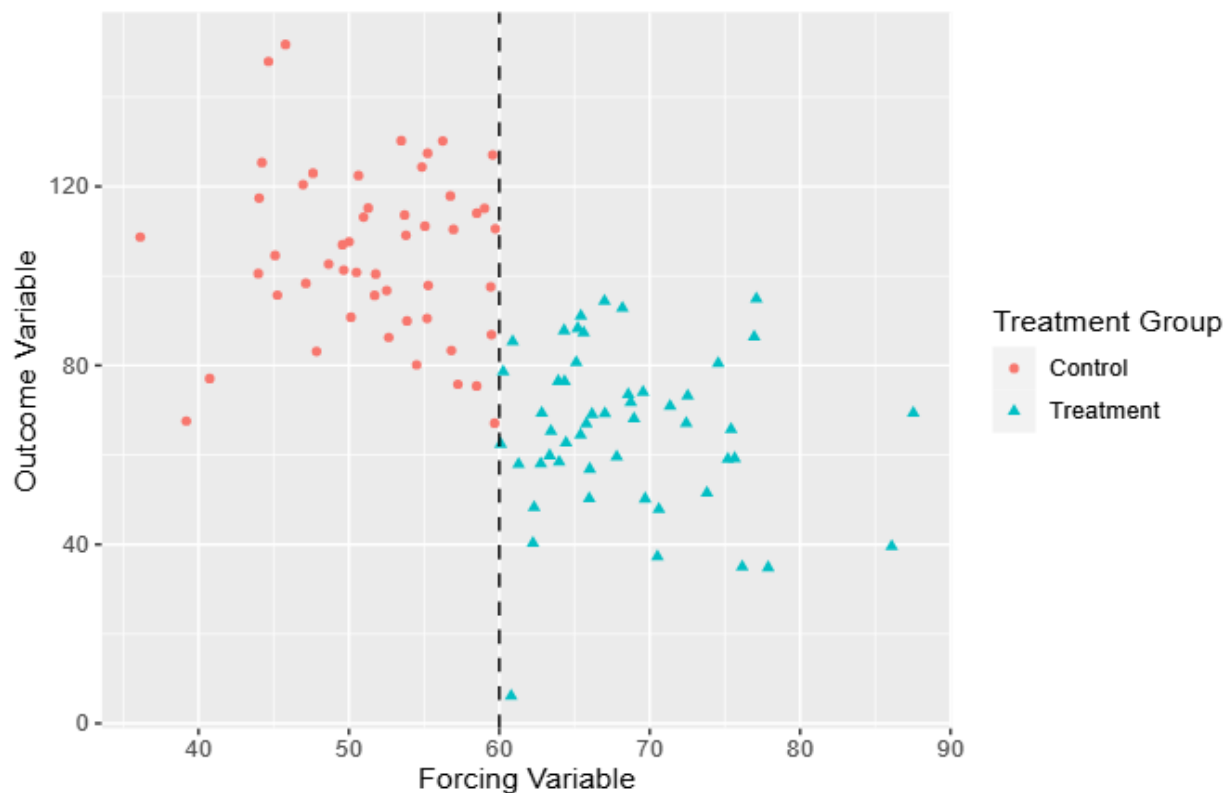
Individuals with a value larger than the cutoff are in the other treatment group.

The treatment group is perfectly predicted by the forcing variable. In this scenario, we cannot rely on other causal inference techniques such as matching or weighting methods because there is not a consistent mixture of treatment and controls across different values of the forcing variable.

#### Instructions

Take a look at the plot provided in the learning environment. Note how points with a value below 60 on the x-axis belong to the control group, while those with values above 60 belong to the treatment group.

#### Example of Sharp RDD with cutoff at 60



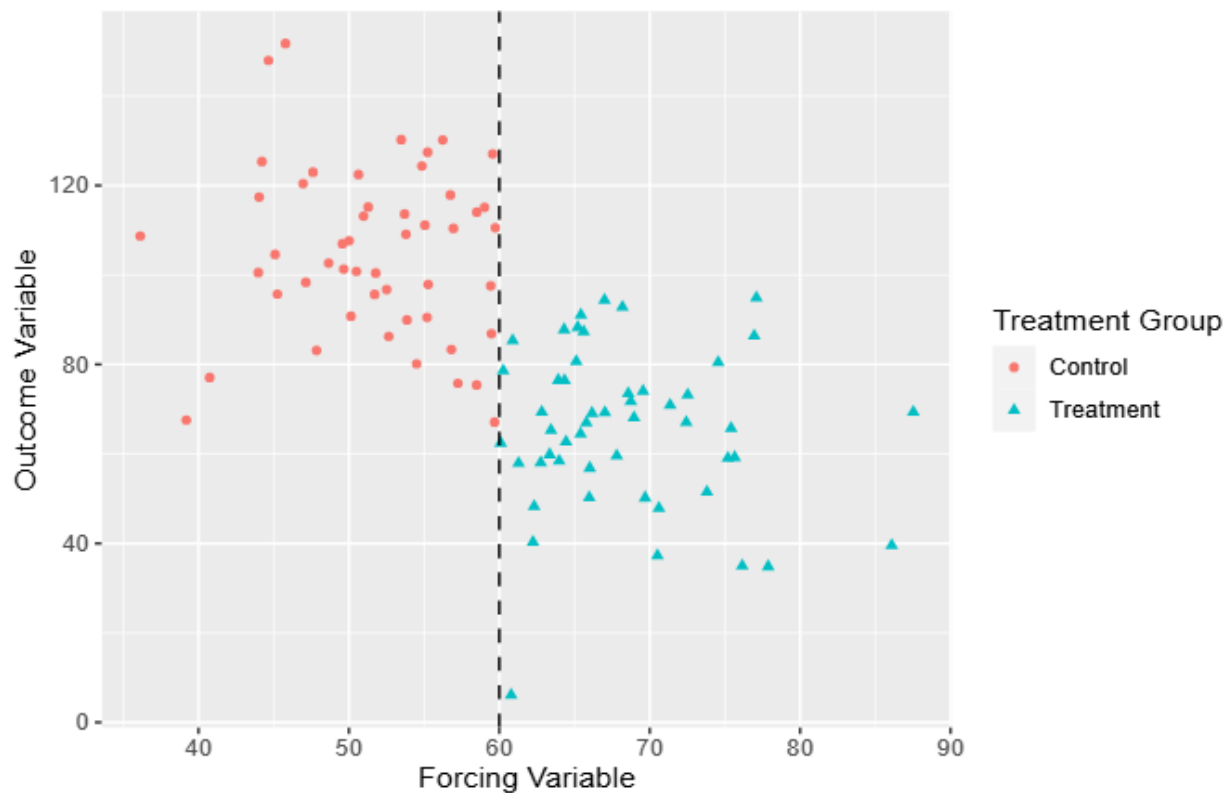
## Regression Discontinuity Design

### Sharp or Fuzzy?

The forcing variable cutpoint can either be exact or not exact:

If the cutpoint is exact, the probability of treatment changes from zero to one at the cutpoint. In other words, all observations on one side of the cutpoint are in the treatment group (and actually received the treatment) and all observations on the other side of the cutpoint are in the control group (and didn't receive the treatment). This is known as a *sharp design*.

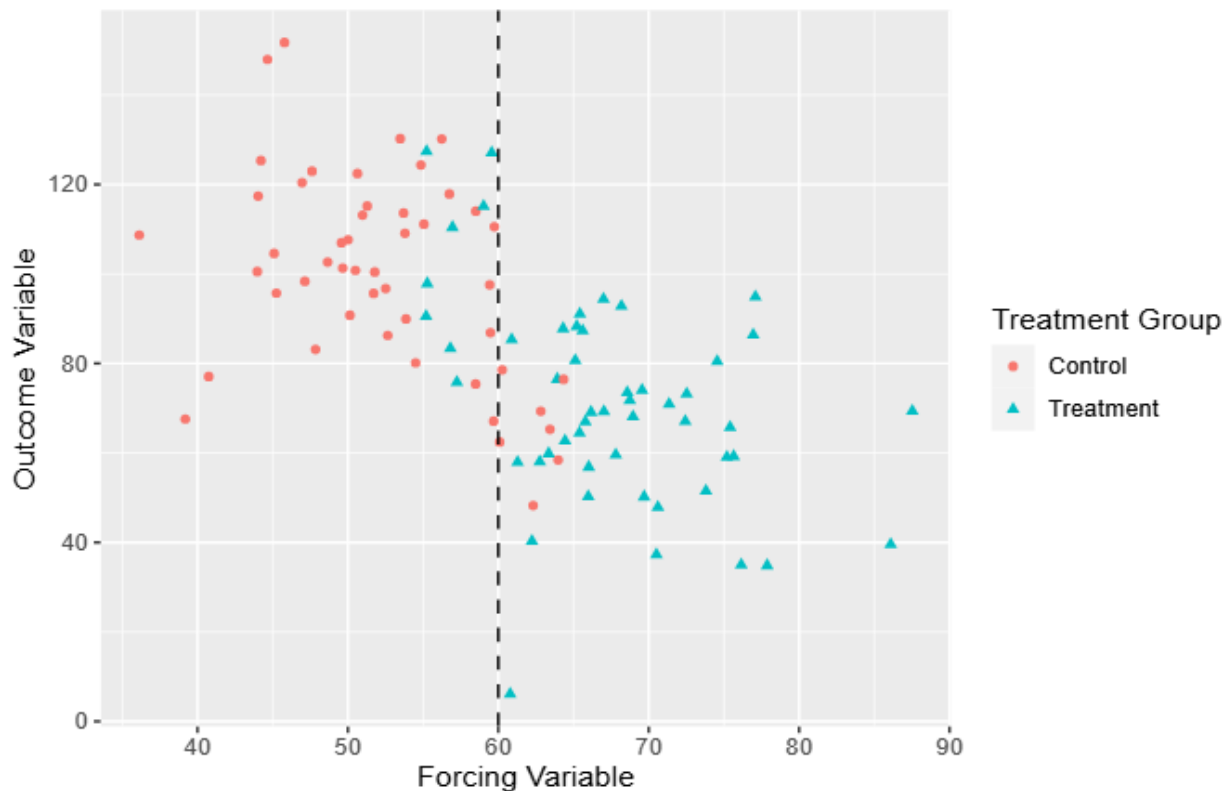
### Example of Sharp RDD with cutoff at 60



If the cutpoint is NOT exact, the probability of treatment doesn't jump from zero to one at the cutpoint. In other words, there are individuals in each treatment group on BOTH sides of the cutpoint. This is known as a *fuzzy design*.

## Example of Fuzzy RDD

with cutoff at 60



While this distinction may seem minor, it is actually incredibly important to recognize when to use sharp RDD as opposed to fuzzy RDD. Not only do the two approaches make different assumptions about the data, but they also require slightly different statistical methods.

We will focus on sharp RDD in this lesson, even though perfect compliance with the treatment assignment is not always realistic.

### Instructions

Play around with the interactive plot in the learning environment to see what different levels of “fuzziness” look like in a regression discontinuity design.

What does this mean with respect to our companies? As you change the degree of fuzziness, which small companies offer contribution matching? Which large companies are in violation of the law?

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-regression-discontinuity-design-and-instrumental-variables/lessons/regression-discontinuity-design/exercises/sharp-or-fuzzy>

## Regression Discontinuity Design

### Defining RDD Assumptions

We saw that our employee contribution example requires a sharp regression discontinuity design: all companies with at least 300 employees have a contribution matching program, and all companies with fewer than 300 employees do NOT have a contribution matching program. Before we decide which companies are similar enough to compare, we must consider some assumptions.

In order to get valid estimates of the treatment effect using a sharp RDD approach, several assumptions have to be met.

The treatment variable impacts the outcome, but not any of the other [variables](#)

Preview: Docs Stores data that can be manipulated and referenced throughout the code.

.

The treatment assignment happens only at ONE cutpoint value of the forcing variable.

Treatment assignment is independent of the potential outcomes within a narrow interval around the cutpoint.

Counterfactual outcomes can be modeled within the interval around the cutpoint.

### Instructions

Take a look at the learning environment and hover over each card to illustrate how each of these assumptions relates to the contribution matching program example.

## Assumption 1

Treatment impacts the outcome, but not any other variables.

## Assumption 1

The contribution matching program impacts the amount that employees contribute, but no other variables.

## Assumption 2

Treatment assignment happens only at ONE cutpoint value of the forcing variable.

## Assumption 2

The contribution matching program is only required for companies with at least 300 employees.

## Assumption 3

Treatment assignment is independent of the potential outcomes within a narrow interval of the forcing variable around the cutpoint.

## Assumption 3

Companies with just under 300 employees and just above 300 employees are similar enough to one another with respect to other variables EXCEPT for the treatment group that the treatment assignment could be considered random.

## Assumption 4

Counterfactual outcomes can be modeled within the interval around the forcing variable's cutpoint.

## Assumption 4

Employee contributions among companies that have slightly fewer or slightly more than 300 employees can be modeled to determine the causal effect of the program.

### Regression Discontinuity Design

### Visual Check

A scatter plot of our data allows us to check certain RDD conditions visually. We can see whether we have a sharp or fuzzy cutoff. We can also use the plot to check for a *discontinuity* — a sudden change in the outcome variable — at the cutoff.

To create this scatter plot with the contribution matching dataset, we can use the `ggplot2` package in R:

```
library(ggplot2) #load ggplot2 package
```

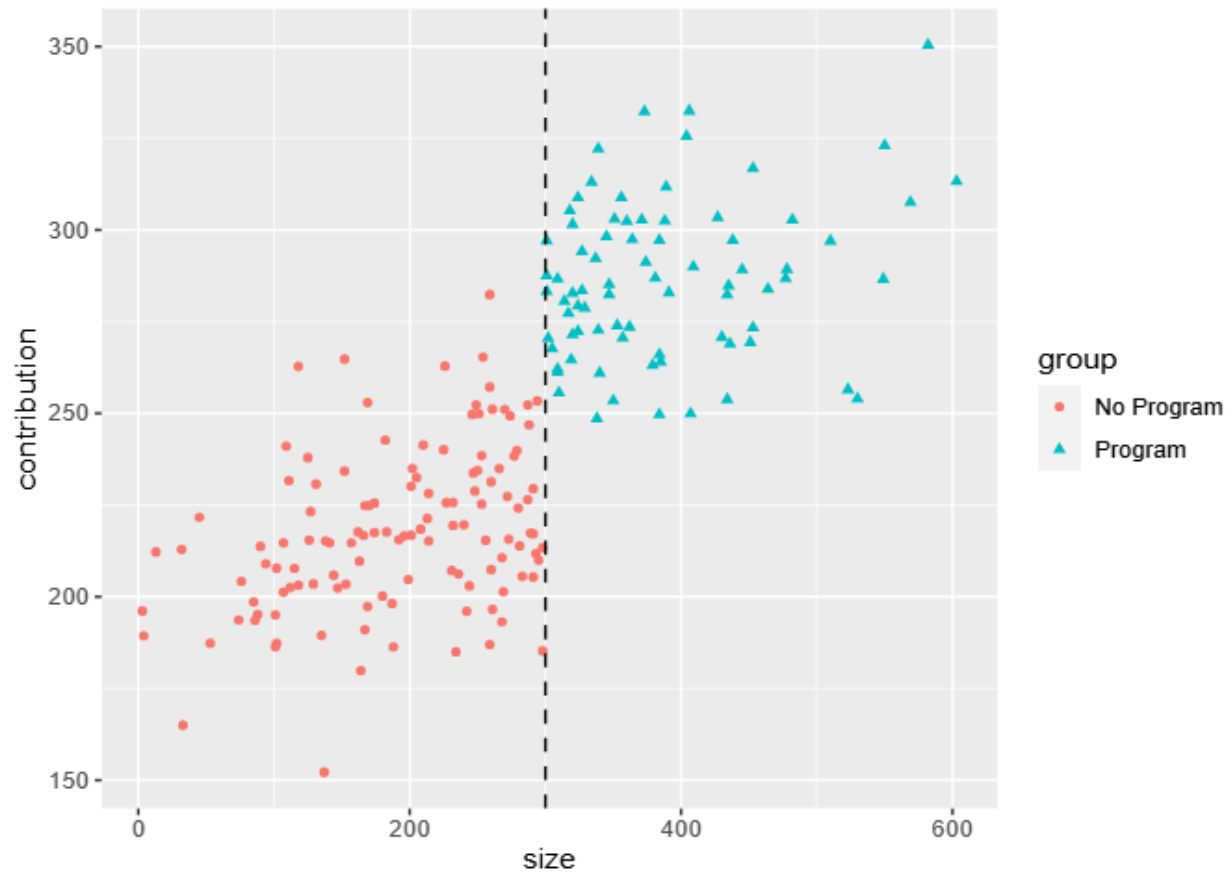
Copy to Clipboard

Our scatter plot should have the number of employees on the x-axis and the contribution amount on the y-axis. The points for treatment and control groups should be different colors and shapes, so we can easily tell the two groups apart. Finally, we add code for a dashed vertical line at the cutoff of 300 employees.

```
create a scatterplot with treatment groups
ggplot(
 data = cont_data,
 aes(
 x = size, # forcing variable
 y = contribution, # outcome variable
 color = group, # sets point color by treatment group
 shape = group # sets point shape by treatment group
)
) +
 geom_point() +
 geom_vline(xintercept = 300, linetype = "dashed") #add line at 300
```

Copy to Clipboard

This plot clearly shows that we have a sharp RDD, not a fuzzy one. The dashed line at 300 employees separates the two groups into companies that offer a matching program (at least 300) and companies that do NOT offer a matching program (fewer than 300).



We can also check to make sure that there is actually a discontinuity in average contributions based on whether or not companies have more than 300 employees. To do this, we can add a separate best fit line for each treatment group using the `geom_smooth()` function. If we had saved our first plot as `rdd_scatter`, we can add the code for the lines as follows:

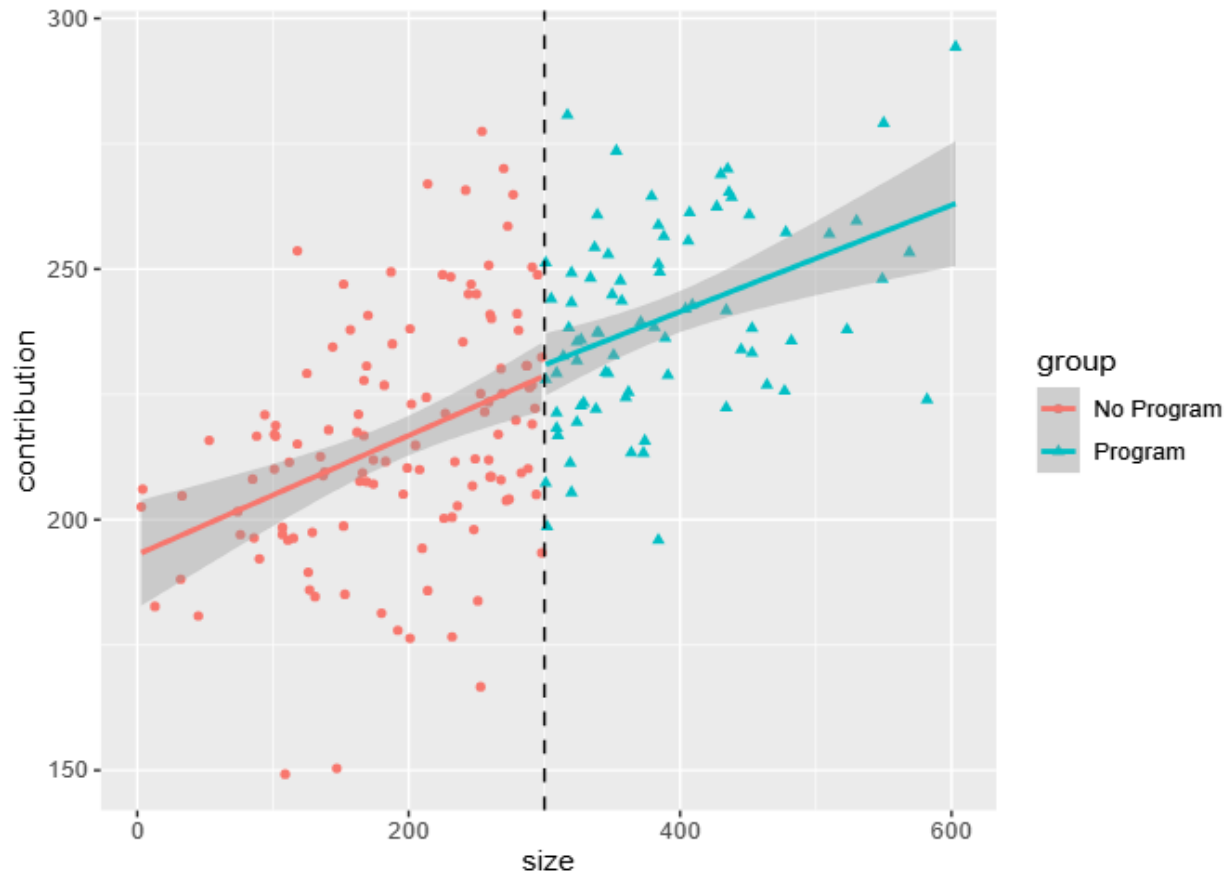
```
add best fit lines for each group to scatter plot
rdd_scatter +
 geom_smooth(
 aes(group = group), #plot separate lines for each group
 method = "lm" #use linear regression
)
```

[Copy to Clipboard](#)





There is an obvious jump in the average contributions at the cutoff point, which means there is a discontinuity present. If there were no discontinuity present, we might see something like this:



Note that there is no jump in the outcome variable here. The lines connect smoothly.

#### Instructions

Checkpoint 1 Enabled

1.

A new policy requires that power plants that have an output of 600 megawatts or more have to install an emissions control device that removes harmful chemicals before releasing exhaust into the air. An environmental group decides to measure ambient air quality one mile away from each power plant to assess whether the emissions control device leads to better air quality.

The `air_data` dataset has been loaded for you in `notebook.Rmd` with the following variables:

`id`: power plant ID.

`watts`: power plant output (megawatts)

`group`: treatment group formed by cutpoint at 600 watts (control = "No Device", treatment = "Emissions Device")

`aqi`: air quality index from 0 (good air quality) to 500 (poor air quality)

Create a scatter plot named `air_scatter`:

Use `watts` as the forcing variable and `aqi` as the outcome variable.

Use different colors and shapes for each treatment group.

Add a dashed vertical line at 600 megawatts.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print `air_scatter` to view the plot. Consider whether this plot shows a sharp or fuzzy cutoff at 600 megawatts.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Add a line of code to `air_scatter` to add best-fit lines for each treatment group to the plot. Save the updated plot to `air_scatter2`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print `air_scatter2`. Do the regression lines appear to show a discontinuity in the outcome variable at the cutoff?

---

```

title: "Visual Check"
output: html-notebook

```{r}
# load ggplot2 library
library(ggplot2)
# import dataset
air_data <- read.csv("air_data.csv")
```

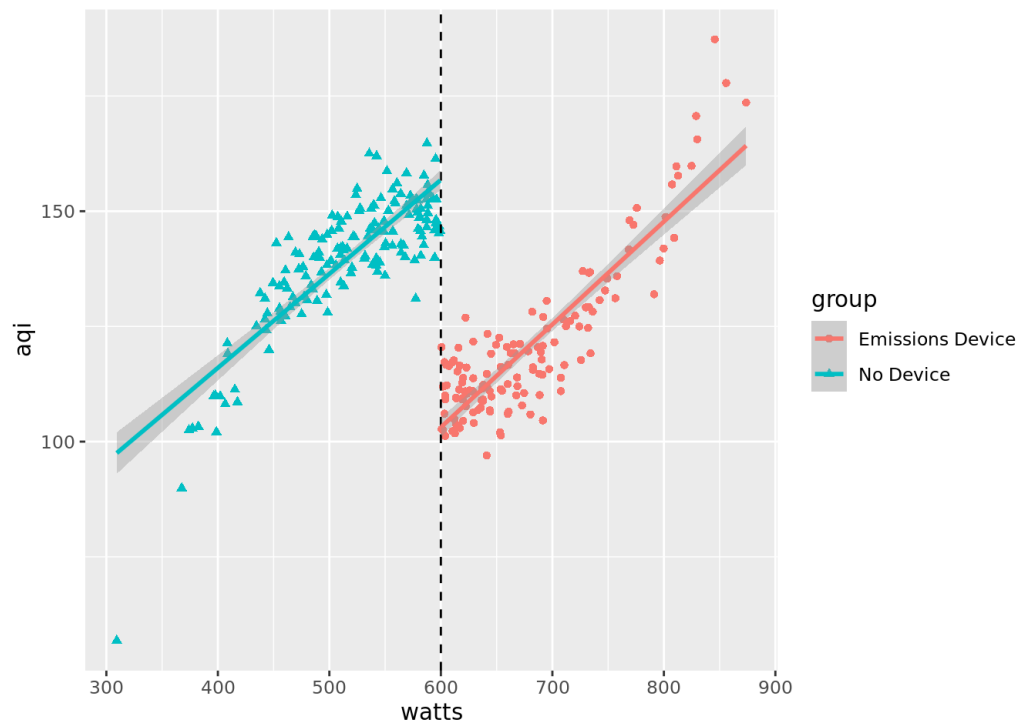
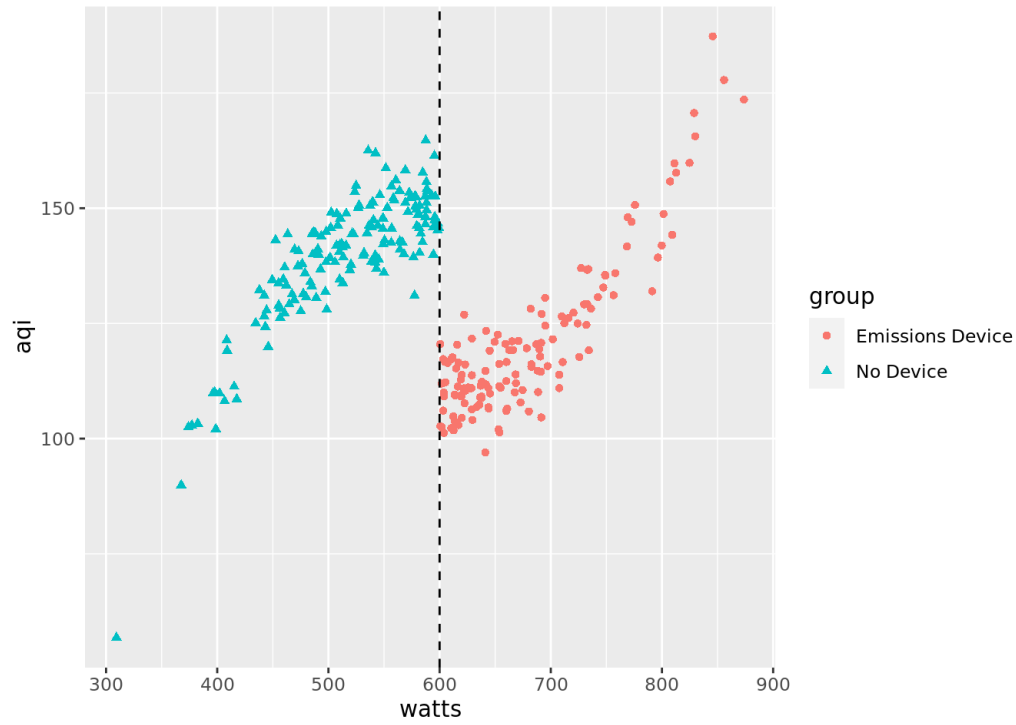
Checkpoint 1
```{r}
# create scatter plot with watts as forcing variable and AQI as outcome
air_scatter <- ggplot(
  data = air_data, #dataset
  aes(
    x = watts, #forcing variable
    y = aqi, #outcome variable
    color = group, #point color by treatment
    shape = group #point shape by treatment
  )
) +
  geom_point() + #add points to plot
  geom_vline(xintercept = 600, linetype = "dashed") #add dashed line
```

Checkpoint 2
```{r}
# print plot
print(air_scatter)
```

Checkpoint 3
```{r}
# add best-fit lines to air_scatter
air_scatter2 <- air_scatter +
  geom_smooth(aes(group = group), method = "lm")
```

Checkpoint 4
```{r, message=FALSE}
# print air_scatter2
print(air_scatter2)
```

```



**Regression Discontinuity Design**

**Choosing a Bandwidth**

In RDD, we know we need to look at points near the cutoff to find treatment and control groups that are similar. But how do we know how close to look?

The *bandwidth* describes the distance on either side of the cutoff we should use to reduce our dataset. Any points that are more than one bandwidth above or below the cutoff are discarded. Choosing the bandwidth can have a serious impact on the results of an RDD analysis:

A wider bandwidth keeps more of the original dataset, so we have more information to estimate the treatment effect with. However, the treatment groups might be too different on confounding

[variables](#)

Preview: Docs Stores data that can be manipulated and referenced throughout the code.  
, which could decrease accuracy.

A narrower bandwidth retains less of the original dataset, so treatment groups will be more alike. However, the smaller sample size means less information to estimate the treatment effect.

We could select the bandwidth based on what we BELIEVE is best. However, an

[algorithm](#)

Preview: Docs An algorithm is a formal process used to solve a problem. They can be represented in several formats but are usually represented in pseudocode in order to communicate the process by which the algorithms solve the problems they were created to tackle.

that optimizes the bandwidth mathematically may be a better choice. A popular choice—which we will use—is the Imbens-Kalyanaraman (IK) algorithm.

The R package `rdd` contains all of the tools needed to calculate the optimal bandwidth and carry out an RDD analysis. To calculate the IK bandwidth using `rdd`, we will use the `IKbandwidth()` function, which requires three arguments:

`x`: the forcing variable  
`y`: the outcome variable  
`cutpoint`: the cutoff value to use.

To calculate the IK bandwidth for the contribution matching dataset, we would use the following code:

```
library(rdd)

calculate IK bandwidth
cont_ik_bw <- IKbandwidth(
 X = cont_data$size, # forcing variable
 Y = cont_data$contribution, # outcome variable
 cutpoint = cont_cutpoint # cutpoint
)

print the IK bandwidth to the console
cont_ik_bw
[1] 13.26322
```

Copy to Clipboard

The reduced dataset used in our RDD analysis will include only the companies that have between 286 and 314 employees ( $300 \pm 13.26$ ). Companies with between 286 and 314 employees are likely to be similar on other variables that may impact employee contributions, such as average salary or insurance costs.

To illustrate the bandwidth visually, we can add bandwidth lines to the scatterplot. We can use `geom_vline()` to add reference lines at the cutpoint  $\pm$  the bandwidth to our scatter plot `rdd_scatter` from earlier:

```
rdd_scatter +
 geom_vline(xintercept = 300 + c(-cont_ik_bw, cont_ik_bw)) # add lines to
 indicate the bandwidth
```

Copy to Clipboard



This plot shows us just how narrow the optimal bandwidth is for the contribution program dataset.

#### Instructions

Checkpoint 1 Enabled

1.

The dataset `air_data` has been loaded for you in **notebook.Rmd**. Calculate the Imbens-Kalyanaraman (IK) optimal bandwidth using the `IKbandwidth()` function. Save the results to `air_ik_bw`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print `air_ik_bw`. Think about whether the bandwidth seems large or small in relation to the scale of the forcing variable.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

A scatter plot of AQI (`aqi`) against power plant output (`watts`) with a dashed line at 600 megawatts has been created for you and saved as `air_scatter`. Modify `air_scatter` to add solid vertical lines for the bandwidth cutoff lines. Save the result to `air_scatter2`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print `air_scatter2`. Do you think the two device groups within this bandwidth will be similar enough to compare? What might be the tradeoff if the bandwidth is too narrow?

```

title: "Choosing a Bandwidth"
output: html-notebook

```

```

```{r, message=FALSE}
# load rdd and ggplot2 libraries
library(rdd)
library(ggplot2)

# import data
air_data <- read.csv("air_data.csv")

# scatter plot with cutpoint
air_scatter <- ggplot(
  data = air_data, #dataset
  aes(
    x = watts, #forcing variable
    y = aqi, #outcome variable
    color = group, #color points by group
    shape = group #shape points by group
  )
) +
  geom_point() + #add points to plot
  geom_vline(xintercept = 600, linetype = "dashed") #add line
...

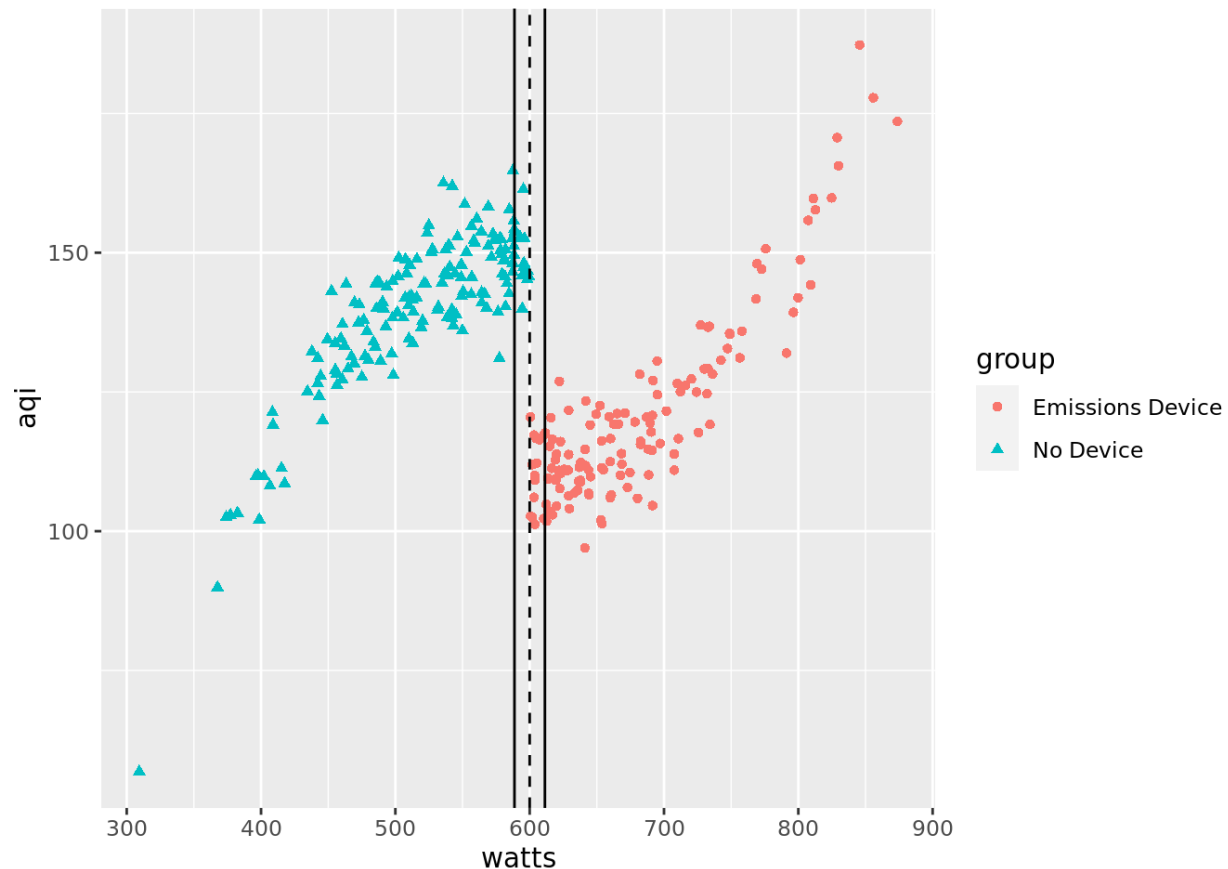
# Checkpoint 1
```{r}
calculate the IK bandwidth
air_ik_bw <- IKbandwidth(
 X = air_data$watts, #forcing variable
 Y = air_data$aqi, #outcome variable
 cutpoint = 600 #cutpoint
)
...

Checkpoint 2
```{r}
# print air_ik_bw
print(air_ik_bw)
...

# Checkpoint 3
```{r}
add vertical bandwidth lines to air_scatter
air_scatter2 <- air_scatter +
 geom_vline(xintercept = 600 + c(-air_ik_bw, air_ik_bw))
...

Checkpoint 4
```{r}
# print updated plot
print(air_scatter2)
...

```



Regression Discontinuity Design

Estimating the Causal Treatment Effect

The use of a bandwidth impacts the type of causal estimand we can calculate in a regression discontinuity design analysis. Because the RDD approach uses a subset of the full dataset, we can only estimate the *local average treatment effect* (LATE). The LATE is the average treatment effect among the subset of data that falls within the range of the bandwidth.

To estimate the LATE in RDD, a regression model that allows for different slopes on each side of the cutpoint is fit. The regression model is then used to get a predicted value of the outcome variable for each treatment group at the cutpoint. The difference between the predicted outcome values of the treatment and control groups is an estimate of the LATE.

We can use the `RDestimate()` function from the `rdd` package as follows to fit the local linear regression model for the contribution matching data:

```
cont_rdd <- RDestimate(
  formula = contribution ~ size, #outcome regression model
  data = cont_data, #dataset
  cutpoint = 300, #cutpoint
  bw = cont_ik_bw #bandwidth
)
```

Copy to Clipboard

The `RDestimate()` function fits the local linear regression model at the provided bandwidth, but also at half of the bandwidth and twice the bandwidth. If the estimate of the LATE is relatively the same across bandwidths, we can be more confident that the estimate is accurate. We see all three estimates when we print the results.


```
Call:
RDestimate(formula = contribution ~ size, data = cont_data,
cutpoint = 300, bw = cont_ik_bw)
```

Coefficients:

LATE	Half-BW	Double-BW
90.60	110.67	71.62

Copy to Clipboard

The model output shows us that the LATE is 90.60, meaning that in this dataset, we can conclude that employer-sponsored retirement contribution matching programs led to an increase in average monthly contributions of \$90.60. However, we see that the estimate changes based on the bandwidth, ranging from \$110.67 at half of the bandwidth to \$71.62 at twice the bandwidth.

Instructions

Checkpoint 1 Enabled

1.

The `air_data` dataset has been loaded for you in **notebook.Rmd**. The IK optimal bandwidth has been saved as `air_ik_bw`. Use the `RDestimate()` function to fit the local linear regression model using `air_ik_bw` as the bandwidth. Save the results to `air_rdd`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the results of the local linear regression. Take note of the estimates of the LATE at the different bandwidths. Are the estimates different or fairly similar? What do you think this says about the reliability of our estimate?

```
---
title: "Estimating the Causal Treatment Effect"
output: html-notebook
---

```{r, message=FALSE}
load rdd and ggplot2 libraries
library(rdd)
library(ggplot2)

import data
air_data <- read.csv("air_data.csv")

calculate the IK bandwidth
air_ik_bw <- IKbandwidth(
 X = air_data$watts, #forcing variable
 Y = air_data$aqi, #outcome variable
 cutpoint = 600 #cutpoint
)
...

Checkpoint 1
```{r}
# fit local linear regression model
air_rdd <- RDestimate(
  formula = aqi ~ watts, #formula for outcome model
  data = air_data, #dataset
  cutpoint = 600, #cutpoint
  bw = air_ik_bw #bandwidth
)
...

# Checkpoint 2
```{r}
print results of local linear regression
print(air_rdd)
...

```

## Regression Discontinuity Design

### Advantages and Disadvantages of RDD

As we've seen, the advantages of regression discontinuity design are that RDD:

- Is a simple

- [method](#)

- Preview: Docs Loading link description to understand and implement.

- Avoids using a complicated regression model for the entire dataset — the local regression model is simple.

- Is useful in cases where there is no overlap on a confounding variable, which may prevent us from using stratification or propensity score analysis.

However, there are several drawbacks to RDD inherent to the method:

- Smaller bandwidths make RDD assumptions more plausible BUT also reduce the sample size.

- The local average treatment effect (LATE) is not an easily interpretable estimand. We calculated the effect of the contribution matching program only among the companies with close to 300 employees. How confident can we be that this effect would be the same in much smaller or larger companies?

Let's explore the tradeoffs with an example. Say we run `RDestimate()` on the contribution data again, but set the bandwidth to 100 instead of the IK optimal bandwidth of 13. We save the results to `rdd_100` and print the output:

```
Call:
RDestimate(formula = contribution ~ size, data = cont_data,
cutpoint = 300, bw = 100)
```

Coefficients:

LATE	Half-BW	Double-BW
56.71	59.90	53.54

[Copy to Clipboard](#)

We can also get information on the number of observations included and the standard error of the LATE for each bandwidth with the code that follows.

```
rdd_100$obs #number of observations
Output
[1] 113 68 178

rdd_100$se #standard errors
Output
[1] 6.647 9.077 5.269
```

[Copy to Clipboard](#)

Let's consider just the half-bandwidth (50) and the double-bandwidth (200). Using the half-bandwidth, we analyze only companies with 250-350 employees, so we may believe these companies are very similar to one another. But this leaves only 68 companies in our sample and a standard error of 9.077 for the LATE. We may ask:

- Can we trust a LATE with a higher standard error?

- Do these findings apply to companies outside the range of 250-350 employees?

At the double-bandwidth, we analyze companies with 100-500 employees, so our sample size is much larger at 178 companies and our standard error is reduced to 5.269. But how confident are we that companies with 100 employees are similar enough to compare to companies with 500 employees?

Regression discontinuity design is a useful method to keep in our causal inference toolbox, but we must be aware of the tradeoffs throughout the process.

#### Instructions

Checkpoint 1 Enabled

1.

A new RDD model with a bandwidth of 50 called `air_rdd_50` has been created for you in **notebook.Rmd**. Print `air_rdd_50` and inspect the results. Are there big differences in the estimated LATE across bandwidths of 50, 25, and 100? Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Check the sample size under each reported bandwidth of `air_rdd_50`. Is there a big difference in sample size when you look at power plants with outputs of 575-625 megawatts (`Half-BW`) and those with outputs of 500-600 megawatts (`Double-BW`)? Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Check the standard errors of the LATE under each reported bandwidth of `air_rdd_50`. Is there a big difference in standard errors when you look at power plants with outputs of 575-625 megawatts (`Half-BW`) and those with outputs of 500-600 megawatts (`Double-BW`)? Does the standard error get larger or smaller with bigger samples?

```

title: "Advantages and Disadvantages of RDD"
output: html-notebook

```{r, message=FALSE}
# load rdd and ggplot2 libraries
library(rdd)
library(ggplot2)

# import data
air_data <- read.csv("air_data.csv")

# fit model with bandwidth of 50
air_rdd_50 <- RDestimate(
  formula = aqi ~ watts, #outcome model
  data = air_data, #dataset
  cutpoint = 600, #cutpoint
  bw = 50 #bandwidth
)
```

Checkpoint 1
```{r}
# Print model results
print(air_rdd_50)
```

Checkpoint 2
```{r}
# Check the number of observations
air_rdd_50$obs
```

Checkpoint 3
```{r}
# Check the standard errors
air_rdd_50$se
```
```

## Regression Discontinuity Design

## Conclusion

In this lesson, we showed that the implementation of an employee-sponsored retirement matching program led to an increase in average monthly employee contributions.

We learned a lot about regression discontinuity design along the way, including:

RDD is a

[method](#)

Preview: Docs Loading link description

used when the treatment assignment is determined by a continuous forcing variable at a specific cutoff point.

An RDD is known as *sharp* if the cutoff is exact and *fuzzy* if the cutoff is not exact.

Individuals within a narrow window on either side of the cutoff are assumed to be similar to each other, except for the treatment group assignment.

Local linear regression can be used to determine the local average treatment effect (LATE) among the individuals in this narrow window.

Disadvantages of RDD include reduced sample size and potential lack of generalizability of the LATE.

### Instructions

#### Conclusion

Great job performing a regression discontinuity design analysis on your own! In this lesson, you verified that RDD was an appropriate method to use, got causal estimates of the LATE using local linear regression, and examined the reliability of your results by using different bandwidths.

So what did this RDD analysis tell us? Based on the results of the local linear regression modeling (as shown in the learning environment), we can conclude that in our sample, emissions control devices led to a decrease in AQI of 36.45 points. This means that the emissions control device achieved its goal of providing cleaner air in close proximity to the power plants in our dataset.

# Instrumental Variables

## Introduction to Instrumental Variables

<https://www.codecademy.com/paths/data-science-inf/tracks/dsinf-causal-inference/modules/dsinf-regression-discontinuity-design-and-instrumental-variables/lessons/instrumental-variables/exercises/assignment-vs-compliance>

Suppose that a city is interested in increasing the per-capita rate of recycling among its citizens while decreasing the city's operating costs.

To encourage this, the city allows individuals to discontinue curbside recycling pickup and instead opt into a rebate program. The city wants to evaluate whether or not this rebate program increases the amount of recycling in the city.

The city compiles the following

### variables

Preview: Docs Loading link description  
in a dataset to evaluate the success of the rebate program:

`recycled`: amount recycled (kg/person).

`rebate`: participation in rebate program (curbside vs. rebate).

`distance`: distance from recycling center (5 miles vs. > 5 miles). An individual who lives less than five miles from a recycling center might be more likely to opt into the rebate program than someone who lives more than five miles from a center. However, distance to a recycling center should not directly impact the amount of waste each person recycles.

We are going to use Instrumental Variables to answer the city's question, but before we dive into that, we need to establish some more tools.

The first is **conditional exchangeability**.

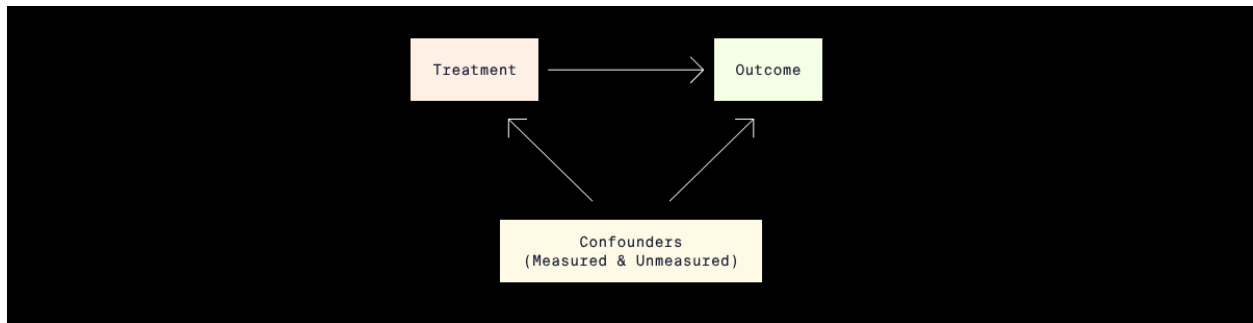
One key assumption made in causal inference methods like weighting or stratification is *conditional exchangeability*. This assumption states that there are no unmeasured confounding variables that have a causal effect on both the treatment assignment and the outcome.

Randomization of the treatment assignment ensures that both measured and unmeasured confounding variables are evenly balanced between treatment groups. In a non-randomized setting, balance is NOT guaranteed. The assumption of conditional exchangeability cannot be tested or verified — in most cases, the best we can do without randomization is to identify and measure as many potential confounding variables as possible.

Instrumental variable (IV) estimation is a causal inference technique that helps us estimate the causal effect of the treatment even in the presence of unmeasured confounding variables. In this lesson, we will learn about the assumptions and potential applications of IV estimation.

### **Instructions**

Take a look at the diagram in the learning environment. This diagram depicts the causal relationships between the treatment, the outcome, and confounding variables in a typical non-randomized study.



## Instrumental Variables

### Not That Kind of Instrument

In observational studies, when randomization is not possible, balance of measured and unmeasured confounding

[variables](#)

Preview: Docs Loading link description

is not guaranteed. Without taking appropriate measures, the causal estimate of the effect of a treatment on an outcome of interest will be biased.

Instrumental variable (IV) estimation is one causal inference

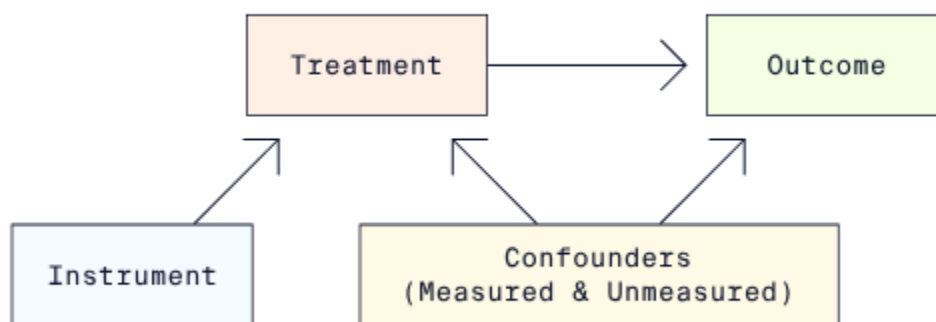
[method](#)

Preview: Docs Loading link description

that uses *instruments* to help reduce bias from both measured AND unmeasured confounding variables.

If you started this lesson hoping to learn about pianos and guitars, you may be disappointed. In IV estimation, an *instrument* (or *instrumental variable*) is a variable that is causally related to an outcome variable ONLY through another variable – typically the treatment variable of interest.

An instrumental variable would be depicted in a causal diagram as follows:

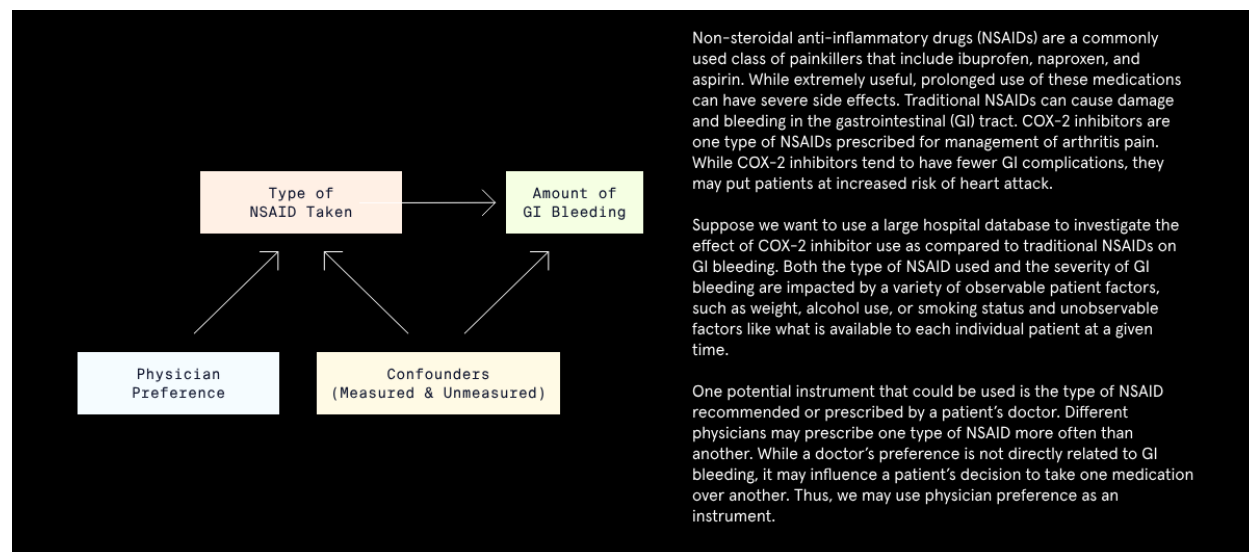


In this diagram, the arrows signify the presence and direction of a causal relationship. For example, there is no arrow directly from the instrument to the outcome because the instrument only impacts the outcome through its causal relationship with the treatment.

#### Instructions

Take a look at the animation in the learning environment. Each frame of the animation has a different example of how IV estimation could be used in practice along with a causal diagram that describes the scenario. Pause the animation as necessary to read and understand each scenario.

Pay particular attention to the third example, which describes the recycling scenario again. We will use it throughout the rest of this lesson to illustrate IV estimation.



## Instrumental Variables

### Assignment vs. Compliance

When treatment group assignments cannot be randomized due to ethical or practical reasons, the best we can do is to encourage compliance. However, encouragement does not guarantee compliance.

Compliance with the treatment assignment can be influenced by many factors, only some of which may be measurable. To account for unmeasured confounders of treatment compliance and the outcome, we could use IV estimation with treatment assignment as the instrument:

When the instrument AND treatment are both binary variables, we can define four types of “compliers”:

- Always takers*: takes the treatment regardless of treatment assignment.
- Never takers*: never takes the treatment regardless of treatment assignment.
- Compliers*: takes the assigned treatment.
- Defiers*: takes the opposite of the assigned treatment.

In the context of the recycling program example, the four types of compliers would be defined as follows:

### Instrument Value

$\leq 5$  miles

$> 5$  miles

|                      |          |          |
|----------------------|----------|----------|
| <b>Always takers</b> | Rebate   | Rebate   |
| <b>Never takers</b>  | Curbside | Curbside |
| <b>Compliers</b>     | Rebate   | Curbside |
| <b>Defiers</b>       | Curbside | Rebate   |

The issue of compliance is related to the fundamental problem of causal inference, which states that we can never observe both potential outcomes. In an IV estimation, we can never observe both potential treatments received for an individual. Thus, we cannot know with certainty what kind of complier an individual is.

Fortunately, we can get around this issue by making additional assumptions about our sample.

#### Instructions

Take a look at the interactive flowchart in the learning environment. The flowchart has three levels: treatment assignment, treatment received, and type of compliance. For each treatment assignment, click on one treatment received. Depending on your selection of treatment received, the type of compliance will appear.

## Instrumental Variables

## Assumptions of IV Estimation

Compliance with the treatment assignment influences which causal estimand we calculate:

When compliance is perfect, the treatment assignment and treatment received can be used interchangeably to get an accurate estimate of the causal effect.

When compliance is NOT perfect, we cannot estimate the average treatment effect (ATE) because treatment assignment and treatment compliance are NOT interchangeable. Instead, the average effect of treatment assignment can be thought of as the effect of the *intention to treat*, or ITT effect.

IV estimation allows us to approximate the ATE by estimating a local average treatment effect (LATE) just among the compliers. This is known as the *compliers' average causal effect* (CACE). To estimate the CACE, we must make four assumptions about the sample:

*Relevance*: the instrument has a causal effect on the treatment received.

*Exclusion*: the instrument affects the outcome ONLY indirectly through the treatment received.

*Exchangeability*: there are no confounders that influence both the instrument AND the outcome.

*Monotonicity*: there are no "defiers" in the sample.

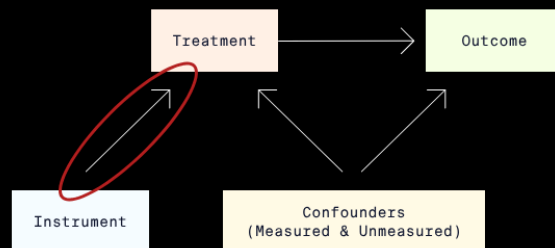
The relevance assumption implies that there are no "never-takers" and "always-takers" in the sample. In these subgroups, the treatment received doesn't depend on the value of the assigned treatment. And in order to estimate the treatment effect among the "compliers," we must also assume that there are no "defiers" in the sample.

#### Instructions

The learning environment contains four pausable slides — one for each IV assumption. Check each slide for additional information about the assumptions, including diagrams showing violations of these assumptions.



The monotonicity assumption essentially states that as the value of the instrument increases, the probability of taking the treatment does NOT decrease. The only group that violates this assumption is the defiers.



## Instrumental Variables

### Two-Stage Least Squares Regression

In a standard ordinary least squares (OLS) regression model, the outcome is predicted directly from the treatment assignment and other confounding variables. Such an OLS model would take the form:

**Outcome =  $\beta_0 + \beta_1 * \text{Treatment Assignment}$ .**

In OLS regression, we would use  $\beta_1$  as an estimate of the treatment effect. To fit the OLS regression model using the recycling data, we would use the following code:

```
lm(recycled ~ rebate, #outcome ~ treatment
 data = recycle_df #dataset
)

Output:

Call:
lm(formula = recycled ~ rebate, data = recycle_df)

Coefficients:
(Intercept) rebate
 126.00 38.04
```

Copy to Clipboard

The OLS estimate suggests that participation in the rebate program leads to an average increase in recycling of 38.04 kilograms/person. However, this estimate is biased because OLS regression does not control bias introduced by unmeasured confounding variables or imperfect compliance.

In IV estimation, we account for unmeasured confounding variables and imperfect compliance via *two-stage least squares* (2SLS) regression. This type of regression predicts the outcome in two separate steps:

In the first stage, treatment received is predicted by the instrument (treatment assignment):

**Predicted Treatment Received =  $\alpha_0 + \alpha_1 * \text{Treatment Assignment}$**

In the second stage, the outcome is predicted as a function of the predicted treatment received from the first stage:

**Outcome =  $\beta_0 + \beta_1 * \text{Predicted Treatment Received}$**

$\beta_1$  from the second stage of the 2SLS regression model is used as the estimate of the CACE.

In this lesson, we focus on 2SLS regression with a continuous outcome, binary instrument, and binary treatment to keep things simple. When this is the case, the first stage uses logistic regression, while the second stage uses linear regression.

### Instructions

Checkpoint 1 Enabled

1.

An online retailer is interested in studying the effect of offering video streaming services on the average amount of money that users spend on the platform. The retailer gives its paying members access to exclusive video content at no additional expense. The online retailer then starts an email campaign for a random subset of users to encourage them to use the streaming services.

The retailer tracks the amount of money spent by its users in a year and compiles the following variables into a dataframe named `video_df`:

`email`: whether or not an individual received the email campaign (received email vs. did not receive email).

`streaming`: whether or not an individual used the streaming services (used service vs. did not use service).

`spending`: the amount of money spent by an individual in a year (dollars)

There are many factors that could influence whether or not an individual on the platform uses the video streaming services. For example, some individuals might have more free time to watch videos. Other individuals might not be interested in the content on the platform. The online retailer has no way to capture this information.

Receiving the email does not directly impact how much someone spends – it only impacts the likelihood of using the streaming platform. Thus, the instrument is whether or not the individual was emailed.

A file named `video_data.csv` has been provided which contains data collected by the online retailer. Import this file as a dataframe and name it `video_df`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment and run the code to examine the first few rows of `video_df`.

```

title: "Two-Stage Least Squares Regression"
output: html-notebook

Checkpoint 1
```{r}
# import data
video_df <- read.csv("video_data.csv")
```

Checkpoint 2
```{r}
# view structure of data
head(video_df)
```
```

## Instrumental Variables

## IV Estimation in R

Performing 2SLS in R is easy if we use the `ivreg()` function from the `AER` package.

The key difference in syntax between `ivreg()` and other regression functions is that the `formula` argument

[Preview: Docs Loading link description](#)

of the `ivreg()` function must include the instrument. If we wanted to perform 2SLS regression with [variables](#)

[Preview: Docs Loading link description](#)

`outcome` as the outcome, `treatment` as the treatment, and `instrument` as the instrument, the model formula would be `outcome ~ treatment | instrument`.

To fit the 2SLS regression using the recycling data, we would use the following code:

```
import library
library(AER)

run 2SLS regression
iv_mod <- ivreg(
 #outcome ~ treatment | instrument
 formula = recycled ~ rebate | distance,
 data = recycle_df
)
```

Copy to Clipboard

To view the coefficients and standard errors, we can use `summary(iv_mod)$coefficients`, which gives the following output (you may need to make this section of the screen wider to view the full table):

|             | Estimate  | Std. Error | t value   | Pr(> t )     |
|-------------|-----------|------------|-----------|--------------|
| (Intercept) | 129.36463 | 0.8683141  | 148.98368 | 0.000000e+00 |
| rebate      | 31.25452  | 1.4629239  | 21.36442  | 5.118885e-68 |

Copy to Clipboard

The results of 2SLS regression show that the estimate of the effect of the rebate program is 31.25, meaning participation in the rebate program led to an average increase in recycling of 31.25 kilograms/person. This only applies to compliers: those individuals who participated in the rebate program because they lived within 5 miles of a recycling center, but who would not have participated otherwise.

You may be wondering why we couldn't just fit the two separate regression models described in the previous exercise using `lm()` or `glm()` functions. The `ivreg()` function is preferred because it automatically corrects standard errors to account for the fact that the second stage regression model uses predicted values of the treatment.

If we use incorrect standard errors, we could make incorrect conclusions about the treatment effect:

Lower standard errors correspond with more precise treatment effect estimates and a greater likelihood that the treatment coefficient will be found to be significantly different from zero.

Higher standard errors correspond with less precise treatment effect estimates and a lesser likelihood that the treatment coefficient will be found to be significantly different from zero.

## Instructions

Checkpoint 1 Enabled

1.

Fit a linear ordinary least squares (OLS) regression model to estimate the effect of use of video streaming services on the amount spent by users of the online retailer. Save this regression model as `ols_model`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Uncomment the code to print a summary of the coefficients from the ordinary least squares (OLS) model.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the `ivreg()` function from the AER package to fit the 2SLS regression model in one step. Make sure to modify the model formula to account for the instrument. Save this model as `iv_mod`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Uncomment the `summary()` function to print the resulting coefficients. How does the estimate differ from the OLS estimate in the last checkpoint?

```

title: "IV Estimation in R"
output: html-notebook

```{r, message=FALSE}
# import library
library(AER)
# load dataset
video_df <- read.csv("video_data.csv")
```

Checkpoint 1
```{r}
# OLS model
ols_model <- lm(spending ~ streaming, data = video_df)
```

Checkpoint 2
```{r}
# print OLS coefficients
summary(ols_model)
```

Checkpoint 3
```{r}
# 2SLS model
iv_mod <- ivreg(
  #outcome ~ treatment | instrument
  formula = spending ~ streaming | email,
  data = video_df
)
```

Checkpoint 4
```{r}
# print 2SLS model
summary(iv_mod)
```

```

## Instrumental Variables

### Interpretation and Considerations of IV Estimation

While IV estimation can be effective in certain circumstances, it also has limitations. One limitation is that the causal estimand (CACE) is not generalizable. The CACE only describes the effect of those who comply with treatment.

Another limitation is that it is difficult to find a suitable instrument that has a strong relationship with the treatment. If an instrument is only weakly related to the treatment, 2SLS regression will produce inaccurate estimates of the CACE.

To illustrate this, suppose that instead of using distance as an instrument for participation in the rebate program, we used another variable, `children`. The variable indicates whether or not an individual has children. Having children wouldn't directly cause a change in recycling, but individuals with children might be less likely to participate in the rebate program. The rebate program requires individuals to take the time to drop off recycling — time that people with children might not have.

Performing the 2SLS regression again with the `ivreg()` function and the `children` variable as an instrument highlights the effect of using a weak instrument:

```

iv_mod_weak <- ivreg(
 formula = recycled ~ rebate | children, #new weak instrument
 data = recycle_df
)

```

```
)
```

Copy to Clipboard

Using `summary(iv_mod_weak)$coefficients` we can view just the coefficient table from the results summary.

|             | Estimate  | Std. Error | t value   | Pr(> t )     |
|-------------|-----------|------------|-----------|--------------|
| (Intercept) | 126.10140 | 8.930242   | 14.120715 | 5.477652e-37 |
| rebate      | 37.84692  | 18.018181  | 2.100485  | 3.631487e-02 |

Copy to Clipboard

The estimate of 37.85 is neither accurate nor precise, as highlighted by the large standard error. This estimate is similar to the estimate from OLS regression, demonstrating that this is a weak instrument.

#### Instructions

##### Checkpoint 1 Enabled

##### 1.

Instead of using the email campaign as an instrument, let's see what happens when we choose a weaker instrument. Imagine that the online retailer decides to pay a search engine company to display an ad for the retailer's video streaming services above the results when anyone uses search terms like "video streaming" or "streaming". Seeing such an ad wouldn't directly increase the amount of money a user spends on the online retailer's website. However, the ad could make a user more likely to use the retailer's video streaming services. Thus, the targeted ads could act as an instrumental variable.

This new variable, `ads`, can be found in a second dataframe called `video_df2` that has been loaded for you in **notebook.Rmd**.

Fit a second 2SLS regression model that uses the `ads` variable as an instrument and save the results to `iv_mod2`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

##### 2.

The original 2SLS model with `email` as an instrument has been saved for you as `iv_mod`.

Print a summary of the results of both 2SLS regression models saved in `iv_mod` and `iv_mod2`. How do the estimates of the treatment effect differ depending on what instrument is used? What about the standard errors?

```

title: "Interpretation and Considerations of IV Estimation"
output: html-notebook

```{r, message=FALSE}
library(AER) #load AER package

# import datasets
video_df <- read.csv("video_data.csv")
video_df2 <- read.csv("video_data2.csv")

# 2SLS model with email as instrument
iv_mod <- ivreg(
  formula = spending ~ streaming | email,
  data = video_df
)
```

Checkpoint 1
```{r}
# 2SLS model with new instrument
iv_mod2 <- ivreg(
  #use new instrument ads
  formula = spending ~ streaming | ads,
  data = video_df2
)
```

Checkpoint 2
```{r}
```

```
# Print summaries for both models
summary(iv_mod)
summary(iv_mod2)
``
```

Instrumental Variables

Conclusion

Congratulations on finishing this lesson on instrumental variable estimation! We covered a lot of topics:

Instrumental variable (IV) estimation is a causal inference technique that can be used to estimate a causal treatment effect even in the presence of unobserved confounding

[variables](#)

Preview: Docs Loading link description

IV estimation can be used in non-randomized studies when compliance with the assigned or encouraged treatment is not perfect.

An instrument is a variable that is related to an outcome of interest ONLY through the treatment variable.

The four assumptions of IV estimation are relevance, exclusion, exchangeability, and monotonicity.

IV estimation is performed via two-stage least squares (2SLS) regression.

The `ivreg()` function in the `AER` package performs 2SLS regression and automatically provides corrected standard errors of the treatment effect.

Instructions

Conclusion:

Great job! The 2SLS regression model using the email campaign as an instrument produced a CACE estimate of 38.056 (as shown in the learning environment). As long as our assumptions hold, this means that using video streaming services increased spending by an average of about 38 dollars. Since this is the CACE, it only applies to compliers: those who used the streaming services because they received the email, but who would not have used streaming otherwise.

Difference in Differences for Causal Inference

Introduction

Imagine that a California state law was passed in 2016 that raised the minimum wage beginning in the year 2017. We are interested in what this law's impact has been on student wages since student jobs often pay low wages.

We have the data on the average student wages at all California public universities from the year 2007 to 2017, displayed in the plot in the learning environment. We note that wages rise and fall from year to year, but we see a particularly large rise from 2016 to 2017.

It's easy to assume the large increase was entirely due to the new minimum wage law. But what would average student wages in 2017 have looked like had the law NOT been passed?

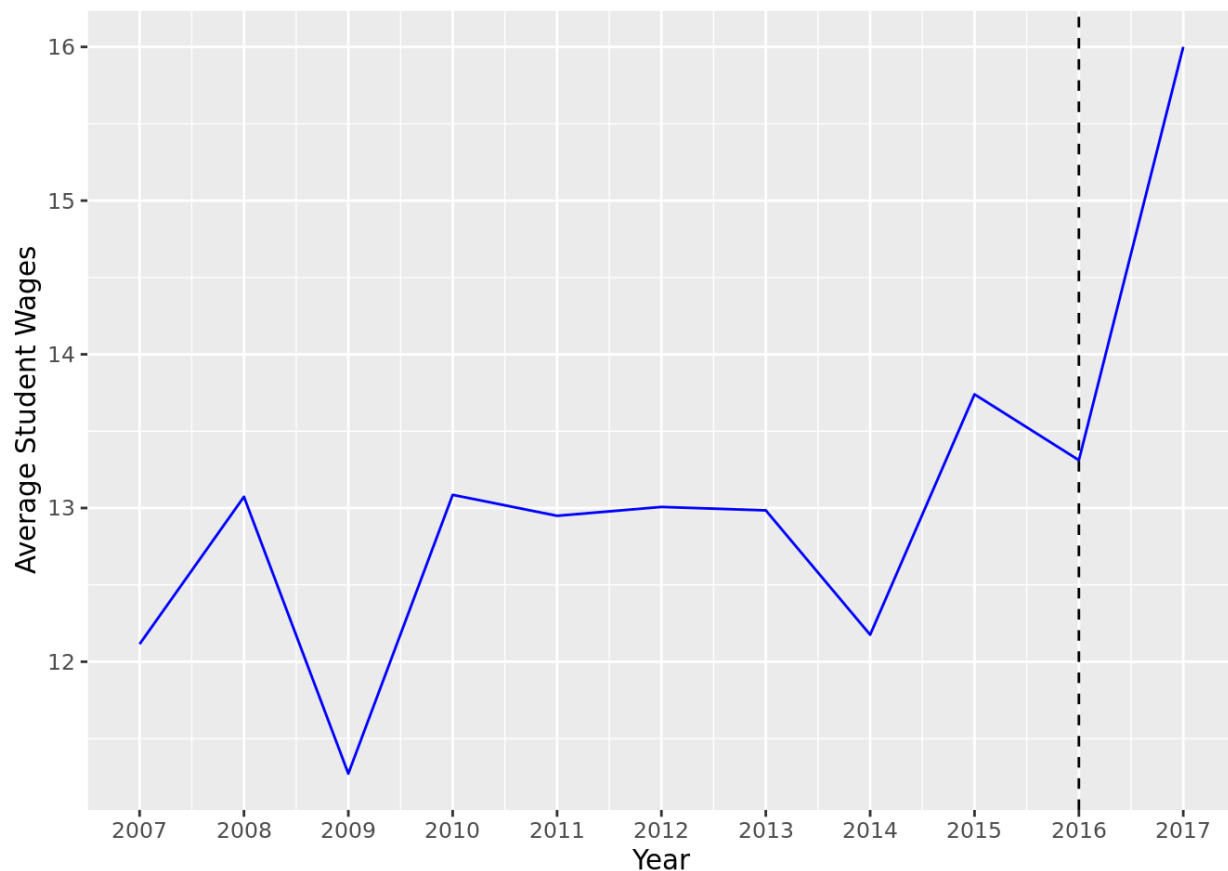
Would wages still have increased a lot, meaning the law had little impact?

Would wages have decreased, meaning the law had an even larger impact than just the difference between 2016 and 2017 wages?

The fundamental principle of causal inference tells us that we can't observe both situations, so we need to use other observed data as a substitute for what student wages in California would have looked like in 2017 had the law not passed.

Difference in differences (DID) is a causal inference technique that estimates a treatment effect by analyzing treatment and control trends over time. In the following exercises, we will walk through how to apply DID to our student wage example, review the different assumptions and limitations of the DID [method](#)

Preview: Docs Loading link description
, and learn how to perform this method using R programming.



Difference in Differences for Causal Inference

Working Example of DID

Let's say we have some data on student wages at the state level in a dataset called `wages`. It has the following variables:

`state`: state where the universities are located
`year`: year the data is from
`avg_wage`: average student wage for all public universities in the state

Start by using the package `ggplot2` in R to re-create the student wage line plot from the last exercise. Since we only want to plot California schools, filter by the `state` variable and then add code for our plot.

```
# import libraries
library(dplyr)
library(ggplot2)
# plot wages versus years
ca_wages <- wages %>%
  #only California schools
  filter(state == "California") %>%
  #wages over time
  ggplot(aes(x = year, y = avg_wage)) +
  #line plot
  geom_line()
```

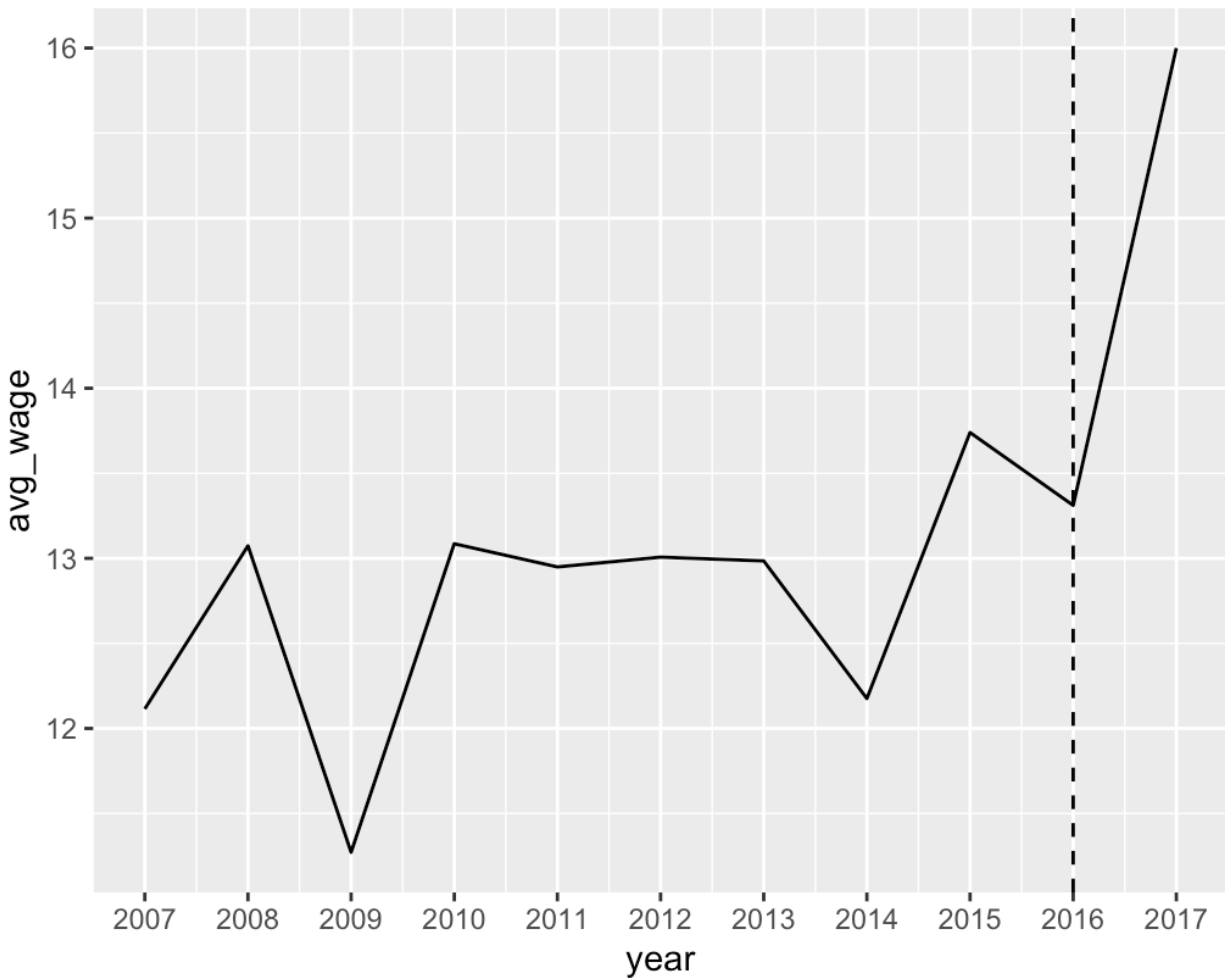
Copy to Clipboard

The situation created by the law is a *natural experiment*. Rather than having a researcher randomly assign treatment and control groups to study minimum wage effects, treatment assignment is decided by some outside force. In this case, that outside force was the minimum wage law that went into effect in 2017.

Let's add a dashed vertical line to our plot to separate the time before and after the law went into effect. We'll also label the x-axis scale to see the years more clearly.

```
ca_wages +
  geom_vline(xintercept = 2016, linetype = "dashed") +
  scale_x_continuous(breaks = c(2007:2017))
```

Copy to Clipboard



We can see that there is some change after the law is implemented in 2016, but we don't know if the change is due to the law or because of other conditions that happened at the same time. We need some data to use as a counterfactual to this situation: what student wages in California would have looked like if the law never happened.

Instructions

Checkpoint 1 Enabled

1.

Let's say there was a new entertainment tax in Sydney starting in 2019. You want to find out if the tax affected movie theater ticket sales. You have data about average annual movie theater ticket sales in Sydney from 2012 through 2019 with the following variables:

`city`: city where the universities are located

`year`: year the data is from

`sales`: average ticket sales for theaters in the city

This data is contained in the dataset `tickets` which has been loaded for you in `notebook.Rmd` with the first few rows printed for you in the workspace.

Make a line plot that shows average movie tickets for Sydney by year. Remember to filter `city` to look at just Sydney. Save the plot as `syd_sales`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add to `syd_sales` a dashed vertical line at `x=2018` and x-axis scale labels for the years 2012 to 2019. What happened to ticket sales in the year the tax was implemented?

```
---
title: "Working Example of DID"
output: html-notebook
```

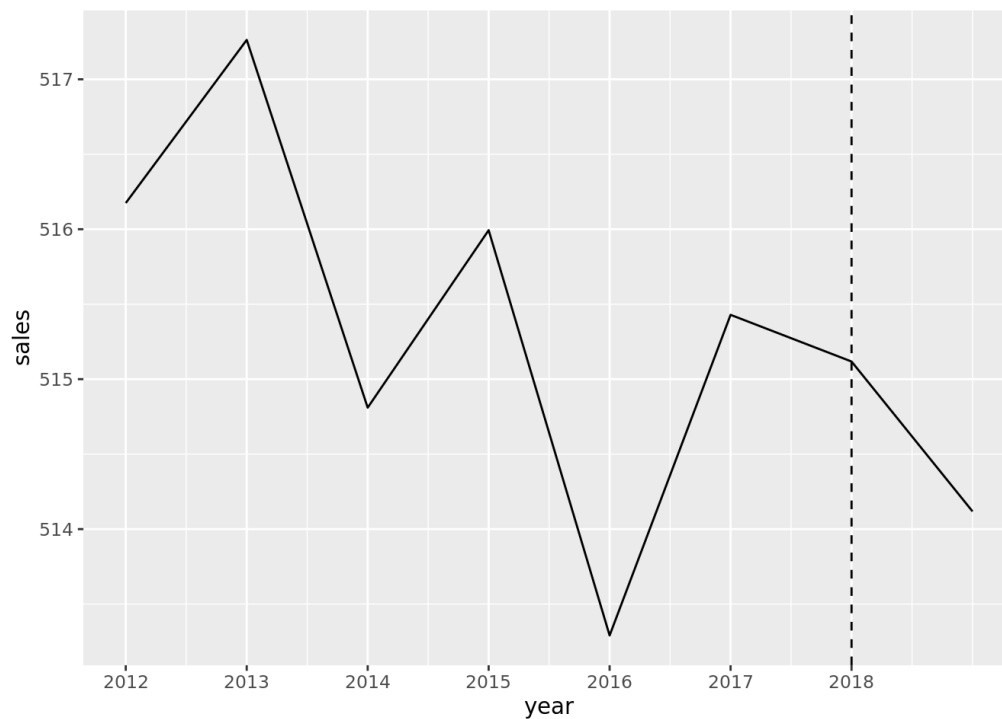
```

---
```{r, message=FALSE}
import libraries
library(dplyr)
library(ggplot2)
import dataset
tickets <- read.csv("ticket_sales.csv")
view dataset
head(tickets)
```

# Checkpoint 1
```{r}
create plot of ticket sales
syd_sales <- tickets %>%
 #only Sydney
 filter(city == "Sydney") %>%
 #sales over time
 ggplot(aes(x = year, y = sales)) +
 #line plot
 geom_line()
```

# Checkpoint 2
```{r}
add line at x=2018 and x-axis label
syd_sales +
 geom_vline(xintercept = 2018, linetype = "dashed") +
 scale_x_continuous(breaks = c(2012:2019))
```

```



Difference in Differences for Causal Inference

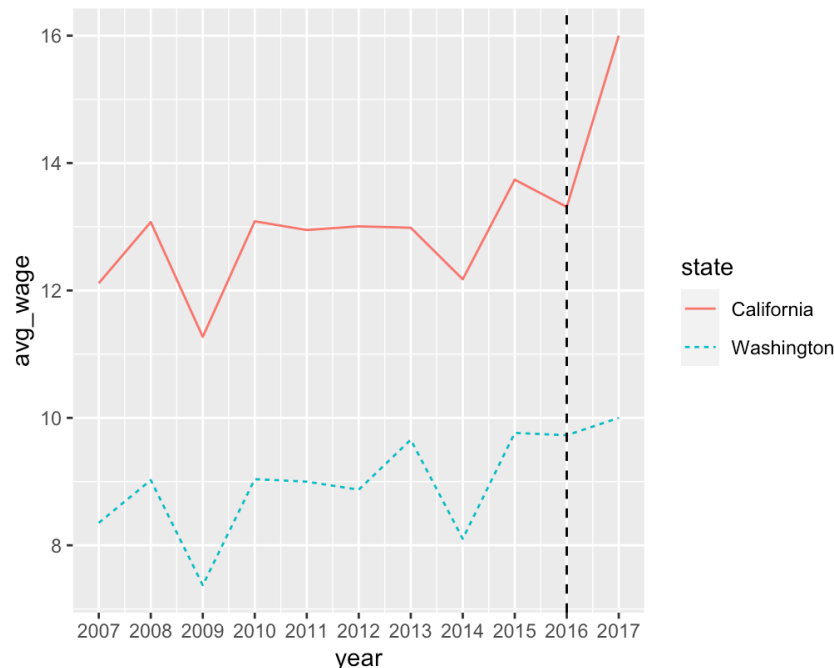
Finding a Control Group

It is impossible to know what truly would have happened in California schools if the law had never happened. However, we can look for another state whose average student wages that follow a similar pattern over time — but did not receive the same treatment. We can use the other state's data to approximate the counterfactual. With DID, we use the *trend* of a control group to estimate the counterfactual trend for the treated. In other words, we use DID to estimate the average treatment effect on the treated (ATT).

In this example, a control trend might be the average student wage of universities in Washington, given that Washington did not implement a new minimum wage law. We can view these two groups in a single plot using the following code:

```
# import library
library(ggplot2)
# plot wages versus years by state
ggplot(data = wages, aes(x = year, y = avg_wage, color = state, linetype = state)) +
  geom_line() +
  #vertical line at 2016
  geom_vline(xintercept = 2016, linetype = "dashed") +
  #x-axis scale 2007 to 2017
  scale_x_continuous(breaks = c(2007:2017))
```

Copy to Clipboard



We can see that in the years leading up to 2016, the average minimum wage at Washington colleges followed a very similar pattern to the California colleges, only shifted lower. We hope that these similar patterns account for other important [variables](#)

Preview: Docs Loading link description

, such as the impact on wages of living costs and necessary student expenses.

Instructions

Checkpoint 1 Enabled

1.

The `tickets` dataset also contains data on average ticket sales in Toronto. Create a line plot that shows the ticket sales for both Toronto and Sydney over the same years. Be sure to change the color and type of line by city. Does Toronto have a similar trend in ticket sales from 2012 to 2018?

Note: you can optionally add a dashed vertical line at 2018 and set the x-axis scale using the functions in the last two lines of code shown in the narrative.

```

title: "Compare Treatment and Control"
output: html-notebook
---

```{r, message=FALSE}
import library
library(ggplot2)
import dataset
tickets <- read.csv("ticket_sales.csv")
```

# Checkpoint 1
```{r}
plot ticket sales for Sydney and Toronto
ggplot(data = tickets, aes(year, sales, color = city, linetype = city)) +
 geom_line() +
 #vertical line at 2018
 geom_vline(xintercept = 2018, linetype = "dashed") +
 #x-axis scale 2007 to 2017
 scale_x_continuous(breaks = c(2012:2019))
```

```

Difference in Differences for Causal Inference

Assumptions

We noted that Toronto may be a good control group for Sydney because the trends in ticket sales follow a similar pattern. This is actually a necessity of DID called the *parallel trends* assumption: trends between the control and treatment groups should be similar prior to the start of treatment.

The specific values between groups do not need to be the same, but the two groups should have the same change over time. We can check this visually by plotting the data by group and verifying that the general trends are similar, as we saw in the previous exercise. A disadvantage to DID is the difficulty of meeting the parallel trends assumption: we need pre-treatment data for both groups AND the data has to show similar trends before treatment.

Another assumption for DID is that the treatment group was selected for the treatment by a factor that is independent of the outcome. This assumption is met for our example: the treatment is determined by state, which is independent of the overall average student wages.

Finally, the assumptions for DID also include the normal assumptions for all causal inference methods:

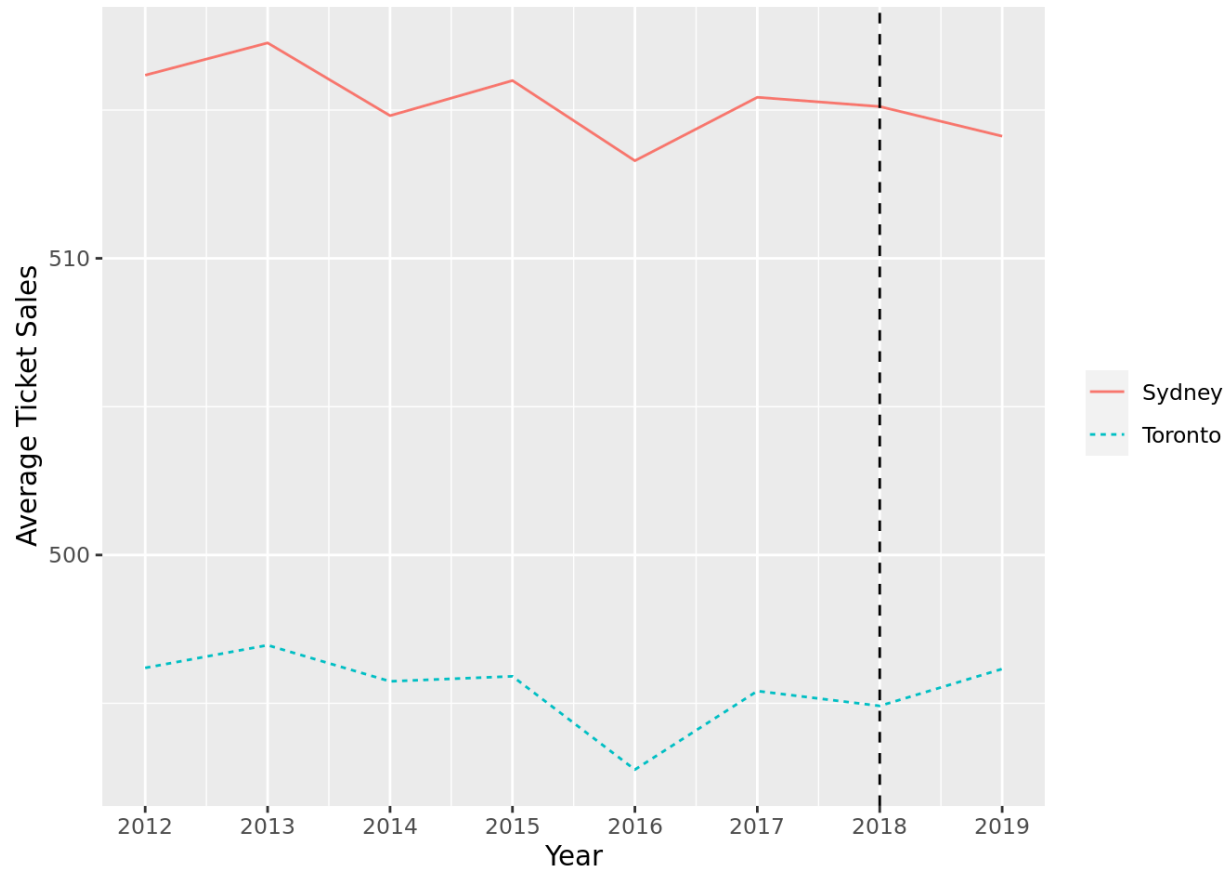
- Conditional exchangeability
- Stable Unit Treatment Value Assumption (SUTVA)
- Overlap

Instructions

The learning environment shows your plot from the ticket sales data in the previous exercise. Take a look at the lines for Toronto and Sydney covering the time from 2012 to 2018, just before the new tax was implemented. Both lines follow a similar trend going up and down slightly each year but with a particular dip in 2016.

Do you think Toronto will make a good approximation of Sydney's counterfactual even though its line is lower on the plot?

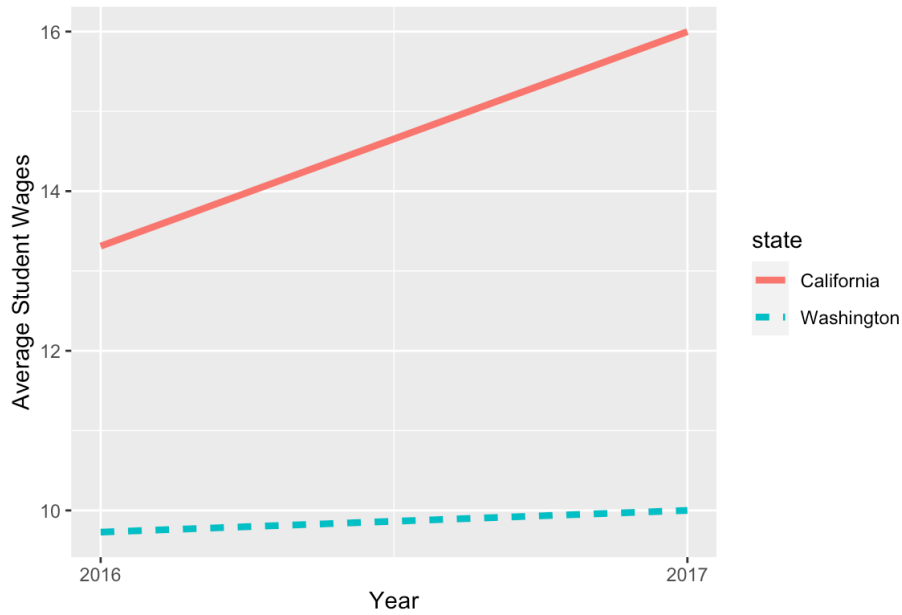
Do you think we've met the other assumptions of DID in this case?



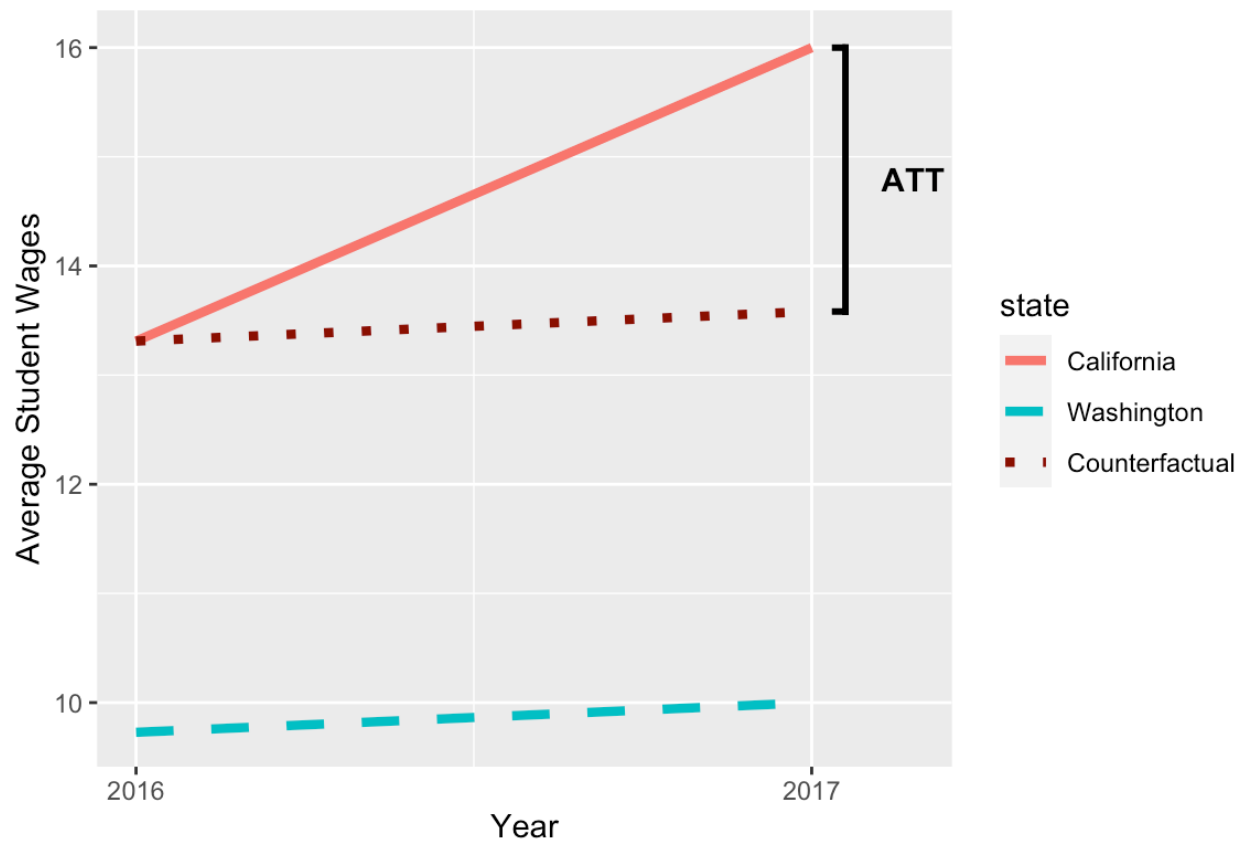
Difference in Differences for Causal Inference

The 2x2 Case

Normally we would want to use all the data we have available to us to perform our DID model. However, that type of analysis can be complex. To keep things simple, we are going to focus on a 2x2 analysis for our student wage data, which means we will look at differences in average wages from the year before the law went into effect (2016) and the year after (2017).



From this plot, we see that the Washington schools had a much smaller increase in average student wages from 2016 to 2017 than the California schools did. We assume that if the law had NOT been passed, the pattern of student wages for the California schools would have been similar to the pattern of wages for the Washington schools. In other words, we assume the slope of the California wages would be the same as that of the Washington wages, as illustrated by the dotted red line in the following plot.



With this new line based on Washington's pattern, we have drawn our approximation of California's counterfactual trend. The difference between California's 2017 observed wages and its 2017 counterfactual wages will be our estimated ATT, as shown in the plot. In the next exercises, we will compute this treatment effect using the approximated counterfactual trend in two ways.

Instructions

Checkpoint 1 Enabled

1.

A new dataset called `tickets2` that only includes the data from 2018 and 2019 has been created for you in **notebook.Rmd**. Uncomment and run the code in the workspace to view a plot of average tickets sales for Sydney and Toronto in those two years.

Based on the plot, consider the following:

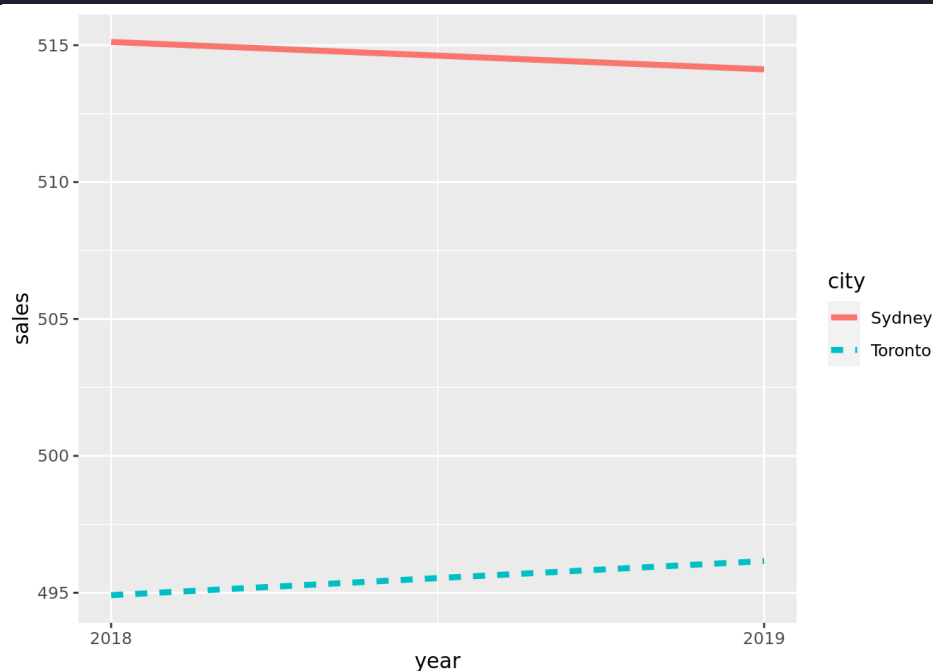
Do you think average ticket sales for Sydney would have increased or decreased had their not been a new tax?

What do you think the impact of the tax was on average ticket sales in Sydney?

```
---
title: "The 2x2 Case"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(ggplot2)
library(dplyr)
import dataset
tickets <- read.csv("ticket_sales.csv")
subset to 2018 and 2019
tickets2 <- tickets %>%
 filter(year > 2017)
```

# Checkpoint 1
```{r}
Uncomment to view plot
ggplot(data = tickets2, aes(x = year, y = sales, color = city, linetype = city)) +
 geom_line(size = 1.5) +
 scale_x_continuous(breaks = c(2018, 2019))
```
```



Difference in Differences for Causal Inference

Estimate Using Difference in Means

One way we can estimate the ATT is by comparing the differences in average wages over both time and treatment groups.

Let's view the mean wages for both states for 2016 and 2017 only.

```
# import library
library(dplyr)
# filter data
wages %>%
  filter(year > 2015) #only 2016 and 2017
```

Copy to Clipboard

The output looks like the following:

```
   state year  avg_wage
1 California 2016 13.311279
2 California 2017 16.000000
3 Washington 2016  9.728146
4 Washington 2017 10.000000
```

Copy to Clipboard

First, let's use the means in our output to take the difference in average wages before and after the law for each state.

California

$\text{mean2017} - \text{mean2016} = \$16.00 - \$13.31 = \2.69

Washington

$\text{mean2017} - \text{mean2016} = \$10.00 - \$9.73 = \0.27

Hypothetically, if the law had no effect on wages, we would expect the difference between the post-treatment mean and the pre-treatment mean for the treatment group would be similar to the difference for the control group. However, these findings match what we saw in our plots before: California experienced a MUCH LARGER change in average wages from 2016 to 2017 than Washington did.

But how much of that California change was due to factors other than the new law? We answer this question by taking the difference of the differences we just found — which is where this method gets its name!

$\text{ATT} = \text{differenceCalifornia} - \text{differenceWashington}$

$\text{ATT} = \$2.69 - \0.27

$\text{ATT} = \$2.42$

According to our basic 2x2 DID that uses mean differences alone, we estimate the average treatment effect on the treated to be \$2.42. But there's still another DID method we might want to try!

Instructions

Checkpoint 1 Enabled

1.

The `tickets` dataset has been filtered to just the years 2018 and 2019 and saved for you as `tickets2` in `notebook.Rmd`. Print `tickets2` and inspect the average ticket sales for each city.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Using the `tickets2` output and either R or a calculator, compute the change in average tickets sales from 2018 to 2019 for each city. Save Toronto's difference as `Tdiff` and Sydney's difference as `Sdiff`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Compute the DID estimate for the impact of the new tax by subtracting Toronto's difference from Sydney's difference. Save this value as `did_means`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print `did_means` and consider its value. What does this tell you about the impact of the new tax on average ticket sales in Sydney?

```
---
title: "Estimate Using Difference in Means"
output: html-notebook
---

```{r, message=FALSE}
import libraries
library(ggplot2)
library(dplyr)
import dataset
tickets <- read.csv("ticket_sales.csv")
subset to 2018 and 2019
tickets2 <- tickets %>%
 filter(year > 2017)
```

# Checkpoint 1
```{r}
print tickets2
print(tickets2)
```

# Checkpoint 2
```{r}
save values of Tdiff and Sdiff
Tdiff <- tickets2$sales[4] - tickets2$sales[3]
Sdiff <- tickets2$sales[2] - tickets2$sales[1]
rounded alternative:
Tdiff <- 496.16 - 494.91
Sdiff <- 514.12 - 515.12
```

# Checkpoint 3
```{r}
subtract Tdiff from Sdiff
did_means <- Sdiff - Tdiff
```

# Checkpoint 4
```{r}
print did_means
print(did_means)
```
```

Difference in Differences for Causal Inference

Estimate Using Linear Regression

While we were able to estimate the ATT through mean differences alone, we can also use linear regression for DID. A simple DID regression model predicts the outcome from the

[variables](#)

Preview: Docs Loading link description

for treatment group and time, along with the interaction of treatment with time.

To simplify our output, we will first transform the `state` variable to a treatment indicator called `treat` (1 for California and 0 for Washington). Then we transform `year` to a time indicator called `time` (1 for 2017 and 0 for 2016).

```
# transform state to treat
wages2$treat <- ifelse(wages2$state=="California",1,0)
# transform year to time
wages2$time <- ifelse(wages2$year==2017,1,0)
```

Copy to Clipboard

We can print and inspect our dataset to make sure our transformations were done correctly.

```
      state treat year time  avg_wage
1 California    1 2016    0 13.311279
2 California    1 2017    1 16.000000
3 Washington    0 2016    0  9.728146
4 Washington    0 2017    1 10.000000
```

Copy to Clipboard

To create a DID regression for our student wage data, we run a model that predicts average student wages from the treatment, time, and the interaction of treatment and time. Note that in R, `treat*time` is equivalent to `treat + time + treat:time`.

```
did_mod <- lm(
  #include interaction
  avg_wage ~ treat*year,
  #use subsetted data
  data = wages2
)
```

Copy to Clipboard

When we print `did_mod` we get the following output. Note that the coefficient on the interaction term `treat:year` is exactly what we computed for the ATT by taking the difference of means. We estimate the impact of the minimum wage law on California student wages to be an increase of about \$2.42.

```
Call:
lm(formula = avg_wage ~ treat * time, data = wages2)
```

```
Coefficients:
(Intercept)      treat        time  treat:time
    9.7281      3.5831      0.2719      2.4169
```

Copy to Clipboard

What do all the other coefficients represent?

`Intercept` indicates the expected value for pre-treatment average student wages for the control group (Washington 2016).
`treat` is the difference between the control group and the treatment group at the pre-treatment time (California 2016 - Washington 2016).
`time` is the difference between the pre-treatment and post-treatment times for the control group (Washington 2017 - Washington 2016).

Combinations of these coefficients give us back all four means from our dataset.

| Mean | Equivalent Coefficient(s) |
|-----------------|---------------------------------------|
| Washington 2016 | Intercept |
| California 2016 | Intercept + treat |
| Washington 2017 | Intercept + time |
| California 2017 | Intercept + treat + time + treat:time |

Instructions

Checkpoint 1 Enabled

1.

The dataset `tickets2` with only 2018 and 2019 data has been saved for you in `notebook.Rmd`. Use the `ifelse()` function to add to `tickets2` the indicator variable `treat` that is 1 for Sydney and 0 for Toronto.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use the `ifelse()` function to add to `tickets2` the indicator variable `time` that is 1 for 2019 and 0 for 2018.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create a linear regression model `did_reg` that predicts average tickets sales from `treat`, `time`, and the interaction of `treat` with `time`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Now print `did_reg` and inspect the coefficient on the interaction term. was the direction of the impact what you expected based on your earlier plots?

```
---
title: "Estimate Using Linear Regression"
output: html-notebook
---

```{r, message=FALSE}
import library
library(dplyr)
import dataset
tickets <- read.csv("ticket_sales.csv")
filter to only 2018 and 2019
tickets2 <- tickets %>%
 filter(year > 2017)
```

# Checkpoint 1
```{r}
add treatment indicator
tickets2$treat <- ifelse(tickets2$city=="Sydney",1,0)
```

# Checkpoint 2
```{r}
add time indicator
tickets2$time <- ifelse(tickets2$year==2019,1,0)
```

```

...

Checkpoint 3
```{r}
# save linear model
did_reg <- lm(
  #include interaction
  sales ~ treat*time,
  #subsetting data
  data = tickets2
)
...

# Checkpoint 4
```{r}
view model results
print(did_reg)
...

```

## Difference in Differences for Causal Inference

### Review

Congratulations! You've completed a lesson on the fundamentals of difference in differences for causal inference. Through applied exercises you learned:

- DID uses the trend of a control group as an approximate counterfactual trend for the treatment group.
- DID requires that we have pre-treatment data for both groups and that we meet the parallel trends assumption.
- We can compute the ATT by taking mean differences or through linear regression.

You may be wondering why we would use linear regression at all when we can just use means. Regression allows us to control for confounding

[variables](#)

Preview: Docs Loading link description

as well as get estimated standard errors. We explored simple models in this lesson to get a strong foundation, but more complex DID analyses benefit from the added flexibility of regression modeling.

## Estimate Using Linear Regression

[Code](#)

```
city year sales treat time
1 Sydney 2018 515.1175 1 0
2 Sydney 2019 514.1180 1 1
3 Toronto 2018 494.9121 0 0
4 Toronto 2019 496.1579 0 1
```

```
Call:
lm(formula = sales ~ treat * time, data = tickets2)
```

```
Coefficients:
(Intercept) treat time treat:time
 494.912 20.205 1.246 -2.245
```

What do all the other coefficients represent?

- **Intercept** indicates the expected value for pre-treatment average outcome for the control group.
- **treat** is the difference between the control group and the treatment group at the pre-treatment time.
- **time** is the difference between the pre-treatment and post-treatment times for the control group.

Combinations of these coefficients give us back all four means from our dataset.

Mean	Equivalent Coefficient(s)
Toronto 2018	Intercept
Sydney 2018	Intercept + treat
Toronto 2019	Intercept + time
Sydney 2019	Intercept + treat + time + treat:time

Done4 / 15 done

### Collect Data

If you didn't already have a dataset in mind, search the internet and see if it's freely available. If data is not available, go back to the previous step and find a new problem that will have data readily available.

We recommend using a business, marketing, or policy dataset that lends itself well to the field of data science.

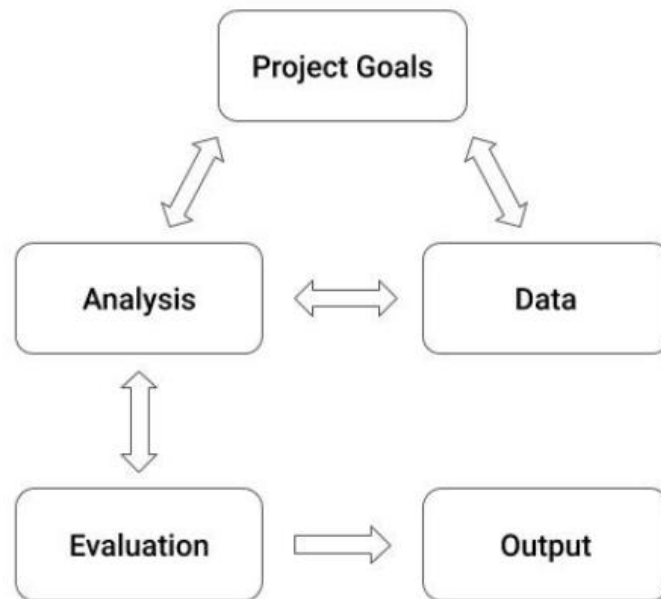
### Select Data Science Solvable Problem

First, choose a topic of interest; then, within your topic, select a problem that you can answer using hypothesis testing or causal inference. Make sure that this problem can be solved in a reasonable amount of time and with available data. If the problem is too complex, it might be worth breaking it down into smaller units.

### Project Scoping

Properly scoping your project will greatly benefit you; scoping creates structure while requiring you to think through your entire project before you begin. You should start with stating the goals for your project, then gathering the data, and considering the analytical steps required. A proper project scope can be a great road map for your project, but keep in mind that some down-stream tasks may become dead ends

which will require adjustment to the scope.



### Setting up your Git Repository

Create a new Git repository for this project.

### Load, Check, and Inspect Data

Once you obtain your data, check to see if the data has the right [unit of observation](#) or granularity. Make sure it has the variables that you need for your exploratory data analysis and that there is enough data.

### Data Wrangling and Tidying

Clean your data and make sure it is ready for analysis.

### Explore the Data

Once your data is cleaned and ready to go, explore it through the use of summary statistics and data visualizations. Make sure to document findings along the way.

### Transform your Variables as Needed

Scale and transform any variables to ensure that you can make valid comparisons.

### Set up Your Hypothesis Tests

Establish your null hypothesis, define your confidence level, and prepare your data to address the questions you are trying to ask.

### Conduct your Tests

Run the associations, and perform the calculations necessary to address your question. Be sure to keep track of the tests you perform so you can record your results later.

**Choose Causal Analysis Method**

If you choose to try to answer a causal question, decide which method is most appropriate. Be sure to check that statistical assumptions have been met.

**Run Causal Model**

If you choose to try to answer a causal question and have decided on an appropriate method, run your model in an R Markdown file to take advantage of the tools available in R for causal inference.

**Communicate Findings**

Make your visualizations and documented findings easy to decipher for potential stakeholders and/or the general public.

**Conclusion****Next Steps**

Lastly, being a data scientist is about making continual progress. What are future-looking takeaways and limitations? If you were to do this project again, how can you extend this analysis?

# Data Scientist: Inference Specialist Career Path Review

## Review what you learned in the Data Scientist: Inference Career Path!

Congratulations on all your hard work building your data science knowledge and statistical and inference fluency. Everything you've learned in this Career Path will continue to serve you for years to come!

At this point, you have the technical knowledge, skills, and abilities you need to get your start as a Data Scientist with a specialty in Inference.

You are now able to:

- Create programs with Python 3 and R
- Query and manipulate data with SQL and Python pandas
- Create data visualizations with Python matplotlib and seaborn
- Summarize and analyze datasets
- Conduct hypothesis testing
- Clean and tidy datasets
- Communicate your findings clearly
- Run experiments and interpret the results
- Perform advanced SQL queries to answer questions with data
- Explore the relationship between variables with Causal Inference

Plus, you now have several portfolio projects that showcase what you've learned in this Career Path and can share these projects with your community and future employers.

Once again, congratulations on finishing a challenging learning journey! We hope that you continue to build on your skills for years to come.

## Next Steps after Data Scientist: Inference Specialist Career Path



## You've finished the Data Scientist: Inference Specialist Career Path – what will you learn next?

Congratulations (again) on completing the Data Scientist: Inference Career Path! There are a lot of places you can go with your new skills: You could work as a data scientist, in strategy, or experimental design, or create hobby projects. As you know, there is always more to learn.

So, now that you have finished this learning journey, you may be asking, "What's next?"

## Interview Preparation

Now that you have completed the Career Path, you are ready to start applying to jobs (though we realize you may have already started this). There is a lot that goes into interviewing for data scientist roles, from developing a portfolio to passing a take home, as well as multiple rounds of interviews. We have you covered with a [Data Scientist Interview Preparation](#) Skill Path, where you will learn about the stages in the interview process, practice sample take-home projects, and review everything you need to know to pass the interview and earn a job as a Data Scientist.

## R

Particularly for Inference Data Scientists, R is the other major programming language. You've already gotten an introduction to R with the R for Programmers track, but we recommend further developing your R skills since its development community is built for Inference Data Scientists. We have a [Learn R](#) course that covers the basics of the language in more detail, or there's the [Analyze Data with R](#) Skill Path where you can get started with R for data analysis.

## Other Areas of Data Science

While you've completed your Data Scientist Career Path with an Inference specialization, there is still a lot more to learn about data science as a field. From here, you can build your visualization skills with [Tableau](#), your Machine Learning Skills, or move into [Natural Language Processing](#).

## Data Science Events

Events give you a chance to connect with other data scientists and learn about the latest topics in the field. There are a lot of events out there including [PyData](#), [rstudio](#), [Women in Analytics](#), [Gartner Data & Analytics Summit](#), [The Data Science Conference](#), and the list goes on! You can also join or create a [Codecademy meetup](#) to connect with other new data analysts.

Once again, congratulations! We're excited to see what you accomplish next.

